ITC327W – Unit 1, 2 & 3 Portfolio

Software Requirements Specification (SRS)

Phehello Secondary School – Online Enrolment System

Group Name: Ctrl-Alt-Defeat

Group members:

224131387 KI. Makhoali
221000537 PIJ. Lebusa
220049867 LR. Mahlatsi
222018199 M. Lepitla
224074632 MM. Motaung
220042034 DT. Adegbola

Module: ITC327W

Table of Contents

Part A – Initial SRS (Activities 1.1 & 1.3)

- 1. Project Title 2. Project Description 3. Problem Statement 4. Purpose 5. Project Scope
- 6. Definitions 7. Stakeholders & Users 8. Current Process 9. Product Perspective 10. Main Product Functions
- Requirements (FR / NFR / SR / IR) Prioritised Requirements Table

Part B – Appendices & Project Setup Evidence (Activities 1.2 & 1.3)

- Appendices A–E & Project Setup Evidence

Part C – Unit 2: Feasibility, Risk & Refined Scope (Activities 2.1–2.3)

- Feasibility Study · Project Risk Register · Refined Scope · Requirements Change Log · Group Reflection

Part D – Unit 3: System Architecture and Design

Part E – Unit 4: System Development and Implementation

- Activity 4.1 – Set Up and Implement the Shared Supabase Backend
- Activity 4.2 – Develop the Flutter Mobile Application
- Activity 4.3 – Develop the ASP.NET Web Application
- Activity 4.4 – Integrate the Mobile and Web Applications
- Development Challenges, Changes and Project Records
- Requirements Implementation Status and Readiness for Unit 5

APPENDIX F – Database Data Dictionary

PART A: INITIAL SRS

1. Project Title

Phehello Secondary Online Enrolment System

2. Project Description

This project is being developed as part of ITC327W Work-Integrated Learning for Phehello Secondary School, a public secondary school whose enrolment process is currently entirely manual and paper-based. The group is developing a prototype online enrolment system consisting of a Flutter mobile application for parents/guardians, an ASP.NET web application for school staff (a Financial Clerk and an Administrator), and a shared Supabase backend. The system will allow parents to apply for enrolment, upload required documents and proof of payment, and track application status online, while staff verify payments and review applications digitally instead of on paper. The stakeholder is the Principal of Phehello Secondary School, who confirmed the current process, problems, and requirements during a stakeholder needs-assessment interview conducted via Microsoft Teams on 24 August 2026 (see Appendix B). The stakeholder was not part of the group's original plans; it was identified while searching for a real stakeholder within the local community, when the group walked past Phehello Secondary and decided to approach it directly rather than continue searching elsewhere. The group chose to proceed with this stakeholder because enrolment is a problem every member has personally experienced - each member's own parents had to make repeated trips back and forth to complete their own high school enrolment - making it a genuine, relatable problem worth investigating.

3. Problem Statement

Phehello Secondary School currently uses a manual, paper-based process to enrol new learners. Parents collect, complete, and physically deliver enrolment forms and supporting documents (a copy of the birth certificate, immunisation records, a progress report from the previous school, and a transfer letter, as specified on the school's enrolment form) to the school in person, and must pay an enrolment fee to the financial clerk before the application is finalised. This results in time-consuming, often repeated trips to the school - particularly when documentation is incomplete or incorrect - slow manual processing split across a financial clerk and an administrator, and no way for parents to track application or payment status. The problem affects parents/guardians (especially young or working parents, for whom repeat trips are costly in time and money) and school staff, since the administrator and financial clerk each spend significant one-on-one time per application, limiting their capacity for other duties. An improved solution is needed to let parents apply, upload documents and proof of payment, and track status online, while letting the financial clerk verify payments and the administrator review, approve/reject, and communicate with parents digitally.

4. Purpose

The purpose of the prototype is to improve how Phehello Secondary School receives, verifies, and manages learner enrolment applications, by allowing parents to apply and upload documents remotely and allowing school staff to review and process applications digitally rather than on paper.

5. Project Scope

Scope element	Group response
Project aim	To develop an integrated prototype that allows parents to apply for their child's enrolment at Phehello Secondary online, including uploading required supporting documents, and allows school administration staff to review, verify, and process applications digitally.
Main objectives	Allow parents to complete and submit an online enrolment application. Allow parents to upload required documents. Allow parents to track application status. Allow admin staff to view, verify and approve/reject applications. Provide a secure record of applications and documents
Deliverables	Flutter mobile application prototype (parent-facing)

	ASP.NET web admin application prototype (school-facing) Supabase backend/database and document storage Project documentation
Acceptance criteria	Parents can complete and submit a complete enrolment application through the mobile application. Parents can upload the required supporting documents. Submitted applications appear in the ASP.NET web administration application. Authorised admin staff can review an application and update its status. Updated application status is visible to the parent through the mobile application. Application information and uploaded documents are stored securely in Supabase.
Exclusions	Integration with Department of Education systems (e.g., SA-SAMS) Online fee/payment processing. The system will not process or transfer money online. It will display the stakeholder-confirmed enrolment fee and school banking details to the parent/guardian and allow the parent/guardian to enter a payment reference and upload proof of payment for verification. Actual payment processing remains outside the system. Online medical consultations Province-wide deployment Production use of real learner records Advanced analytics beyond basic application information/status Full school management functionality beyond enrolment
Constraints	Project must be completed within the academic project period. Development limited to Flutter, ASP.NET and Supabase Testing relies on sample/mock data. Access to the school's actual infrastructure may limit on-site testing. The school will not provide real learner/student personal information for the project
Assumptions	Principal and staff will be available for scheduled meetings. School will provide a sample of its current enrolment form. Target parents have access to a smartphone and basic connectivity

6. Definitions and Abbreviations

Term / Abbreviation	Meaning
SRS	Software Requirements Specification
Flutter	Framework used to develop the mobile application
ASP.NET	Framework used to develop the web application
Supabase	Shared online backend and database
WIL	Work-Integrated Learning

7. Stakeholders and User Groups

Stakeholder / user	Role in the proposed system	Responsibilities/Interest
Maleshoane Gweneth Adegbola	Principal, Phehello Secondary School - project sponsor and primary point of contact	Provides overall stakeholder direction, confirms the school's needs and approves important project findings and decisions.
Zukiswa Khondlo	Secretary to the Principal and delegated stakeholder representative responsible for working with the project group and participating in requirements elicitation.	Provided information about the current enrolment process, participated in the requirements interview, confirmed the findings on behalf of the school.
Parent/Guardian	Primary user of the mobile enrolment application	Registers, completes enrolment applications, uploads required documents and proof of payment, and tracks application status.
Financial Clerk	School staff member user	Reviews submitted proof of payment and verifies payment information.
Administrator	Staff member user	Reviews applications, verifies documents, approves/rejects applications, communicates with parents and manages the enrolment process.

Stakeholder Delegation: The Principal, Maleshoane Gweneth Adegbola, is the official stakeholder and project sponsor for the project. During the initial scoping meeting on 20 August 2026, the principal confirmed her willingness to act as the group's official stakeholder. Due to her work commitments and limited availability, she delegated Zukiswa Khondlo, her secretary to work with the group as her stakeholder representative. Zukiswa therefore participated in the formal requirements interview and provided information about the school enrolment process and requirements and confirmed the interview findings on behalf of Phehello Secondary School on the 24 August 2026.

Stakeholder Analysis Table

Stakeholder	Role	Interest	Influence	Main need
Maleshoane Gweneth Adegbola	Official Stakeholder Project Sponsor	High	High	Successful digital enrolment and alignment with the school needs
Zukiswa Khondlo	Delegated Stakeholder Representative	High	Medium	Accurate requirements and an enrolment process that reflects the school's current procedures
Administrator	System user	High	High	Review, verify, approve/reject and manage applications
Financial Clerk	System user	High	Medium	Verify submitted proof of payments and payment information.
Parents/Guardians	End user	High	Medium	Apply remotely and track status

8. Current Process

Parents/guardians who wish to enrol a child at Phehello Secondary must obtain a paper enrolment form, complete it by hand, and submit it in person at the school together with the required supporting documents. According to the enrolment form provided by Phehello Secondary School, the required supporting documents for an accepted learner are a copy of immunisation records, a copy of the birth certificate, a progress report from previous school and a transfer letter from the previous school. Before enrolment is finalised, a fee is paid to the financial clerk, who issues a payment slip. The administrator then reviews the application and verifies the submitted documentation.

People involved

- Parents/guardians: complete and submit the enrolment form, supporting documents, and payment.
- Financial Clerk: receives the enrolment fee and issues a payment slip and verifies the proof of payment submitted with the application.
- Administrator: reviews the application, checks the forms are completed correctly, verifies the documentation and manages the application process.

Information used

- Enrolment/registration form (paper-based)
- Copy of immunisation records
- Copy of birth certificate
- Progress report from the previous school
- Transfer letter from the previous school
- Proof of payment (payment slip issued by the financial clerk)

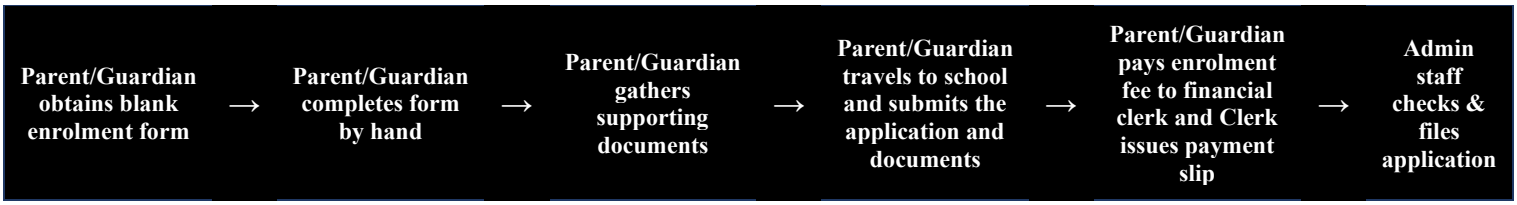
Current tools or methods

- Paper enrolment forms and physical document submission, payment slips and manual filing; no digital system currently used for enrolment.

Problems experienced

- **Parents /Guardians:** incorrect or incomplete documentation may require parents to travel back to the school to correct their applications, resulting in additional time and transport costs.
- **Staff workload:** The financial Clerk and Administrator spend significant time handling applications and assisting parents individually, reducing the time available for their other responsibilities.
- **Paper-based records:** Paper forms and supporting documents can be lost, damaged, or incompletely filled in, making applications difficult to manage and retrieve.
- **No status tracking**: Parents cannot easily check the progress of their enrolment or payment status without contacting or returning to the school.

Current Process Diagram



9. Product Perspective

The proposed prototype will consist of two connected applications supported by a shared Supabase backend. The Flutter mobile application will be used by parents or guardians to register, complete and submit learner enrolment applications, upload required supporting documents and proof of payment, and track application status. The ASP.NET web application will be used by authorised school staff, including the Financial Clerk and Administrator, to verify payment information, review supporting documents, process applications, and update application statuses. Both applications will use the same Supabase backend so that information submitted through the mobile application can be accessed by authorised staff through the web application and status updates can be reflected in the parent's application view.

10. Main Product Functions

Component	Main functions
Flutter mobile application	Register/sign in, complete and submit enrolment application, upload required documents, view the stakeholder-confirmed enrolment fee and school banking details, enter a payment reference, upload proof of payment, and view application/payment status.
ASP.NET web application – Financial Clerk	Sign in, view submitted proof of payment, verify payment information proof of payment against school records and update payment verification status.
ASP.NET web application - Administrator	Sign in, view submitted applications, review and verify documents, approve/reject/request corrections, leave notes, message parents about incorrect documents, send deadline reminders, view a running application count, generate confirmation letters.
Supabase backend	Store parent and learner information, enrolment applications, uploaded documents, proof of payment, application status, each learner's unique student number and provide shared data access for the mobile and web applications.

11. Requirements

Functional Requirements (FR)

The following functional requirements define what the proposed online enrolment system must do. The requirements are based on the current enrolment process and the findings confirmed during the formal stakeholder requirements interview on 24 August 2026.

ID	Requirement
FR1	The parent must be able to register and sign in through the Flutter application.
FR2	The parent must be able to complete and submit an enrolment application through the Flutter application.
FR3	The parent must be able to upload the required supporting documents (copy of birth certificate, a progress report from the previous school, a copy of immunisation records and a transfer letter from the previous school) through the Flutter application.
FR4	The parent must be able to upload proof of payment (payment slip) through the Flutter application.
FR5	The parent must be able to view the status of a submitted application, including payment verification status, through the Flutter application.
FR6	The Financial Clerk must be able to verify a parent's proof of payment against the school's payment records through the ASP.NET application.
FR7	The Administrator must be able to view submitted applications through the ASP.NET application.
FR8	The Administrator must be able to approve, reject, or request corrections on an application through the ASP.NET application.
FR9	The Administrator must be able to leave notes on an application through the ASP.NET application.
FR10	The Administrator must be able to message a parent about incorrect or missing documents through the ASP.NET application.
FR11	The Administrator must be able to send reminders to parents about application deadlines.
FR12	The Administrator shall be able to preview and generate a confirmation letter for an approved or rejected application, so that the parent/guardian receives a formal record of the application outcome.

FR13	The Administrator must be able to view a running count of applications received through the ASP.NET application.
FR14	The system must store submitted applications, documents, and proof of payment in Supabase.
FR15	The parent/guardian must be able to view the stakeholder-confirmed enrolment fee and school banking details through the Flutter application and enter a payment reference before uploading proof of payment.

Non-Functional Requirements (NFR)

ID	Requirement
NFR1	Only authorised staff (Financial Clerk or Administrator, per their role) must be able to access the ASP.NET administration dashboard.
NFR2	Uploaded documents and proof of payment must be stored securely and only accessible to authorised users.
NFR3	The prototype must use test/mock data instead of real learner records.
NFR4	Error messages shown to parents and staff must be clear and understandable.
NFR5	The mobile interface must display correctly on a standard Android smartphone.
NFR6	The system must enforce role-based access control. The Financial Clerk must only be able to access payment-related information and verify proof of payment. The administrator must be able to access applications and supporting documents and must be the only staff role authorised to approve, reject, or request corrections on an application.

System Requirements (SR)

ID	Requirement
SR1	The mobile application must be developed using Flutter.
SR2	The web application must be developed using ASP.NET.
SR3	The shared backend must use Supabase.
SR4	The source code must be stored in a GitHub repository.

Integration Requirements (IR)

ID	Requirement
IR1	Applications and documents submitted through Flutter must be stored in Supabase.
IR2	Applications stored in Supabase must be displayed in the ASP.NET application.
IR3	Status updates made through ASP.NET must be reflected in the Flutter application.

Prioritised Requirements Table

ID	Requirement	Type	Priority	Related stakeholder needs
FR1	Register/sign in via Flutter	Functional	Must Have	Needed for any parent to use the mobile application
FR2	Submit enrolment application via Flutter	Functional	Must Have	Reduce in-person form submission

FR3	Upload required documents (ID, clinic card, etc.) via Flutter	Functional	Must Have	Reduce reliance on physical document submission and repeat trips for incorrect documentation
FR4	Upload proof of payment via Flutter	Functional	Must Have	Fee must be paid before enrolment is finalised; financial clerk needs to verify it
FR5	View application/payment status via Flutter	Functional	Must Have	Parents currently cannot check status without returning to the school
FR6	Financial Clerk verifies payment via ASP.NET	Functional	Must Have	Matches the financial clerk's actual role in the current process
FR7	Administrator views applications via ASP.NET	Functional	Must Have	Reduce manual filing/checking of paper forms
FR8	Administrator approves/rejects/requests corrections via ASP.NET	Functional	Must Have	Speed up application processing
FR9	Administrator leaves notes on an application	Functional	Should Have	Supports the administrator's existing review workflow
FR10	Administrator messages parents about incorrect documents	Functional	Must Have	Directly addresses the #1 challenge raised: incorrect documentation causing repeat trips
FR11	Administrator sends deadline reminders	Functional	Should Have	Requested during the stakeholder requirements interview to help manage the enrolment process
FR12	System generates confirmation letters	Functional	Must Have	Explicitly requested - includes school logo, learner name, ID number, student number
FR13	Administrator views running application count	Functional	Should Have	Explicitly requested reporting need
FR14	Store applications & documents in Supabase	Functional	Must Have	Maintain a searchable digital record
FR15	View enrolment fee and school banking details and enter payment reference via Flutter	Functional	Must Have	Parents/guardians need the required payment information before making payment and submitting proof of payment. The payment reference allows the submitted payment to be associated with the relevant payment record.
NFR1	Restrict admin dashboard to authorised staff	Non-functional	Must Have	Protect learner personal information
NFR2	Secure document & payment storage	Non-functional	Must Have	Protect sensitive documents and payment proof

NFR3	Use test data in prototype	Non-functional	Must Have	POPIA / data-privacy compliance
NFR6	Separate Financial Clerk and Administrator permissions	Non-functional	Must Have	Matches the two distinct staff roles identified in the interview
SR1	Mobile app built in Flutter	System	Must Have	Module requirement
SR2	Web app built in ASP.NET	System	Must Have	Module requirement
SR3	Shared backend in Supabase	System	Must Have	Module requirement
IR1	Flutter submissions stored in Supabase	Integration	Must Have	Applications must reach staff without manual re-entry
IR2	Supabase records shown in ASP.NET	Integration	Must Have	Staff need to see applications and payments as they arrive
IR3	ASP.NET status updates reflected in Flutter	Integration	Should Have	Parents need up-to-date status

PART B: APPENDICES & PROJECT SETUP

Appendices

Appendix	Evidence	Status
Appendix A	Stakeholder interview questions	See Part B below
Appendix B	Stakeholder responses / meeting notes	Attached - full interview conducted 24 August 2026 (Microsoft Teams); see Interview Record and Minutes Summary below
Appendix C	Evidence of stakeholder engagement	Attached - stakeholder identity, WhatsApp/Teams scheduling evidence, and Meeting Charter & Minutes below
Appendix D	Stakeholder confirmation of findings	Attached - findings summary, confirmation table, and evidence below
Appendix E	Additional current-process evidence	Attached - see images below

Project Setup Evidence

Item	Evidence	Status
Microsoft Project schedule	Original .mpp file with 3 phases, tasks, dates, dependencies, milestones	Attached separately
GitHub repository	Working link, README, all members with access	Attached
Supabase project	Screenshot showing project exists	Attached
Project Progress Tracker	Completed ITC327W Excel workbook	Attached separately
Group Reflection	300–400 words, completed in Excel workbook	Attached

APPENDIX A: STAKEHOLDER INTERVIEW QUESTIONS

About the current enrolment process

1. Could you explain, in your own words, how the current enrolment process works from start to finish?
2. Who is involved in the process, from the time a parent submits a form to the time enrolment is confirmed?

About the challenges you currently experience

3. What difficulties do you or your administration staff experience with the current enrolment process?
4. Which of these problems causes the greatest difficulty for you or your staff?

About whom would use a new system

5. Who would use a mobile application to submit enrolment applications?
6. Who at the school would use the web application to review and manage applications?

About required information and functionality

7. What information about a learner and their application needs to be recorded and kept?
8. What should parents be able to do using the mobile application (for example, submit an application, upload documents, or check status)?
9. What would you like staff to be able to do through a web-based system (for example, approve, reject, or request corrections)?

About reporting and access

10. What reports or summaries would help you manage enrolment (for example, number of applications received, pending, or approved)?
11. Who should be allowed to view or update an application once it has been submitted?

About measuring success

12. How will you know that a new proposed system has successfully addressed the challenges you face today?

APPENDIX B: INTERVIEW RESPONSE & MEETING NOTES

Phehello Secondary School – Online Enrolment System

Date of interview: 24 August 2026
Location / platform: Microsoft Teams
Stakeholder present: Zukiswa Khondlo - Secretary to the Principal, Phehello Secondary School
Group members present: KI. Makhoali, PIJ. Lebusa, LR Mahlatsi, M. Lepitla, MM. Motaung, DT. Adegbola

Interview Record

#	Question	Stakeholder response	Follow-up required?	Confirmed?
1	1.Could you explain, in your own words, how the current enrolment process works from start to finish?	Parents collect and fill in enrolment forms. Before the process is complete, a fee is paid to the financial clerk, who issues a payment slip as proof. Parents then submit the forms together with supporting documents (ID/birth certificate, clinic card, etc.).	No	Yes
2	2.Who is involved in the process, from the time a parent submits a form to the time enrolment is confirmed?	Three people: the parent, the financial clerk, and the administrator.	Yes- answered live	Yes
3	(Follow-up) In your words, what would you say is the role of the financial clerk and the administrator in the application process?	The financial clerk receives the payment and issues the payment slip. The administrator reviews the application, ensures the forms are filled in properly, and checks that the documentation is correct.	No	Yes
4	3.What difficulties do you or your administration staff experience with the current enrolment process?	For young and working parents: documentation is sometimes incorrect, requiring them to travel back and forth - difficult given distance, and some travel on foot. For staff: long queues and a high amount of human interaction limits their time for other duties.	No	Yes
5	4.Which of these problems causes the greatest difficulty for you or your staff?	For parents: the back-and-forth travel. For admin staff: the amount of individual human interaction since it takes focus away from other responsibilities.	No	Yes
6	5. Who would use a mobile application to submit enrolment applications?	Young parents and working parents.	No	Yes
7	6. Who at the school would use the web application to review and manage applications?	The financial clerk and the administrator. The clerk checks in the system whether payment has been made, comparing proof of payment to the school's records. The administrator approves or rejects applications and leaves a comment/message for parents if a document is incorrect.	No	Yes

8	7. What information about a learner and their application needs to be recorded and kept?	To be confirmed against the actual enrolment form, which the school is going to send the group so they can see exactly what questions are asked and what information/documents are required.	Yes - awaiting the enrolment form from the school	Yes
9	(Follow-up) So in it, does it also list the required documents?	Yes	No	Yes
10	8. What should parents be able to do using the mobile application (for example, submit an application, upload documents, or check status)?	Fill in the required information, attach the correct documents, and check the status of their application.	No	Yes
11	9. What would you like staff to be able to do through a web-based system (for example, approve, reject, or request corrections)?	Approve or reject applications; check that uploaded documents are correct and leave notes for parents where necessary; retrieve proof of payment to check against school records; and alert applicants to how much time is left to apply and when applications close.	No	Yes
12	10. What reports or summaries would help you manage enrolment (for example, number of applications received, pending, or approved)?	A running count/spreadsheet of applications received, so the school can track progress against numbers. Once accepted, a confirmation letter should be produced including the learner's name and ID/student number.	No	Yes
13	(Follow-up) So you guys have student numbers at high school?	Yes - every student has a special/unique student number attached to them.	No	Yes
14	11. Who should be allowed to view or update an application once it has been submitted?	The applicant (parent) can log in and out and view their status. The administration can oversee the whole process.	Yes - answered live	Yes
15	(Follow-up) Who is the administrator that is going to be in charge - secretary, chief secretary, or someone else?	Schools have more than one administrator with different roles; the person responsible for the school system specifically will be the one in charge.	No	Yes
16	(Follow-up) How do we make sure a confirmation letter is official?	The school will provide its logo for use on the letter. The learner's unique student number is also used as a check - if someone arrives claiming to be enrolled without the correct number, or using someone else's number, this can be verified (or flagged as unsuccessful) against the school's system.	No	Yes
17	12. How will you know that a new proposed system has successfully addressed the challenges you face today?	Shorter queues and less one-on-one human interaction with administration, freeing staff to focus on their own work, since interacting with people individually takes up a lot of time.	No	Yes

Stakeholder Meeting Charter & Minutes (Appendix B)

Phehello Secondary School - Online Enrolment System

Meeting Details

Field	Details
Date	24 August 2026
Time	11:00
Platform	Microsoft Teams
Stakeholder	Zukiswa Khondlo - Secretary to the Principal, Phehello Secondary School (the Principal, Maleshoane Gweneth Adegbola, was not present for this session but assigned us her secretary as her delegated representative)
Meeting type	Full formal stakeholder needs-assessment interview (Activity 1.2) - all 12 interview questions from Appendix A were covered

Attendees

Name	Role
Zukiswa Khondlo	Secretary to the Principal - answered on the school's behalf
K.I. Makhoali	Group member
P.I.J. Lebusa	Group member
M. Lepitla	Group member
MM. Motaung	Group member
DT. Adegbola	Group member
LR. Mahlatsi	Group member

Meeting Objectives

1. Work through the full 12-question stakeholder needs-assessment interview (Appendix A)
2. Understand the current enrolment process, its challenges, and who is involved
3. Establish who would use the proposed mobile and web systems, and how
4. Confirm required learner/application information and system functionality
5. Capture evidence that the formal interview took place

Minutes Summary

The full interview record (all 12 questions, responses, and follow-up questions) is captured in Appendix B. Key points discussed:

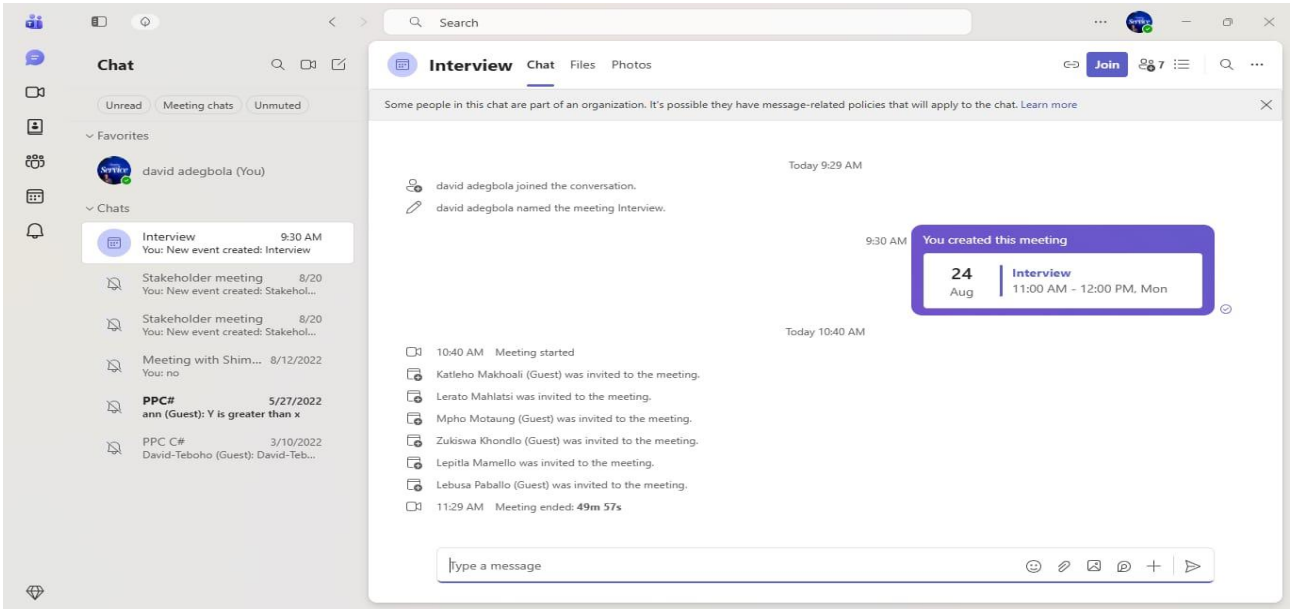
- Current process: forms + supporting documents + a payment to the financial clerk before submission.
- Three roles involved: parent, financial clerk, administrator.
- Main challenges: incorrect documentation causing repeat trips for parents, and high staff time spent on one-on-one interactions.
- A mobile app would be used mainly by young/working parents; a web system by the financial clerk and administrator.
- Required learner information will be confirmed once the school sends the group its actual enrolment form.
- Reporting needs include an application count and confirmation letters carrying the school logo and each learner's student number.

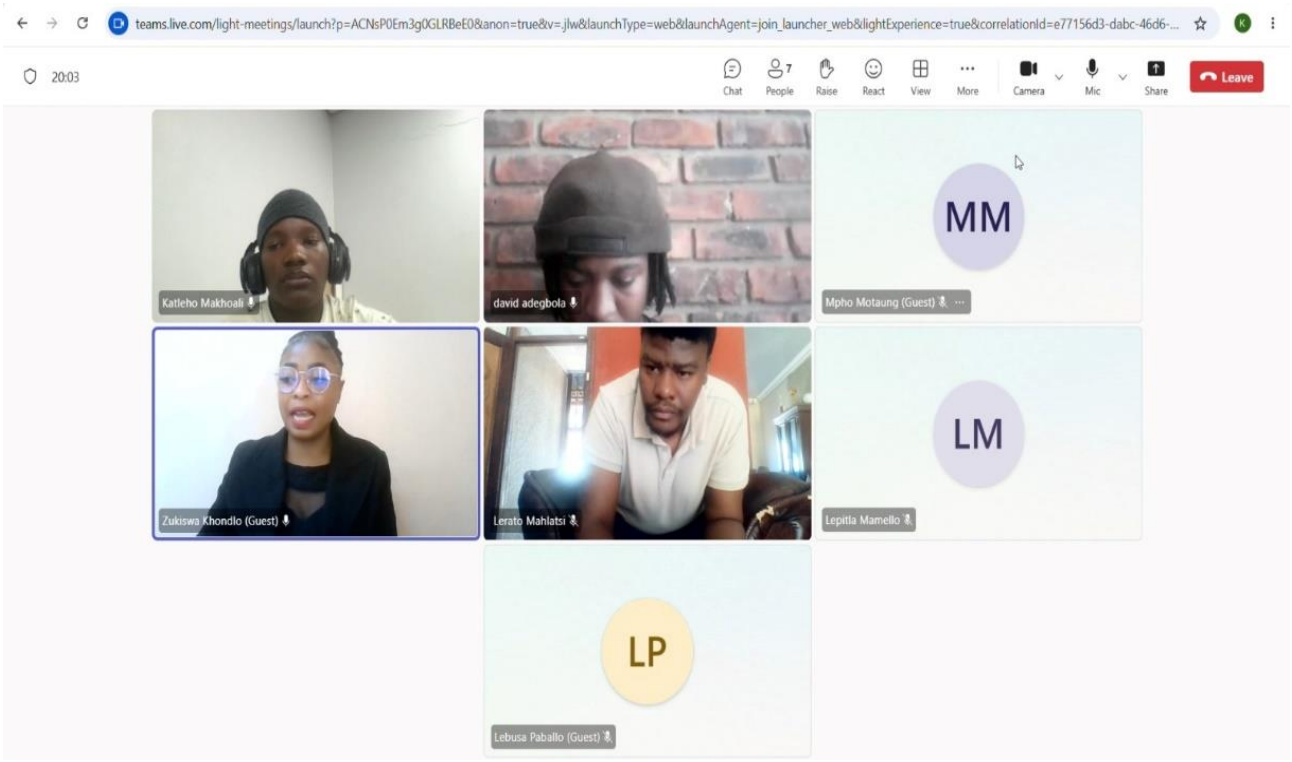
Success would be measured by shorter queues and reduced one-on-one staff time

Outcomes

All 12 questions covered? Yes.
Findings confirmed by the stakeholder's delegated representative. Yes- Zukiswa Khondlo confirmed the findings on behalf of Phehello Secondary School.

Interview Evidence




teams.live.com/light-meetings/launch?p=ACNsP0Em3g0GLRBeE0&anon=true&v=.jlw&launchType=web&launchAgent=join_launcher_web&lightExperience=true&correlationId=e77156d3-dabc-46d6-...
20:03
Chat
People
Raise
React
View
More
Camera
Mic
Share
Leave
Katleho Makhoali
david adegbola
MM
Mpho Motaung (Guest)
Zukiswa Khondlo (Guest)
Lerato Mahlatsi
LM
Lepitla Mamello
LP
Lebusa Paballo (Guest)

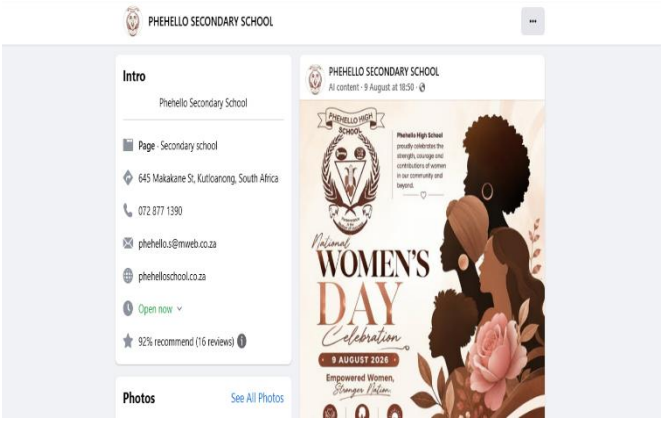
APPENDIX C: EVIDENCE OF STAKEHOLDER ENGAGEMENT

The following items support that the stakeholder identified in the Stakeholder Profile is a real, verifiable person and that the group has engaged with them directly.

Item	Purpose	Status
Screenshot: stakeholder identity	Confirms the principal is a real, identifiable person (e.g., official school Facebook page)	Attached
Meeting confirmation	Calendar invite, message, or email confirming a scheduled meeting date/time	Attached
Meeting evidence	Photo or attendance record from the actual meeting	Attached

1. Stakeholder Identity

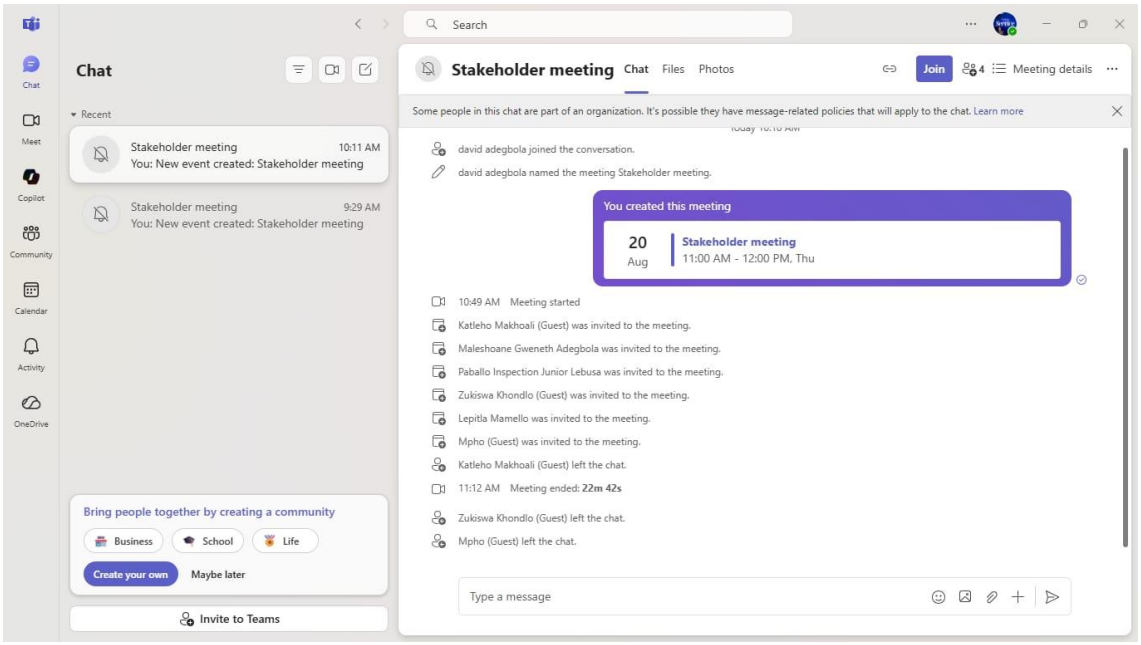


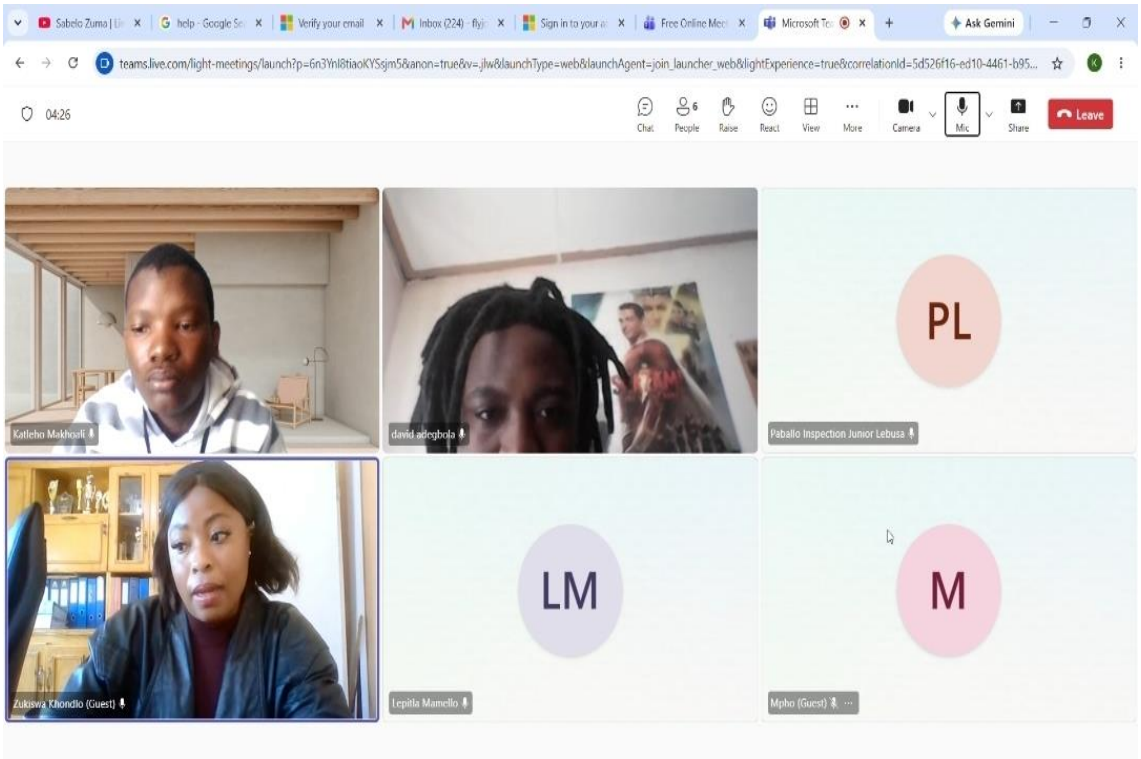


3. Meeting Confirmation



4. Meeting Evidence





Stakeholder Meeting Charter & Minutes (Appendix C)

Phehello Secondary School - Online Enrolment System

Meeting Details

Field	Details
Date	20 August 2026
Time	11:00
Platform	Microsoft Teams
Stakeholder	Maleshoane Gweneth Adegbola - Principal, Phehello Secondary School (present for the first part of the meeting; call continued with Zukiswa Khondlo, Secretary to the Principal, after the principal had to leave)
Meeting type	First introductory / scoping meeting (not the full formal interview)

Attendees

Name	Role
Maleshoane Gweneth Adegbola	Principal, Phehello Secondary School - Stakeholder (present for introductions, project explanation, and confirmation of stakeholder role; left partway through)
Zukiswa Khondlo	Secretary to the Principal - continued the meeting on the principal's behalf after she left
K.I. Makhoali	Group member
P.I.J. Lebusa	Group member
M. Lepitla	Group member
MM. Motaung	Group member
DT. Adegbola	Group member

Meeting Objectives

1. Formally introduce the group and explain the Work-Integrated Learning project in full
2. Confirm the stakeholder's willingness to participate as the group's official stakeholder
3. Gain a first-pass understanding of the current enrolment process at Phehello Secondary
4. Agree on a date for the full, formal stakeholder needs-assessment interview
5. Capture evidence that this meeting took place

Planned Agenda

Time	Item	Notes
5 min	Introductions	Each group member states name and role
10 min	Explain the project	What the project is, why, what a prototype means here

2 min	Confirm stakeholder role	Ask directly for agreement to act as the group's stakeholder
15 min	Light current-process discussion	2–3 open questions only - not the full interview
2 min	Schedule formal interview	Propose a date/time for the full 9-question interview
2 min	Close & thank	

Minutes

To be completed live during the meeting.

Topic	Discussion / Response	Decision / Action
Introductions	Group members introduced themselves and their roles. The principal introduced Zukiswa Khondlo, her secretary.	-
Project explanation	Group explained the Work-Integrated Learning project, the purpose of the prototype, and what would be expected of the stakeholder.	Principal confirmed her understanding of the project.
Confirm stakeholder role	Group asked the principal directly whether she was willing to act as the group's official stakeholder for the project.	Agreed - Yes, confirmed by the principal.
Principal's availability	Principal explained she needed to leave for another commitment shortly after confirming her role.	Handed the remainder of the meeting to Zukiswa Khondlo, her secretary.
Current process discussion (with secretary)	Zukiswa gave a brief, informal overview of how enrolment forms are currently received and filed at the school.	Treated as preliminary only and subsequently verified and expanded upon during the formal Activity 1.2 interview with Zukiswa Khondlo, the principal's delegated representative.
Schedule formal interview	Group discussed timing for the full stakeholder needs-assessment interview.	Completed - the formal interview was held on 24 August 2026 via Microsoft Teams (see Appendix B).
Close & thank	Group thanked Zukiswa for her time and confirmed next steps.	-

Outcomes

Stakeholder agreement confirmed?

Yes — the principal confirmed directly, before she had to leave the call, that she is willing to act as the group's official stakeholder for this project.

Key points learned about the current process

Preliminary, secretary-level notes were gathered from Zukiswa Khondlo after the principal left this scoping call. These were later fully superseded by the formal Activity 1.2 interview held on 24 August 2026 (see Appendix B), whose findings were confirmed by Zukiswa Khondlo on behalf of Phehello Secondary School.

Date agreed for formal interview

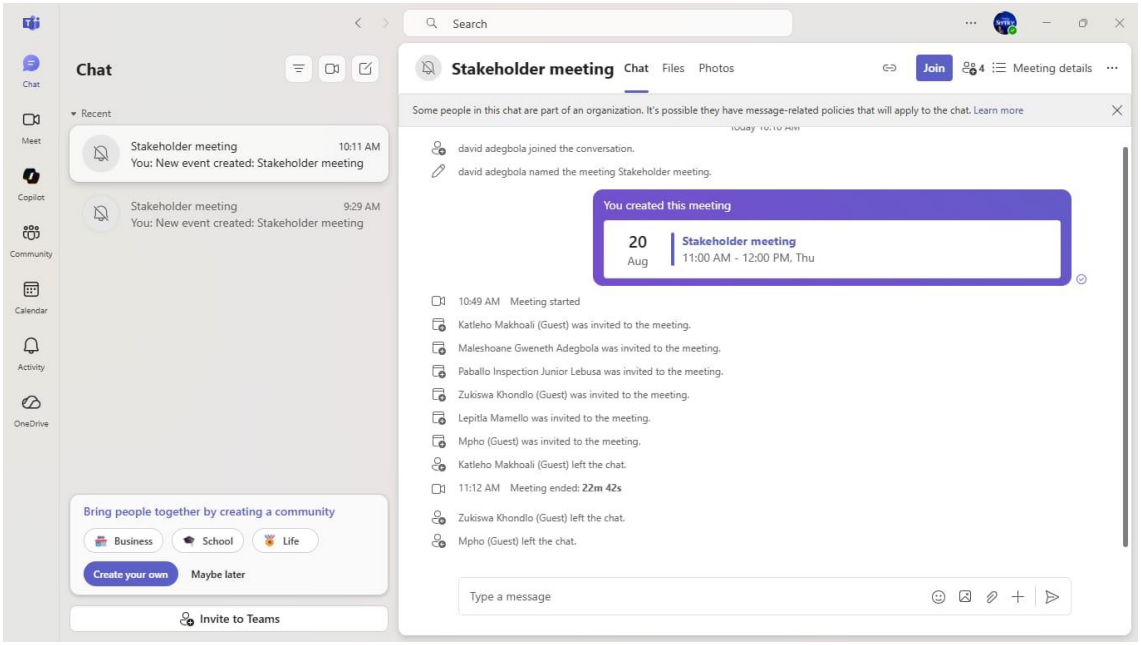
Confirmed and completed - the full stakeholder needs-assessment interview took place on 24 August 2026 via Microsoft Teams with Zukiswa Khondlo (Secretary to the Principal); see Appendix B for the full interview record, confirmed by the principal.

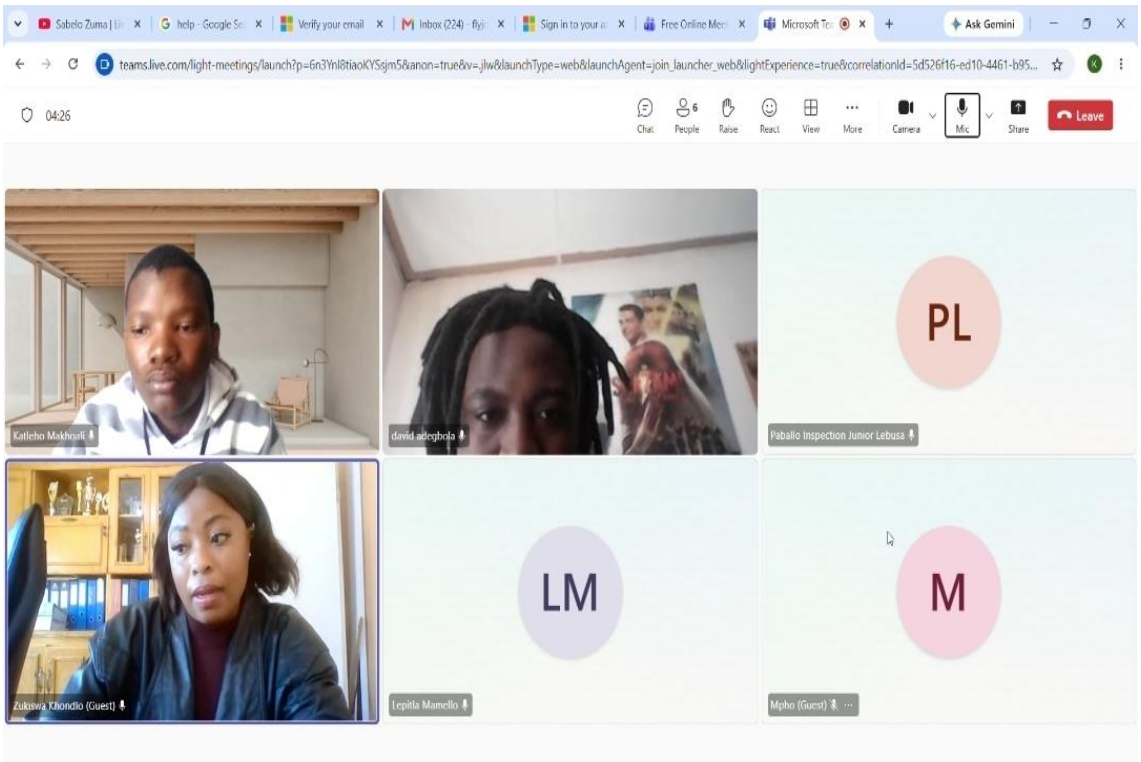
Other notes / next steps

The principal was present for the introductions, project explanation, and confirmed her role as stakeholder, but had to leave shortly afterwards for another commitment. The group subsequently held the full stakeholder needs-assessment interview on 24 August 2026 via Microsoft Teams (Appendix B), whose findings were confirmed by the Secretary (Appendix D).

Evidence

Screenshot of the Microsoft Teams meeting record confirming the date, time and participants. The record shows the principal, Maleshoane Gweneth Adegbola, joined the call together with the group and Zukiswa Khondlo.





APPENDIX D: STAKEHOLDER CONFIRMATIONS OF FINDINGS

Summary of Findings Presented to the Stakeholder

Current process: Parents collect and fill in enrolment forms, then submit supporting documents (ID, clinic card, etc.). Before the process is completed, a fee is paid to the financial clerk, who issues a payment slip.

People involved: The parent, the financial clerk, and the administrator. The financial clerk receives payment and issues the slip. The administrator reviews the application, checks the forms are filled in correctly, and verifies the documentation.

Challenges: Parents (especially young or working parents) sometimes submit incorrect documentation and have to travel back and forth, which is difficult given distance and cost. For staff, the biggest challenge is the amount of one-on-one time each application takes, which limits their ability to focus on other duties.

Who would use the system: Young and working parents would use a mobile app to apply? At the school, the financial clerk and administrator would use a web system - the clerk to check payments against proof submitted, and the administrator to approve or reject applications and message parents about incorrect documents.

Information and functionality: The learner information required matches what's already on the current enrolment form (which you'll be sending us), including the required documents. Parents should be able to fill in details, upload documents, and check their status. Staff should be able to approve/reject, leave notes, check proof of payment against school records, and send reminders about application deadlines.

Reporting: The administrator would benefit from a running count of applications received, and confirmation letters (with the school logo and the learner's name, ID number, and student number) once accepted. Every student already has a unique student number, which would also help prevent fraudulent enrolments.

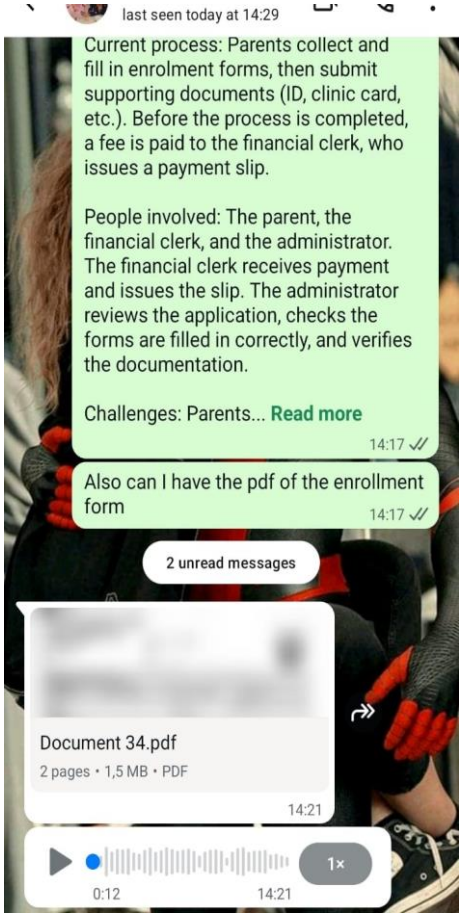
Access: Parents would log in to view their own status; the administrator responsible for the school system would oversee the whole process.

Measuring success: Fewer queues and less time spent on individual, in-person interactions, so staff can focus on their other responsibilities.

Confirmation

Item	Details
Date of confirmation	24 August 2026
Method	WhatsApp voice note
Stakeholder name and role	Zukiswa Khondlo - Secretary to Principal, Phehello Secondary School
Findings confirmed as, correct?	Yes
Any corrections made by the stakeholder	None

Evidence


last seen today at 14:29
Current process: Parents collect and fill in enrolment forms, then submit supporting documents (ID, clinic card, etc.). Before the process is completed, a fee is paid to the financial clerk, who issues a payment slip.
People involved: The parent, the financial clerk, and the administrator. The financial clerk receives payment and issues the slip. The administrator reviews the application, checks the forms are filled in correctly, and verifies the documentation.
Challenges: Parents... Read more
14:17
Also can I have the pdf of the enrollment form
14:17
2 unread messages
Document 34.pdf
2 pages • 1,5 MB • PDF
14:21
1×
0:12
14:21

APPENDIX E: ADDITIONAL CURRENT-PROCESS EVIDENCE

APPLICATION FOR ADMISSION TO SCHOOL

PHEHELLO SECONDARY SCHOOL

645 MAKAKANE STREET
ODENDAALSRUS
9480

Telephone: 057 - 1010296
Fax:
Year: _____

Note: This form must be completed in full. All changes to be initialed or signed by parent / guardian. Completing the form does not necessarily mean that the learner has been accepted into the school.

Grade Applied For:		Highest Grade Passed		Year When Grade was passed:		Accession No:	

Surname:		Initials:	Nick Name:
First Name:		Other Names:	
Date Of Birth: YYYY	MM	DD	Gender: Male: Female:
Race:		Identification or Passport No:	
Country of Residence:		Citizenship:	
If SA, indicate province of residence:			

Physical Address:		Home Telephone:	
		Emergency Telephone:	
City/Suburb		Learner Cell:	
Code:	Learner Email Address:		
Home Language:		Preferred Language of Instruction	
Boarder	Yes / No		
Deceased Parent	Mother / Father / Both	Mode of transport:	
Religion:	For Grade 1 only: Indicate pre-primary education	None / Non Formal / Formal	

Previous School Information

Name of Previous School:			
Previous School Address:			
Code:	Province:	Country:	

Learner Medical Information

Medical Aid Number:		Medical Aid Name:	
Medical Aid Main Member:		Doctor Name:	
Doctor's Address:		Doctor Telephone Number:	
Medical Condition:			
Special Problems Requiring Counseling:			
Dexterity of Learner:	Right Handed / Left Handed / Ambidextrous	Reg. Social Grant YES / NO:	
		Rec. Social Grant YES / NO:	

If the learner is accepted, the following documents must be submitted to the school:

1. Copy of Immunisation Records.
2. Copy of Birth Certificate
3. Progress Report from Previous School
4. Transfer Letter from Previous School

APPLICATION FOR ADMISSION TO SCHOOL

Siblings

Number of other Children at this school: | Position in the family (e.g first):

Please supply full names below:

Name: | Grade:

Name: | Grade:

Name: | Grade:

Parent / Guardian Information Complete a SEPARATE parent form for each parent living at a different physical address

Title: | Initials: | Surname:

First Name: | Gender: Male: Female:

Home Language: | Race:

Identification Number: | Or Passport number | Account Payer: Yes No

Residential Street Address:

City/Suburb | Code:

Occupation: | Employer:

Surname of Spouse: | First Name:

Occupation of Spouse: | Learner resides with this parent/s Yes No

Spouse ID Number: | Relationship to Learner:

Marital status of parent:

Correspondence Details

Title: | Surname:

Postal Address:

City/Suburb | Code:

Other Contact Details

Home Telephone | Work Telephone

Fax Number : | Cell Number :

Spouse Work Telephone Number: | Spouse Cell Number :

E-Mail Address: | Spouse E-Mail Address:

I hereby declare that to the best of my knowledge, the above information as supplied is accurate and correct.

Name of Parent / Guardian (Please Print) : _____

Signature of Parent / Guardian _____

Date: ----------/----------/----------

Office use only:

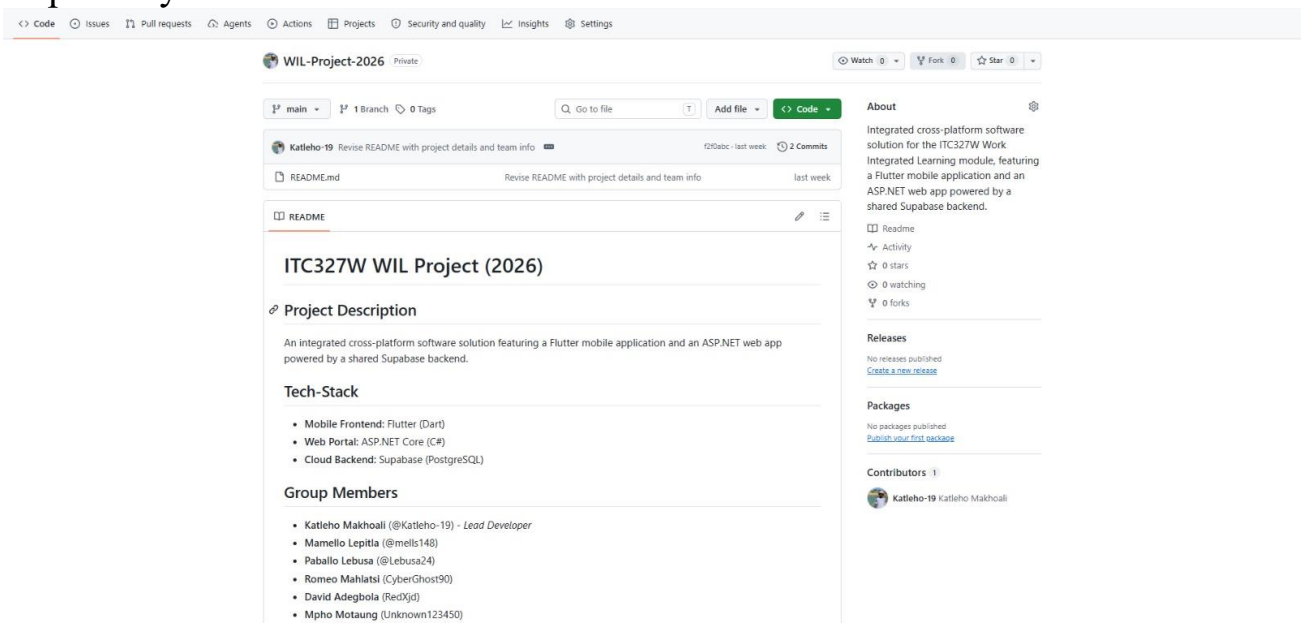
1. Date:	2. Accepted:	3. Accession Number:
4. Rejected:	5. Reason for Rejection:	
6. Documentation Received:	6a Immunisation Record:	6b. Birth Certificate:
6c. Progress Report from Previous School:	6d. Transfer Letter from Previous School:	

PROJECT SETUP EVIDENCE

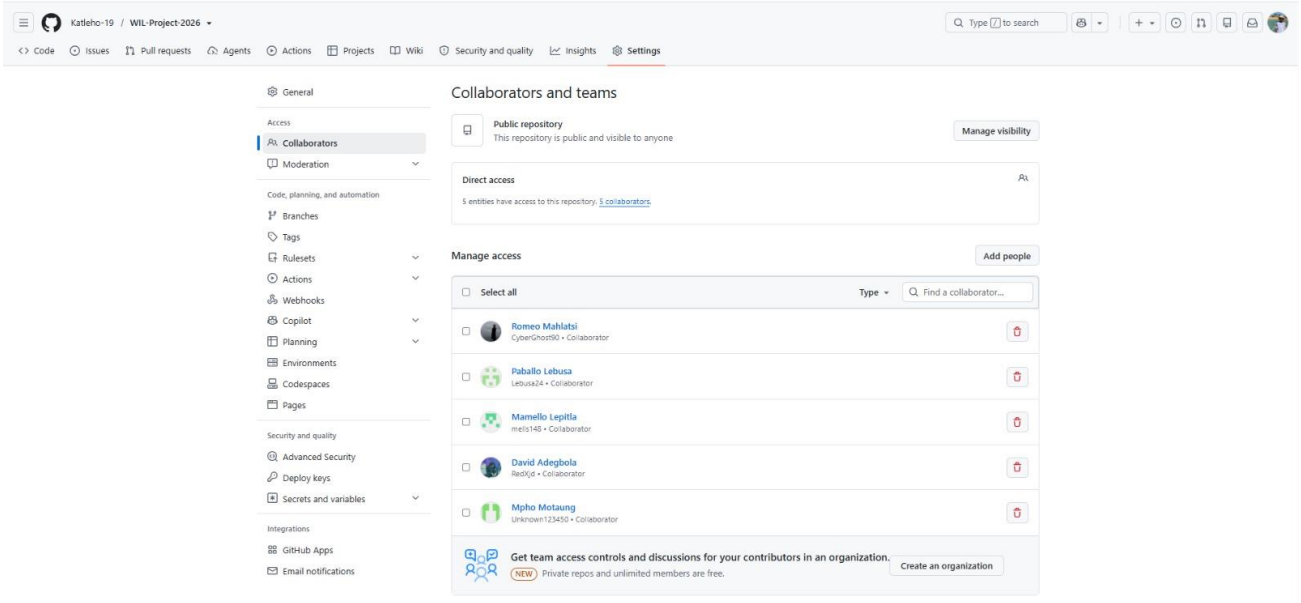
GITHUB REPOSITORY SETUP

Repository Name: WIL-Project-2026
URL: https://github.com/Katleho-19/WIL-Project-2026
Visibility: Private
Primary Branch: main

Repository README & Overview

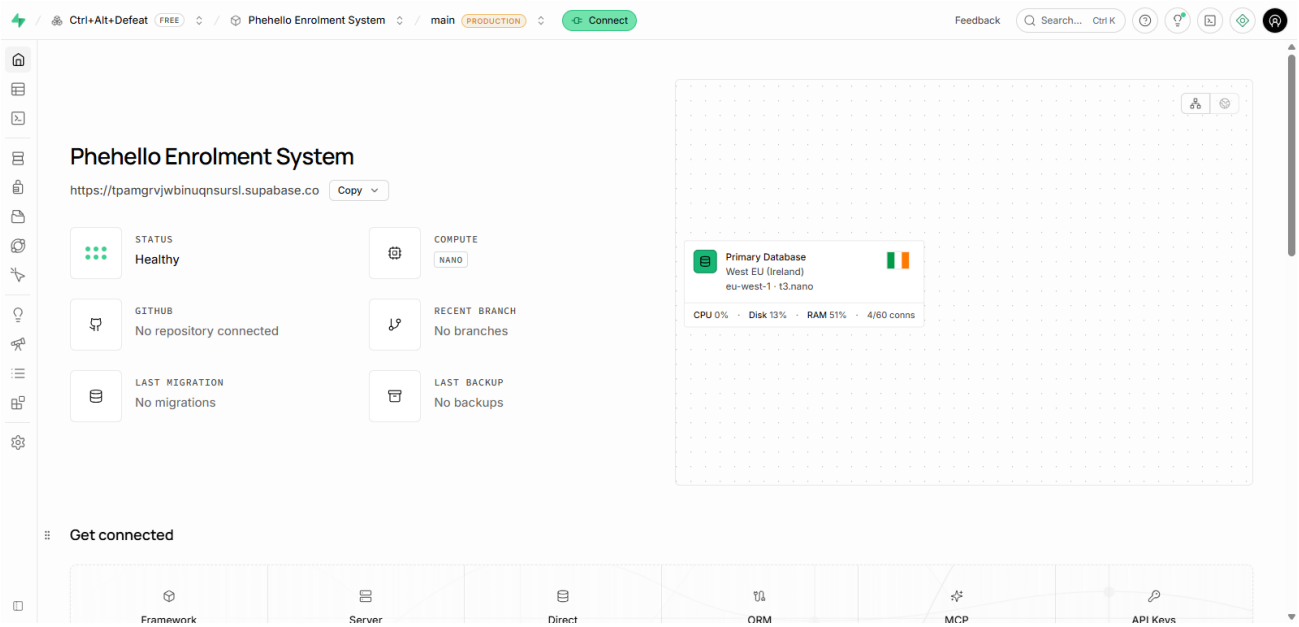


Team Collaborators & Access Control

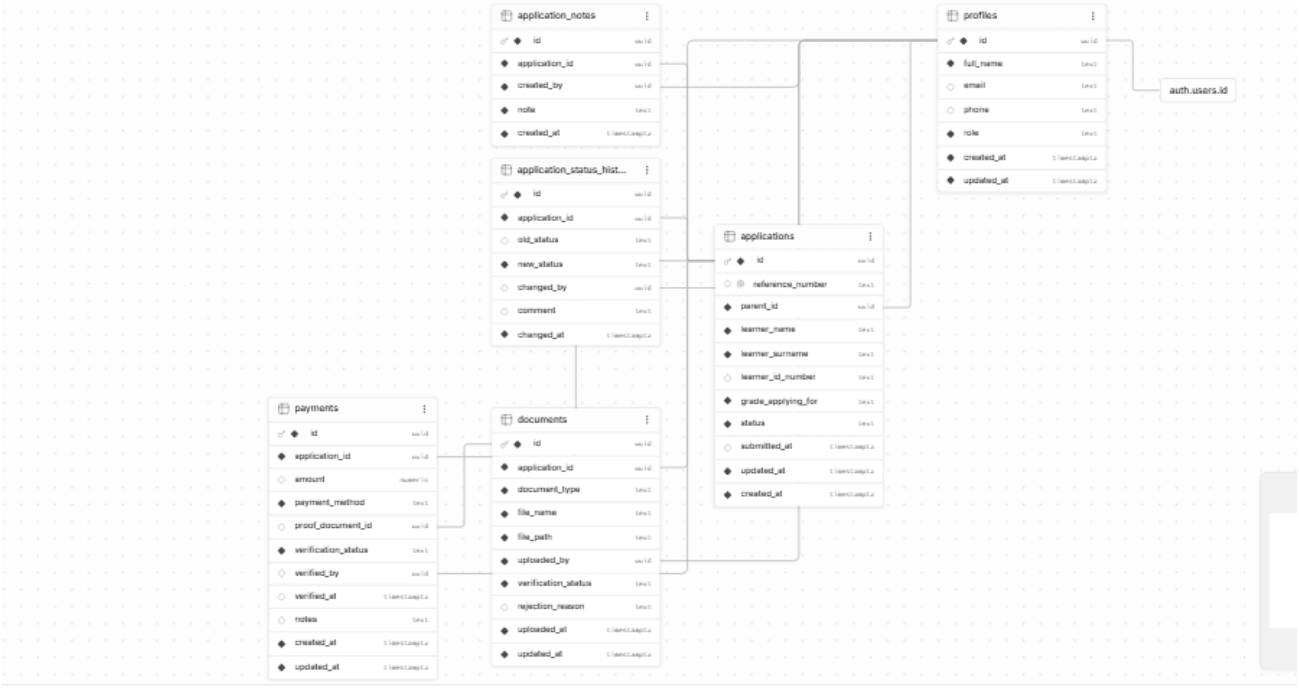


SUPABASE BACKEND SETUP

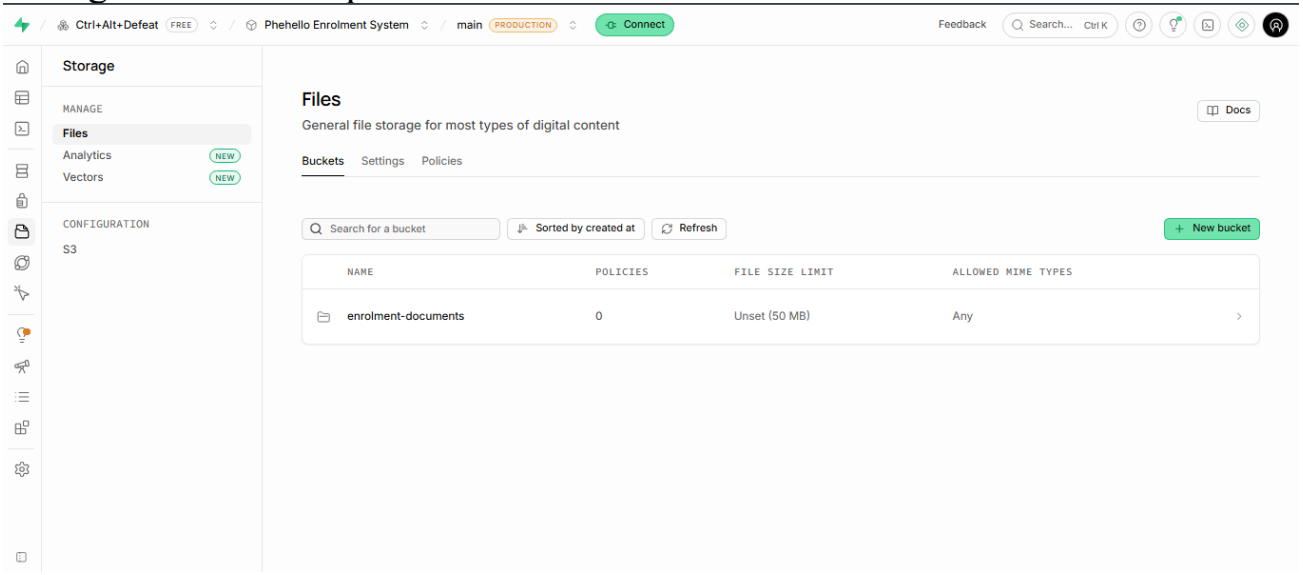
Supabase Project Overview



Database Schema/Table editor



Storage Buckets Setup



PART C: UNIT 2 FEASIBILITY AND RISK ANALYSIS

Review of the Current Project

Question	Group response
What is the main problem being addressed?	Enrolment at Phehello Secondary relies entirely on paper forms and an in-person payment step, causing lost paperwork, slow processing, no status tracking, and repeated trips to the school when documentation is incorrect.
What are the main Flutter functions?	Register/sign in, submit an enrolment application, upload documents and proof of payment, view application/payment status.
What are the main ASP.NET functions?	Financial Clerk verifies payment; Administrator views, approves/rejects, leaves notes, messages parents, sends reminders, generates confirmation letters, views application counts.
What information will Supabase manage?	Parent/learner accounts, enrolment applications, uploaded documents, proof of payment, application status, and each learner's unique student number.
Which functions appear most difficult?	Coordinating payment verification (Financial Clerk) with application approval (Administrator) as two separate steps on one shared record.
Which functions require skills the group still needs to develop?	The group has prior experience with each technology individually but has not previously implemented this complete Flutter-ASP.NET-Supabase workflow together.
Which functions may require more time than is available?	The four Should Have functions (detailed status view, notes, reminders, running count) are the first candidates to simplify or postpone if time is tight.

1. Technical Feasibility

Question	Group finding
Does the group have the required Flutter skills?	Yes - the group has used Flutter since second-year, second semester, with prior project experience.
Does the group have the required ASP.NET skills?	Yes - the group built an ASP.NET project last semester.
Can the group work with Supabase?	Yes - the group has used Supabase since the first semester of this year; the project's schema and storage buckets are already set up.
Can Flutter and ASP.NET connect to the same Supabase backend?	Yes - Supabase serves both applications through its REST/Auth/Storage APIs.
Are suitable development computers available?	Yes - all members have personal laptops.
Are suitable test devices available?	Yes - all members have Android smartphones.
Is reliable internet access available?	Internet access is available to all group members, although reliability varies by location; this is managed by scheduling development sessions around confirmed connectivity.
Which technical skills still need to be learned?	None of the three technologies individually - the untested part is combining all three for this specific enrolment-and-payment workflow.
Which functions may be too technically difficult?	None identified at Must Have priority; the integration between Flutter, ASP.NET and Supabase for payment verification is the most complex part and will be tested early.

Technical Feasibility Decision: Feasible. The group has direct experience with all three technologies and has already begun backend setup. Testing the full three-way integration early in Unit 3 is recommended as good practice, not because a skill is missing.

2. Operational Feasibility

Question	Group finding
Will the prototype address the stakeholder's problem?	Yes - it replaces the in-person, paper-and-payment-slip process with online submission, payment proof upload, and digital review.
Who will use the mobile application?	Parents/guardians - particularly young or working parents, per the confirmed interview findings.
Who will use the web application?	Financial Clerk (payment verification) and Administrator (application review and approval).
Do the intended users have suitable devices?	Assumed yes for the prototype/demonstration; a real deployment would need to confirm this for all parents.
Do users have suitable internet access?	Assumed yes for the prototype/demonstration; noted as a limitation for a real deployment rather than a blocker for this prototype.
Will the proposed solution fit the organisation's current activities?	Yes - it digitises the existing enrolment activity rather than replacing broader school administration.
Will users require guidance or training?	Minimal - the Financial Clerk and Administrator dashboards are scoped to their existing responsibilities.
Are any proposed functions unsuitable for the intended users?	No - the proposed functions match what parents and staff already do in the current process, just digitised.

Operational Feasibility Decision: Feasible. The solution was designed around the actual confirmed process and the two real staff roles involved, not assumptions.

3. Economic Feasibility

Resource	Available?	Action required
Internet access or data	Yes, variable by member	Schedule development sessions around reliable connectivity
Transport to the stakeholder	Yes	Not regularly required - stakeholder meetings are conducted via Microsoft Teams/WhatsApp
Development computers	Yes	None - all members have personal laptops
Test devices	Yes	None - all members have Android smartphones
Required software	Yes (free tiers)	Monitor Supabase free-tier storage limits as document/payment uploads are added
Supabase resources	Yes - project, schema and storage already set up	Monitor usage as tables and storage buckets grow
Presentation resources	Yes	None - existing lecture/presentation venue and equipment
Other project resources	None identified	N/A

Economic Feasibility Decision: Feasible. All resources needed are already in place at no cost.

4. Schedule Feasibility

Question	Group finding
Is enough time available for system design?	Yes, if design begins immediately after Assessment 1 feedback is received.
Is enough time available for Flutter development?	Yes, for the Must Have functions (register/sign in, submit application, upload documents and proof of payment, view status).
Is enough time available for ASP.NET development?	Yes, for the Must Have functions (view applications, verify payment, approve/reject/request corrections).

Is enough time available for Supabase integration?	Yes, provided the three-way integration is tested early rather than left until the end.
Is enough time available for testing and corrections?	Limited - a dedicated testing period needs to be reserved in the Microsoft Project schedule rather than testing only at the end.
Is enough time available for stakeholder feedback?	Yes - the stakeholder has shown consistent availability (20 Aug scoping meeting, 24 Aug formal interview, same-day confirmation).
Are too many functions currently included?	No - the confirmed Must Have functions (11 of 14 FRs) are realistic for one semester.
Which functions may need to be simplified, postponed or removed?	The four Should Have functions (detailed status view, administrator notes, deadline reminders, running application count) are the first candidates if the schedule tightens.

Schedule Feasibility Decision: Feasible with changes. Reserve an explicit testing period and an early Flutter–Supabase–ASP.NET integration test in the Microsoft Project schedule and treat the four Should Have requirements as the first candidates to simplify or postpone if the schedule tightens.

5. Legal Feasibility

Question	Group finding
Will the prototype collect personal information?	The design allows for learner/parent information and documents, but the prototype will be tested with mock data only (NFR3), not real learner records.
Is all the information necessary?	Yes - the fields requested match what the current paper enrolment form already requires; no additional information is collected.
Who should be allowed to access the information?	Only authorised staff (NFR1): the Financial Clerk sees payment information; the Administrator sees full applications; parents can only view their own application.
Will test data be used during development?	Yes - the prototype will use mock/test learner and parent information, not real learner records, per NFR3.
How will passwords and private keys be protected?	Supabase service keys and credentials are intended to be kept out of GitHub, using environment variables/local configuration only.
Does the project involve confidential stakeholder information?	Only the stakeholder's own contact details for coordination purposes, which are not published in project documentation or GitHub.
Is private stakeholder information being kept out of GitHub?	Yes, per the group's stated rule.
Is the GitHub repository's current visibility appropriate?	The repository (WIL-Project-2026) was initially created as Public, which was identified as a security and legal risk because configuration files or credentials could accidentally be exposed. The repository has since been changed to Private, with all group members given access. This is the appropriate visibility for the project.

Legal Feasibility Decision: Feasible. The repository visibility issue identified during the feasibility review has been addressed by changing the repository from Public to Private. The group will also continue to ensure that API keys, .env files, Supabase service-role keys and other sensitive credentials are not committed to GitHub.

6. Ethical Feasibility

Question	Group finding
Does the stakeholder understand the purpose of the project?	Yes. The project was explained to the principal during the initial scoping meeting, where she agreed to act as the group's official stakeholder. The formal requirements interview was subsequently conducted with Zukiswa Khondlo, the Principal's delegated representative.
Is unnecessary personal information being collected?	No - only information already required by the existing paper process and confirmed enrolment form.
Could the proposed solution negatively affect intended users?	A parent without smartphone/data access could be disadvantaged in a real deployment; acknowledged as a limitation of a semester prototype, not something this project needs to solve.
Is information being used only for its intended purpose?	Yes. The system is designed only for learner enrolment and application processing. During development and testing, mock data will be used instead of real learner records.
Does the prototype accurately reflect stakeholder needs?	Yes - requirements were revised directly from the confirmed 24 August interview findings, not assumptions.
Are users being treated fairly and responsibly?	Yes. The system is used to reduce unnecessary travel and queues while maintaining separate access permissions for parents, the Financial Clerk and the Administrator. A real deployment would also need to consider parents who do not have reliable smartphone or internet access.

Ethical Feasibility Decision: Feasible.

Feasibility Summary

Feasibility Area	Decision	Main Reason	Action Required
Technical	Feasible	The team already has hands-on experience with Flutter, ASP.NET and Supabase from prior coursework, and the Supabase backend (project, schema and storage) has already been created and configured. The main outstanding risk is untested three-way integration between the mobile app, web app and shared backend.	Test the full 3-way integration early in Unit 3
Operational	Feasible	The proposed system was designed directly around the process and roles confirmed in the 24 August stakeholder interview - including the Financial Clerk and Administrator roles and the payment-proof step - rather than assumptions, so it should fit how the school actually works.	None
Economic	Feasible	Flutter, ASP.NET and Supabase are all free to use at the project's current scale, and the group already has the laptops and internet access needed to complete the prototype. No paid services or infrastructure are required.	Monitor Supabase free-tier limits
Schedule	Feasible with changes	The confirmed scope grew from the original draft to 14 functional requirements after the stakeholder interview, increasing the amount of	Reserve a dedicated testing period;

		development and testing work needed within the remaining semester weeks.	simplify Should Have items first if needed
Legal	Feasible	The GitHub repository was identified as Public during the risk assessment, which could have exposed configuration files or credentials; this has since been corrected.	Completed - repository is now set to Private. Continue to audit commit history for any accidentally committed secrets.
Ethical	Feasible	The prototype only collects information the stakeholder confirmed is actually used in the enrolment process, and testing will use fictional/test data rather than real learner or financial records.	None

Overall Feasibility Decision

The project is feasible with changes.

The proposed project is feasible with changes. The main strengths are that it addresses a real, stakeholder-confirmed problem, the group already has hands-on experience with all three required technologies, and Supabase setup is already underway. The main actions required before proceeding further are: (1) reserve dedicated testing time and test the three-way integration early in Unit 3, and (2) switch the GitHub repository to Private and audit it for accidentally committed credentials. Neither action requires removing any confirmed stakeholder requirement.

PROJECT RISK REGISTER

Risk Rating Scale

Likelihood: 1= Low 2=Medium 3=High **Impact:** 1=Low 2=Medium 3=High

Risk Score: Likelihood * Impact

Risk Level: 1-2=Low 3-4=Medium 6-9=High

ID	Risk statement	Category	L	I	Score	Level	Mitigation	Contingency	Owner	Status
R01	Although each technology is individually familiar, Flutter, ASP.NET and Supabase have not yet been connected together for this specific enrolment/payment workflow.	Technical	2	2	4	Medium	Build and test a small end-to-end integration example (submit → store → display → status update) as the first Unit 3 task.	Isolate and simplify the affected integration point while other development continues.	Mamello Lepitla	Open
R02	The GitHub repository is currently Public, which could expose configuration files or credentials if accidentally committed.	Data and security	2	3	6	Medium	Switch the repository to Private immediately; audit commit history now for any committed secrets.	Rotate any exposed Supabase keys immediately and remove the exposed file from history.	Katleho Makhoali	Resolved - repository switched to private; continue monitoring repository history for exposed credentials
R03	The confirmed scope grew after the interview (14 functional requirements vs. the original draft), which may leave insufficient time for testing.	Schedule	2	3	6	Medium	Reserve a dedicated testing period in the Microsoft Project schedule; treat Should Have requirements as first to simplify if needed.	Postpone or simplify Should Have functions (status view detail, notes, reminders, running count) to protect the Must Have core.	Paballo Lebusa	Open
R04	Incorrectly implemented role separation could let the Financial Clerk see more than payment information or let the Administrator bypass the Clerk's verification.	Non-functional/security	2	3	6	Medium	Implement and test role-based access (NFR6) as part of initial Supabase/ASP.NET setup, not as an afterthought.	Restrict access immediately and review the access-control implementation.	Lerato Mahlatsi	Open
R05	There is a risk that changes to the school's	Requirements	1	2	2	Low	Confirm any future changes to the school's	Update the FR3 and the document-	Mamello Lepitla	Open

	required enrolment documents could occur after the requirements have been confirmed, resulting in changes to the documents-upload function.						enrolment form with the stakeholder before implementing or finalising the document-upload function.	upload function through the requirements change process if the school introduces additional or different document requirements.		
R06	A group member may not complete assigned tasks on time, delaying dependent activities.	Team	2	2	4	Medium	Assign clear individual tasks with due dates in the Project Progress Tracker; review weekly.	Reassign urgent tasks and record the contribution concern in the tracker.	Katleho Makhoali	Open
R07	Limited testing of the Flutter–Supabase–ASP.NET integration could let defects (e.g., payment status not syncing) go unnoticed until late in the project.	Quality	2	3	6	Medium	Test each integration point as soon as it is built, not only during a final testing phase.	Roll back to the last working version and fix the specific integration point.	Lerato Mahlatsi	Open
R08	Poor or inconsistent communication within the group could cause misunderstandings about task ownership.	Team	2	2	4	Medium	Hold a short weekly check-in; record decisions in the Project Progress Tracker.	Escalate to the module facilitator if disagreements cannot be resolved within the group.	Mamello Lepitla	Open
R09	Unreliable internet access for one or more members could disrupt development sessions or stakeholder calls.	Resources	2	1	2	Low	Schedule key sessions at times/locations with confirmed reliable connectivity.	Reschedule the affected session and share notes with absent members.	David Adegbola	Open
R10	The group could be tempted to add functions the stakeholder never requested (e.g., automated document verification), increasing scope beyond what is achievable.	Scope	1	2	2	Low	Check any new function idea against the confirmed requirements list before starting it.	Remove or postpone the added function if it threatens Must Have delivery.	Mpho Motaung	Open

R11	The principal may become less available for future confirmations now that the main interview is complete.	Stakeholder	1	2	2	Low	Confirm any future changes in writing (WhatsApp/e mail) rather than relying only on live meetings.	Use previously confirmed findings as the baseline and request written follow-up.	David Adegbola	Open

Three Highest Risks

Highest risk	Why is this important?	What must the group do now?
R02 - Public GitHub repository	A credential leak is one of the few risks here that could cause real, irreversible harm (exposed Supabase keys), not just project delay.	Switch the repository to Private this week and audit commit history for secrets.
R03 - Scope grew after the interview	More confirmed requirements than originally drafted means less slack for testing if not planned for now.	Add an explicit testing period to the Microsoft Project schedule before development ramps up.
R04 - Financial Clerk / Administrator role separation	Getting this wrong expose payment information to the wrong role or lets approvals bypass payment verification - a core part of the confirmed process.	Design and test role-based access as part of initial setup, not as a late add-on.

Review of Feasibility and Risk Findings

Finding	**Does the project need to change?**	**What change is required?**
Technical feasibility	No	None - group already has the required skills; proceed with early integration testing as planned
Operational feasibility	No	None - solution matches the confirmed real process and users
Economic feasibility	No	None - all resources already available at no cost
Schedule feasibility	Yes	Add a dedicated testing period and an early three-way integration test to the Microsoft Project schedule
Legal feasibility	Yes	Switch the GitHub repository from Public to Private; audit for committed secrets
Ethical feasibility	No	None - only necessary information is collected, using test data
High project risks (R02, R03, R04)	Yes	Repository visibility change (R02); dedicated testing period (R03); role-based access implemented and tested early (R04)

Requirements Change Log

Requirement / Item	Original assumption	Change / Status	Reason
FR3 — required documents	Draft assumed birth certificate, transfer letter and school report	Updated to documents confirmed on the school's enrolment form: immunisation records, birth certificate, progress report from previous school, and transfer letter from previous school.	The actual Phehello Secondary School enrolment form was received after the initial requirements draft and provides the school's confirmed document requirements.
Stakeholders table	Draft assumed one generic "admin staff" role	Split into Financial Clerk and Administrator as two distinct roles with different permissions (NFR6)	Confirmed in the interview - payment verification and application approval are handled by different staff
GitHub repository visibility	Repository created as Public	Recommended change to Private	Identified as a legal-feasibility risk (R02) - reduces exposure of credentials/config files
Microsoft Project schedule	N/A	Add a dedicated testing period and an early three-way integration test	Identified through Technical and Schedule feasibility findings and Risk R01/R03/R07
FR15	No requirement for the Flutter application to display the enrolment fee or school banking details or capture a payment reference before proof-of-payment upload.	Added FR15 requiring the parent/guardian to view the stakeholder-confirmed enrolment fee and school banking details and enter a payment reference before uploading proof of payment.	The stakeholder provided the enrolment fee and school banking details through WhatsApp on **21 September 2026**. The updated Flutter design now includes a Payment Information step and captures a payment reference before proof-of-payment upload.
Database fields for the Flutter enrolment form	The approved ERD had no columns for grade applied for, initials, other names, school history notes, siblings at school or a notification title	Added applications.grade_applied_for, learners.initials, learners.other_names, learners.school_history_notes, learner_medical_info.siblings_at_school and notifications.title. submit_enrolment updated to save them	These fields appear in the approved Flutter wireframes(Learner Personal Information, Academic & School History, Medical & Family, and Messages screens) but were missing from the ERD. Found while building the Flutter app.

Refined Project Scope

Function / Requirement	Decision	Reason	Stakeholder informed?
FR1–FR8, FR14 (core application, payment, review)	Keep	Must Have - directly addresses the confirmed problem	Yes - confirmed 24 Aug interview
FR9–FR11, FR13 (notes, messaging, reminders, running count)	Keep (Should Have)	Valuable stakeholder-requested features, but they are the secondary to the core enrolment, payment and application-review functions. They are the first features to simplify or postpone if the schedule becomes tight.	Yes - explicitly requested during the stakeholder requirements interview.
FR12 (confirmation letters)	Keep	Explicitly requested; matches existing student-number fraud-prevention practice	Yes
NFR6 (Financial Clerk / Administrator permission split)	Keep	Matches the two distinct staff roles confirmed in the interview	N/A - internal design decision
Automated document verification / OCR	Remove	Never requested; adds technical complexity outside available time	N/A - was never part of the scope
Full online payment processing	Remove (unchanged from Unit 1)	Only proof-of-payment upload and verification was requested, not payment processing itself	Yes - confirmed as an original exclusion

Stakeholder Confirmation of Important Changes

Item	Details
Date of communication or meeting	24 August 2026 (during and immediately after the stakeholder needs-assessment interview)
Changes discussed	Confirmed the payment/proof-of-payment step, the Financial Clerk role, and the documents actually required (ID, clinic card, and others per the school's enrolment form)
Reason for the changes	These reflect the stakeholder's confirmed current process, not the group's original Unit 1 assumptions
Stakeholder comments	The Secretary, Zukiswa Khondlo confirmed the findings summary as accurate (see Appendix D) on behalf of the school.
Changes accepted?	Yes - confirmed by Zukiswa Khondlo
Evidence available	See Appendix B (interview record) and Appendix D (confirmation of findings)
Outstanding item	The repository-visibility change (Public → Private) and the added testing period are internal project-management decisions arising from the feasibility study, not changes to stakeholder-facing functions, so no separate stakeholder confirmation is required for them

PART D: SYSTEM ARCHITECTURE AND DESIGN

Main System Components

Component	Purpose
Flutter Mobile Application	Provides parents/guardians with a mobile interface to register, complete and submit enrolment applications, upload required documents and proof of payment, view the stakeholder-provided payment information, and track application and payment-verification status.
ASP.NET Web Application	Provides authorised school staff (Financial Clerk and Administrator) with a web interface to verify payments, review and process applications, and manage the digital enrolment workflow.
Supabase Authentication	Manages sign-in for parents (via Flutter) and role-based sign-in for the Financial Clerk and Administrator (via ASP.NET), enforcing the role separation required by NFR1 and NFR6.
Supabase Database	Stores parent/learner accounts, enrolment applications, application status, payment-verification status, and each learner's unique student number, shared between both applications.
Supabase Storage	Stores uploaded supporting documents (birth certificate, immunisation record, previous-school progress report, transfer letter) and proof-of-payment files, per FR3, FR4 and NFR2.

Who Uses Each Application

User/Role	Uses Flutter?	Uses ASP.NET?	Main purpose
Parent (Applicant)	Yes	No	Register, submit an enrolment application, upload documents and proof of payment, and track application status (FR1–FR5).
Financial Clerk	No	Yes	Verify a parent's proof of payment against the school's payment records (FR6).
Administrator	No	Yes	View applications, approve/reject/request corrections, leave notes, message parents, send deadline reminders, generate confirmation letters, and view the running application count (FR7–FR13).

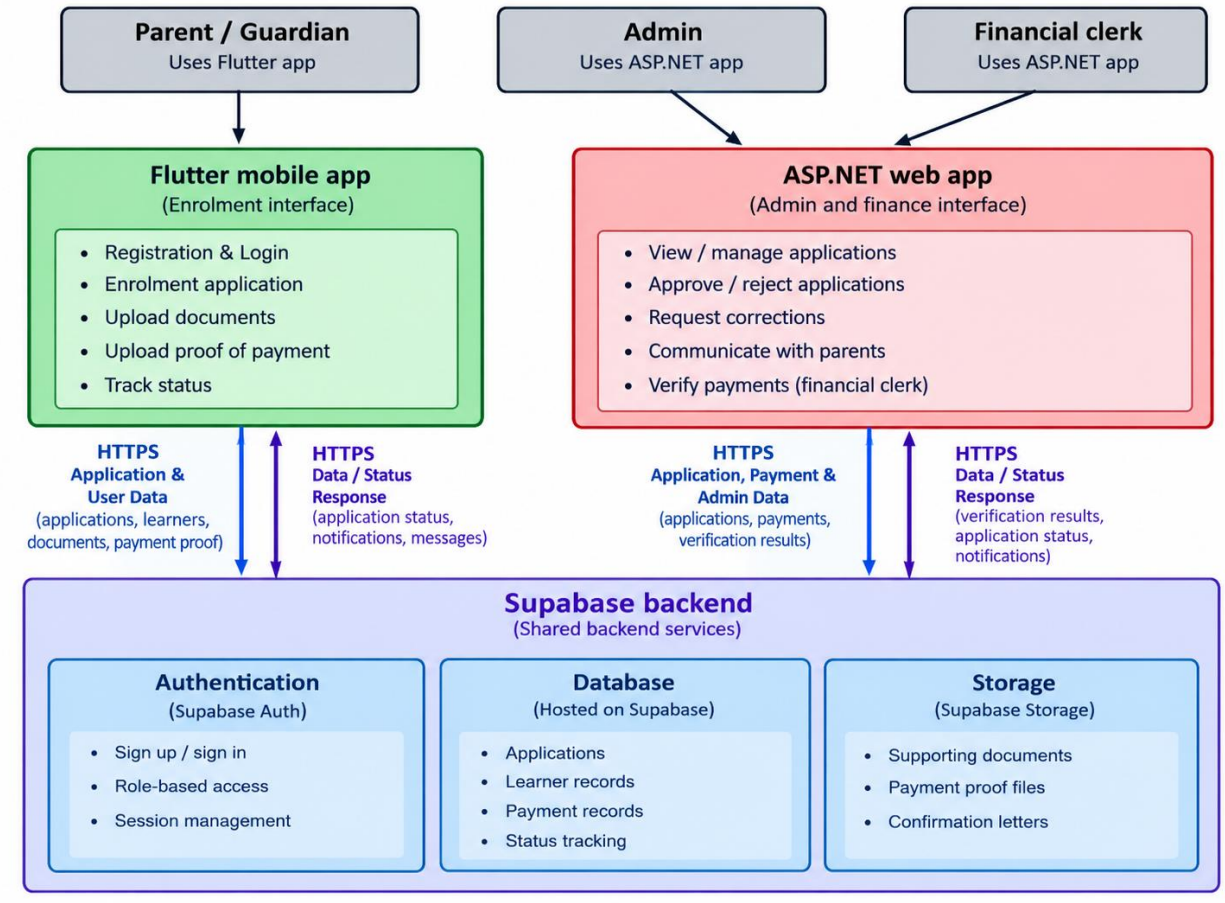
Responsibility of Each Component

Component	Main responsibilities
Flutter	Allow parents to register and sign in; complete and submit enrolment applications; upload required supporting documents and proof of payment; view real-time application and payment-verification status.
ASP.NET	Provide two role-based dashboards on one web application: the Financial Clerk verifies proof of payment against school records; the Administrator reviews applications, approves/rejects/requests corrections, leaves notes, messages parents about incorrect documents, sends deadline reminders, generates confirmation letters, and views the running application count.
Supabase Authentication	Verify user identity at login and assign the correct role (Parent, Financial Clerk or Administrator) so each user only sees what their role permits.
Supabase Database	Store and synchronise parent/learner accounts, enrolment applications, application status, payment-verification status and student numbers, so updates made via ASP.NET are reflected back to the parent in Flutter.
Supabase Storage	Securely store uploaded supporting documents and proof-of-payment files, accessible to authorised staff through ASP.NET and to the submitting parent through Flutter.

Main Data Flows

From	To	Information exchanged
Parent/Guardian	Flutter	Registration details, application form data, uploaded documents, proof of payment.
Flutter	Supabase Authentication	Parent sign-up / login credentials.
Flutter	Supabase Database	New enrolment application record; requests for current status.
Flutter	Supabase Storage	Uploaded supporting documents and proof-of-payment files.
Supabase Database	ASP.NET (Financial Clerk)	Submitted proof of payment and application records for verification.
ASP.NET (Financial Clerk)	Supabase Database	Payment-verification status update.
Supabase Database	ASP.NET (Administrator)	Submitted applications, documents, payment-verification status, running application count.
ASP.NET (Administrator)	Supabase Database	Approval/rejection decision, notes, parent messages, deadline reminders, confirmation-letter data.
Supabase Database	Flutter	Updated application and payment-verification status shown to the parent.

System Architecture Diagram



Architecture Explanation

Parents/guardians use the Flutter mobile application to register, submit enrolment applications, upload the required supporting documents and proof of payment, and track their application and payment status.
School staff use the ASP.NET web application: the Financial Clerk verifies submitted proof of payment against the school's records, and the Administrator reviews applications, approves, rejects or requests corrections, leaves notes, messages parents, sends deadline reminders, generates confirmation letters, and monitors the running application count.
Supabase is responsible for the shared backend of the system. Supabase Authentication manages sign-in and enforces role-based access for parents, the Financial Clerk and the Administrator. Supabase Database stores all enrolment, learner and status information, and Supabase Storage holds uploaded documents and proof-of-payment files.
Flutter communicates with Supabase directly through Supabase's Authentication, Database and Storage APIs: parents authenticate, submit application data, and upload files, all of which are written straight to the shared backend.
ASP.NET communicates with Supabase in the same way - the Financial Clerk and Administrator sign in through Supabase Authentication, and their dashboards read and write application, payment and document data directly to the same Supabase Database and Storage used by Flutter.
Because both applications read from and write to the same Supabase backend, the two applications form one integrated system: an application and documents submitted by a parent on Flutter immediately become visible to staff on ASP.NET, and status or verification updates made by staff on ASP.NET are immediately reflected back to the parent on Flutter (IR1–IR3).

Use Case Diagram

Actors

Actor	What this actor needs to do?
Parent/Guardian	Register and sign in, view the enrolment fee and school banking details, submit an enrolment application (including uploading supporting documents and proof of payment), and track application/payment status.
Financial Clerk	A specialized staff member who signs in and verifies a parent's proof of payment against the school's payment records.
Administrator	A specialized staff member who signs in, views submitted applications, approves/rejects/requests correction, leaves notes, messages parents, sends deadline reminders, generates confirmation letters, and views the running application count.

Use Cases

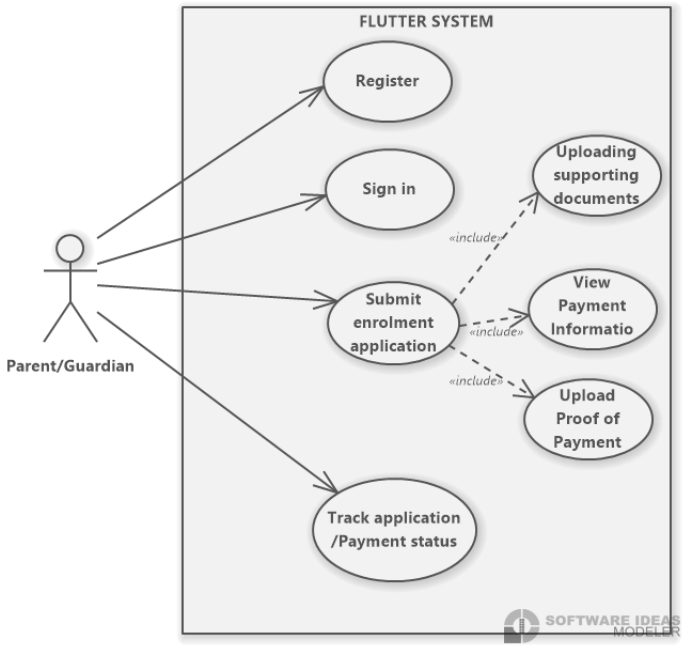
Actor	Use Case
Parent/Guardian	Register
Parent/Guardian	Sign In
Parent/Guardian	Submit Enrolment Application
Parent/Guardian	Upload Supporting Documents
Parent/Guardian	Upload Proof of Payment
Parent/Guardian	View Application/Payment Status
Parent/Guardian	View Payment Information
Financial Clerk	Sign In
Financial Clerk	Verify Proof of Payment
Administrator	Sign In
Administrator	View Submitted Applications

Actor	Use Case
Administrator	Approve/Reject Application
Administrator	Request Corrections
Administrator	Leave Notes on Application
Administrator	Message Parent
Administrator	Send Deadline Reminders
Administrator	Generate Confirmation Letter
Administrator	View Application Count

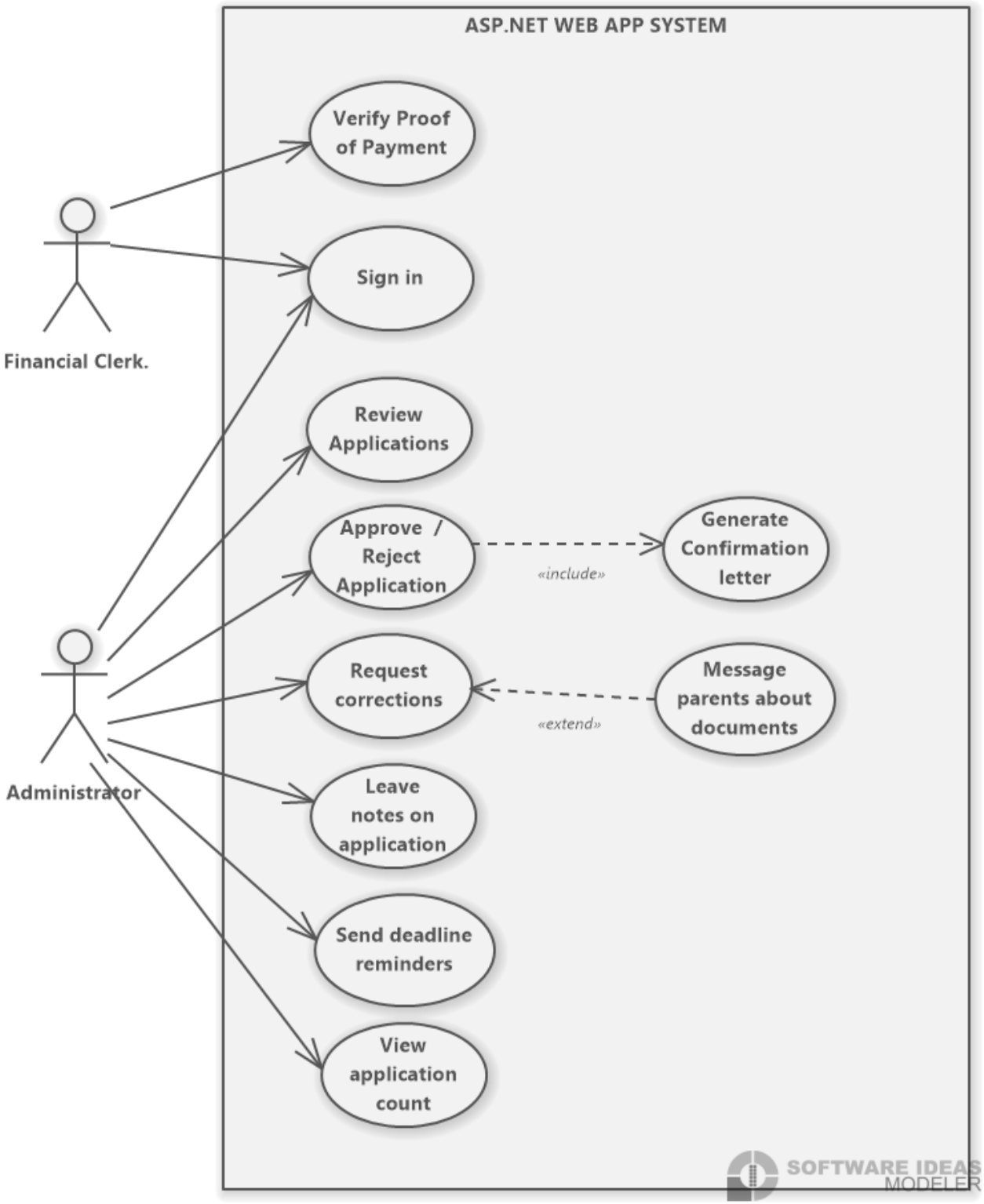
Use Case Relationships

Relationship	Where used	Why
<<include>>	Submit Enrolment Application **<<include>>** Upload Supporting Documents	Uploading the required supporting documents forms part of completing a complete enrolment application.
<<include>>	Submit Enrolment Application **<<include>>** Upload Proof of Payment	Proof of payment is required before the enrolment process can be finalised.
<<extend>>	Message Parent **<<extend>>** Request Corrections	The Administrator only messages the parent when corrections are actually needed - conditional behaviour.
<<include>>	Approve/Reject Application **<<include>>** Generate Confirmation Letter	A confirmation letter is generated after the Administrator makes the final application decision, whether the application is approved or rejected so that the parent/guardian receives a formal record of the outcome.
<<include>>	Submit Enrolment Application <<**include**>> View Payment Information	The parent must always see the enrolment fee and school banking details and enter a payment reference before proof of payment is uploaded (FR15).

Use Case Diagram 1 – Parent/Guardians

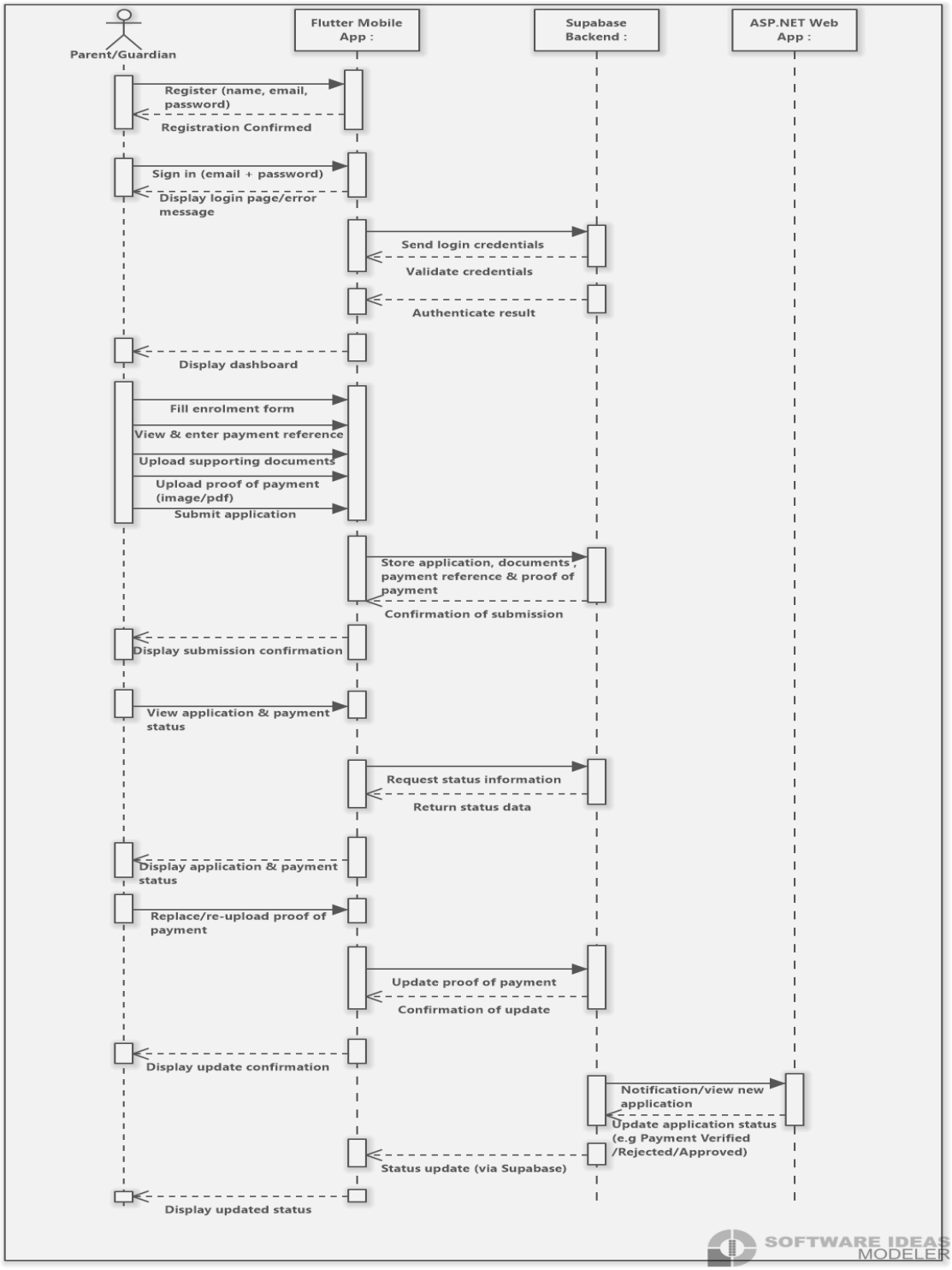


Use Case Diagram 2 – Staff



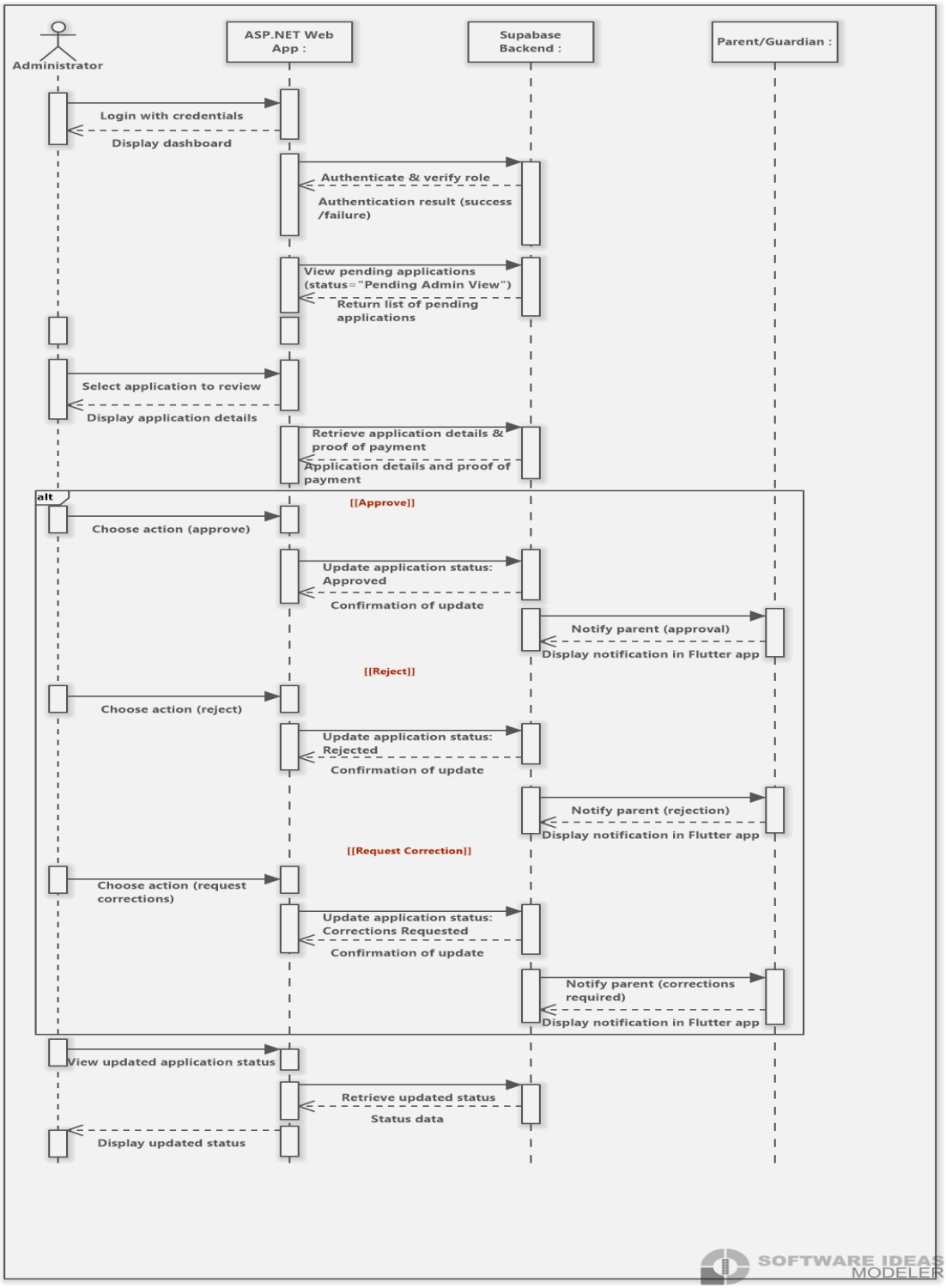
Sequence Diagram

Sequence Diagram 1 – Submit Enrolment Application (Flutter)



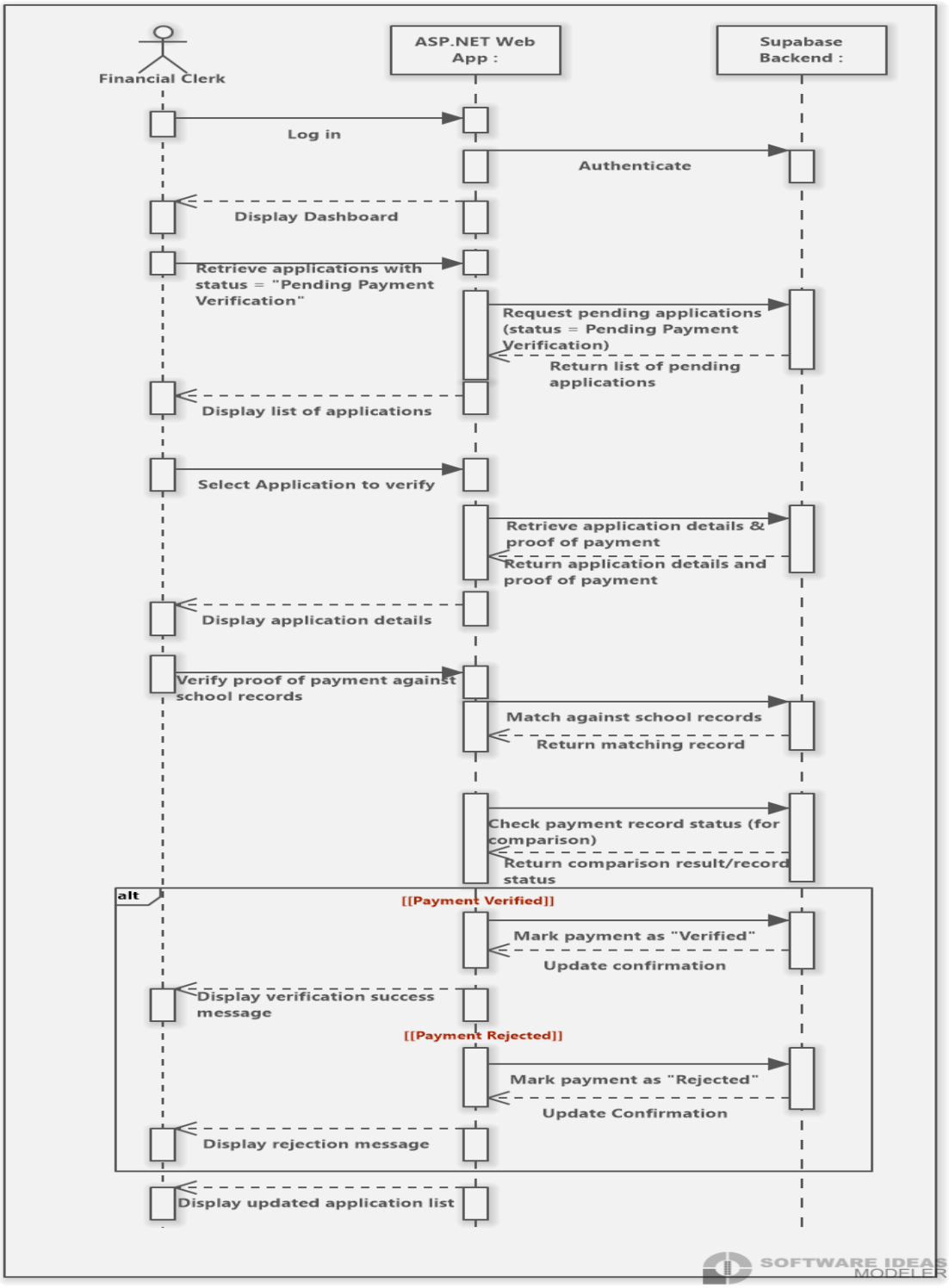
Participant	Role
Parent/Guardian	Starts the process by completing and submitting the application.
Flutter Mobile App	Validates input and captures the application, documents, and proof of payment.
Supabase Backend	Stores the new application record and confirms creation.
ASP.NET Web App	Staff-side application that updates the application/payment status in Supabase; the update then reaches the parent through Supabase.

Sequence Diagram 2 – Approve or Reject Application (ASP.NET)



Participant	Role
Administrator	Starts the review and makes the approval decision.
ASP.NET Web App	Retrieves the application for review and applies the status update.
Supabase Backend	Stores the updated status and serves it back to whichever app requests it.
Parent/Guardian	Receives the outcome notification, which Supabase delivers to the Flutter application (Notifications/Messages screen).

Sequence Diagram 3 – Verify Proof of Payment (ASP.NET)



Participant	Role
Financial Clerk	Starts the process by opening the application to check payment.
ASP.NET Web App	Retrieves the uploaded proof of payment and applies the verification update.
Supabase Backend	Stores the verification status and confirms the update.
School Payment Records	External record the Financial Clerk manually checks proof of payment against.

Database Design

Entities and Attributes

Entity (table)	Key Attributes	Keys
PROFILES	full_name, email, role (Parent / Financial Clerk / Administrator), created_at, updated_at, phone_number, relationship_to_learner, occupation	PK: id (matches Supabase auth.users.id)
LEARNER_MEDICAL_INFO	medical_aid_name, medical_aid_number, medical_aid_main_member, doctor_name, doctor_address, medical_condition	PK: medical_info_id · FK: learner_id
LEARNERS	first_name, middle_name, surname, id_number, date_of_birth, gender, student_number, created_at, updated_at, previous_school_name, highest_grade_passed, previous_grade_passed	PK: learner_id · FK: parent_id
APPLICATIONS	application_date, status, payment_verification_status, submitted_at, reviewed_at, created_at, updated_at	PK: application_id · FK: learner_id, submitted_by, reviewed_by
APPLICATION_DOCUMENTS	document_type, file_name, storage_path, uploaded_at, verification_status, verified_at	PK: document_id · FK: application_id, uploaded_by, verified_by
PAYMENTS	payment_reference, amount, proof_file_name, proof_storage_path, verification_status, verified_at, verification_notes, created_at, updated_at	PK: payment_id · FK: application_id, school_payment_record_id, verified_by
SCHOOL_PAYMENT_RECORDS	payment_reference (unique), learner_name, learner_id_number, amount, payment_date, payment_method, created_at	PK: school_payment_record_id
APPLICATION_NOTES	note_text, created_at	PK: note_id · FK: application_id, created_by
APPLICATION_STATUS_HISTORY	old_status, new_status, change_reason, changed_at	PK: status_history_id · FK: application_id, changed_by

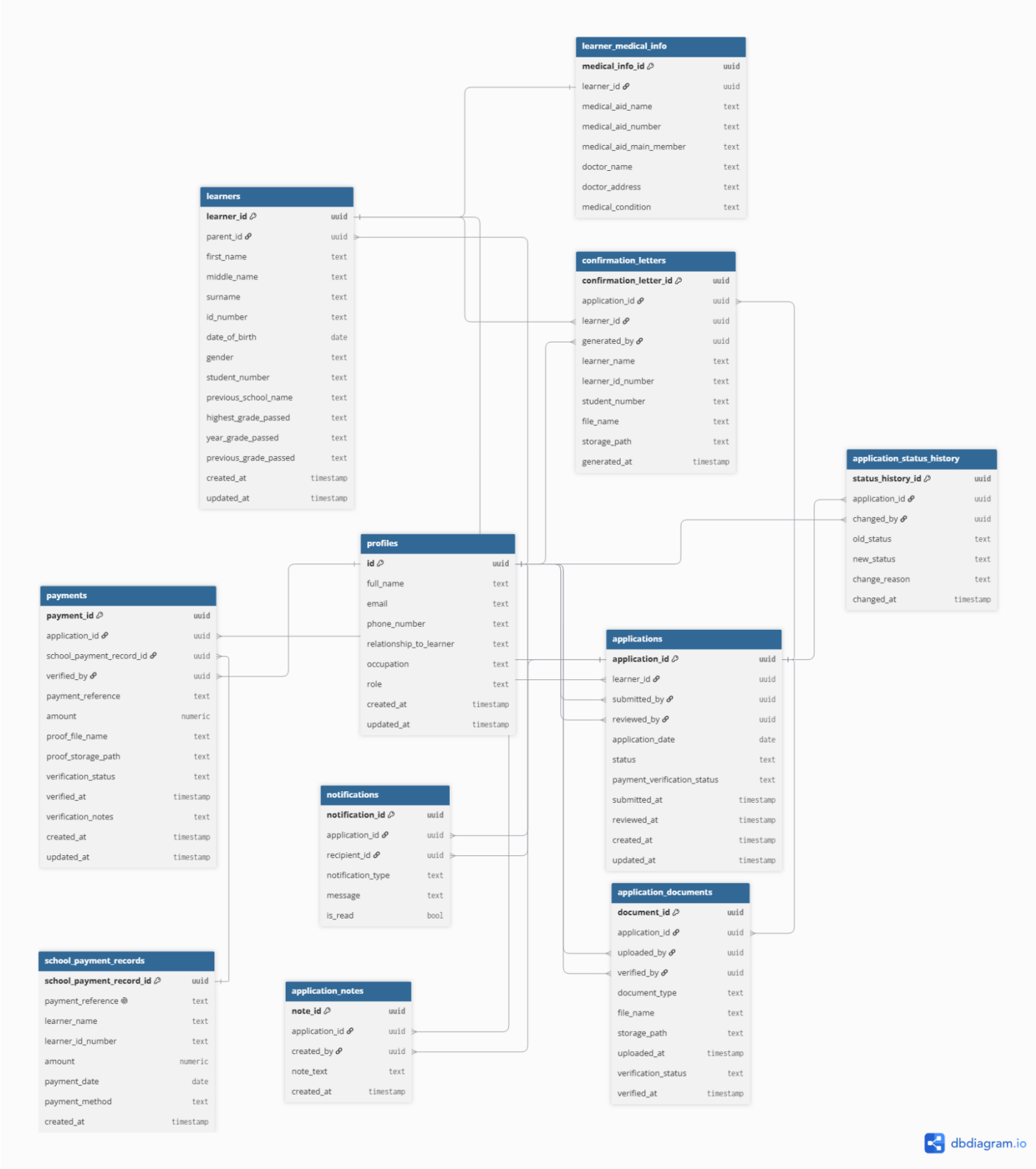
CONFIRMATION_LETTERS	learner_name, learner_id_number, student_number, file_name, storage_path, generated_at	PK: confirmation_letter_id · FK: application_id, learner_id, generated_by
NOTIFICATIONS	notification_type, message, is_read	PK: notification_id · FK: application_id, recipient_id

Relationships and Cardinality

Relationship	Cardinality	Meaning
auth.users - PROFILES	1 to 1	Every Supabase Auth account has one profile that stores the user's role.
PROFILES (parent) - LEARNERS	1 to many	A parent may be guardian to one or more learners.
LEARNERS - APPLICATIONS	1 to many	A learner may have multiple applications over time (e.g. re-applications).
PROFILES (parent) - APPLICATIONS (submitted_by)	1 to many	A parent may submit several applications.
PROFILES (Administrator) - APPLICATIONS (reviewed_by)	1 to .many	An application is reviewed by at most one Administrator; one Administrator reviews many applications.
APPLICATIONS - APPLICATION_DOCUMENTS	1 to many (mandatory)	Every application must have at least one supporting document.
APPLICATIONS - PAYMENTS	1 to many (mandatory)	Every application must have a proof-of-payment record; a new row is inserted for each re-upload (see Decision 1).
SCHOOL_PAYMENT_RECORDS - PAYMENTS	1 to many	A payment attempt is matched to a school payment record through payment_reference during verification.
PROFILES (Financial Clerk) - PAYMENTS (verified_by)	1 to many	A Financial Clerk verifies many payment records.
APPLICATIONS - APPLICATION_NOTES	1 to many	An application may accumulate several notes (FR9).
PROFILES (Administrator) - APPLICATION_NOTES	1 to many	An Administrator may write many notes.
APPLICATIONS - APPLICATION_STATUS_HISTORY	1 to many	Every status change on an application is recorded as an audit entry.
APPLICATIONS - CONFIRMATION_LETTERS	1 to 1	A confirmation letter is generated only for an approved or rejected application (FR12).
APPLICATIONS - NOTIFICATIONS	1 to many	Staff messages and reminders about an application are stored as notifications (FR10, FR11).

PROFILES (parent) - NOTIFICATIONS (recipient_id)	1 to many	A parent may receive many notifications.
LEARNERS – LEARNER_MEDICAL_INFO	1 to 1	A learner may have one medical information record, capturing the medical aid and doctor details required by the school's enrolment form.

ERD Diagram



Database Design Decisions and Refinements

Reviewing the implemented Supabase schema against the approved SRS and sequence diagrams identified the following decisions, which the group has agreed to keep so that the schema, sequence diagrams, and written design remain consistent.

1. Payment resubmission history.
The Parent sequence diagram allows a parent to replace/re-upload proof of payment after a rejection. To preserve a full history of payment attempts, the Application–Payment relationship is **one-to-many**: a new payments row is inserted for each re-upload rather than overwriting the previous attempt. The application-level payment verification status is maintained from the payment records, with a **Verified** payment taking priority, followed by **Pending** and then **Rejected**.

2. Status values are constrained, not free text.
applications.status, applications.payment_verification_status, payments.verification_status, and application_documents.verification_status use text values that follow the exact status vocabulary defined in the approved SRS and sequence diagrams. The application and payment workflow is further controlled by the implemented database logic and Row Level Security (RLS) policies so that only authorised roles can perform the relevant operations.

3. Two payment-status fields are kept in sync deliberately.
verification_status on payments records the verification result of each payment attempt, while payment_verification_status on applications provides an application-level summary for efficient status retrieval. A database trigger updates the application-level payment status when a payment attempt changes, with a **Verified** payment taking priority over **Pending** and **Rejected** attempts.

4. Uniqueness on school payment records.
school_payment_records.payment_reference has a uniqueness constraint because the Financial Clerk's manual verification process depends on matching the uploaded proof of payment to the correct school payment record.

5. Per-document verification fields.
application_documents retains verification_status, verified_by, and verified_at as forward-looking fields. The group's approved scope for this phase only includes verifying proof of payment, not individual supporting documents. These columns are therefore not currently written to by a use case and are documented here to show that their omission from the current workflow is a deliberate scope decision rather than an inconsistency.

6. Learner details are intentionally duplicated on confirmation_letters and school_payment_records.
Both tables store the learner's name and ID number as plain text rather than only referencing learners through a foreign key. This is deliberate because a confirmation letter and a school payment record should reflect the learner's details as they were when the record was created, rather than changing retroactively if the learner's profile is edited later.

7. Delete behaviour on audit and financial tables.
Foreign-key relationships involving payments, confirmation_letters, and application_status_history do not use cascading deletes. This prevents deleting a parent, learner, or application from silently removing financial or audit history.

8. Academic, medical and guardian fields added after wireframe review.
The Flutter Learner Details/Enrolment Form wireframes (Academic & School History, Medical & Family, Parent/Guardian screens) captured data not originally represented in the schema. LEARNERS was extended with academic-history fields, anew LEARNER_MEDICAL_INFO table (one-to-one with LEARNERS) was added for medical aid and doctor details, and PROFILES was extended with parent contact fields, so that every field shown on the interfaces has a corresponding column in the database.

Alignment Check Across Designs

Each row confirms that the same approved function is represented consistently across all three artefacts, per Step 14 of the activity brief.

Requirement	Use Case Diagram	Sequence Diagram	ERD
Parent submits application	Submit Enrolment Application	Fig. 2 - Parent → Flutter → Supabase	APPLICATIONS, LEARNERS, APPLICATION_DOCUMENTS and PAYMENTS tables
Administrator approves application	Approve / Reject Application	Fig. 3 - Administrator → ASP.NET → Supabase	APPLICATIONS table - status attribute
Parent tracks application/payment status	Track Application/Payment Status	Fig. 2 - Parent → Flutter → Supabase	APPLICATIONS - status and payment_verification_status; PAYMENTS — verification_status
Financial Clerk verifies payment	Verify Proof of Payment	Fig. 4 - Financial Clerk → ASP.NET → Supabase	PAYMENTS - verification_status; SCHOOL_PAYMENT_RECORDS - payment_reference

User Interfaces and System Design

Approved System Context

The system consists of a Flutter mobile application for Parent/Guardian users, an ASP.NET web application for the Financial Clerk and Administrator, and a shared Supabase backend/database and document storage. The design supports online enrolment, document and payment-proof uploads, application-status tracking, payment verification, administrative review, and communication.

Required Flutter Screens

Flutter Screen	User	Purpose	Related Requirement/Use Case
Register	Parent/Guardian	Create a parent account	FR1 – Register / Sign In
Login	Parent/Guardian	Authenticate the parent	FR1 – Register / Sign In
Home/Dashboard	Parent/Guardian	Access enrolment functions and application status	FR5
Learner Details/Enrolment form (includes Academic + Medical + Parent/Guardian information)	Parent/Guardian	Enter learner information and complete the application	FR2 – Complete/Submit Enrolment
Supporting Documents	Parent/Guardian	Upload required learner documents	FR3 – Upload Supporting Documents
Payment Information	Parent/Guardian	Display the stakeholder-confirmed enrolment fee and school banking details and allow the parent/guardian to enter a payment reference before proceeding to proof-of-payment upload.	FR15 – View Payment Information / Submit Enrolment Application
Proof of payment	Parent/Guardian	Upload proof of payment after entering the payment reference.	FR4 – Upload Proof of Payment
Application Status	Parent/Guardian	View application and payment-verification status	FR5 – View Application/Payment Status
Application Details	Parent/Guardian	View submitted application information and status	FR5
Notifications/Messages	Parent/Guardian	Receive staff messages and reminders	FR10, FR11
Profile	Parent/Guardian	View/manage account information	Authenticated user support
Help & Support	Parent/Guardian	View guidance on applying, required documents and contacting the school	Supports FR1–FR5 (user guidance)

Required ASP.NET Pages

ASP.NET Page	User	Purpose	Related Requirement/Use Case
Login	Administrator/Financial Clerk	Authenticate staff and provide the appropriate role-based access path for the Administrator or Financial Clerk	Sign In (NFR1, NFR6)
Financial Clerk Dashboard	Financial Clerk	Provide access to payment-verification functions and a summary of payment-related activity.	FR6 - Verify Proof of Payment
Payment Details	Financial Clerk	Display the payment information and supporting details needed to verify the parent's proof of payment.	FR6 – Verify proof of payment
Administrator Dashboard	Administrator	View application summary counts, application trend and recent applications	FR13 – View Application Count
Applications	Administrator	View all submitted applications with search and filters	FR7 – View Submitted Applications
Approved / Rejected / Pending Applications	Administrator	View applications filtered by status	FR7
Application Review	Administrator	View learner/application details and documents; approve, reject or request corrections	FR7, FR8
Application Notes	Administrator	Add and view internal notes on an application	FR9 – Leave Notes on Application
Message Parent	Administrator	Message a parent about missing or incorrect documents	FR10 – Message Parent About Documents
Send Reminder	Administrator	Send deadline reminders to parents	FR11 – Send Deadline Reminders
Confirmation Letter	Administrator	Preview and generate the confirmation letter for an approved or rejected application	FR12 – Generate Confirmation Letter
Payment Verification	Financial Clerk	Review proof of payment next to the matching school payment record and verify it	FR6 – Verify Proof of Payment
Proof of Payment	Financial Clerk	Opened by clicking View from Payment Verification; shows the uploaded proof of payment so that the clerk can approve or reject it	FR6 – Verify Proof of Payment
Verified Payments	Financial Clerk	Lists payments the Financial Clerk has already verified, kept for their own record-keeping	FR6 – Verify Proof of Payment

Navigation Flows

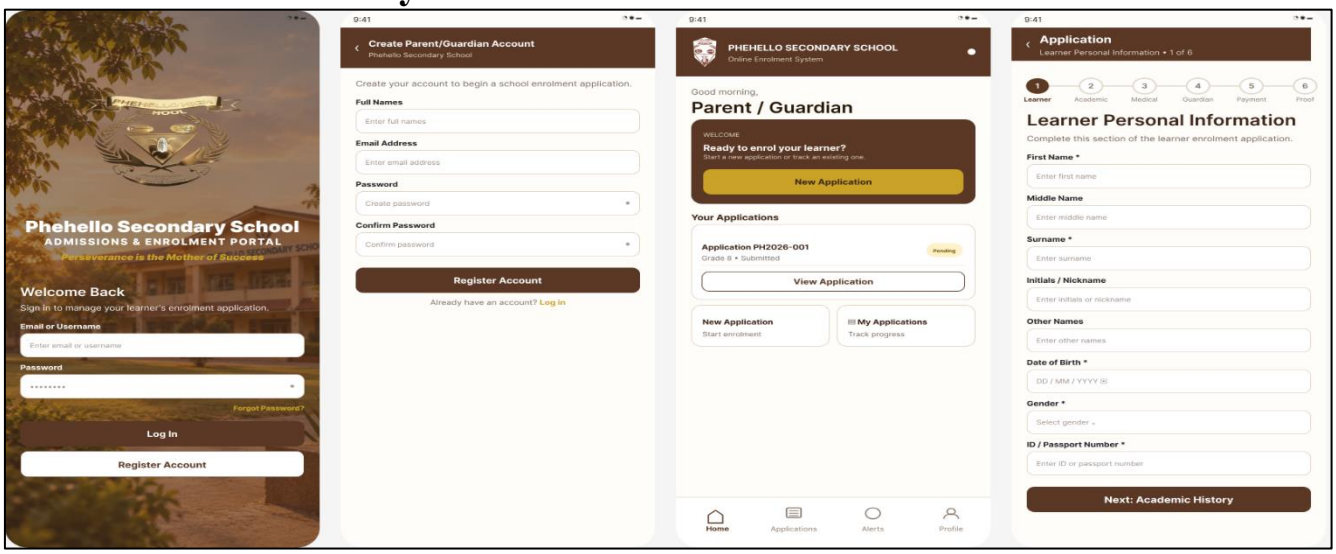
Parent/Guardian: Register → Login → Home → Learner Details/Enrolment Form → Supporting Documents → Payment Information → Proof of Payment → Submit Application → Application Status → Notifications / Messages

Financial Clerk: Login → Financial Clerk Dashboard → Payment Details → Payment Verification →View (opens Proof of Payment) → Verified Payments/Payment Details

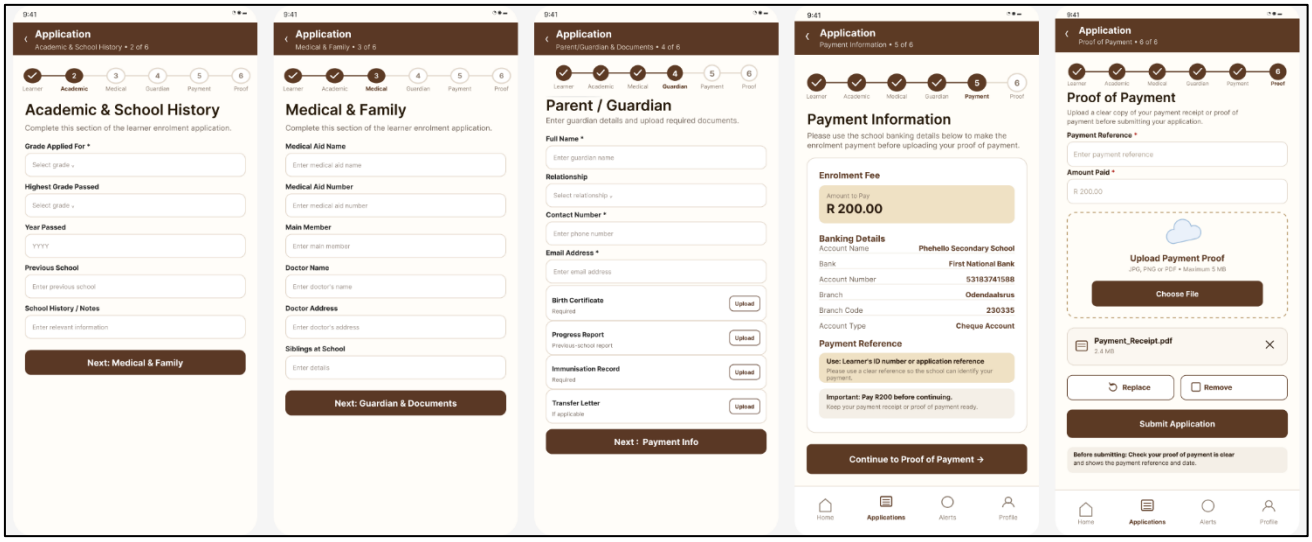
Help: Help

Administrator: Login → Administrator Dashboard → Applications (All/Pending/Approved/Rejected) → Application Review → Approve / Reject / Request Corrections. From Application Review the Administrator can open Application Notes and Message Parent. Approved or rejected applications may continue to Confirmation Letter. Send Reminder is opened from the menu.

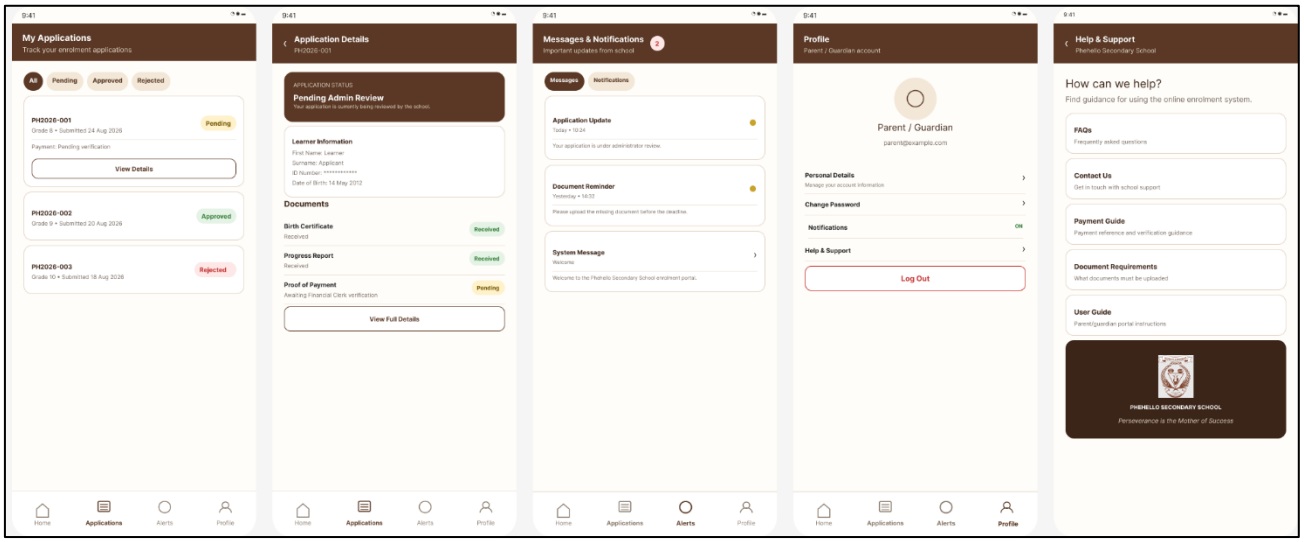
Illustrative low-fidelity Flutter wireframes for Parents/Guardians



Login, Register, Home Dashboard and Learner Personal Information screens

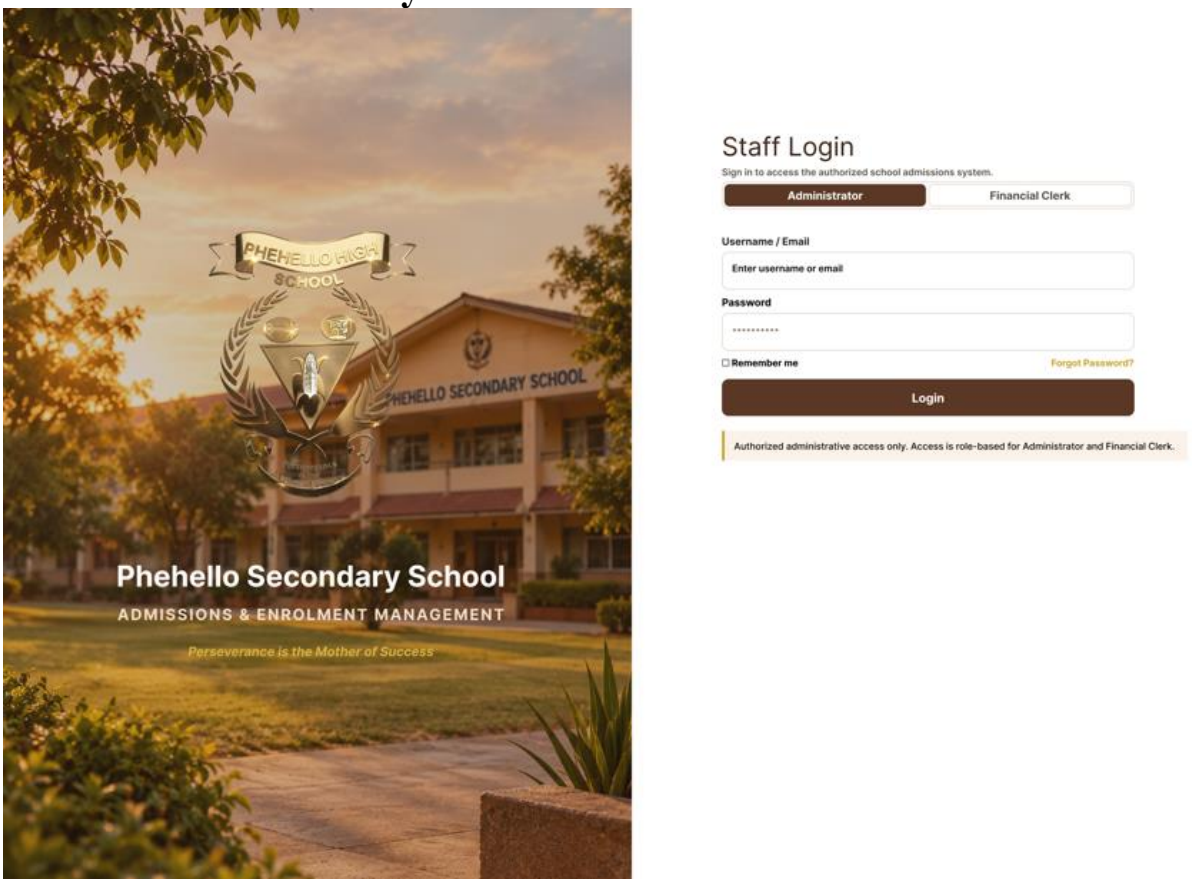


Academic & School History, Medical & Family , Parent/Guardian, Payment Information and Proof of payment screens

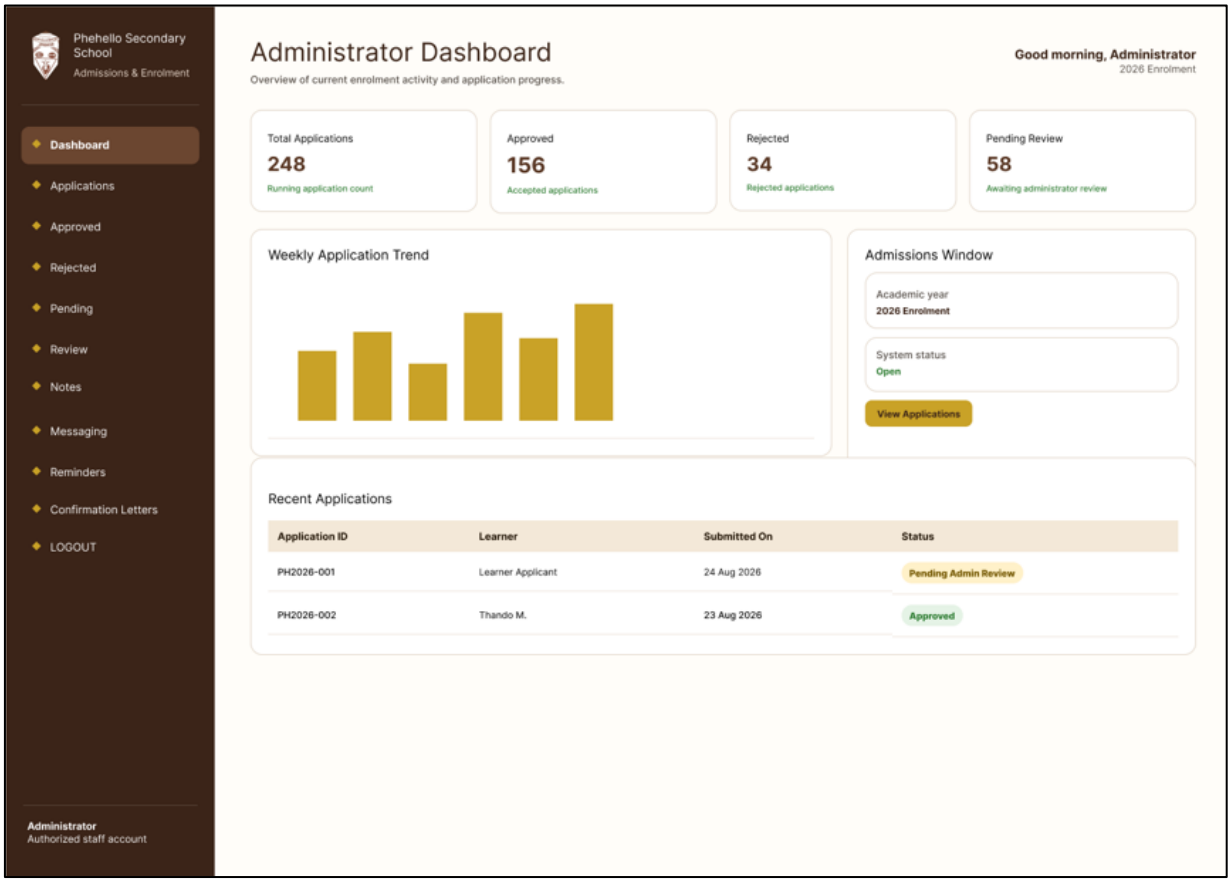


My applications, Application Details, Messages and Notifications, Profile, Help & Support screeens

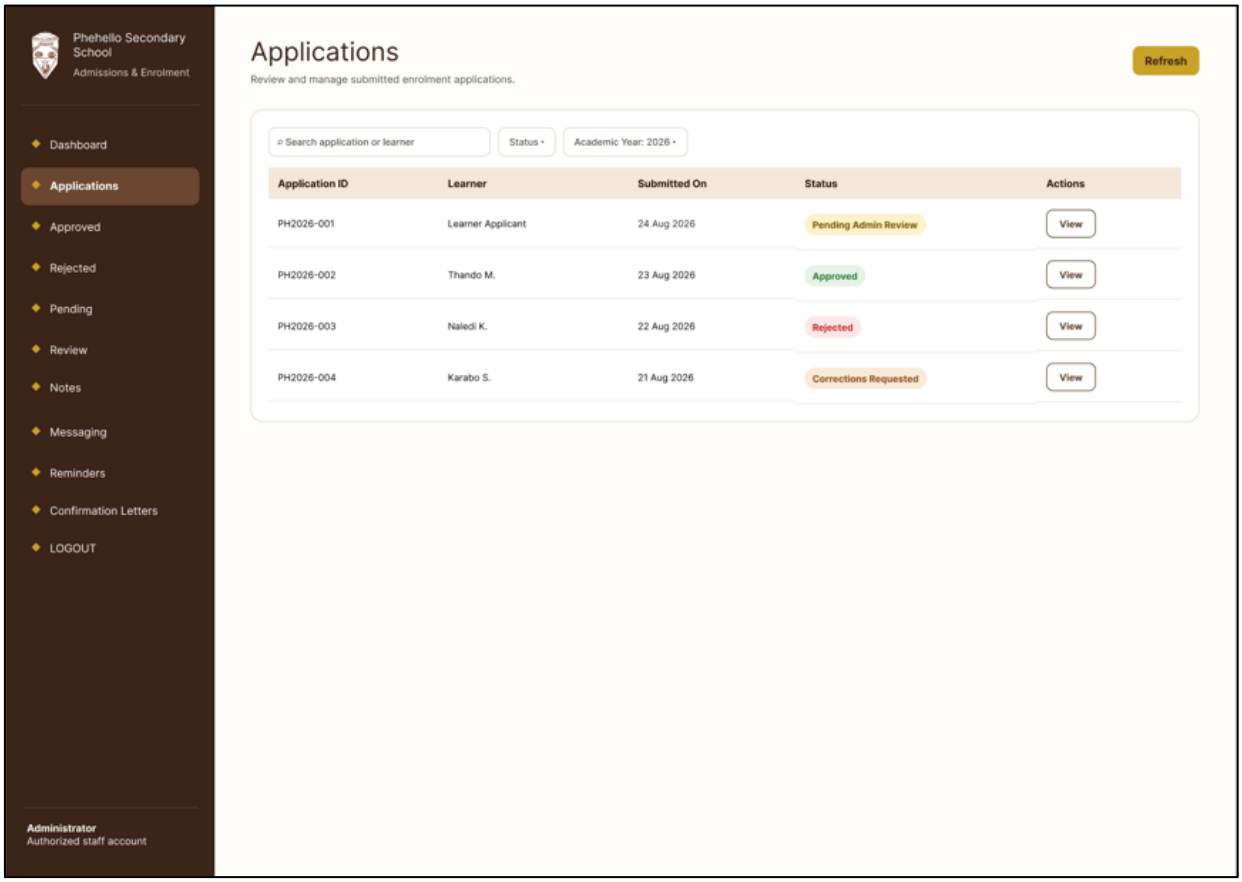
Illustrative low-fidelity ASP.NET wireframes for Administrator



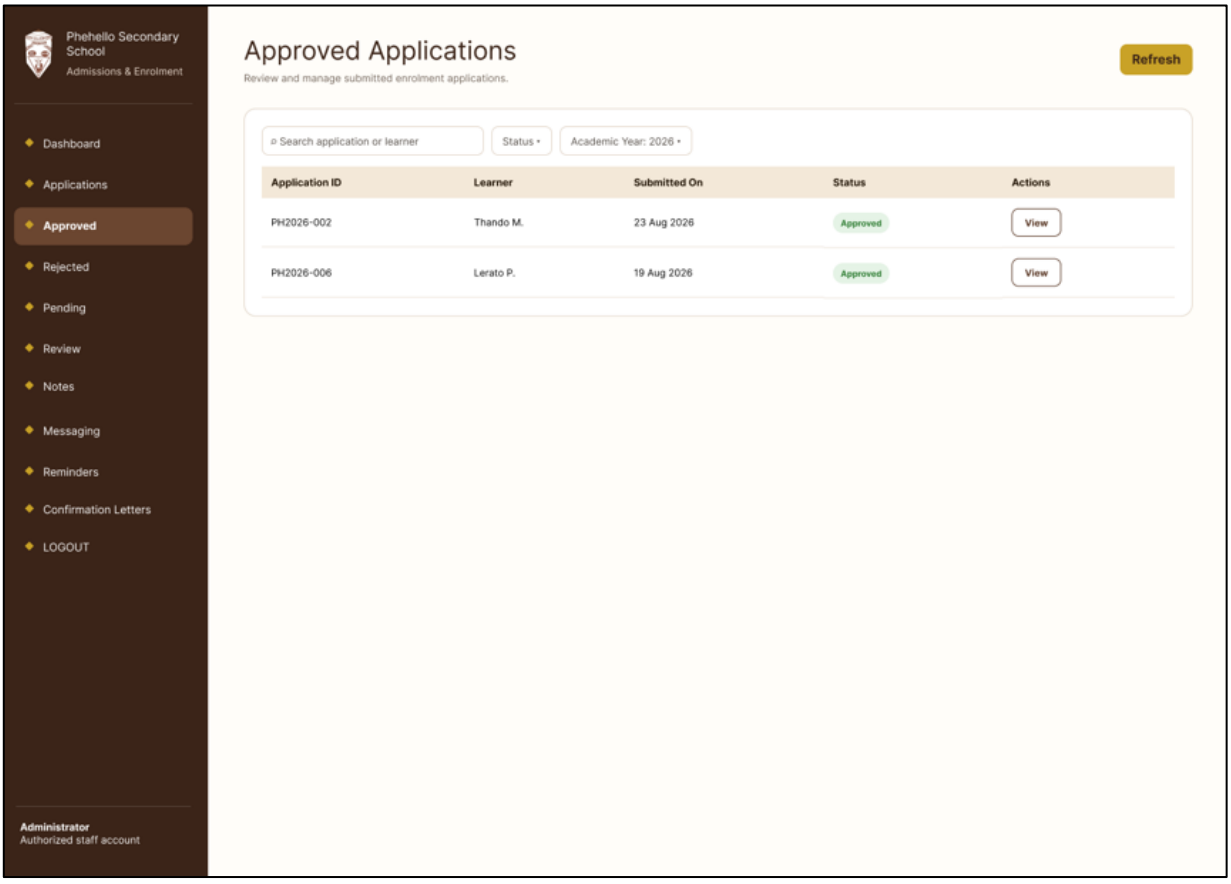
Login/Sign in screen



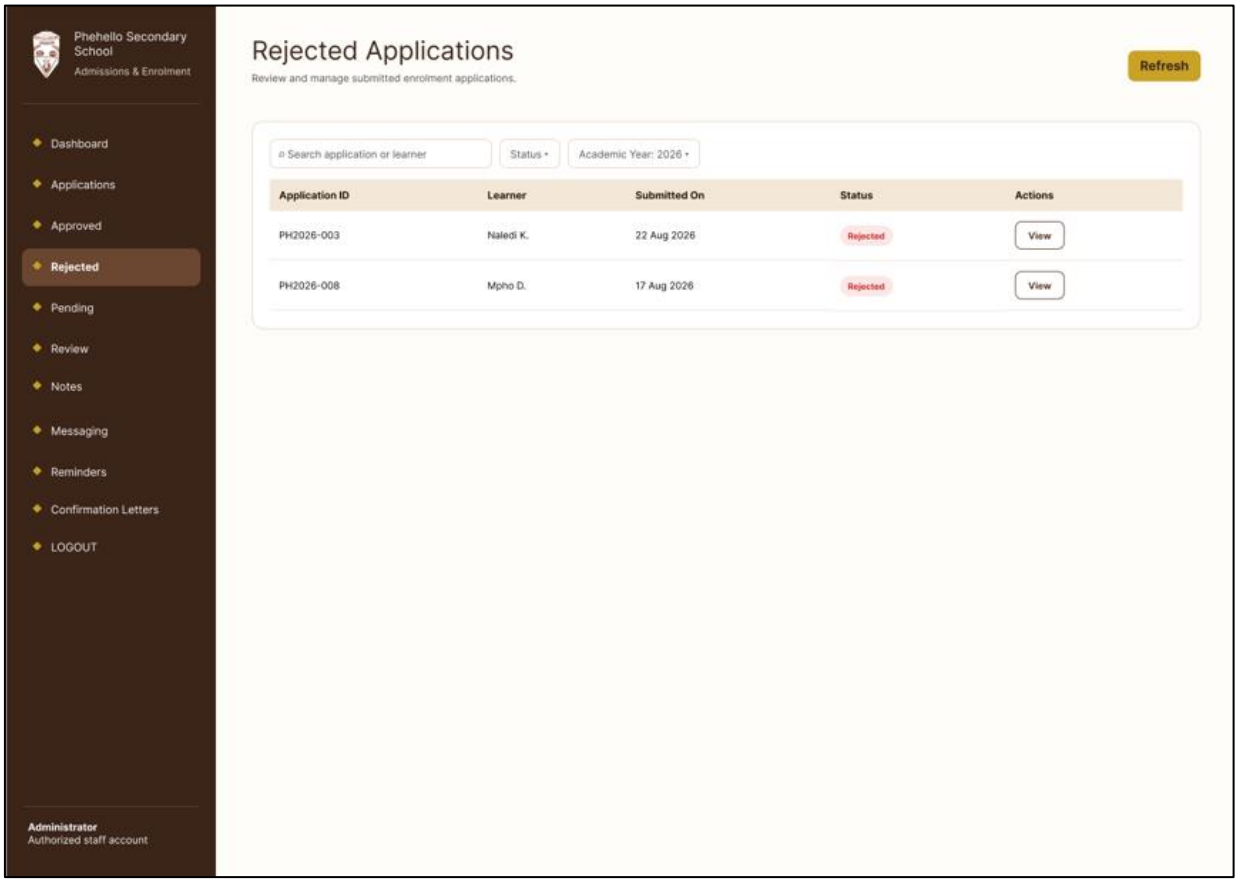
Administrator Dashboard screen



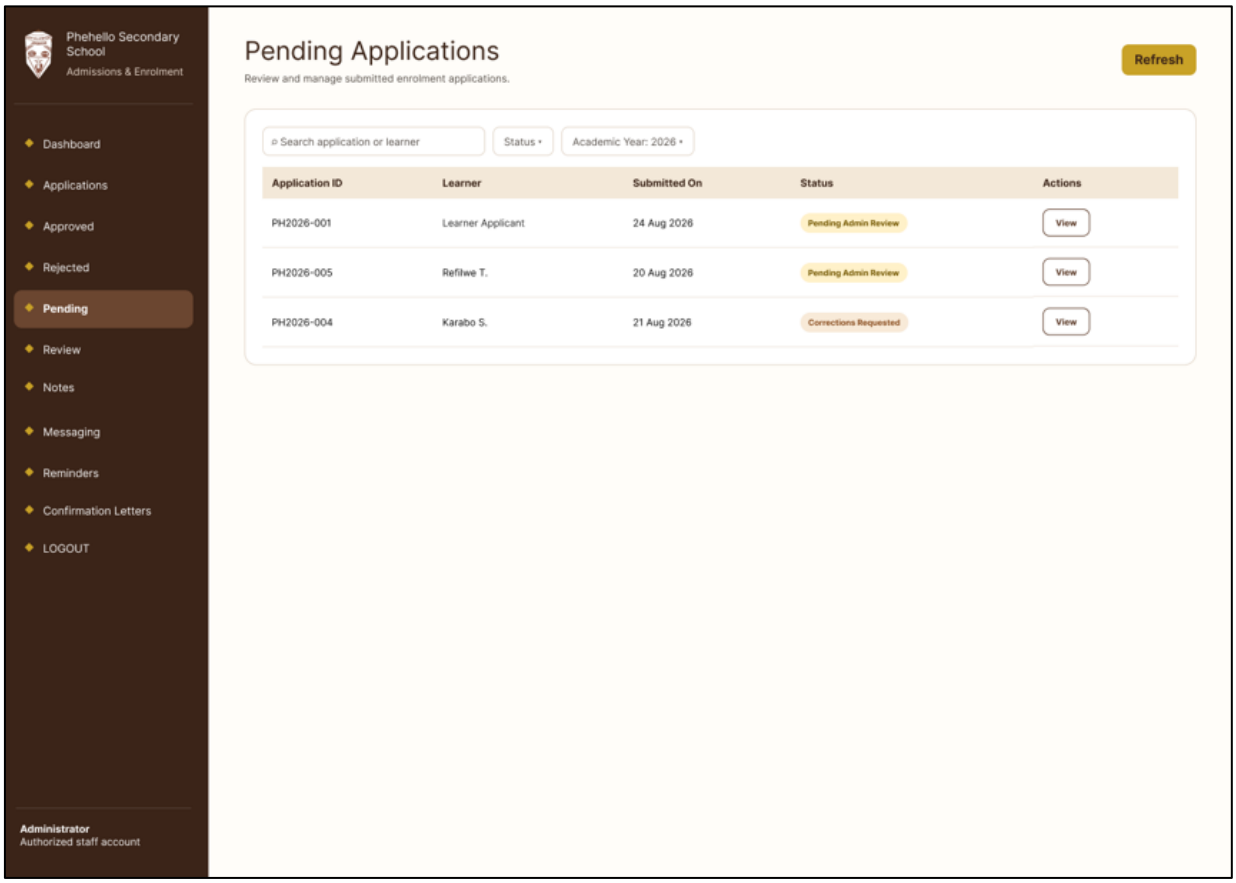
Applications screen



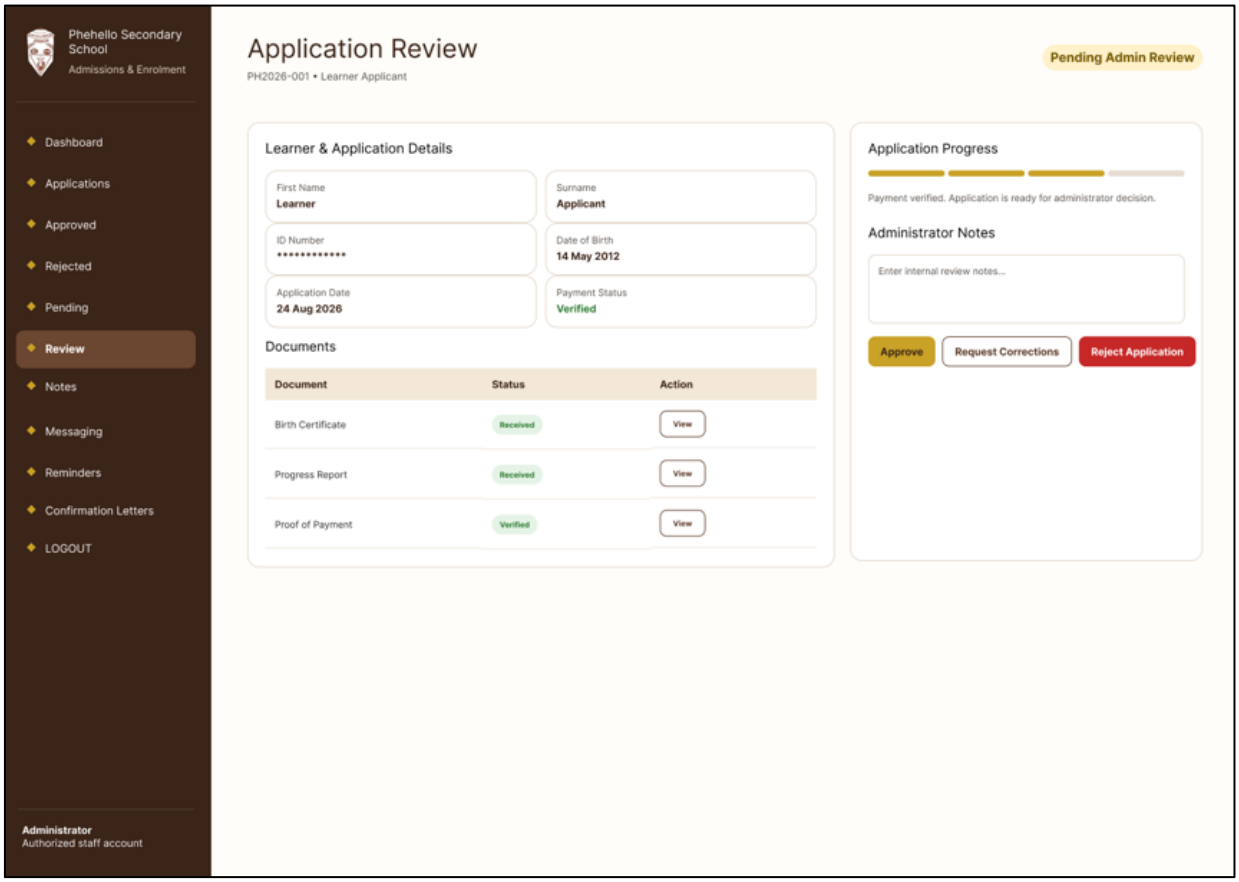
Approved Applications screen



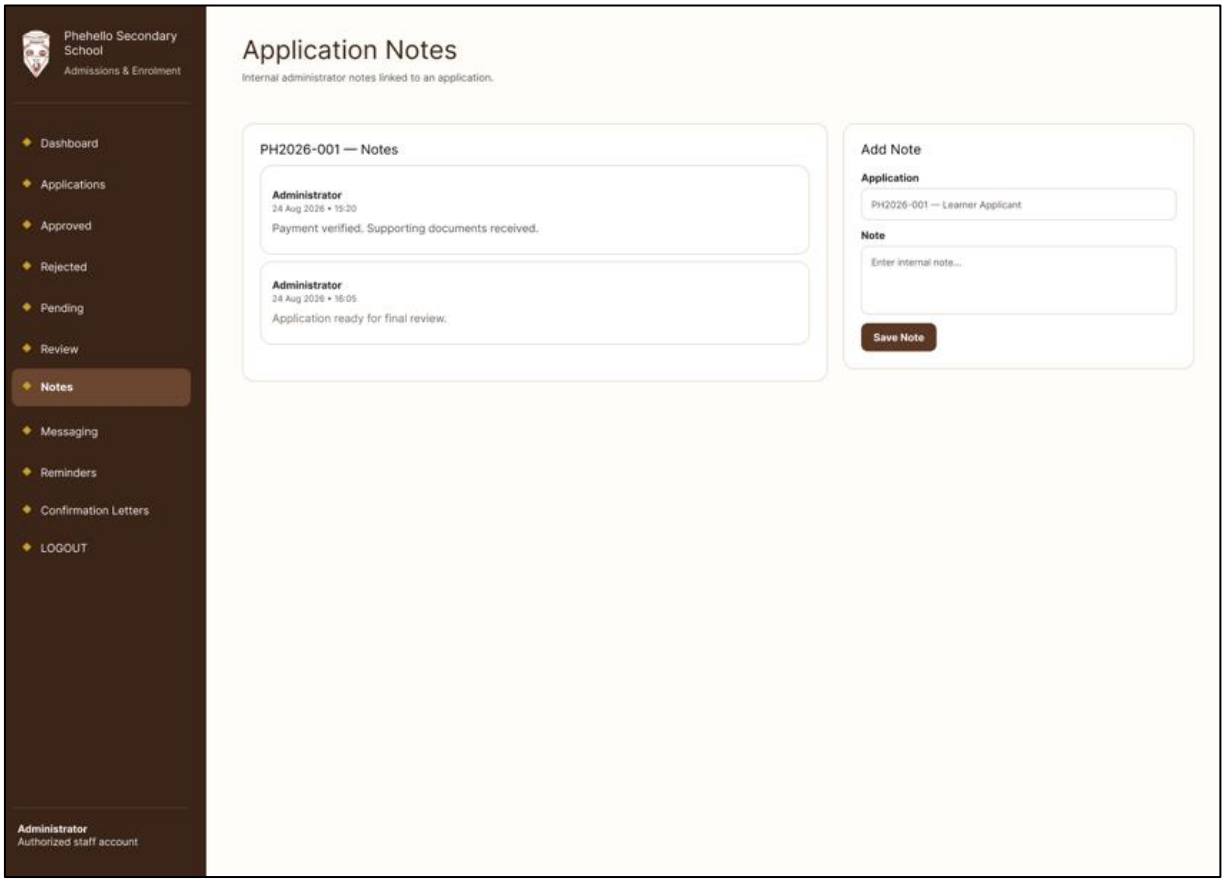
Rejected Applications screen



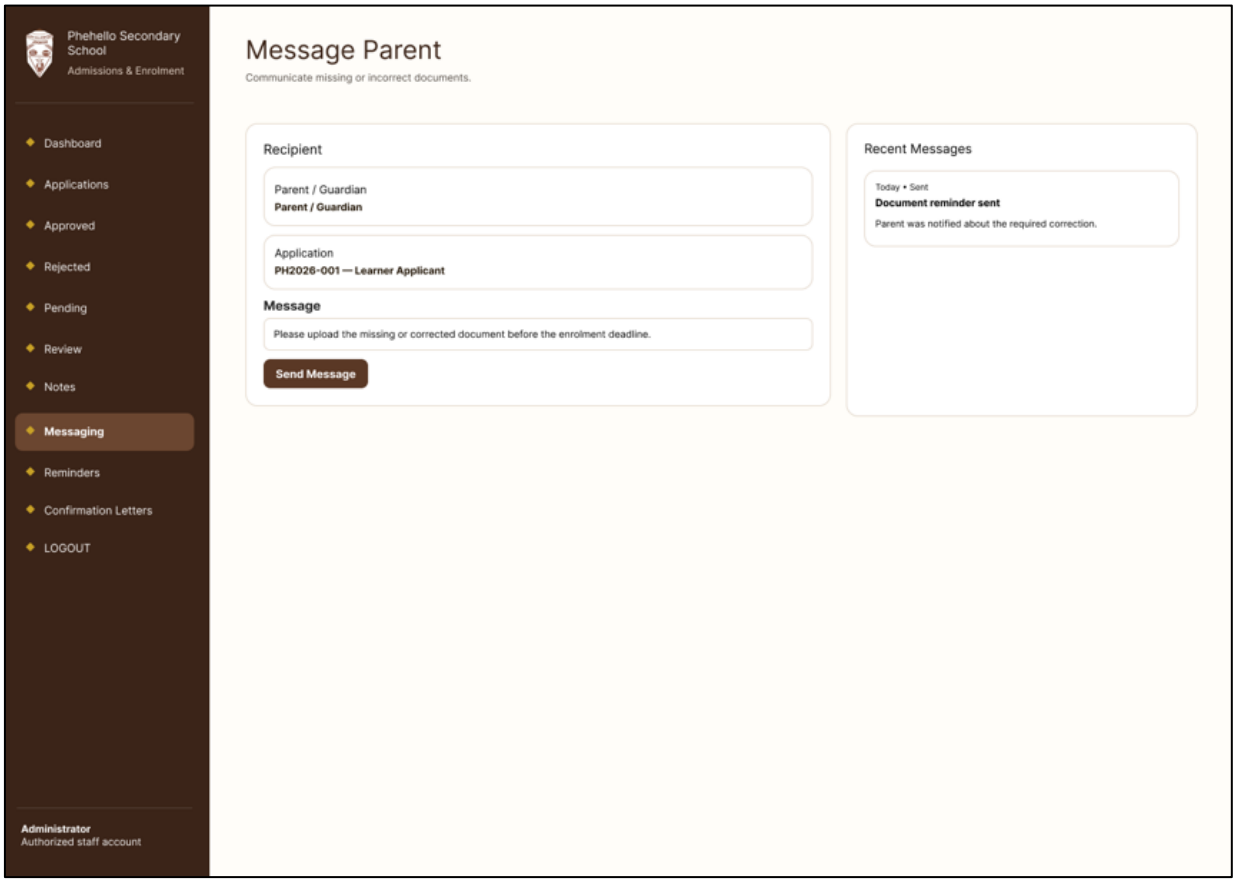
Pending Applications screen



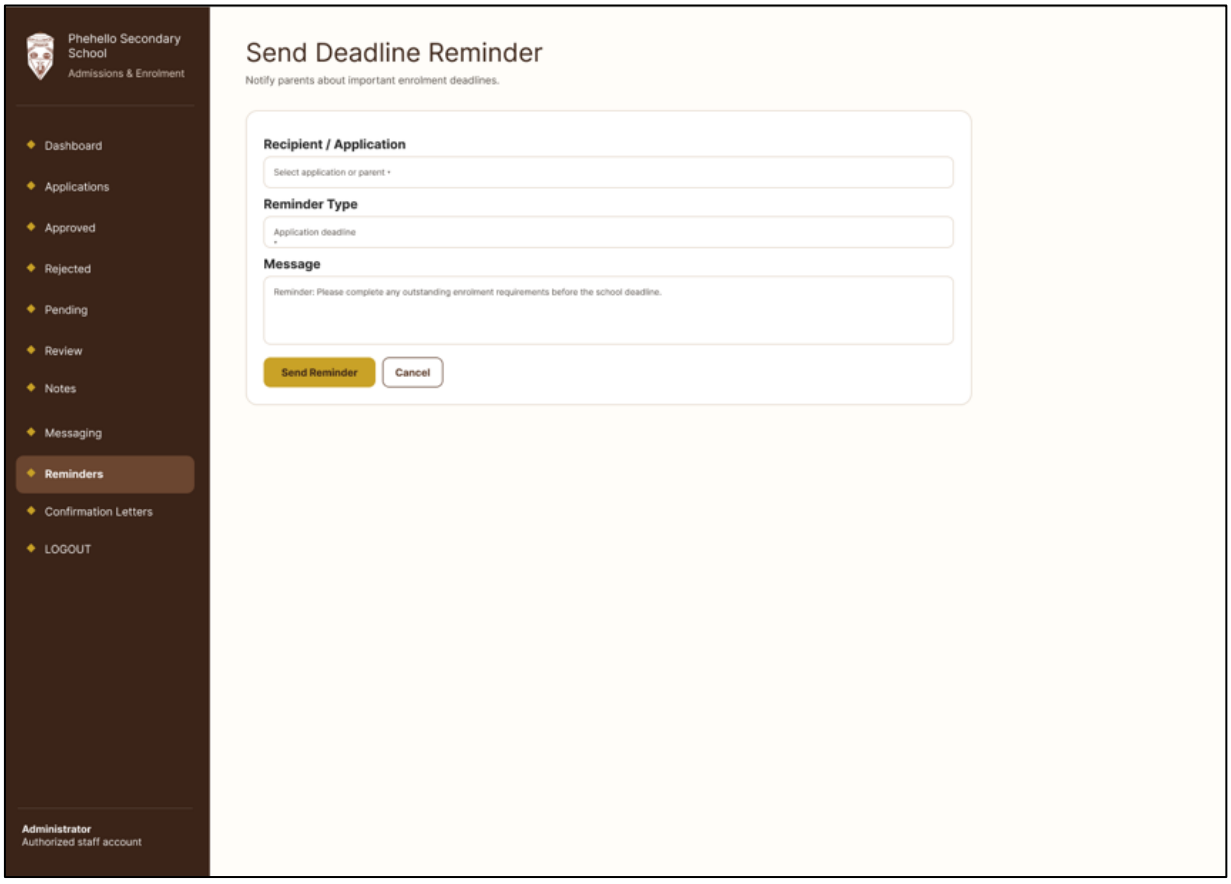
Application Review screen



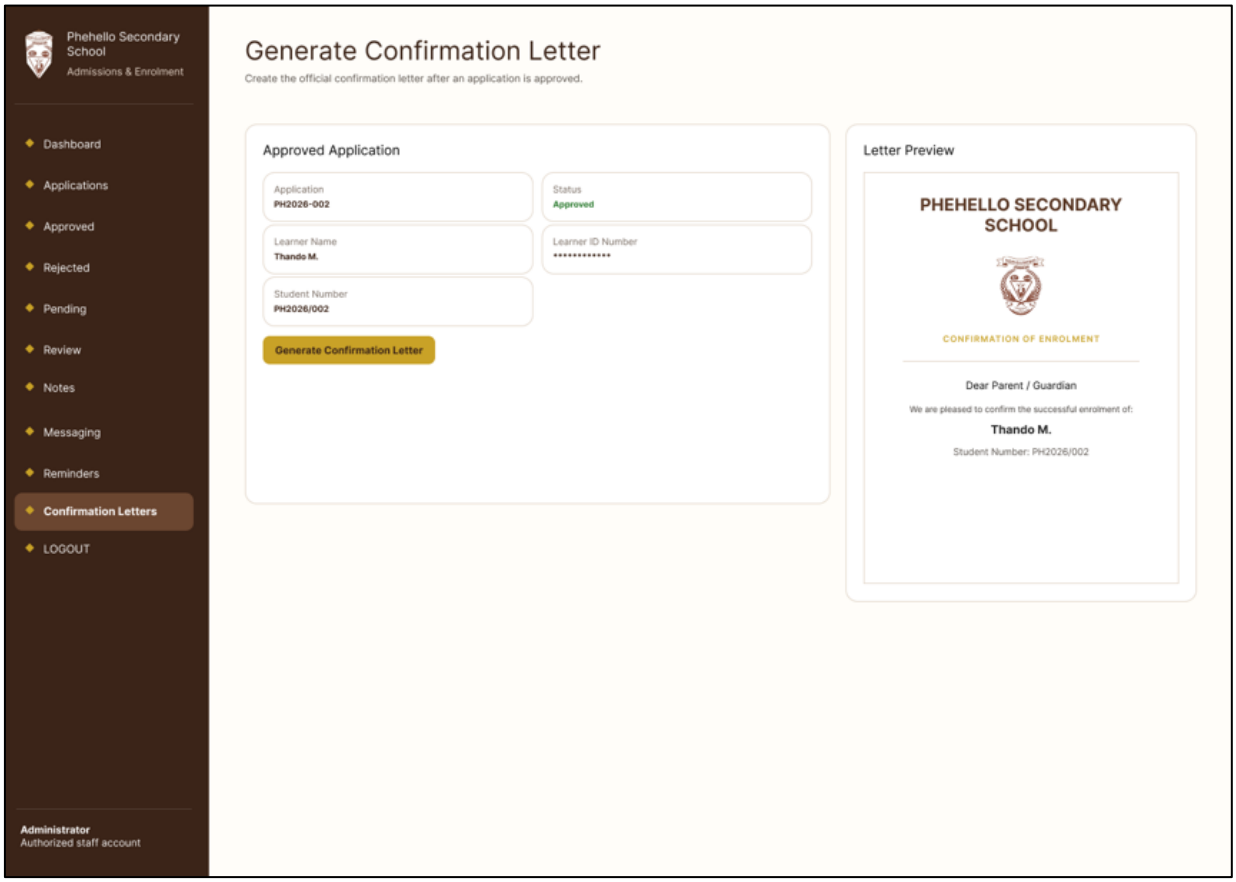
Application Notes screen



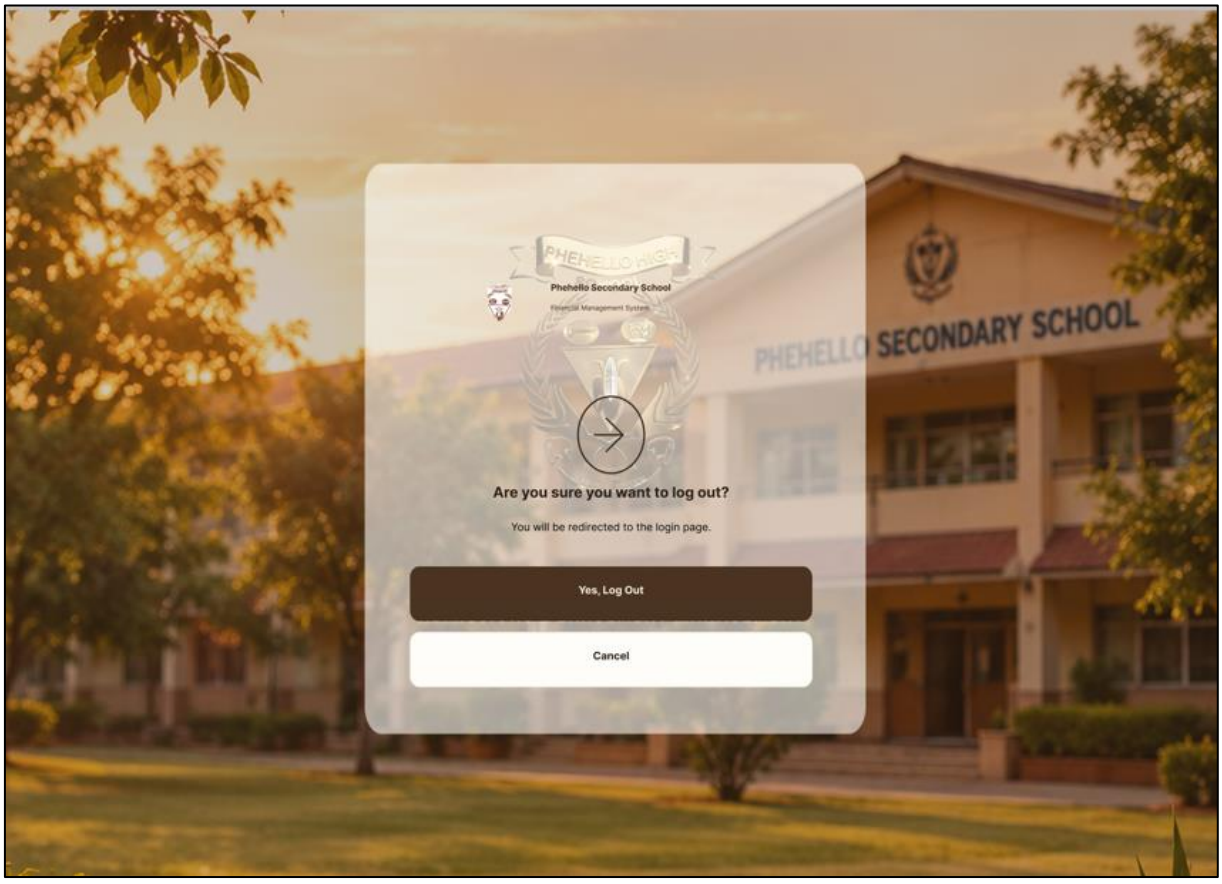
Message Parent screen



Send Reminder screen

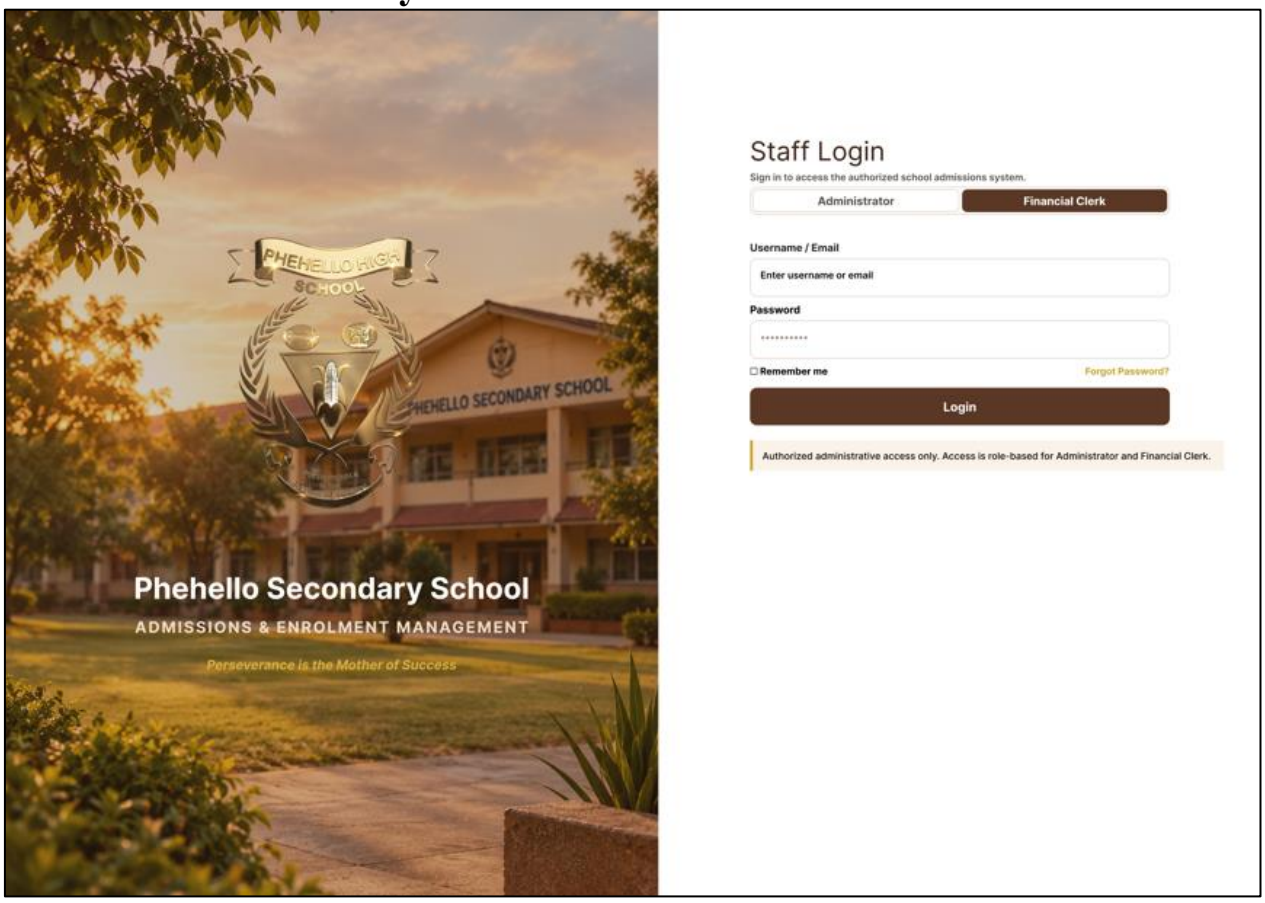


Generate Confirmation Letter screen

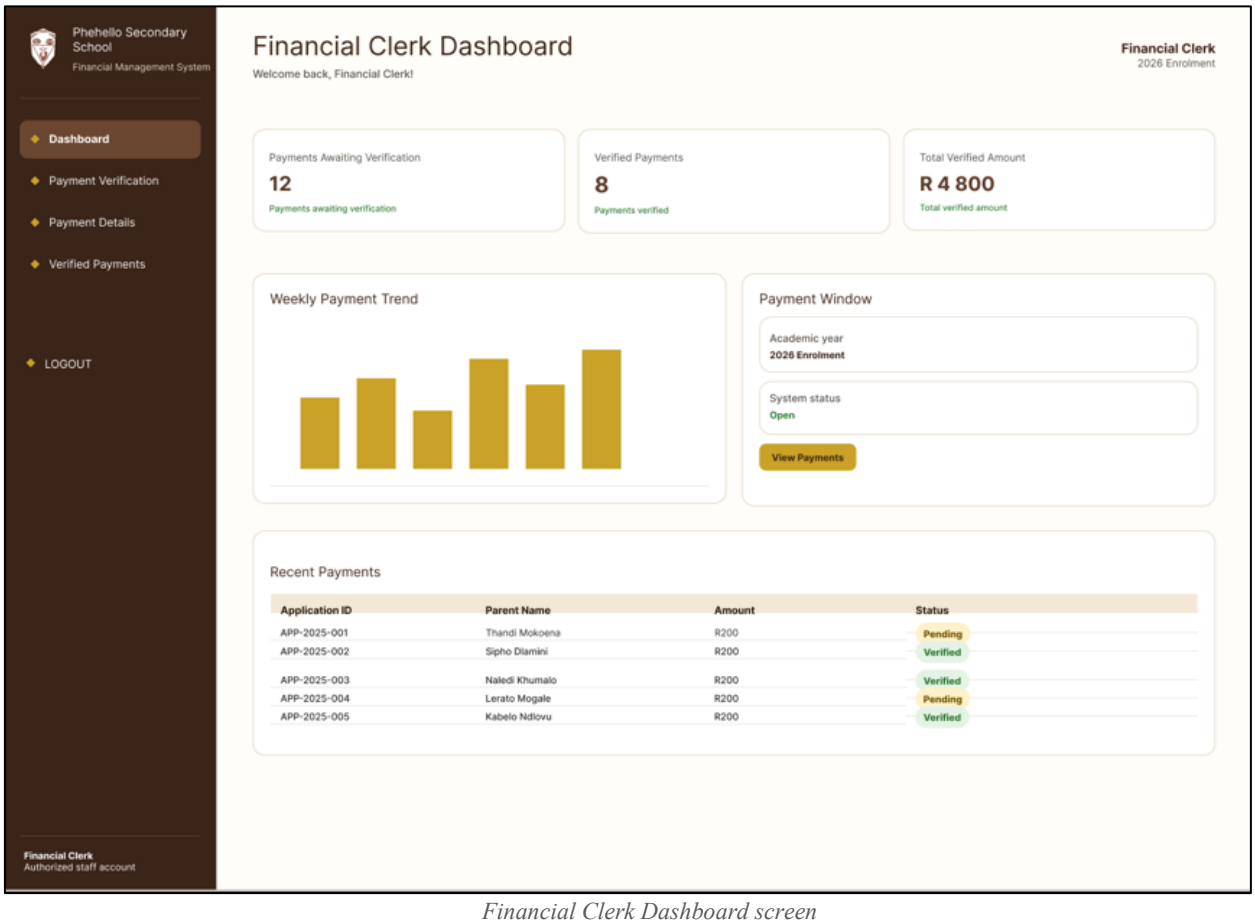


Logout screen

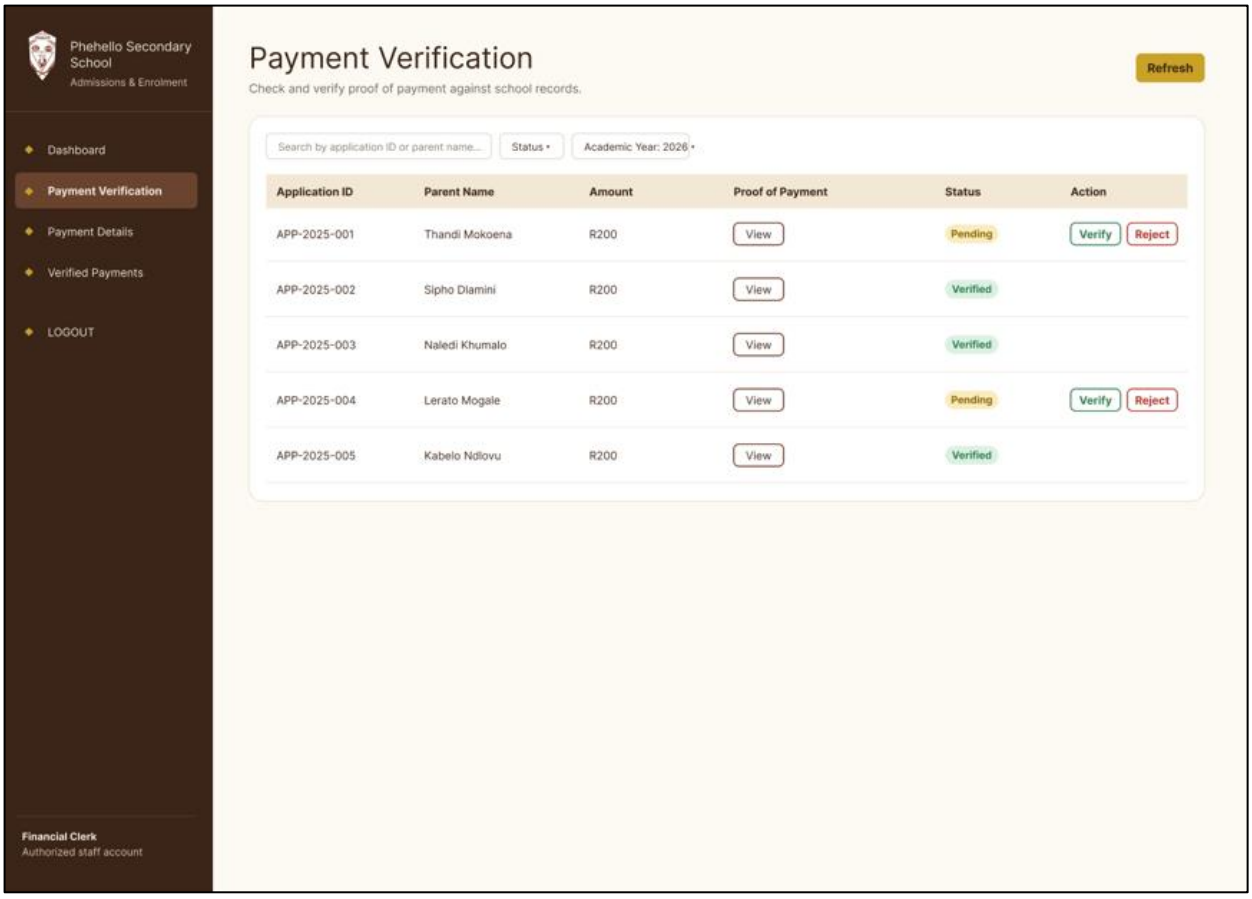
Illustrative low-fidelity ASP.NET wireframes for Financial Clerk



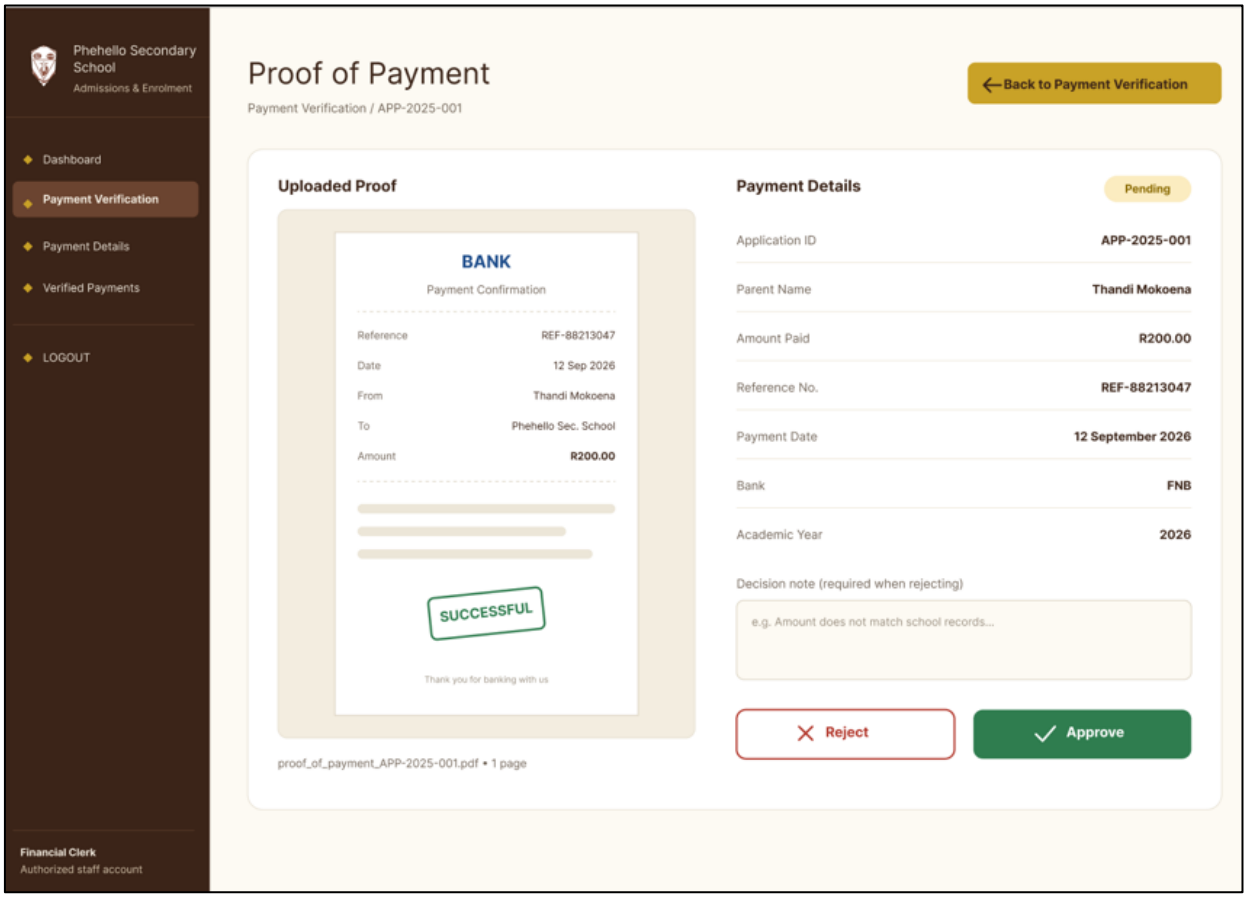
Login/Sign in sreen



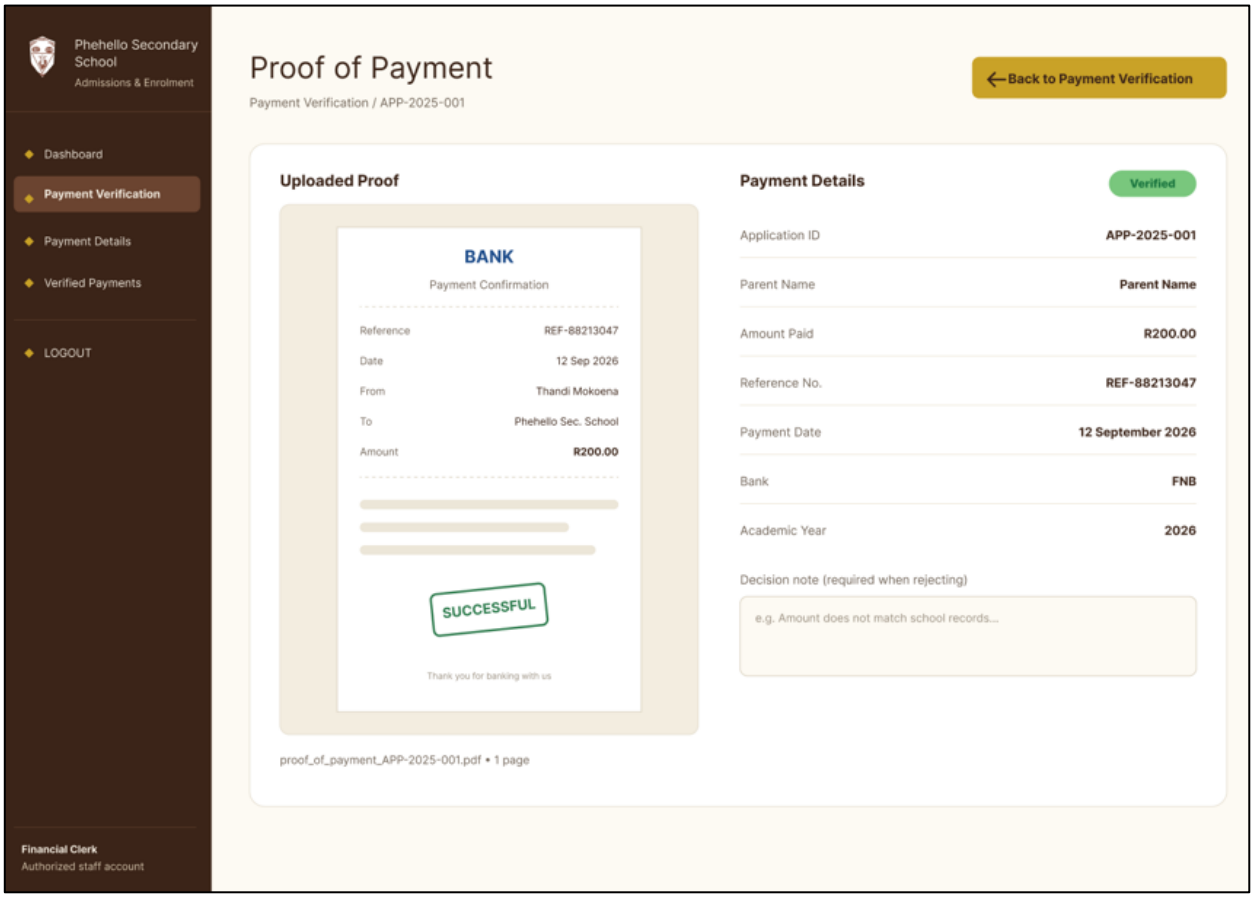
Financial Clerk Dashboard screen



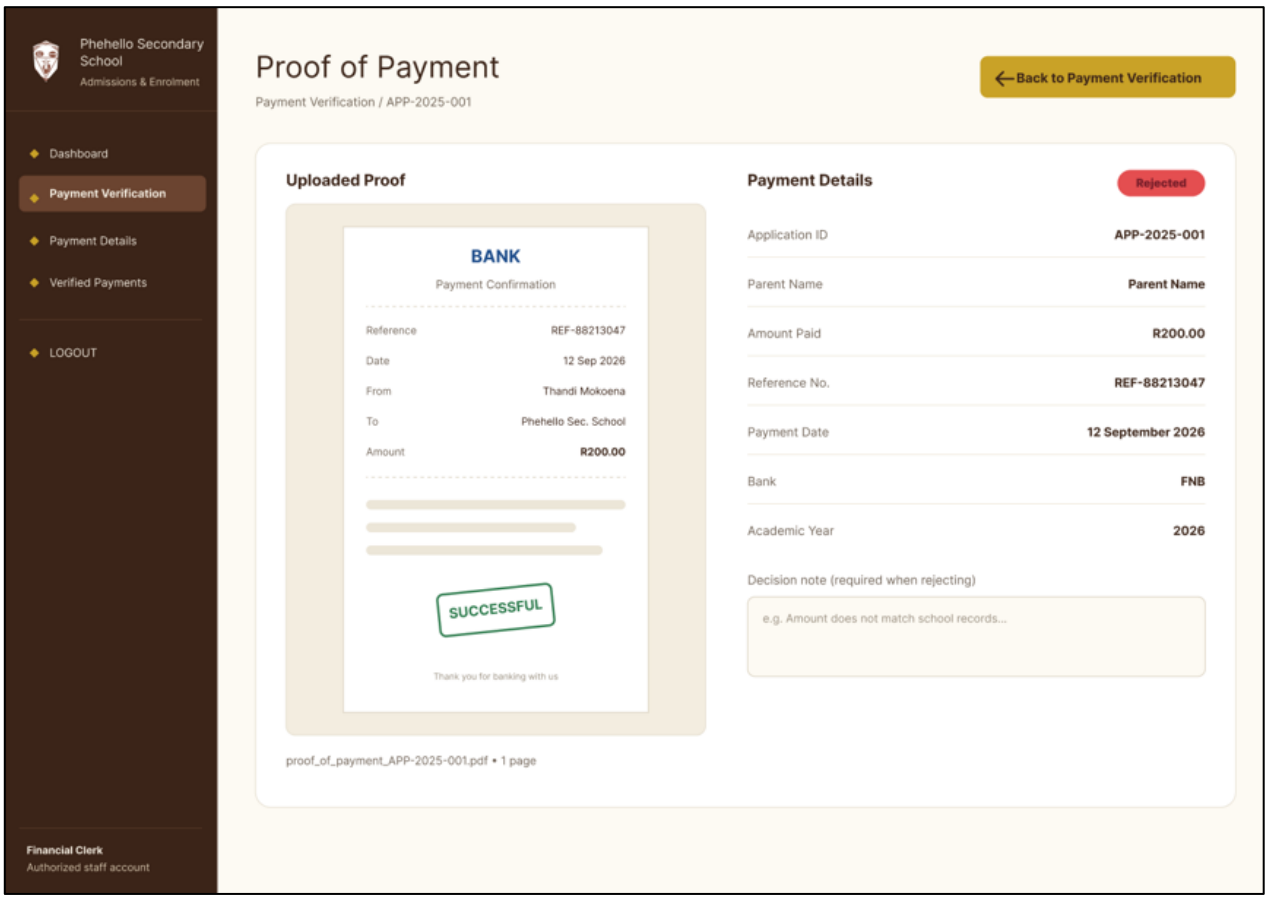
Payment Verification screen



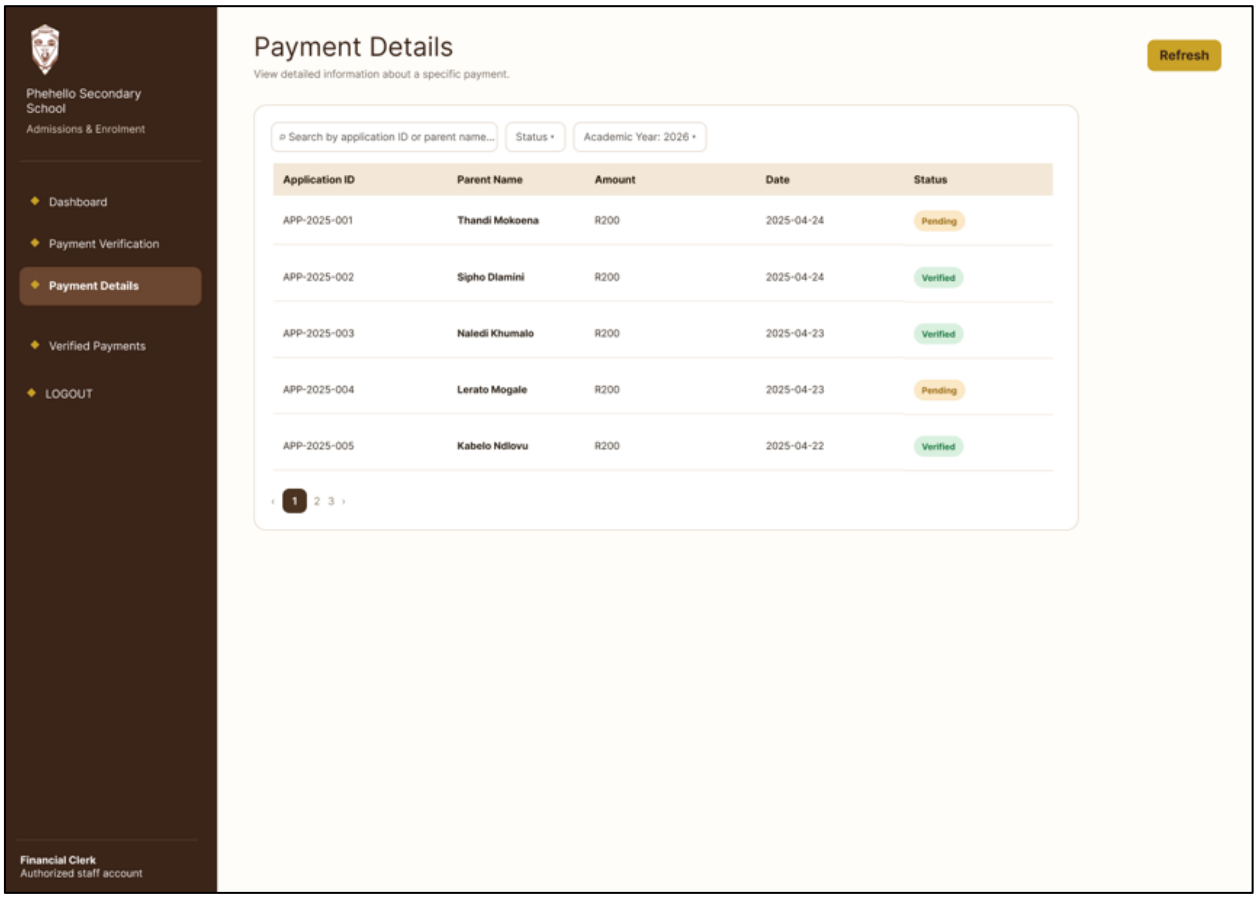
Proof of payment screen when the status is pending



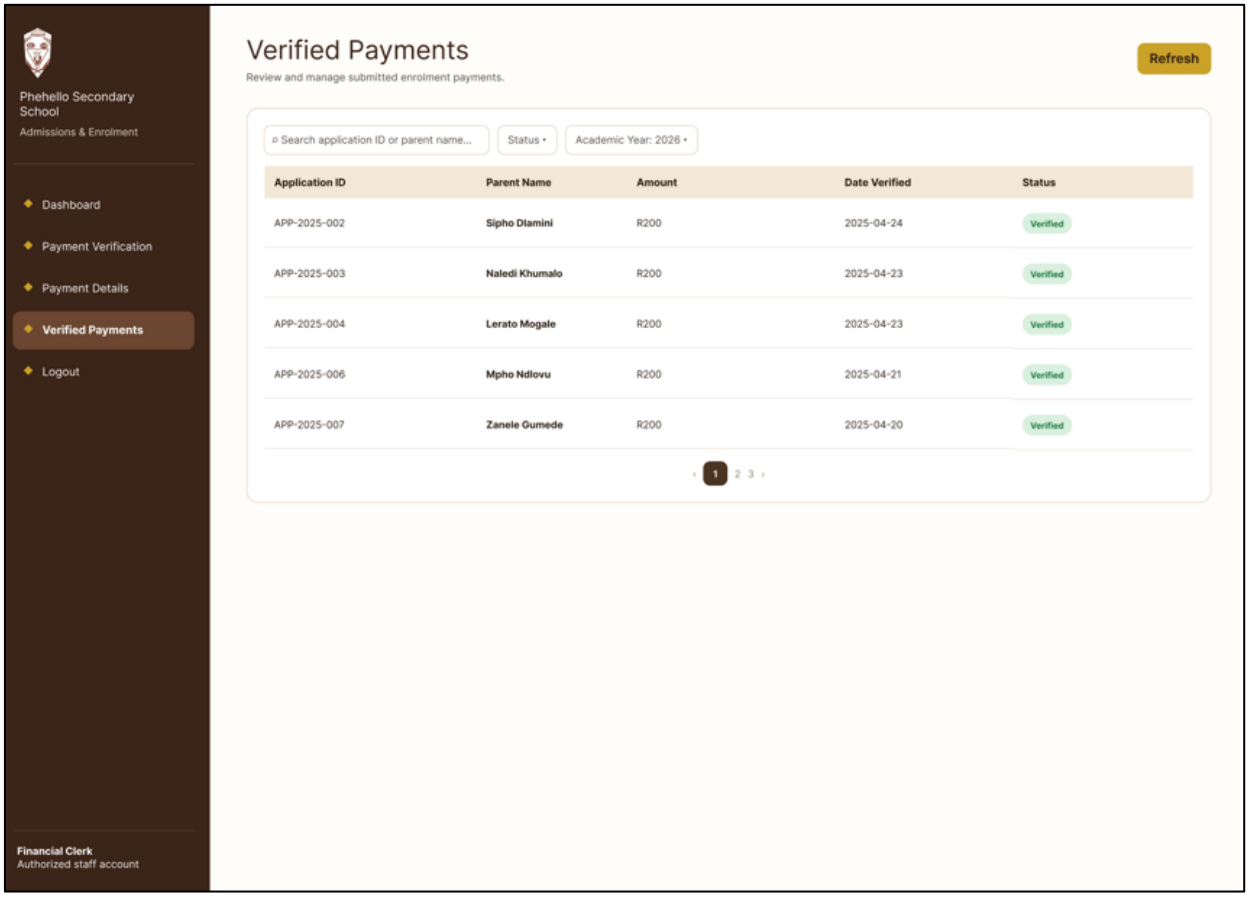
Proof of payment screen when the status is verified



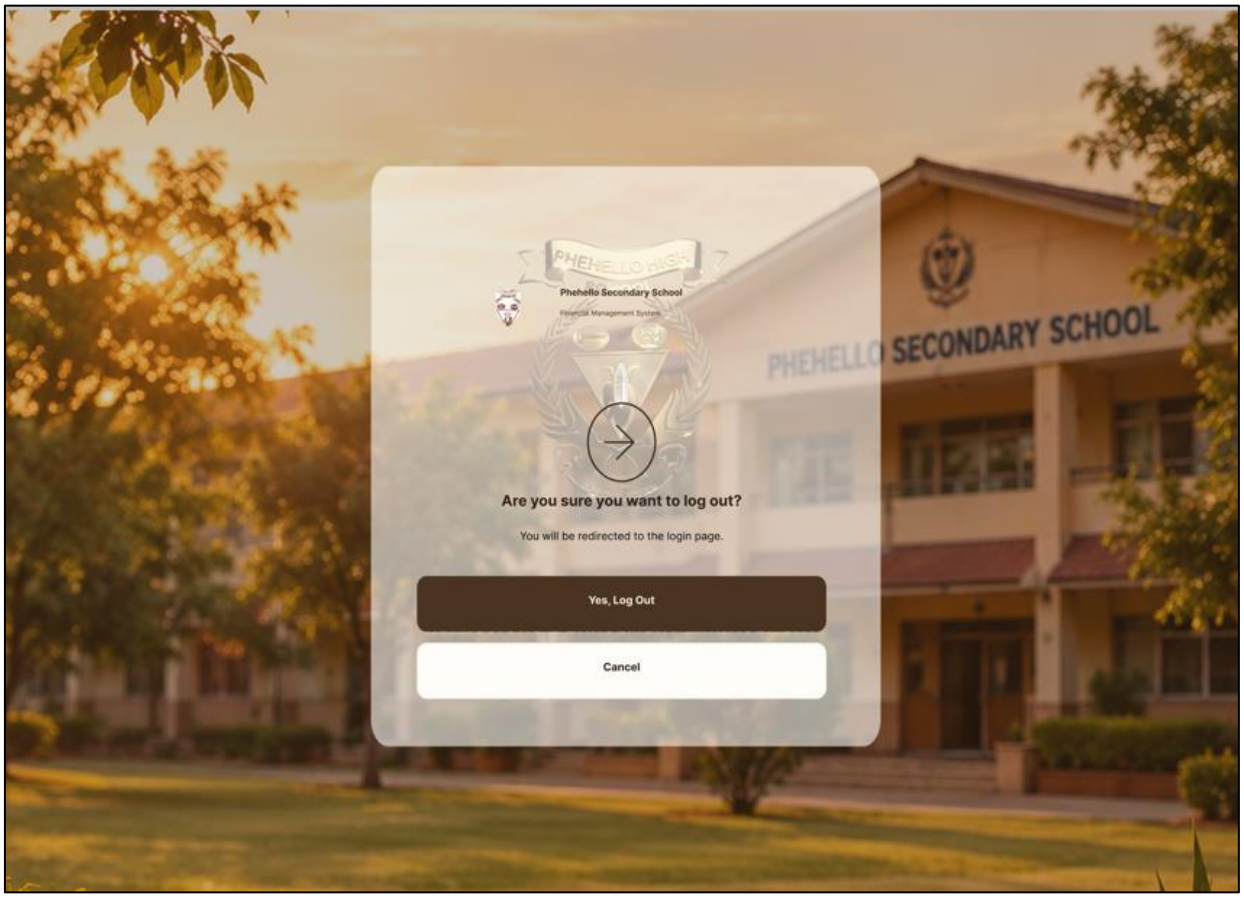
Proof of payment screen when the status is rejected



Payment Details screen



Verified Payments screen



Logout screen

Mobile and Web Integration Check

Flutter Action	Supabase Data	ASP.NET Action
Parent submits application	APPLICATIONS / LEARNERS created or updated	Administrator views submitted application
Parent upload documents	APPLICATION_DOCUMENTS / Supabase Storage	Administrator can review relevant application information/documents
Parent views payment information, enters payment reference and uploads payment proof	PAYMENTS / Supabase Storage	Financial Clerk verifies payment against school payment records
Parent view status	APPLICATIONS.status + payment_verification_status	Staff actions update shared data

Interface-to-ERD Alignment

Interface Information	Entity/Table	Attribute/Field
Parent name	PROFILES	full_name
Learner name	LEARNERS	first_name, middle_name, surname
Learner ID number	LEARNERS	id_number
Date of birth	LEARNERS	date_of_birth
Student number	LEARNERS	student_number
Academic/school history	LEARNERS	previous_school_name, highest_grade_passed, year_grade_passed, previous_grade_passed
Medical information	LEARNER_MEDICAL_INFO	medical_aid_name, medical_aid_number,

		medical_aid_main_member, doctor_name, doctor_address, medical_condition
Parent/guardian contact details	PROFILES	phone_number, relationship_to_learner, occupation
Application status	APPLICATIONS	status
Payment verification status	APPLICATIONS	payment_verification_status
Financial Clerk Dashboard	PAYMENTS, APPLICATIONS, LEARNERS	verification_status, payment_verification_status, status, student_number
Payment Details	PAYMENTS, APPLICATIONS, LEARNERS	payment_reference, proof_file_name, proof_storage_path, verification_status, status, student_number
Administrator Dashboard	APPLICATIONS	status, application_date
Applications	APPLICATIONS, LEARNERS, APPLICATION_DOCUMENTS	status, application_date, student_number, file_name, document_type
Payment verification result	PAYMENTS	verification_status
Payment reference	PAYMENTS / SCHOOL_PAYMENT_RECORDS	payment_reference
Proof of payment file	PAYMENTS	proof_file_name, proof_storage_path
Supporting document	APPLICATION_DOCUMENTS	document_type, file_name, storage_path
Administrator notes	APPLICATION_NOTES	note_text, created_at
Messages and reminders	NOTIFICATIONS	notification_type, message, is_read
Confirmation letter	CONFIRMATION_LETTERS	file_name, storage_path, learner_name, learner_id_number, student_number

Complete System Design Alignment

Requirement	Use Case	Sequence Diagram / Architecture	ERD	Interface
FR1 Register / sign in	Register; Sign In	SD1 (sign-in steps)	PROFILES	Flutter: Register, Login
FR2 Submit enrolment application	Submit Enrolment Application	SD1	LEARNERS, APPLICATIONS	Flutter: Learner Details/Enrolment Form
FR3 Upload supporting documents	Upload Supporting Documents	SD1	APPLICATION_DOCUMENTS (+ Storage)	Flutter: Supporting Documents
FR15 View fee/banking details, enter payment reference	View Payment Information	SD1	PAYMENTS.payment_reference	Flutter: Payment Information
FR4 Upload proof of payment	Upload Proof of Payment	SD1	PAYMENTS (+ Storage)	Flutter: Proof of Payment
FR5 View application/payment status	View Application/Payment Status	Architecture data flow (Supabase	APPLICATIONS.status, payment_verification_status; PAYMENTS.verification_status	Flutter: Home, Application Status,

		→ ASP.NET)		Application Details
FR6 Verify proof of payment	Verify Proof of Payment	SD3	PAYMENTS.verification_status, verified_by; SCHOOL_PAYMENT_RECORDS.payment_reference	ASP.NET: Payment Verification
FR7 View submitted applications	View Submitted Applications	SD2	APPLICATIONS, LEARNERS, APPLICATION_DOCUMENTS	ASP.NET: Applications, Application Review
FR8 Approve / reject / request corrections	Approve/Reject Application; Request Corrections	SD2	APPLICATIONS.status, reviewed_by; APPLICATION_STATUS_HISTORY	ASP.NET: Application Review
FR9 Leave notes	Leave Notes on Application	Architecture data flow (ASP.NET → Supabase)	APPLICATION_NOTES	ASP.NET: Application Notes
FR10 Message parent about documents	Message Parent About Documents	SD2 (corrections branch → notification to Flutter)	NOTIFICATIONS	ASP.NET: Message Parent; Flutter: Notifications/Messages
FR11 Send deadline reminders	Send Deadline Reminders	Architecture data flow (ASP.NET → Supabase → Flutter)	NOTIFICATIONS	ASP.NET: Send Reminder; Flutter: Notifications/Messages
FR12 Generate confirmation letter	Generate Confirmation Letter	Architecture data flow (ASP.NET → Supabase)	CONFIRMATION_LETTERS; LEARNERS.student_number	ASP.NET: Confirmation Letter
FR13 View application count	View Application Count	Architecture data flow (Supabase → ASP.NET)	APPLICATIONS (count by status)	ASP.NET: Administrator Dashboard
FR14 Store applications, documents and proof of payment	Supports all use cases	Architecture (Supabase Database and Storage)	All tables + Storage buckets	Back-end (no screen)

Stakeholder Feedback and Design Changes Record

Item	Details
Date of feedback	14 September 2026
Designs shown	Flutter parent wireframes; ASP.NET Administrator and Financial Clerk wireframes; navigation flow.
Stakeholder comments	The secretary Zukiswa Khondlo selected brown and white as the preferred colours for the system interfaces.
Changes requested	Use **brown** and **white** as the main colours in the interface designs.
Changes made	Updated/refined the Flutter and ASP.NET wireframes using brown and white as the primary colour scheme, with supporting neutral/accent colours where needed for readability and status indicators.
Evidence available	WhatsApp communication between David and the secretary dated 14 September 2026, together with the revised wireframes.

Evidence: Stakeholder Communication

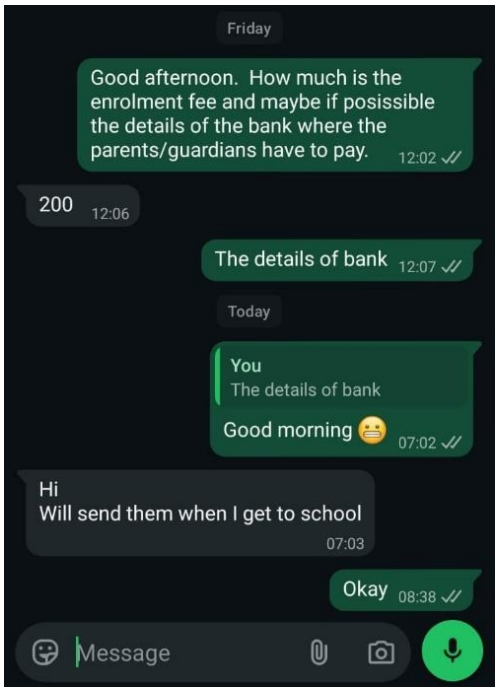


WhatsApp exchange, 21 September 2026 – enrolment fee (R200) confirmed and banking details requested

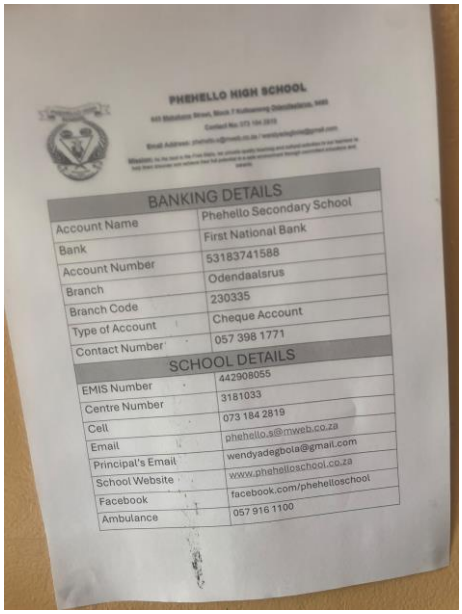
On 14 September 2026, David Adegbola contacted the school secretary via WhatsApp to obtain stakeholder feedback on the interface designs. The secretary selected brown and white as the preferred colours. This feedback was incorporated into the revised Flutter and ASP.NET wireframes.

Item	Details
Date of feedback	21 September 2026
Information requested	WhatsApp exchange between David Adegbola and the school contact, asking for the enrolment fee and the school's banking details.
Information confirmed	Enrolment fee: R200. Banking details supplied as a photo of the school's printed banking-details sheet (First National Bank, Phehello Secondary School account).
Changes made	Added FR15; the Flutter Payment Information screen now displays the fee and banking details and captures a payment reference (Figures shown below).
Evidence available	WhatsApp chat and banking-details photo, dated 21 September 2026 (below).

Evidence: Enrolment Fee and Banking Details



WhatsApp exchange, 21 September 2026 – enrolment fee (R200) confirmed and banking details requested



PHEHELLO HIGH SCHOOL

BANKING DETAILS	
Account Name	Phehello Secondary School
Bank	First National Bank
Account Number	53183741588
Branch	Odendaalsrus
Branch Code	230335
Type of Account	Cheque Account
Contact Number	057 398 1771
SCHOOL DETAILS	
EMIS Number	442908055
Centre Number	3181033
Cell	073 184 2819
Email	phehello.s@mweb.co.za
Principal's Email	wendyadegbola@gmail.com
School Website	www.phehelloschool.co.za
Facebook	facebook.com/phehelloschool
Ambulance	057 916 1100

Phehello Secondary School banking-details sheet, received 21 September 2026

On 21 September 2026, David Adegbola asked the school for the enrolment fee and its banking details via WhatsApp. The school confirmed a fee of R200 and supplied a photo of its banking-details sheet (First National Bank). These details are now shown to the parent on the Payment Information screen before proof of payment is uploaded (see FR15 and the updated interface designs).

Original Design Source Links

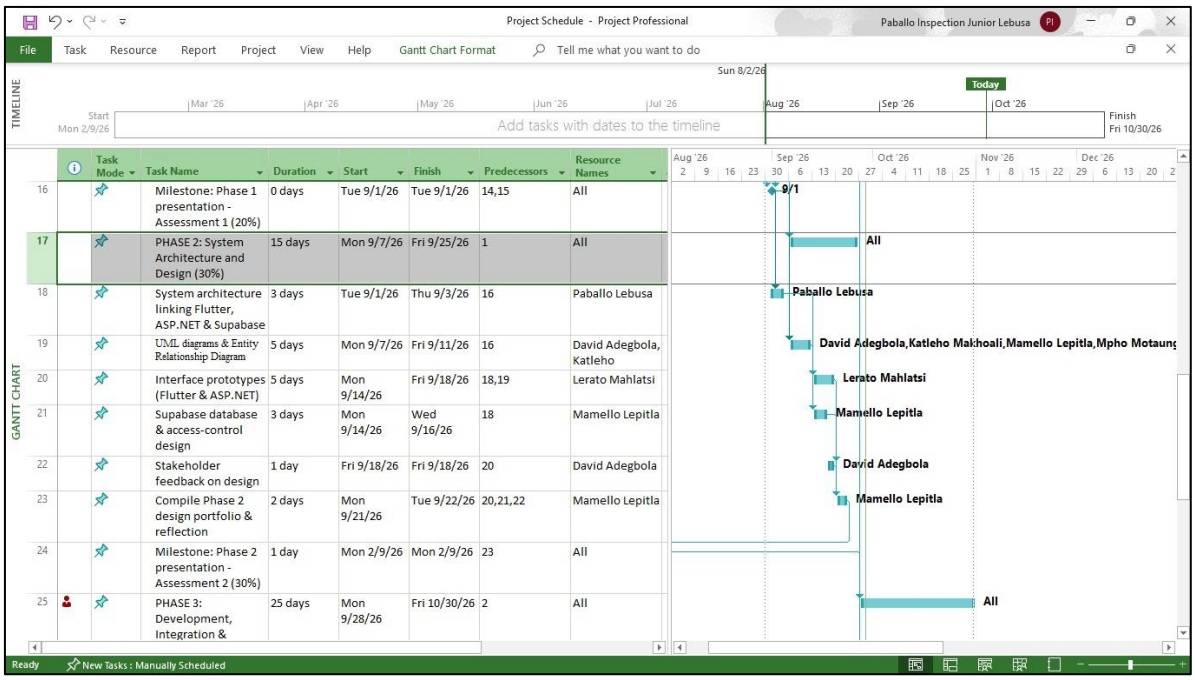
Each design in this section was created in the tool listed below. Lecturers can open the original editable version using the link, or the editable file named in the submission ZIP.

Design	Tool used	Source link / editable file name
System architecture diagram	Software Ideas Modeler	System Architecture.simp
Use Case Diagrams (Parent/Guardian; Staff)	Software Ideas Modeler	Ctrl-Alt-Defeat.simp
Sequence Diagrams 1–3	Software Ideas Modeler	Ctrl-Alt-Defeat.simp
ERD	dbdiagram.io	https://dbdiagram.io/d/ERD-6ab5c1300f25a52d0100e2b7
Flutter wireframes (design)	Figma (page 1)	https://www.figma.com/design/afkx2OMta474iXNnlscqO5/Flutter-and-ASP.NET-interfaces?t=oVbnTCBoJvalux20-1
Flutter wireframes (interactive)	Figma	https://www.figma.com/proto/afkx2OMta474iXNnlscqO5/Flutter-and-ASP.NET-interfaces?node-id=0-1&t=vQCReBfJkRq6vP8o-1
ASP.NET wireframes (design)	Figma (page 2)	https://www.figma.com/design/afkx2OMta474iXNnlscqO5/Flutter-and-ASP.NET-interfaces?t=oVbnTCBoJvalux20-1
ASP.NET wireframes (interactive)	Figma	https://www.figma.com/proto/afkx2OMta474iXNnlscqO5/Flutter-and-ASP.NET-interfaces?node-id=2002-731&t=90hyZ2mnTPvLvaZk-1
GitHub repository	GitHub	https://github.com/Katleho-19/WIL-Project-2026

Note on architecture diagram colour scheme: The architecture diagram was created in Software Ideas Modeler (source file: System Architecture.simp, included in zip). The colour scheme shown in the SRS was subsequently adjusted using Claude AI for visual clarity; the editable file in the ZIP reflects the tool's original colour scheme rather than adjusted version shown here.

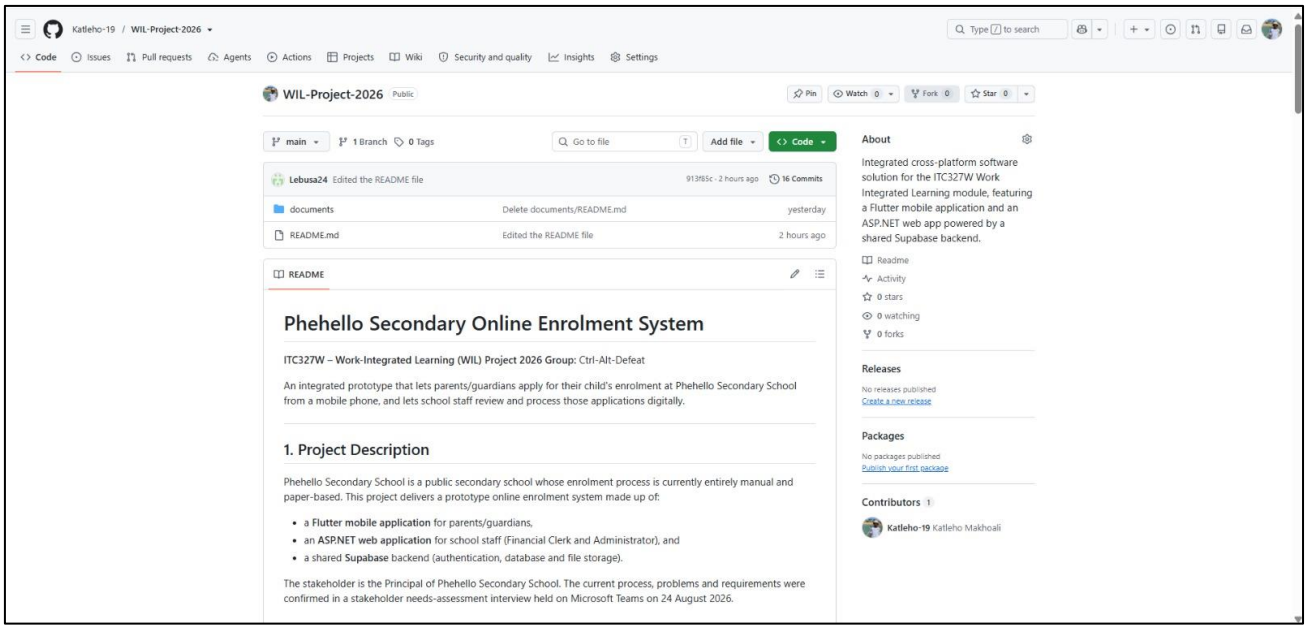
Microsoft Project Plan

The updated original editable Microsoft Project file is submitted with this SRS (file name: Project Schedule.mpp). The screenshot below shows the schedule at the end of Phase 2, including completed Unit 3 design activities, dates, dependencies, responsibilities, milestones and percentage complete.



GitHub Repository and Project Progress Tracker

GitHub repository: https://github.com/Katleho-19/WIL-Project-2026. The repository contains the current architecture, UML, ERD and interface documentation and the design source links. No passwords, API keys or confidential stakeholder information are stored in it.



The updated ITC327W Project Progress Tracker is submitted as a separate Excel workbook.

PART E: SYSTEM DEVELOPMENT AND IMPLEMENTATION

Set Up and Implement the Shared Supabase Backend

The updated ERD below is the design implemented in Supabase. It contains eleven application tables in the public schema, plus the Supabase-managed auth.users table.

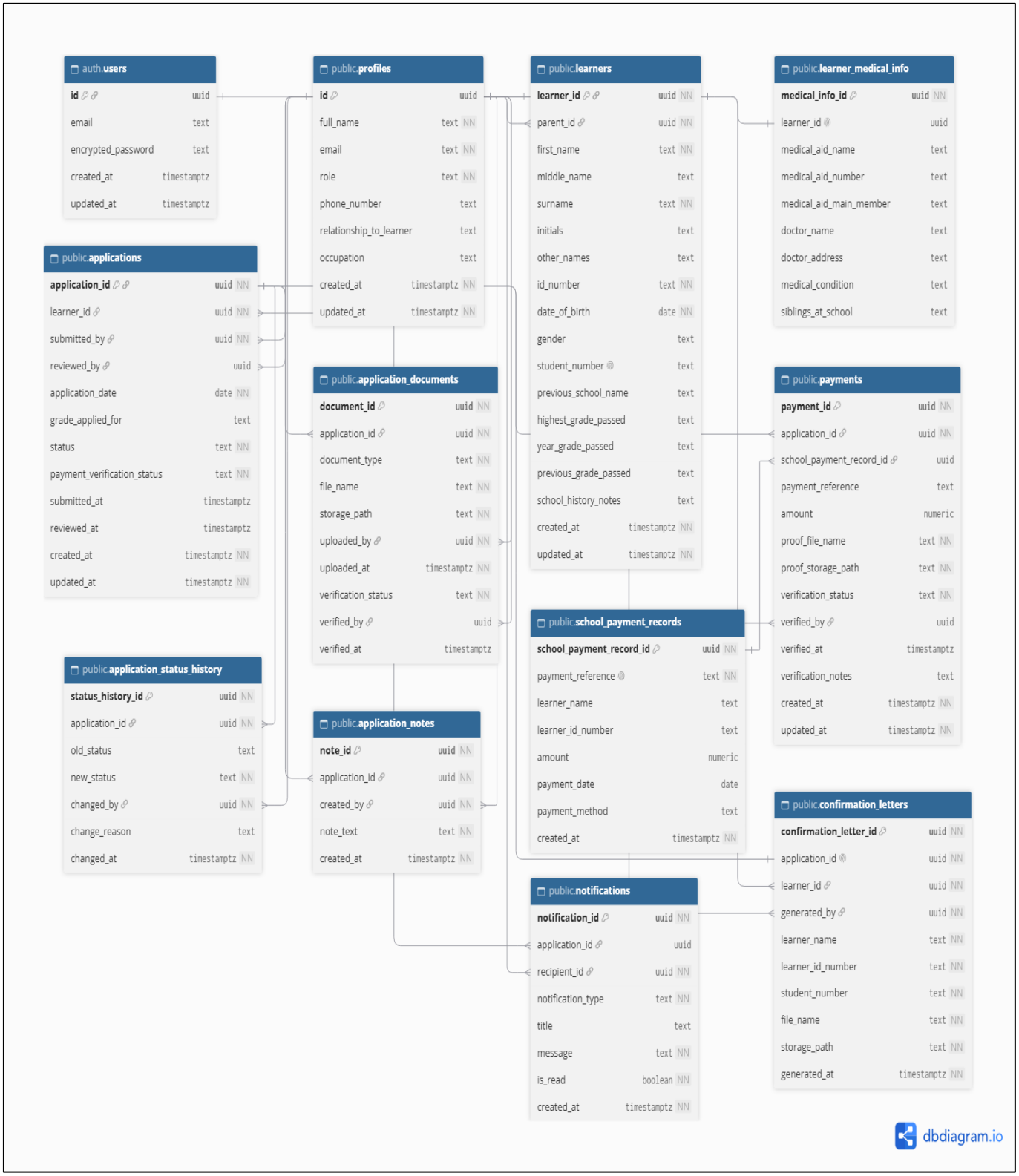


Figure 4.1 – Updated ERD implemented in Supabase (dbdiagram.io export)

Entity / Table	Purpose	Main fields
auth.users	Supabase-managed sign-in accounts for all users	id, email, encrypted_password, created_at, updated_at
profiles	Stores each user's name, contact details and role (Parent, Financial Clerk or Administrator)	id, full_name, email, phone_number, relationship_to_learner, occupation, role
learners	Stores learner personal and previous-school information	learner_id, parent_id, first_name, middle_name, surname, id_number, date_of_birth, gender, student_number, previous_school_name, highest_grade_passed, year_grade_passed, previous_grade_passed, initials, other_names, school_history_notes
learner_medical_info	Stores the medical aid and doctor details required by the school's enrolment form	medical_info_id, learner_id, medical_aid_name, medical_aid_number, medical_aid_main_member, doctor_name, doctor_address, medical_condition
applications	Stores each enrolment application and its application and payment status	application_id, learner_id, submitted_by, reviewed_by, application_date, status, payment_verification_status, submitted_at, reviewed_at, grade_applied_for
application_documents	Stores records of uploaded supporting documents (files are held in Supabase Storage)	document_id, application_id, uploaded_by, verified_by, document_type, file_name, storage_path, uploaded_at, verification_status, verified_at
payments	Stores each proof-of-payment attempt and its verification result	payment_id, application_id, school_payment_record_id, verified_by, payment_reference, amount, proof_file_name, proof_storage_path, verification_status, verified_at, verification_notes
school_payment_records	Holds the school's own payment records that the Financial Clerk matches proof of payment against	school_payment_record_id, payment_reference (unique), learner_name, learner_id_number, amount, payment_date, payment_method
application_notes	Internal Administrator notes on an application (FR9)	note_id, application_id, created_by, note_text, created_at
application_status_history	Audit trail of every application status change	status_history_id, application_id, changed_by, old_status, new_status, change_reason, changed_at
confirmation_letters	Records generated confirmation letters (FR12)	confirmation_letter_id, application_id, learner_id, generated_by, learner_name, learner_id_number, student_number, file_name, storage_path, generated_at
notifications	Administrator messages and deadline reminders shown to parents (FR10, FR11)	notification_id, application_id, recipient_id, notification_type, title, message, is_read

Shared Supabase project

Item	Detail
Project name	Phehello Enrolment System
Organisation / plan	Ctrl+Alt+Defeat, Free plan (monitored per the economic feasibility decision)
Database region	West EU (Ireland)
Number of Supabase projects	One. The same project is used by both the Flutter and the ASP.NET applications
Services used	Authentication, PostgreSQL database, Storage (bucket: enrolment-documents)
Authorised group members with access	KI. Makhoali, PIJ. Lebusa, LR. Mahlatsi, M. Lepitla, MM. Motaung, DT. Adegbola
Where project details are recorded	This document and the GitHub README. The project URL and keys are held in secure local configuration and are not stored in GitHub

Storage bucket configuration:

Bucket	Contents	Access
enrolment-documents	Supporting documents (birth certificate, immunisation record, progress report, transfer letter), proof-of-payment files and generated confirmation letters	Private bucket (not public). Parents: upload and read their own files only. Financial Clerk: read proof-of-payment files. Administrator: read supporting documents and write confirmation letters.

Database tables, keys and relationships

The tables were implemented from the approved ERD. The full list of fields and data types is in Appendix F. Keys and design rules are summarised below.

Table	Primary key	Foreign keys	Configuration notes
profiles	id (uuid)	id → auth.users.id	One profile per sign-in account; role stores Parent, Financial Clerk or Administrator
learners	learner_id	parent_id → profiles.id	student_number is the learner's unique school student number
learner_medical_info	medical_info_id	learner_id → learners.learner_id	One record per learner (1 to 1)
applications	application_id	learner_id → learners; submitted_by → profiles; reviewed_by → profiles	status and payment_verification_status are text fields that use the agreed status values (Design Decision 2)
application_documents	document_id	application_id → applications; uploaded_by → profiles; verified_by → profiles	verification fields kept as forward-looking columns (Design Decision 5)
payments	payment_id	application_id → applications; school_payment_record_id → school_payment_records; verified_by → profiles	A new row is inserted for each re-upload (Decision 1); a trigger keeps applications.payment_verification_status in sync (Decision 3)
school_payment_records	school_payment_record_id	None	payment_reference has a uniqueness constraint (Decision 4)
application_notes	note_id	application_id → applications; created_by → profiles	Several notes per application
application_status_history	status_history_id	application_id → applications; changed_by → profiles	No cascading deletes (Decision 7)
confirmation_letters	confirmation_letter_id	application_id → applications; learner_id → learners; generated_by → profiles	Learner details deliberately stored as text at the time of generation (Decision 6); no cascading deletes
notifications	notification_id	application_id → applications; recipient_id → profiles	One parent may receive many notifications

Relationships implemented

Relationship	Cardinality	Implemented through
auth.users – profiles	1 to 1	profiles.id
profiles (parent) – learners	1 to many	learners.parent_id
learners – applications	1 to many	applications.learner_id
learners – learner_medical_info	1 to 1	learner_medical_info.learner_id
profiles – applications (submitted_by, reviewed_by)	1 to many	applications.submitted_by, applications.reviewed_by
applications – application_documents	1 to many	application_documents.application_id
applications – payments	1 to many	payments.application_id
school_payment_records – payments	1 to many	payments.school_payment_record_id
profiles (Financial Clerk) – payments	1 to many	payments.verified_by
applications – application_notes	1 to many	application_notes.application_id
applications – application_status_history	1 to many	application_status_history.application_id
applications – confirmation_letters	1 to 1	confirmation_letters.application_id
applications – notifications	1 to many	notifications.application_id
profiles (parent) – notifications	1 to many	notifications.recipient_id

Authentication and user access

Supabase Authentication provides email-and-password sign-in. Parents register themselves through the Flutter application. Staff accounts for the Financial Clerk and Administrator are created by the group for the prototype, and each account's role is stored in profiles.role. The ASP.NET login only admits staff roles (NFR1).

User role	Required access (from the SRS)
Parent/Guardian	Register and sign in; create and view their own learner details, applications, documents and payments; view their own notifications and confirmation letter
Financial Clerk	View submitted proof of payment and the matching school payment records; update payment verification status. No access to approve, reject or request corrections on applications (NFR6)
Administrator	View all applications, learner details and supporting documents; approve, reject or request corrections; add notes; send messages and reminders; generate confirmation letters. Cannot mark a payment as verified (NFR6)

Access matrix enforced through database permissions and Row Level Security (C = create, R = read, U = update, D = delete; "own" means only records belonging to that user):

Table	Parent/Guardian	Financial Clerk	Administrator
profiles	R (own)	R (own)	R (own)
learners	C, R, U (own children)	No access	R, U (e.g. student number)
learner_medical_info	C, R, U (own children)	No access	R
applications	C, R (own)	No access	R, U (status, reviewed_by, reviewed_at)
applications_documents	C, R (own)	No access	R

payments	C, R (own)	R, U (verification fields only)	R
school_payment_records	No access	R	No access
application_notes	No access	No access	C, R
application_status_history	R (own)	No access	R (rows written automatically when status changes)
confirmation_letters	R (own)	No access	C, R
notifications	R, U (own, is_read only)	No access	C, R

The matrix above shows the policies in force at the end of Activity 4.1. The following are left for Activities 4.2 and 4.3, when the screens that need them are built: creating and editing a profile at registration; Parent updates to applications and documents when corrections are requested; Administrator read access to parents' contact details; and, if the ASP.NET payment screen needs it, a restricted Financial Clerk view of the payment-related fields of learners and applications.

No Delete (D) permission is granted to any application role, because the approved requirements contain no delete function and the SRS requires financial and audit history to be preserved (Design Decision 7). The database is not open to public access: every table is protected by Row Level Security, so a user cannot read or change another user's records simply by connecting to Supabase.

Preparing the backend for both applications

Application	Information it sends	Information it retrieves
Flutter	Registration and sign-in credentials; learner, medical, academic and guardian details; application record; supporting document files; payment reference, amount and proof-of-payment file; notification read status	Own applications and status; payment verification status; payment information shown on the Payment Information screen; messages, reminders and notifications; own profile
ASP.NET	Staff sign-in credentials; payment verification result; application decision (approve, reject, request corrections); notes; messages and reminders; confirmation letter data and file	Submitted applications, learner details and documents; proof of payment and matching school payment records; application counts by status

Handling of keys and credentials:

Key or credential	Where it may be used	Rule
Project URL and publishable key	Flutter configuration and ASP.NET server-side configuration	Used for client access only together with Row Level Security
Secret / service-role key	Nowhere in the client applications	Never placed in the Flutter app, browser code or GitHub. Not used as a substitute for role-based access
Configuration and .env files	Developer machines only	Excluded from GitHub through .gitignore; commit history audited for committed secrets (Risk R02)

Test data

Mock data was used throughout, as required by NFR3. No real learner or parent information was entered.

Test account	Role	Purpose
Test Parent 1	Parent/Guardian	Submits an application and uploads documents and proof of payment
Test Parent 2	Parent/Guardian	Used to check that one parent cannot access another parent's records
Test Financial Clerk	Financial Clerk	Verifies or rejects proof of payment
Test Administrator	Administrator	Reviews applications, adds notes, sends messages, generates letters

Application	Learner (mock)	Application status	Payment status
Application 1	Test LearnerOne	Pending Admin Review	Pending
Application 2	Test LearnerTwo	Pending Admin Review	Verified

One mock supporting document was added to each application (a birth certificate and an immunisation record). A matching row (reference TEST-REF-002, R500.00) was added to school_payment_records for Application 2 so that the Financial Clerk's matching step could be checked. The payment for Application 1 (reference TEST-REF-001, R500.00) has no matching school record, which tests the unmatched case.

Initial backend checks

Check	Result	Action required
Required tables have been created	Pass: 11 of 11 tables confirmed by query and Schema Visualizer (Figures 4.3, 4.4, 4.6)	None
Primary keys are configured	Pass: every table has a primary key (Figure 4.5)	None
Foreign keys and relationships work	Pass: foreign keys match the ERD, and joins return linked data (Figures 4.5, 4.6, 4.13)	None
Test records can be added	Pass: dummy learners, applications, documents, school payment record and payments inserted (Figure 4.13)	None
Test records can be retrieved	Pass: join query returned learner, application and payment rows (Figure 4.13)	None
Records can be updated where required	Pass: payment status updated to Verified and the application payment status updated automatically through the trigger; the application status change was recorded in application_status_history with the Administrator as changed_by (Figures 4.14, 4.15)	None
Authentication works, where required (Parent, Financial Clerk, Administrator accounts)	Pass: Parent, Clerk and Administrator test accounts created in Supabase Auth and linked to roles in profiles (Figures 4.7, 4.8)	Confirm real sign-in from Flutter and ASP.NET in Activities 4.2 and 4.3
Access permissions are appropriate (Row Level Security and Storage policies)	Pass: RLS enabled on all 11 tables. The Financial Clerk could read payments but could not read or change applications, a Parent could read only their own applications, and neither could approve an application. The Storage bucket is private with five role-based Storage policies (Figures 4.9 to 4.12, 4.16, 4.17)	Test real file upload and download from the apps in Activities 4.2 and 4.3

Project documentation and tools updated

Document or tool	What was updated	Status
SRS	Replace the ERD in Part D with the updated ERD (Figure 4.1) and record any further approved database changes in the Requirements Change Log	Done
GitHub	Database documentation and SQL scripts (schema, RLS policies, test data) added to the repository	Done
Microsoft Project	Backend activities and progress recorded in Project Schedule.mpp	Done
Project Progress Tracker	Completed tasks, contributions and challenges recorded	Done

Evidence

Each figure below is a screenshot taken from the shared Supabase project using test data only. No secret keys, passwords or real personal information are shown. Figure 4.1 (the ERD) appears earlier in this section.

Evidence	What it shows	Status
Screenshot of the shared Supabase project	One project used by the group	Figure 4.2
Screenshots of the implemented tables	Table structure and fields	Figure 4.3 and 4.4
Primary and foreign key evidence	Keys and relationships configured	Figure 4.5 and 4.6
Authentication and access configuration	Sign-in settings, RLS policies and Storage policies (no secret keys visible)	Figure 4.7 to 4.12
Initial checks using dummy data	Records added, retrieved and updated	Figure 4.13 to 4.17
GitHub database documentation	SQL scripts and notes in the repository	https://github.com/Katleho-19/WIL-Project-2026.
Updated Microsoft Project and Progress Tracker	Current backend progress	Project Schedule.mpp and Progress Tracker submitted with this document

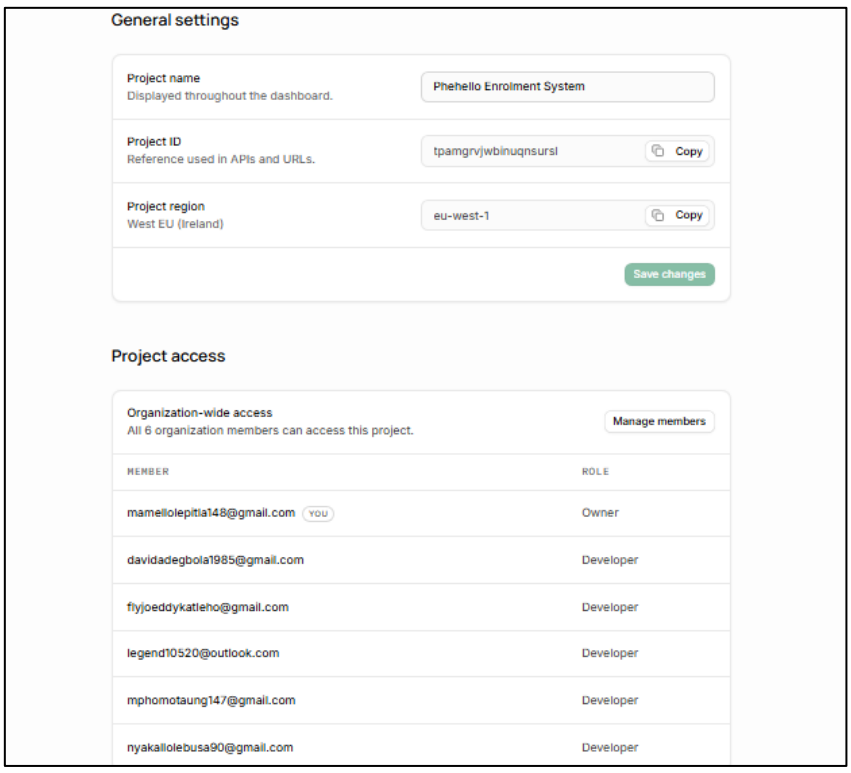
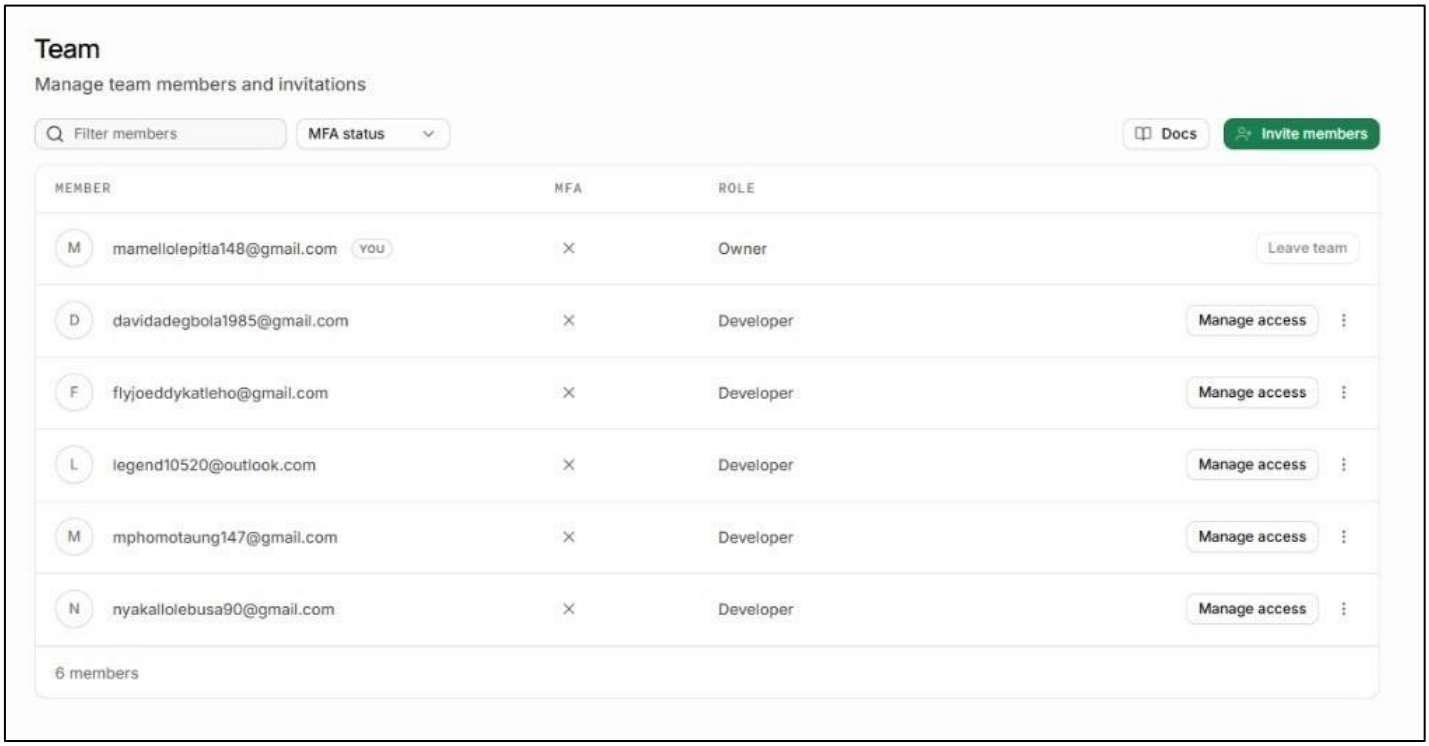


Figure 4.2 – Shared Supabase project: (a) team members (no keys shown). This shows the project settings and access list (project name Phehello Enrolment System, region West EU (Ireland))



4.2 (b) project settings and access list. This shows the team members who have access to it. No keys are displayed.

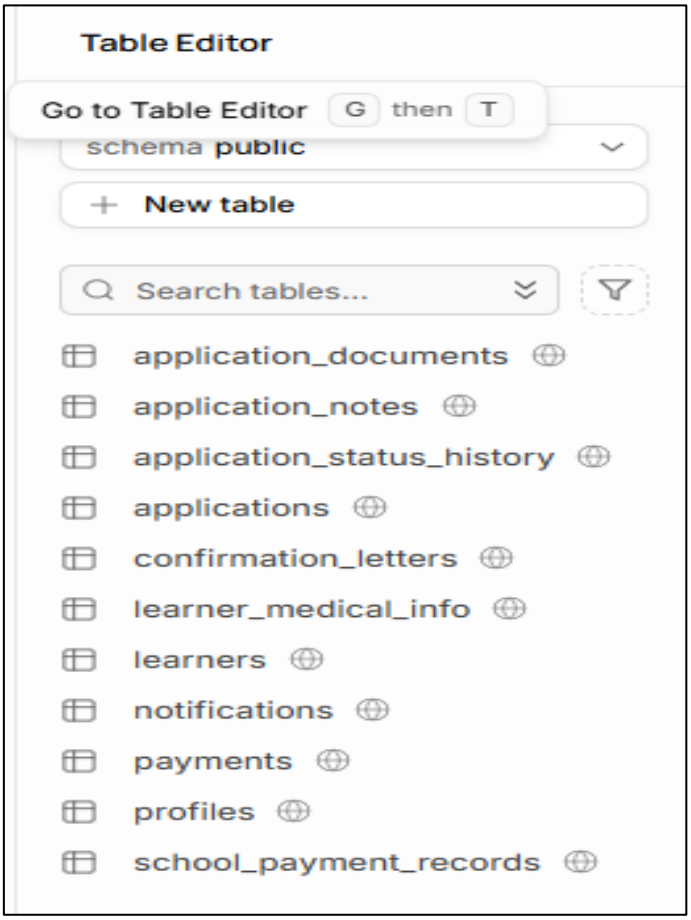


Figure 4.3 – Table Editor showing all 11 tables in the public schema. The Supabase Table Editor listing all 11 tables created in the public schema. This confirms that every table in the ERD exists in the database.

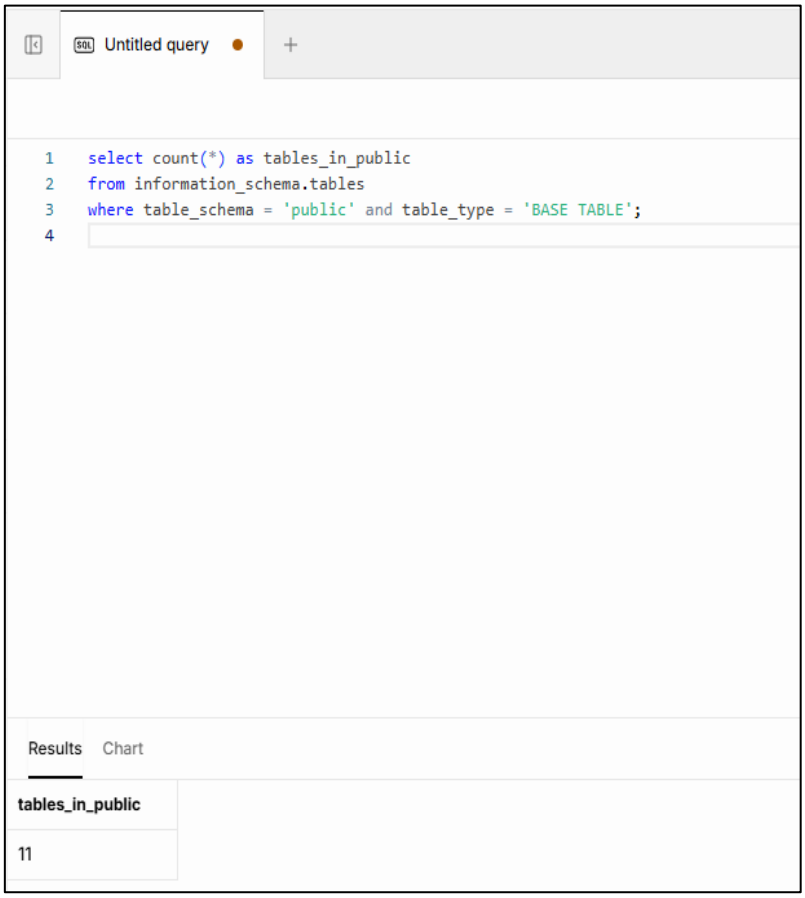


Figure 4.4 – Query confirming 11 tables in the public schema. **A** SQL query that counts the base tables in the public schema. The result is 11, which matches the number of tables in the ERD.

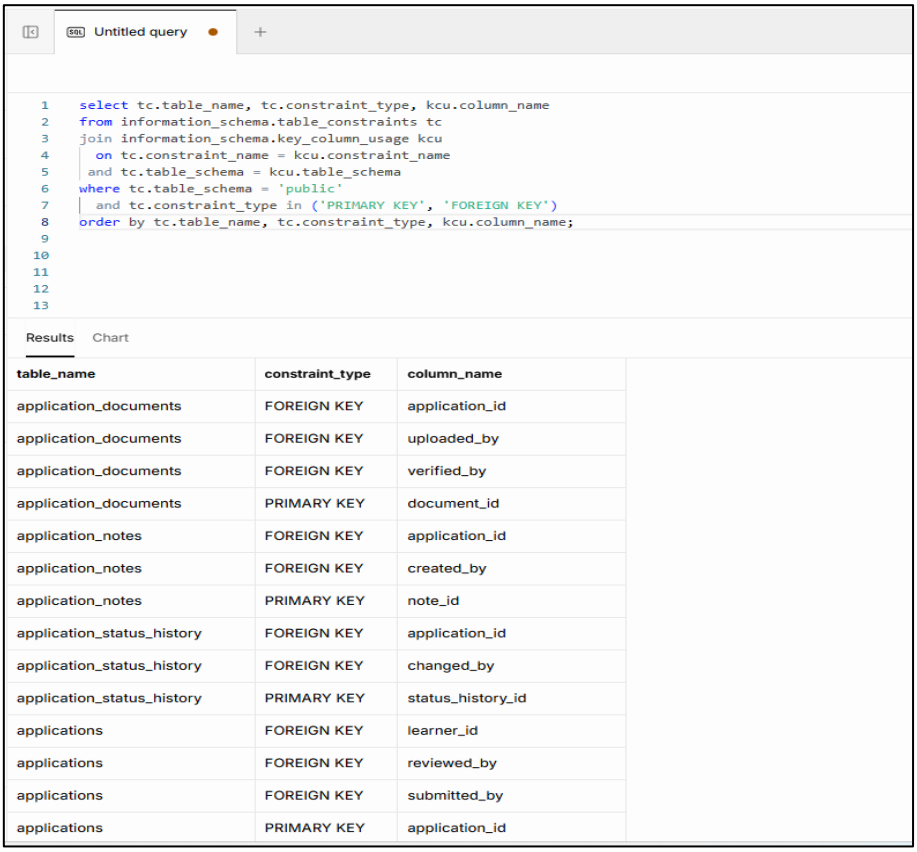
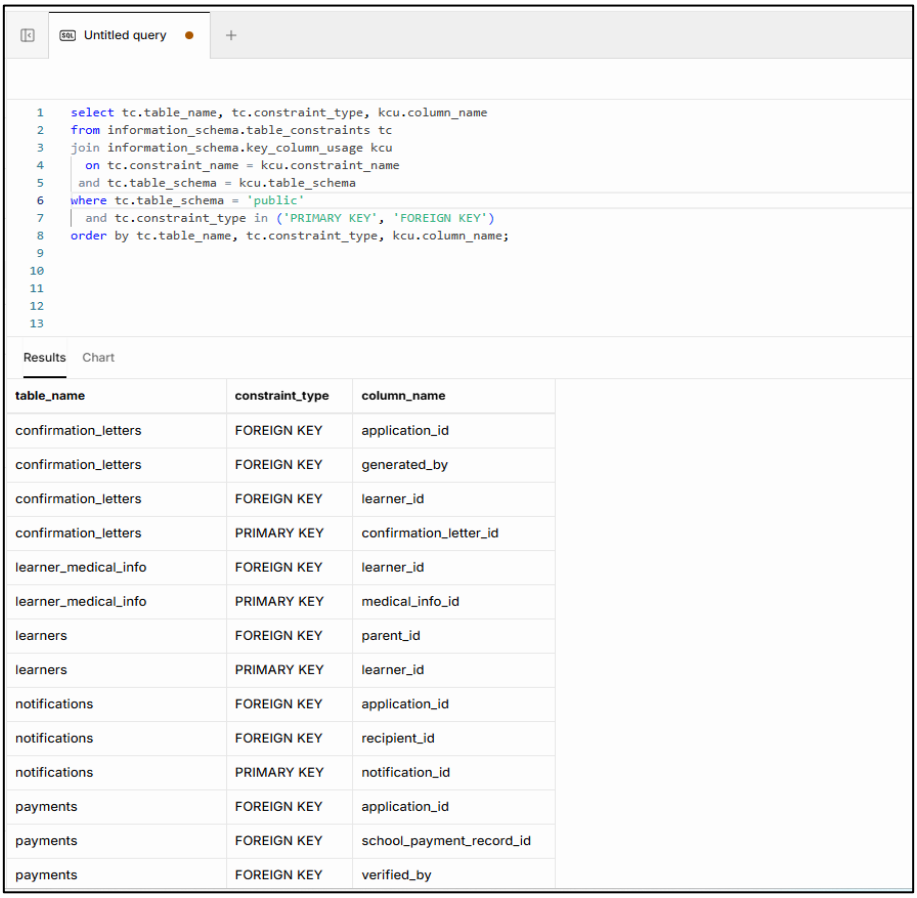
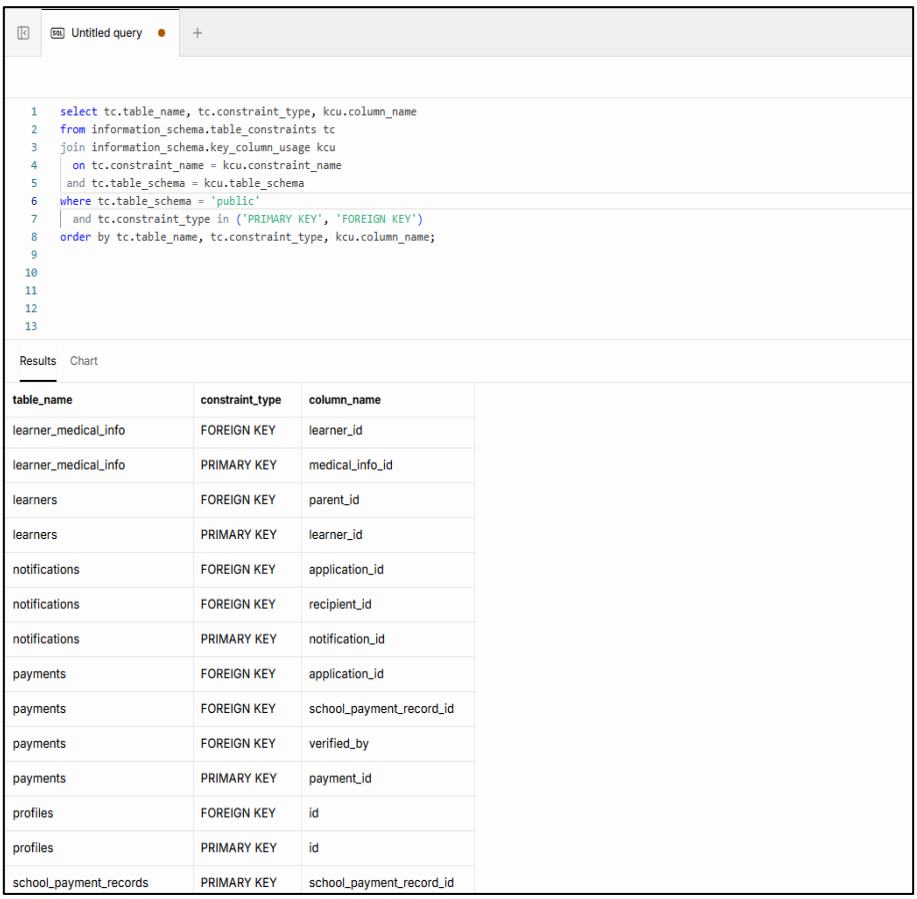


Figure 4.5 – Primary and foreign key constraints for all tables (SQL Editor, shown over three screenshots). A SQL query that lists every primary key and foreign key constraint in the public schema, with its table and column. The result is long, so it is shown over three screenshots. It is used to check that the keys match the ERD.



table_name	constraint_type	column_name
confirmation_letters	FOREIGN KEY	application_id
confirmation_letters	FOREIGN KEY	generated_by
confirmation_letters	FOREIGN KEY	learner_id
confirmation_letters	PRIMARY KEY	confirmation_letter_id
learner_medical_info	FOREIGN KEY	learner_id
learner_medical_info	PRIMARY KEY	medical_info_id
learners	FOREIGN KEY	parent_id
learners	PRIMARY KEY	learner_id
notifications	FOREIGN KEY	application_id
notifications	FOREIGN KEY	recipient_id
notifications	PRIMARY KEY	notification_id
payments	FOREIGN KEY	application_id
payments	FOREIGN KEY	school_payment_record_id
payments	FOREIGN KEY	verified_by

(continued)



table_name	constraint_type	column_name
learner_medical_info	FOREIGN KEY	learner_id
learner_medical_info	PRIMARY KEY	medical_info_id
learners	FOREIGN KEY	parent_id
learners	PRIMARY KEY	learner_id
notifications	FOREIGN KEY	application_id
notifications	FOREIGN KEY	recipient_id
notifications	PRIMARY KEY	notification_id
payments	FOREIGN KEY	application_id
payments	FOREIGN KEY	school_payment_record_id
payments	FOREIGN KEY	verified_by
payments	PRIMARY KEY	payment_id
profiles	FOREIGN KEY	id
profiles	PRIMARY KEY	id
school_payment_records	PRIMARY KEY	school_payment_record_id

(continued)

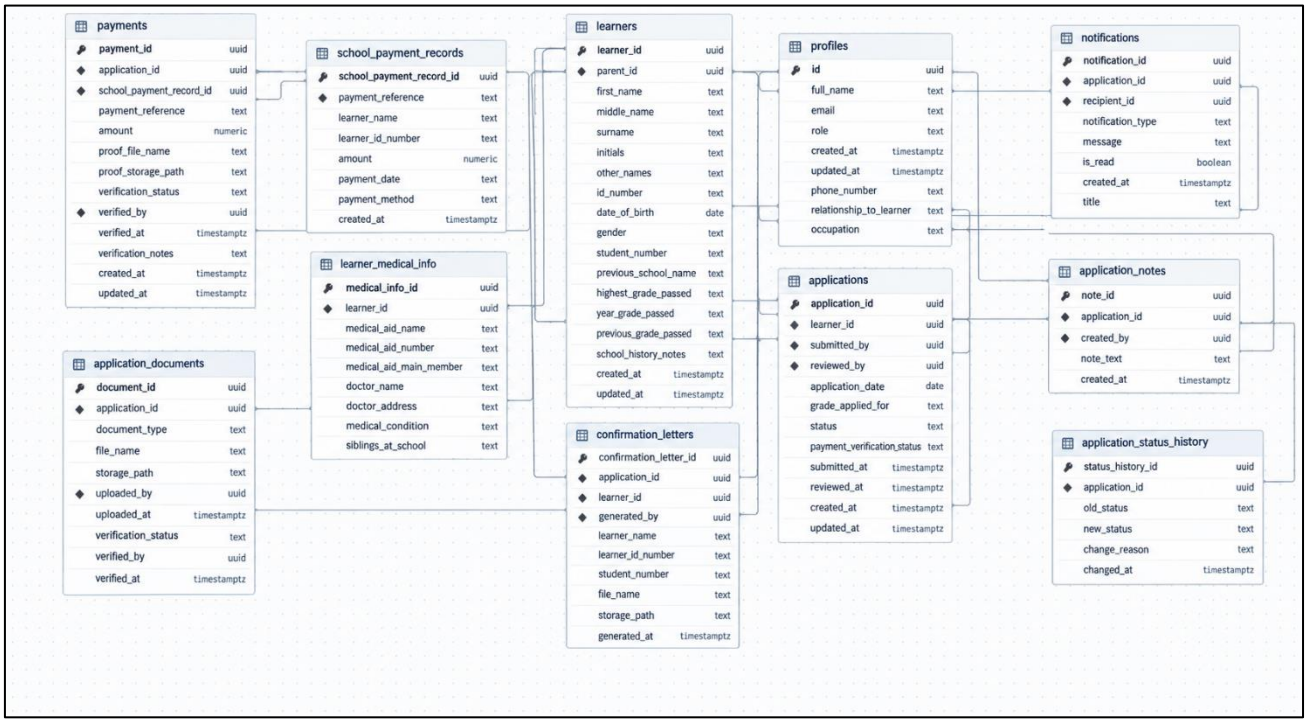


Figure 4.6 – Schema Visualizer showing the implemented tables and relationships. The Schema Visualizer view of the implemented database, showing each table and the foreign-key relationships between them as they exist in Supabase.

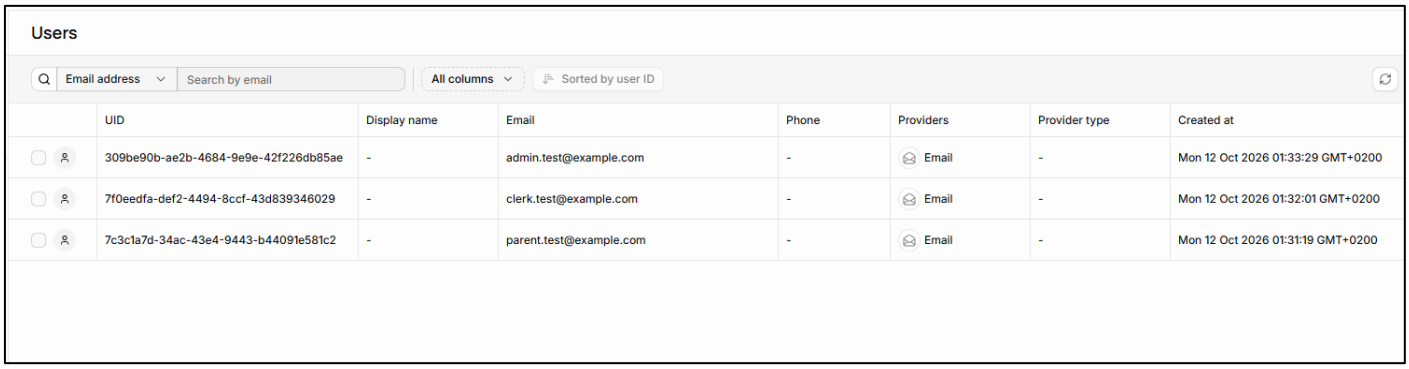


Figure 4.7 – Supabase Auth users (test accounts only). The Supabase Authentication users page, showing the three test accounts (Administrator, Financial Clerk and Parent) created for testing. Only test accounts are shown.

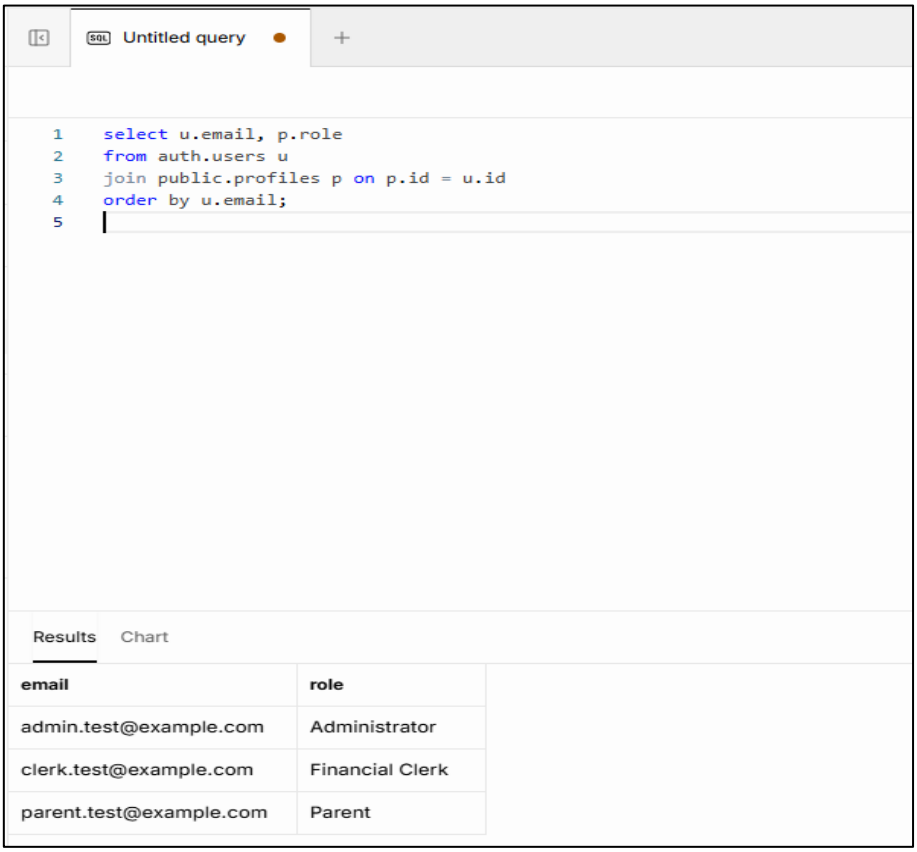


Figure 4.8 – Test accounts linked to their roles in profiles. A SQL query that joins auth.users to profiles to show each test account's email and the role stored for it (Administrator, Financial Clerk or Parent). This confirms that every sign-in account is linked to a role.

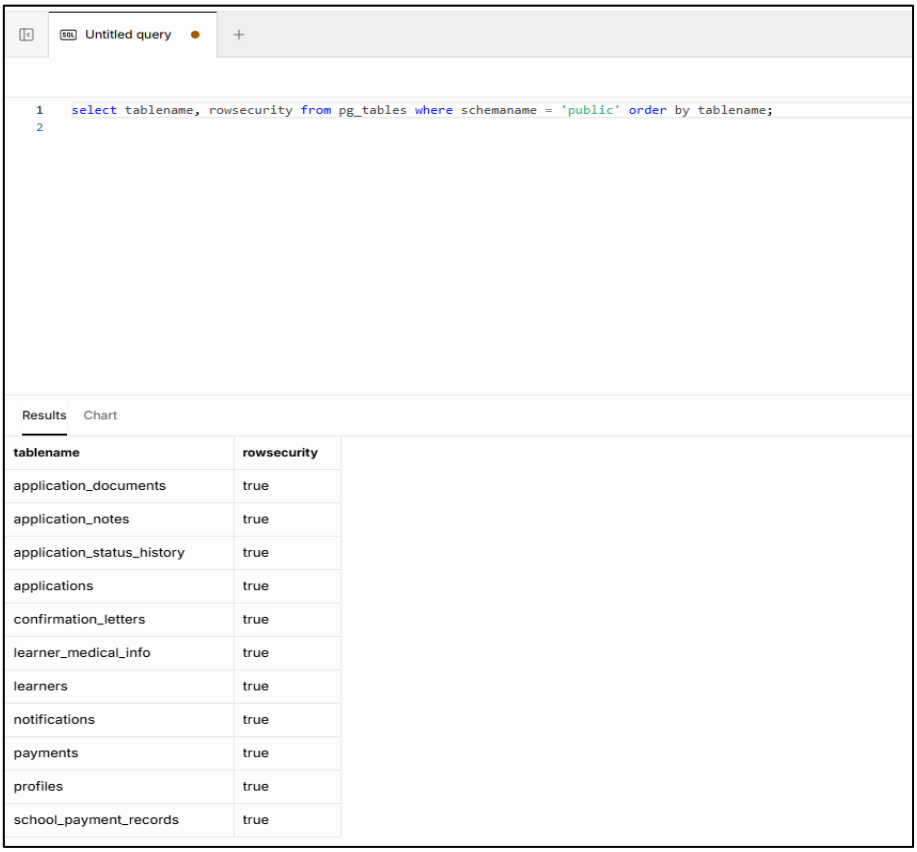
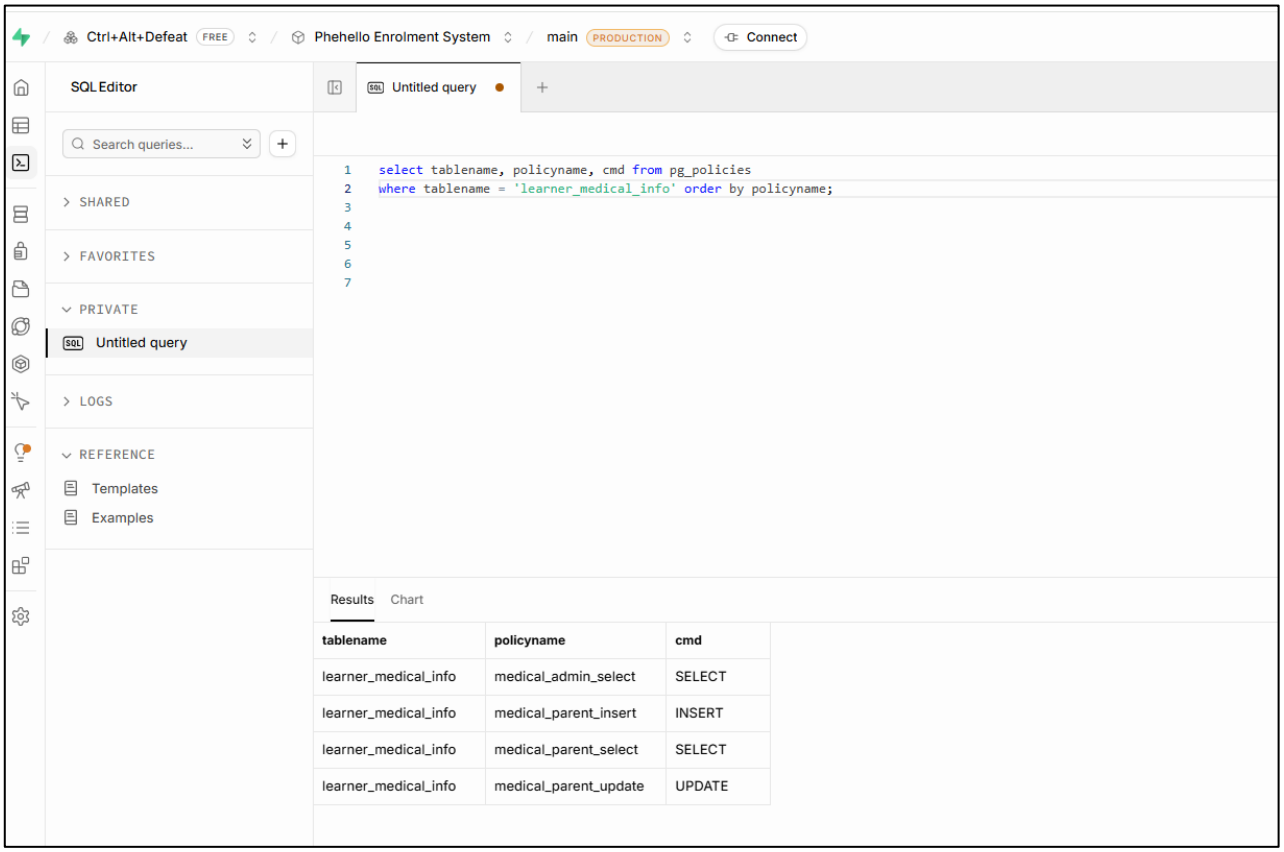
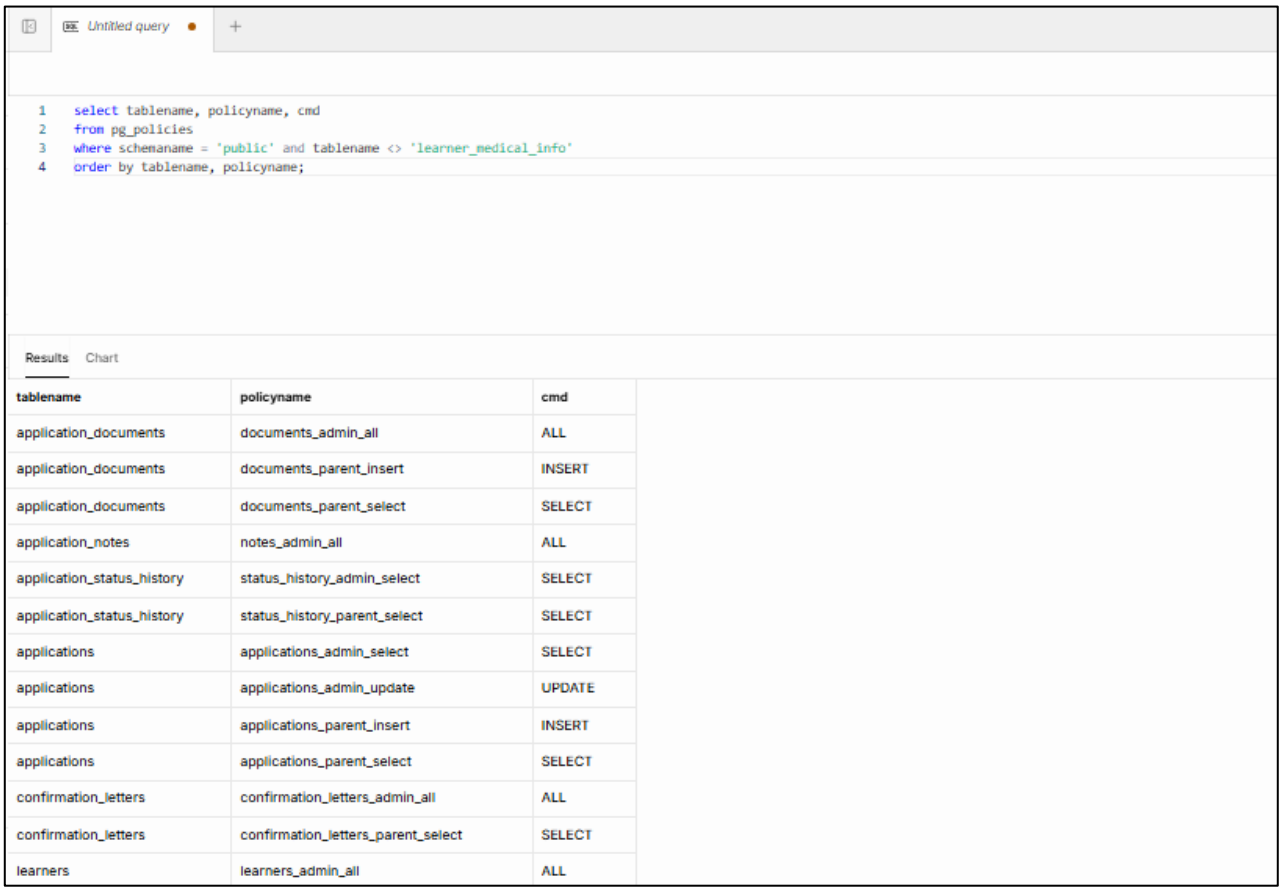


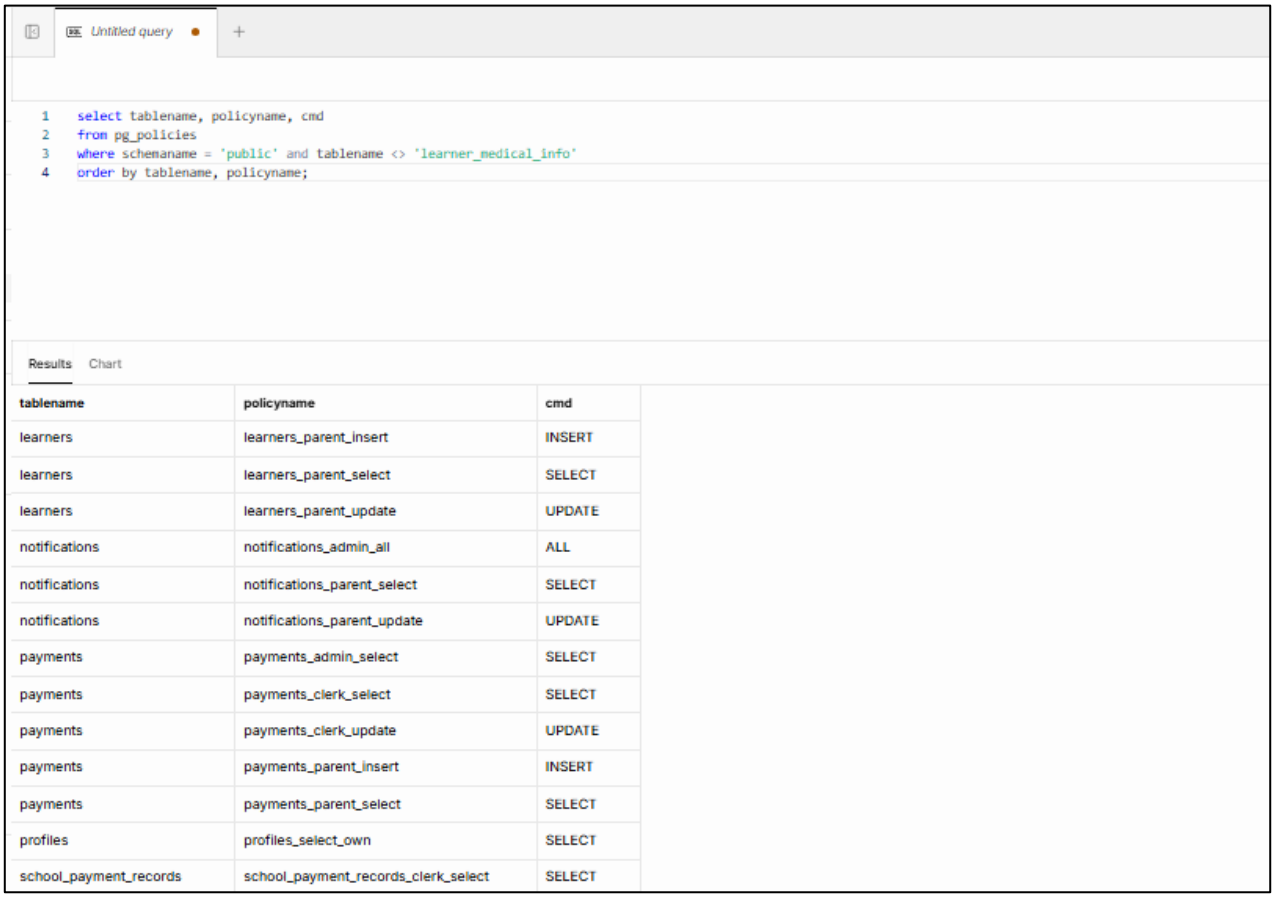
Figure 4.9 – Row Level Security enabled on all 11 tables. A SQL query on the database tables showing that Row Level Security (rowsecurity = true) is switched on for all 11 tables, so no table is open to unrestricted access.



4.10 (a) learner_medical_info policies. The Row Level Security policies in the database, listed with their table, policy name and command (SELECT, INSERT, UPDATE or ALL). Part (a) shows the policies on learner_medical_info



4.10 (b) policies on all other tables shown over two screenshots. The Row Level Security policies in the database, listed with their table, policy name and command (SELECT, INSERT, UPDATE or ALL). part (b) shows the policies on all the other tables, over two screenshots.



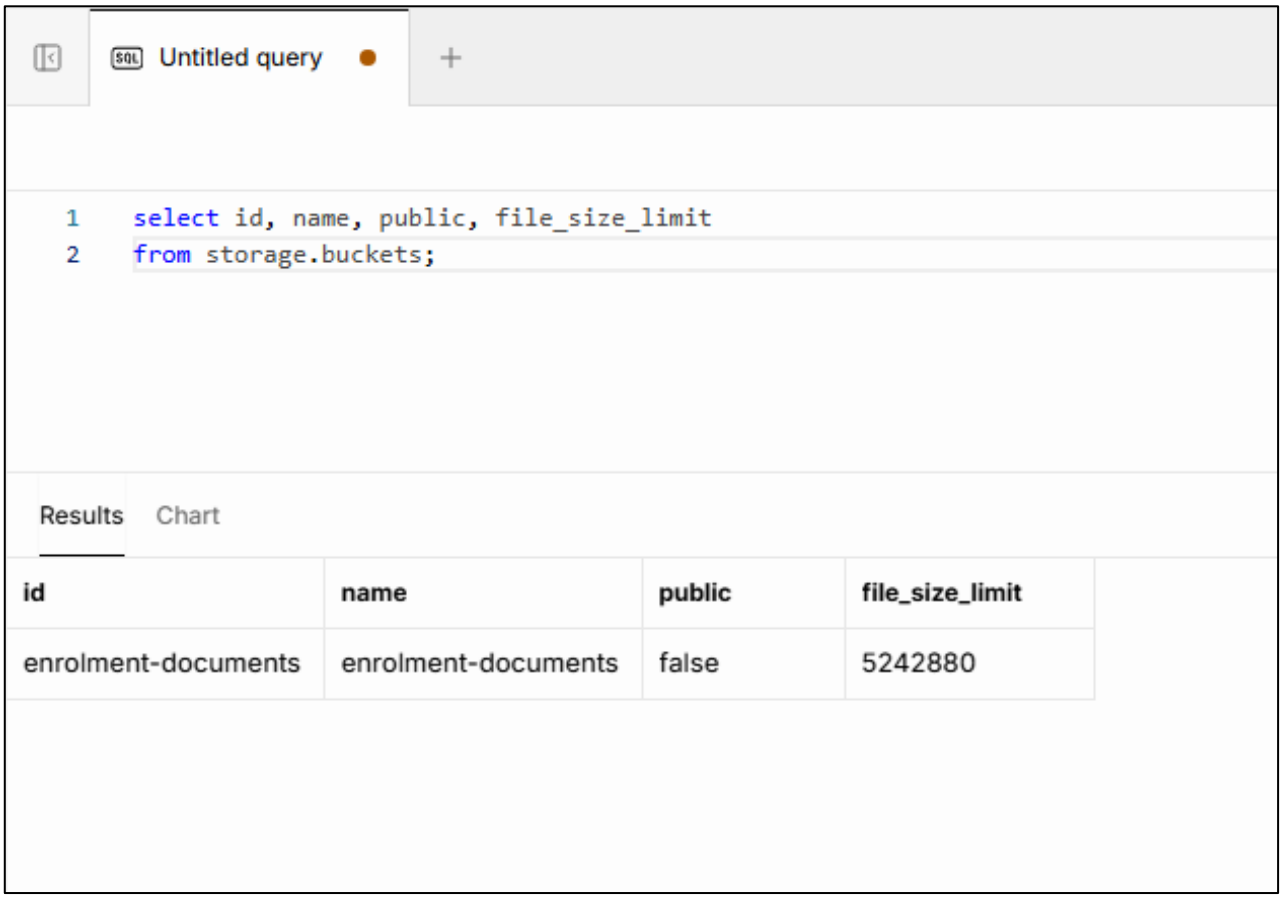
Untitled query

```
select tablename, policyname, cmd
from pg_policies
where schemaname = 'public' and tablename <> 'learner_medical_info'
order by tablename, policyname;
```

Results Chart

tablename	policyname	cmd
learners	learners_parent_insert	INSERT
learners	learners_parent_select	SELECT
learners	learners_parent_update	UPDATE
notifications	notifications_admin_all	ALL
notifications	notifications_parent_select	SELECT
notifications	notifications_parent_update	UPDATE
payments	payments_admin_select	SELECT
payments	payments_clerk_select	SELECT
payments	payments_clerk_update	UPDATE
payments	payments_parent_insert	INSERT
payments	payments_parent_select	SELECT
profiles	profiles_select_own	SELECT
school_payment_records	school_payment_records_clerk_select	SELECT

(continued)



Untitled query

```
select id, name, public, file_size_limit
from storage.buckets;
```

Results Chart

id	name	public	file_size_limit
enrolment-documents	enrolment-documents	false	5242880

Figure 4.11 – Storage bucket enrolment-documents set to private. A query on the storage buckets showing that the enrolment-documents bucket exists, is private (public = false) and has a 5 MB file-size limit.

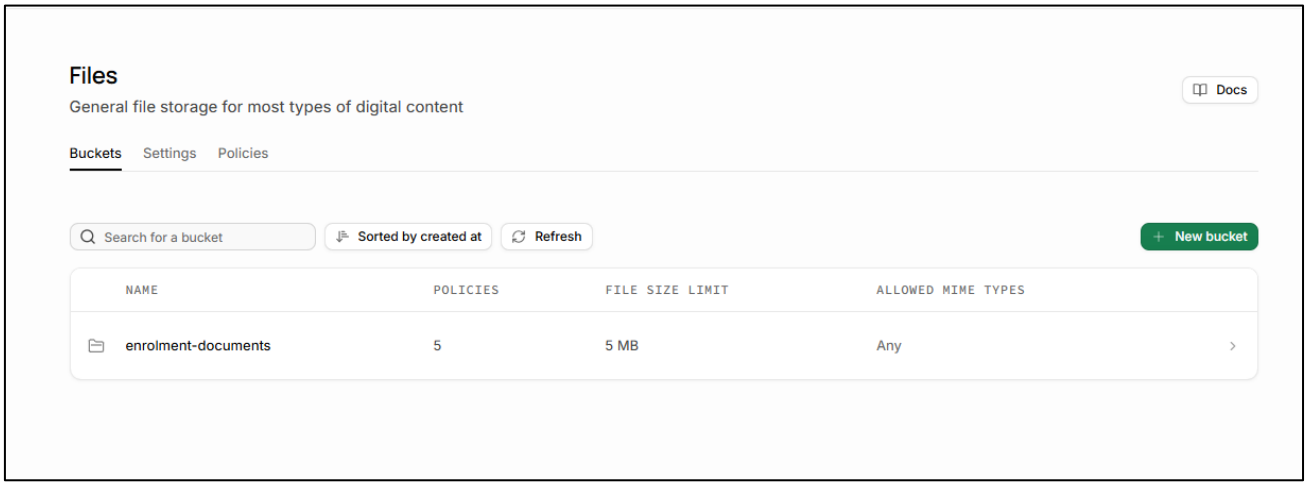
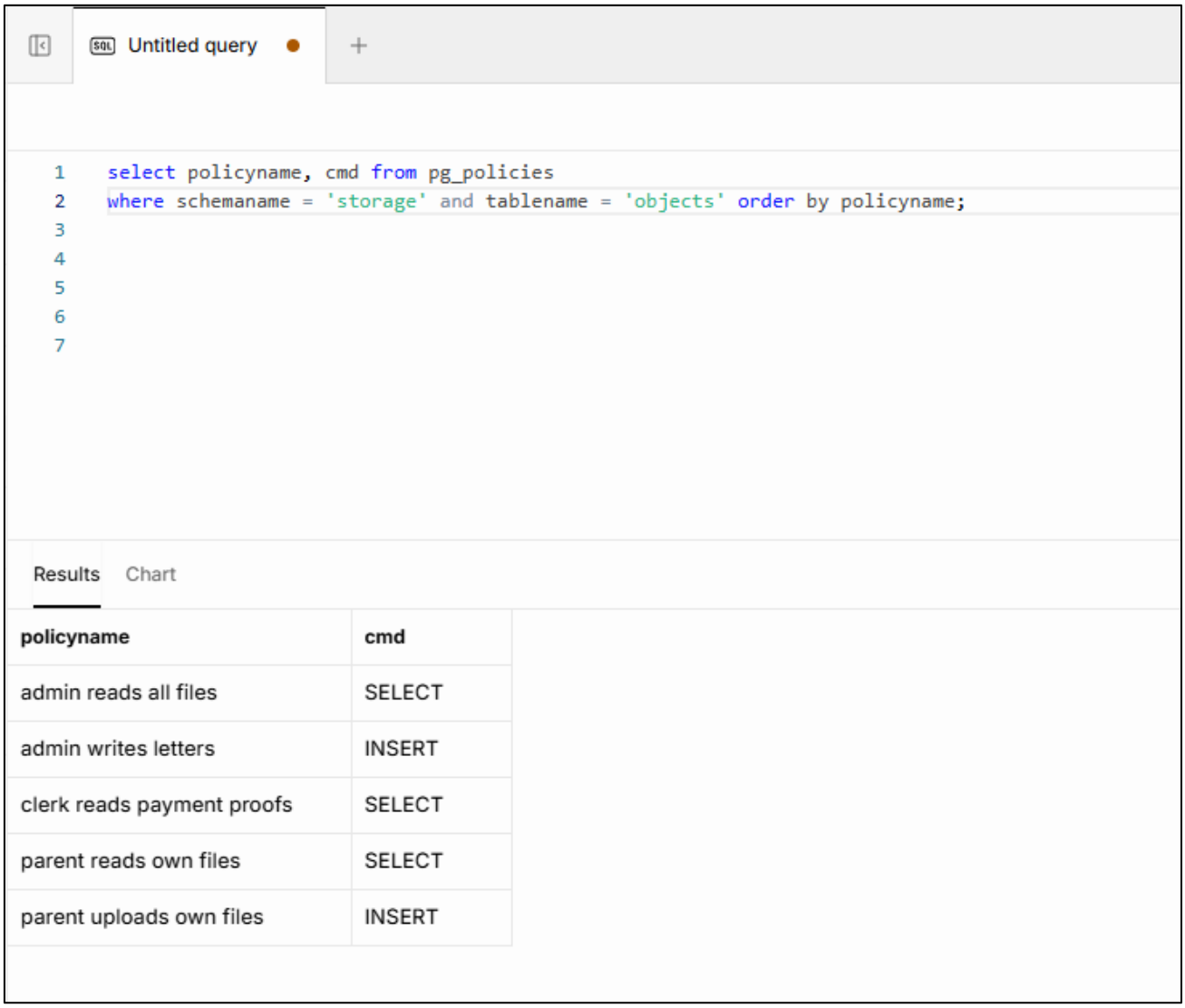


Figure 4.12 – Storage policies for the enrolment-documents bucket: (a) bucket list showing five policies. The Storage policies for the enrolment-documents bucket. Part (a) shows the bucket with its five policies



(b) ***policy names and commands.*** The Storage policies for the enrolment-documents bucket. part (b) lists the policy names and commands: admin reads all files, admin writes letters, clerk reads payment proofs, parent reads own files and parent uploads own files.

Untitled query

Unsaved edits | Database

```
1  select l.first_name, l.surname, a.status,
2         a.payment_verification_status as app_payment_status,
3         p.payment_reference, p.verification_status as payment_status
4  from public.applications a
5  join public.learners l on l.learner_id = a.learner_id
6  left join public.payments p on p.application_id = a.application_id
7  where l.first_name = 'Test'
8  order by l.surname;
9
```

Results | Chart | 2 rows (Limited to o

first_name	surname	status	app_payment_status	payment_reference	payment_status
Test	LearnerOne	Pending Admin Review	Pending	TEST-REF-001	Pending
Test	LearnerTwo	Pending Admin Review	Verified	TEST-REF-002	Verified

Figure 4.13 – Test records retrieved with a join across learners, applications and payments. A join query across learners, applications and payments that returns the two test applications with their application status, payment reference and payment status. It shows that the tables are linked correctly and that test records can be retrieved.

Untitled query

```
1  update public.payments
2  set verification_status = 'Pending'
3  where payment_reference = 'TEST-REF-002';
4
5  select a.payment_verification_status as app_payment_status,
6         p.verification_status as payment_status
7  from public.applications a
8  join public.payments p on p.application_id = a.application_id
9  where p.payment_reference = 'TEST-REF-002';
```

Results | Chart

app_payment_status	payment_status
Pending	Pending

Figure 4.14 – Payment verification update: (a) before payment status update. A test of payment verification. Part (a) shows the application payment status and the payment status both still Pending.

Untitled query

```
update public.payments
set verification_status = 'Verified'
where payment_reference = 'TEST-REF-002';

select a.payment_verification_status as app_payment_status,
       p.verification_status as payment_status
from public.applications a
join public.payments p on p.application_id = a.application_id
where p.payment_reference = 'TEST-REF-002';
```

Results | Chart

app_payment_status	payment_status
Verified	Verified

(b) after the payment status update. A test of payment verification. Part (b) shows both changed to Verified after the payment is marked Verified, which proves that the trigger updates the application automatically.

Untitled query

```
select h.old_status, h.new_status, p.email as changed_by, h.changed_at
from public.application_status_history h
join public.profiles p on p.id = h.changed_by;
```

Results | Chart

old_status	new_status	changed_by	changed_at
Pending Payment Verification	Pending Admin Review	admin.test@example.com	2026-10-12 10:04:12.937449+00

Figure 4.15 – Application status change recorded in application_status_history with the Administrator as changed_by. A query on application_status_history joined to profiles. It shows that a status change (Pending Payment Verification to Pending Admin Review) was recorded automatically, with the Administrator's email as the person who made the change.

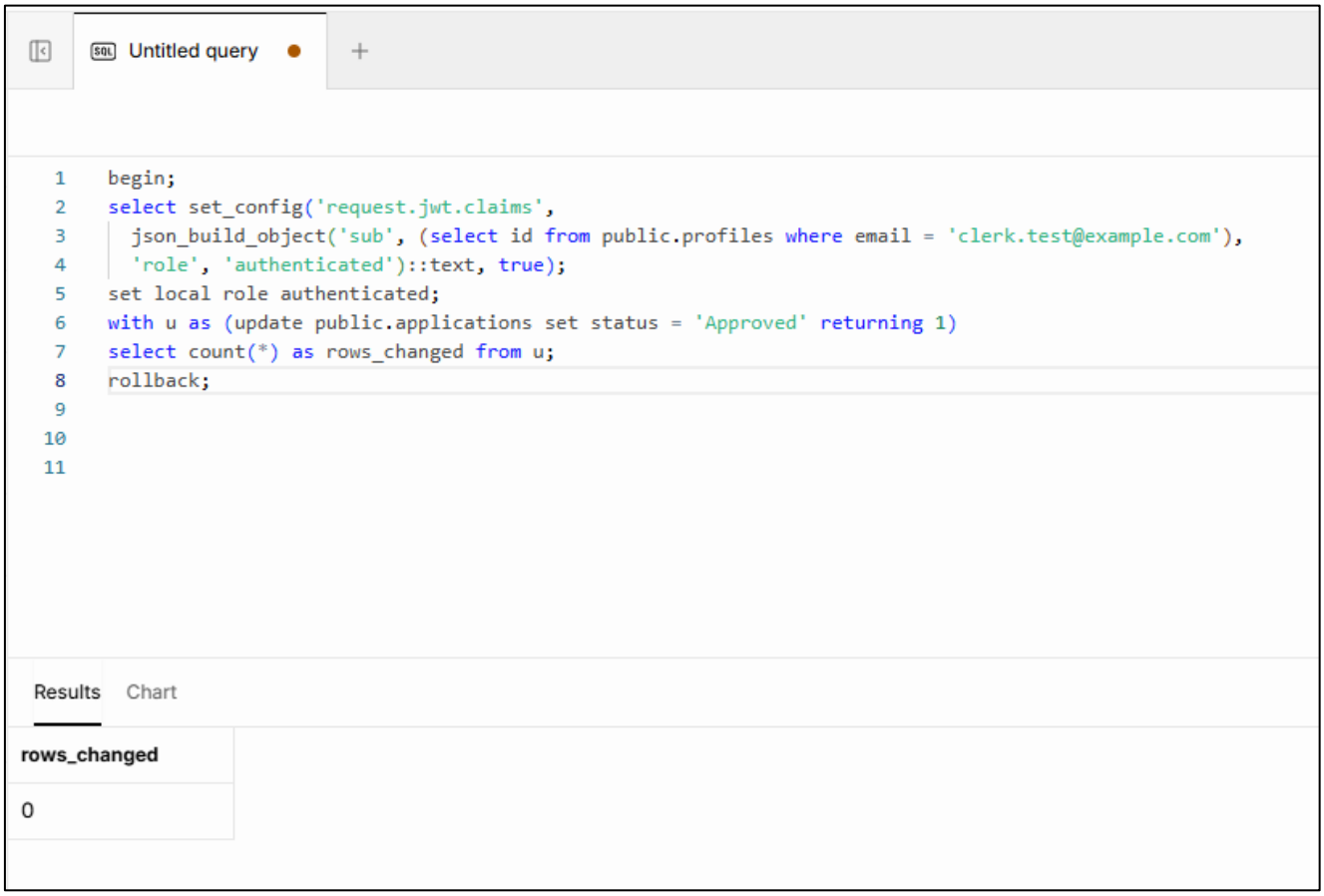
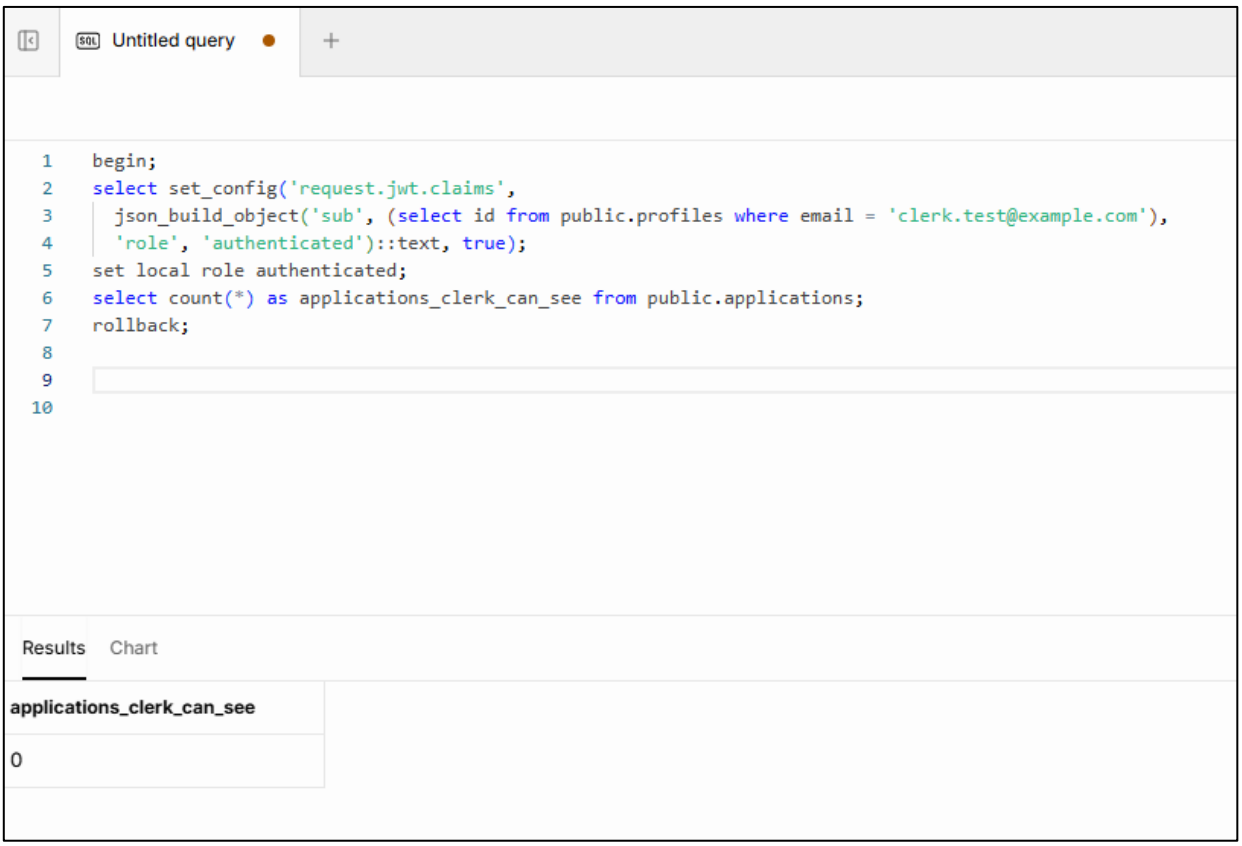
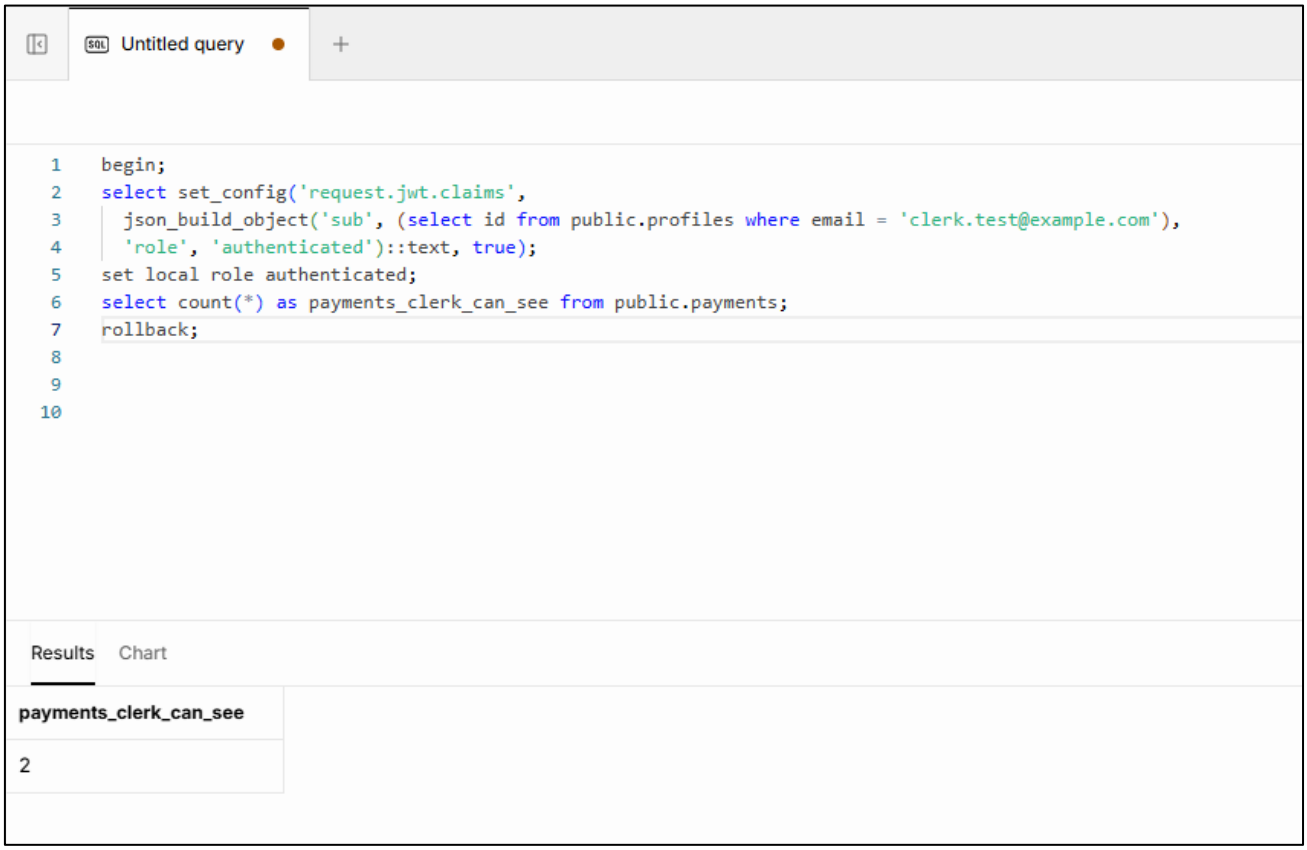


Figure 4.16 – Financial Clerk access tests: (a) cannot read applications (o rows). Access tests run as the Financial Clerk test account (each test is rolled back afterwards). The Clerk (a) can see 0 applications.



4.6 (b) Clerk cannot change an application (0 rows changed). Access tests run as the Financial Clerk test account (each test is rolled back afterwards). The Clerk (b) cannot change an application (0 rows changed).



4.6 (c) Clerk can read payments. Access tests run as the Financial Clerk test account (each test is rolled back afterwards). The Clerk (c) can read payments (2 rows), which matches the role separation in NFR6.

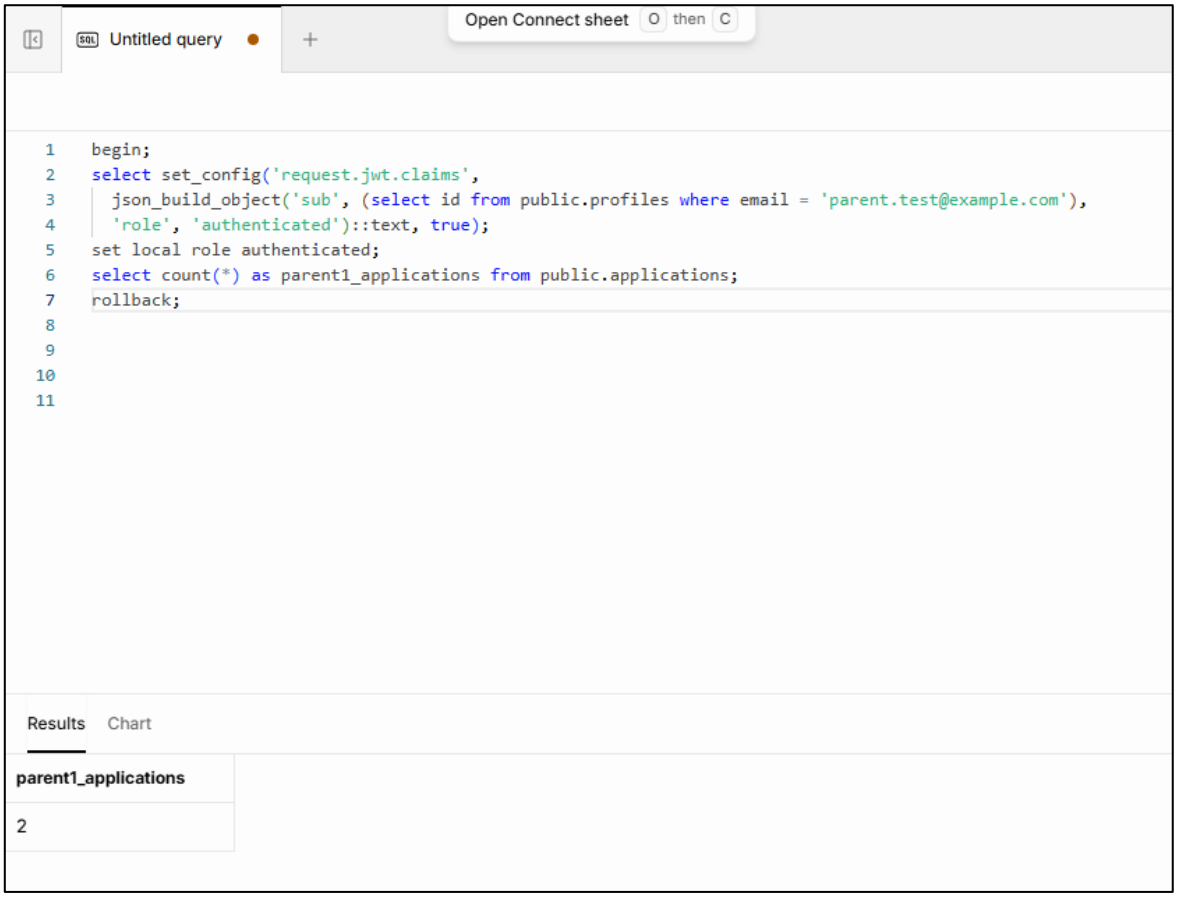
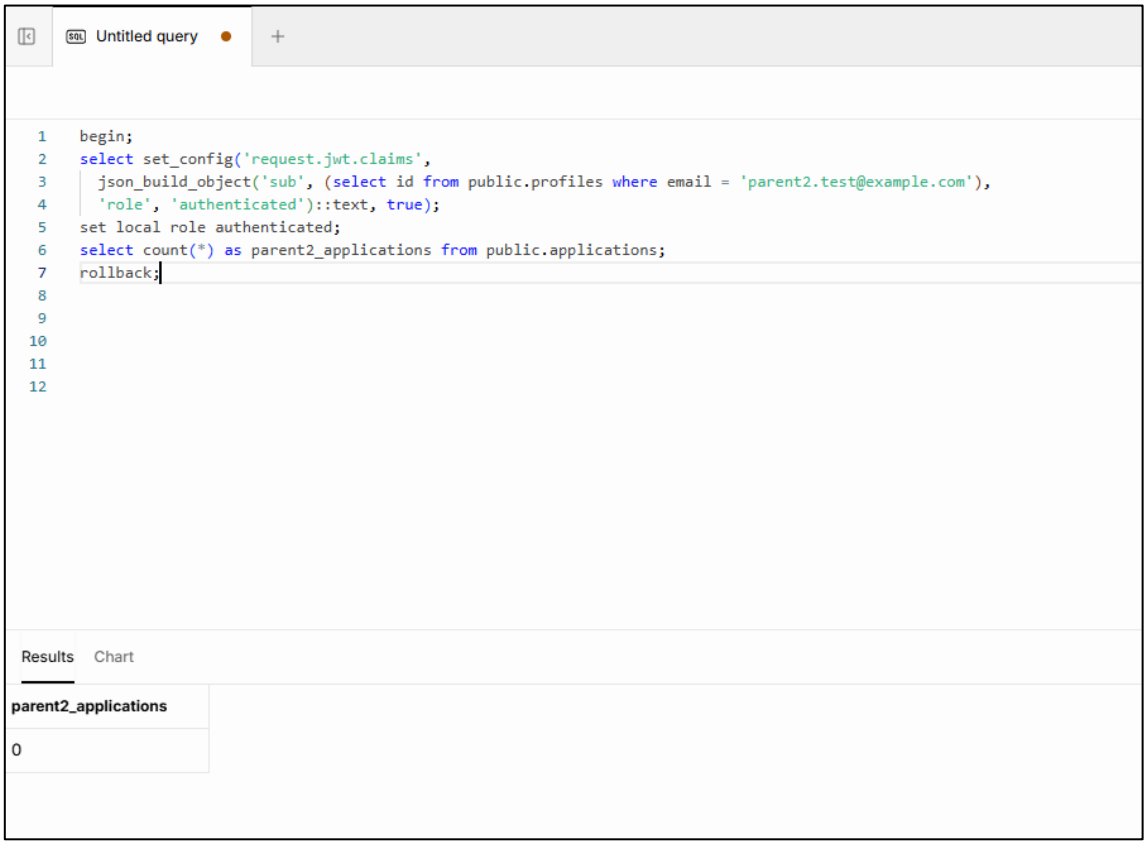
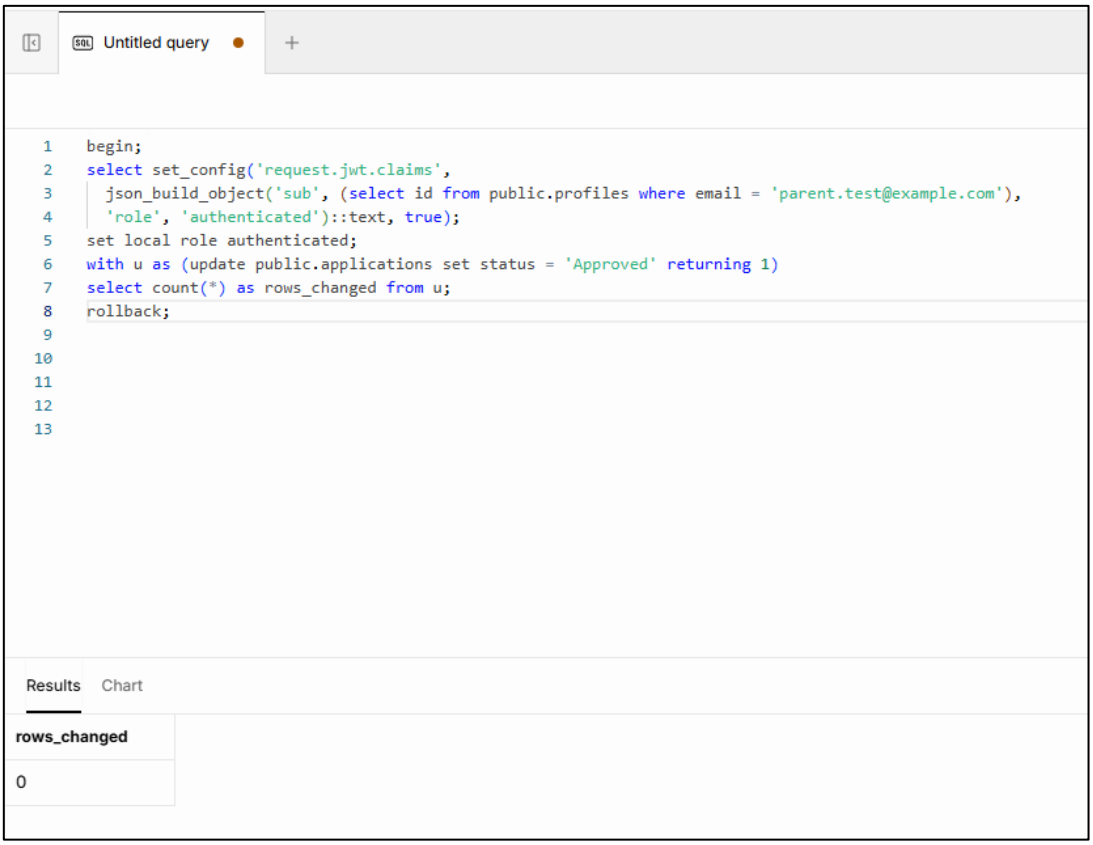


Figure 4.17 – Parent access tests: (a) reads own applications (2 applications). Access tests run as the Parent test accounts. (a) Parent One can read their own 2 applications.



(b) Parent Two cannot read Parent One's applications (0 rows). Access tests run as the Parent test accounts. (b) Parent Two cannot read Parent One's applications (0 rows)



(c) Parent cannot approve own application (o rows). Access tests run as the Parent test accounts. Parent cannot approve an application (0 rows changed).

Develop the Flutter Mobile Application

Approved Mobile Application Requirements

Mobile function	Related requirement or use case	Required screen
Register	FR1, Register	Create Parent/Guardian Account
Sign in and out	FR1, Sign In	Welcome / Login, Profile (log out)
Submit enrolment application	FR2, Submit Application	Application wizard steps 1 to 4, review sheet
Upload supporting documents	FR3, Upload Documents	Wizard step 4: Parent/Guardian and Documents
View enrolment fee and banking details	FR15, View Payment Information	Wizard step 5: Payment Information
Upload proof of payment	FR4, Upload Proof of Payment	Wizard step 6: Proof of Payment
View application and payment status	FR5, View Application Status	Home, My Applications, Application Details
View messages and reminders	FR10, FR11	Messages and Notifications
Fix requested corrections	FR8, Corrections	Application Details

Initial Function Checks

Function or check	Result	Action required
Application launches successfully	Pass	None
Main screens display correctly	Pass	None
Navigation works	Pass	None
Forms validate input	Pass	None
Mobile application connects to Supabase	Pass	None
Required records can be saved	Pass	None
Required records can be retrieved	Pass	None
Authentication works, where required	Pass	None
Access restrictions work as intended	Pass	None

Commit Your Work and Update Project Records

Document or tool	What to update	Status
GitHub	Flutter project committed to the mobile/ folder, with meaningful commit messages from each contributor	[Commit reference]
Microsoft Project	Mobile development tasks and progress recorded in Project Schedule.mpp	Updated
Project Progress Tracker	Completed tasks, responsible group members, progress and challenges	Updated
SRS	Any approved requirement or design change recorded in the Requirements Change Log	Updated

Changes made during implementation (for the SRS change log)

Requirement / Item	Original assumption	Change / Status	Reason
Database fields for the Flutter enrolment form	The approved ERD had no columns for grade applied for, initials, other names, school history notes, siblings at school or a notification title	Added applications.grade_applied_for, learners.initials, learners.other_names, learners.school_history_notes, learner_medical_info.siblings_at_school and notifications.title. The submit_enrolment function updated to save them	These fields appear in the approved Flutter wireframes but were missing from the ERD. Found while building the Flutter app
The enrolment form had to be completed and submitted in one sitting	The enrolment form had to be completed and submitted in one sitting	Added a Save & Close button on the enrolment form. A saved application appears under Your Applications on the Home screen with Continue and Delete options	A parent may not have all the information at once. Advised by the lecturer. [State whether drafts are kept on the device or in Supabase]

Evidence

Mobile application screenshots

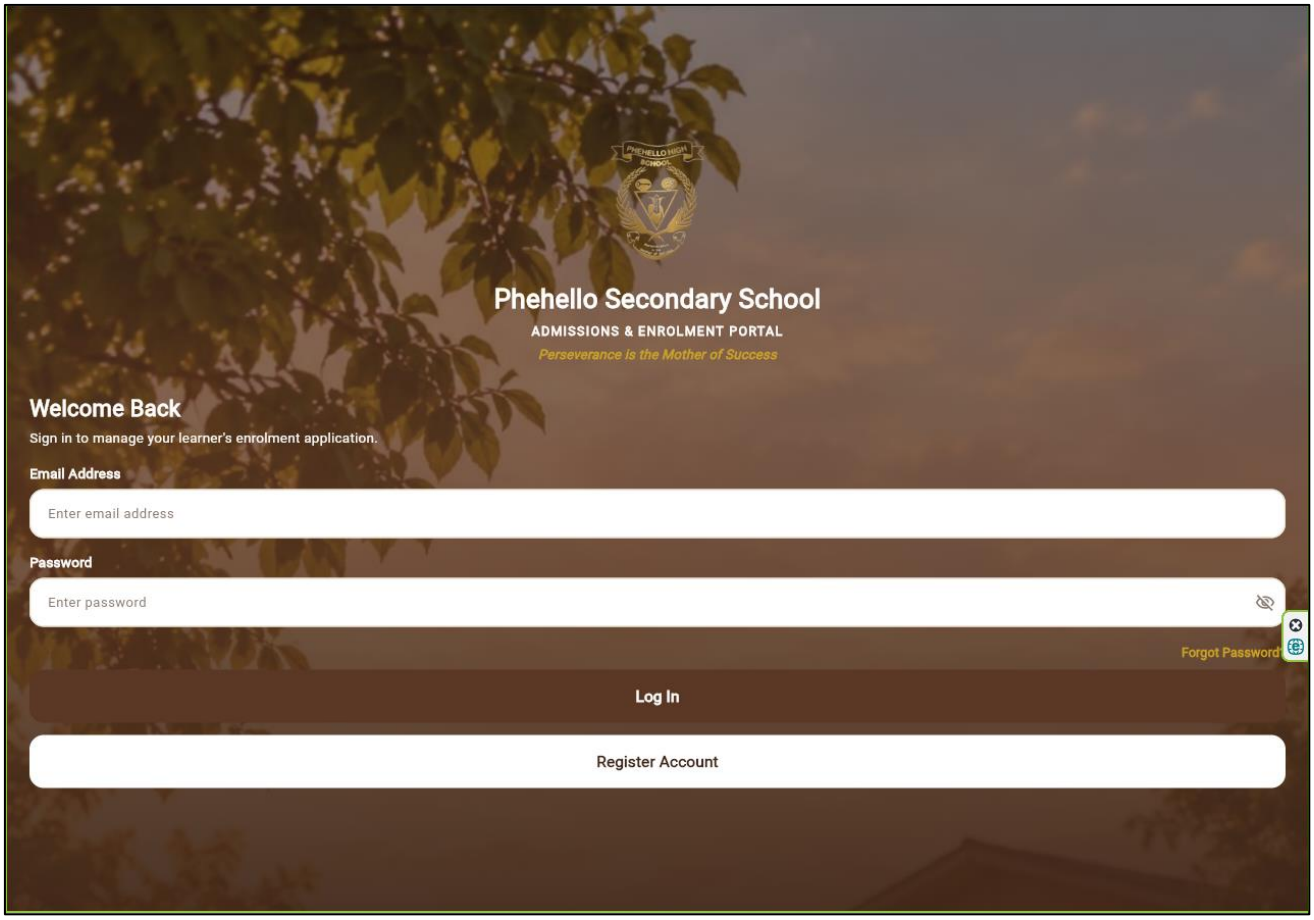


Figure 4.18: Welcome/Login screen. The Welcome/Login screen of the Flutter app, where a parent or guardian enters an email address and password to sign in, or taps Register Account to create a new account.

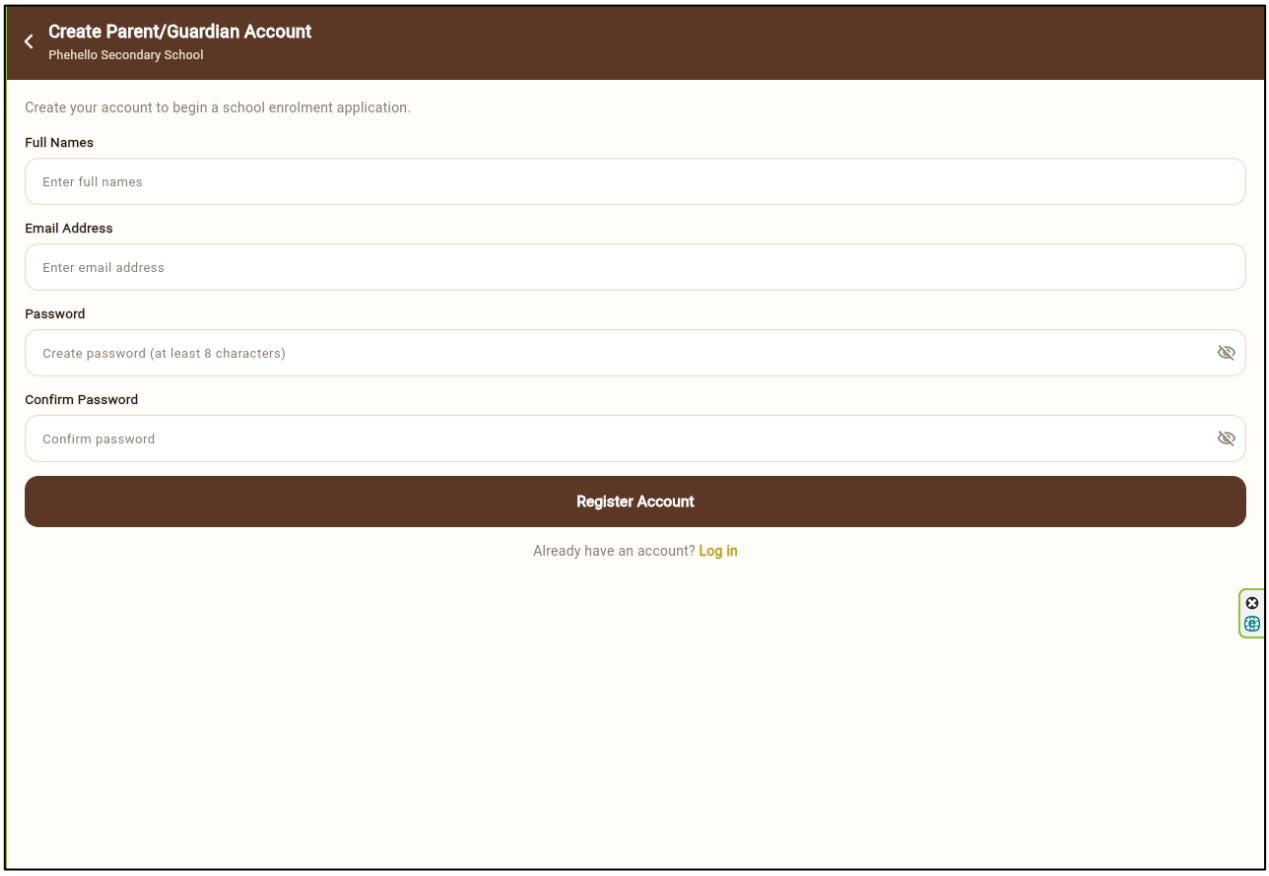


Figure 4.19: Create Parent/Guardian Account screen. The Create Parent/Guardian Account screen, where a new user enters their full name, email address, password and password confirmation to register (FR1).

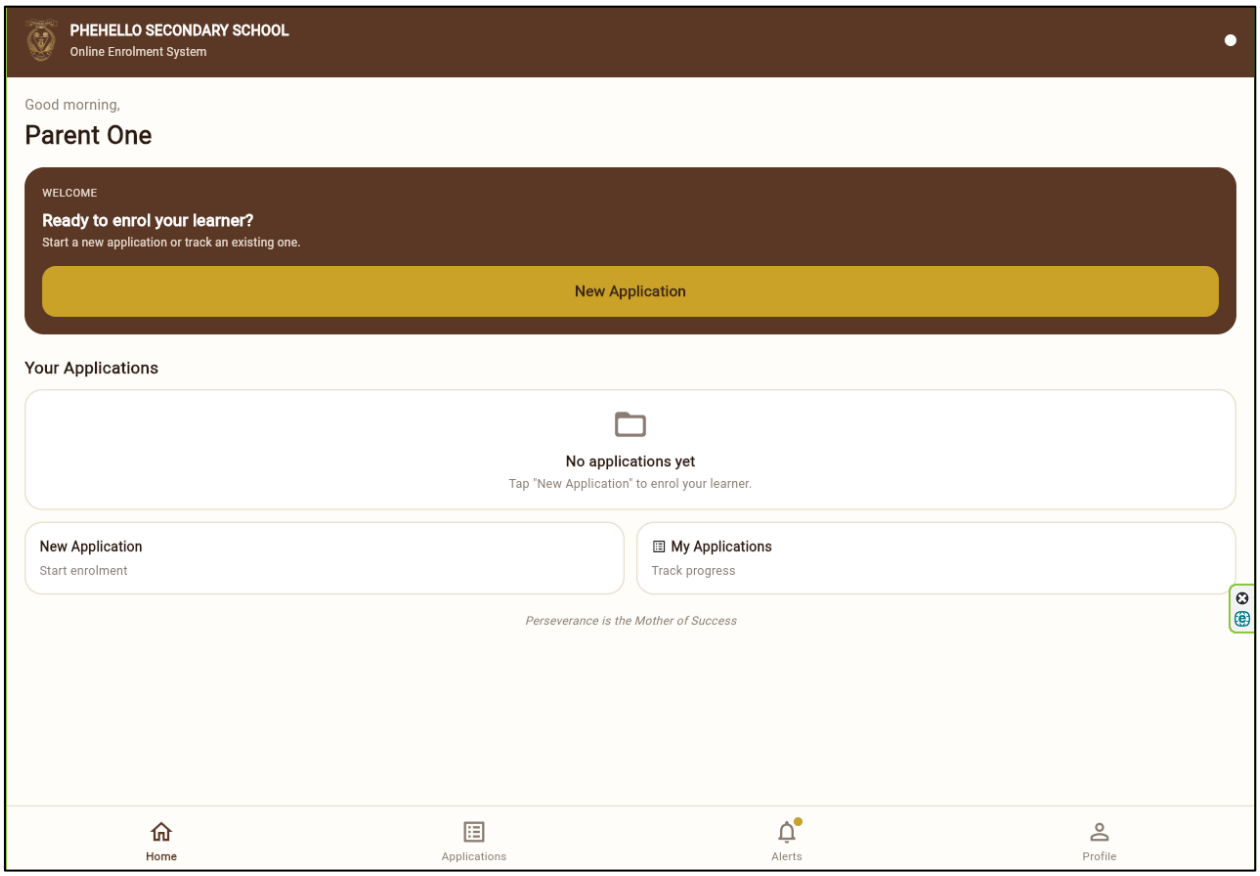


Figure 4.20: Home screen. The Home screen shown after sign-in. It greets the parent, offers a New Application button, lists their applications and provides navigation to Applications, Alerts and Profile.

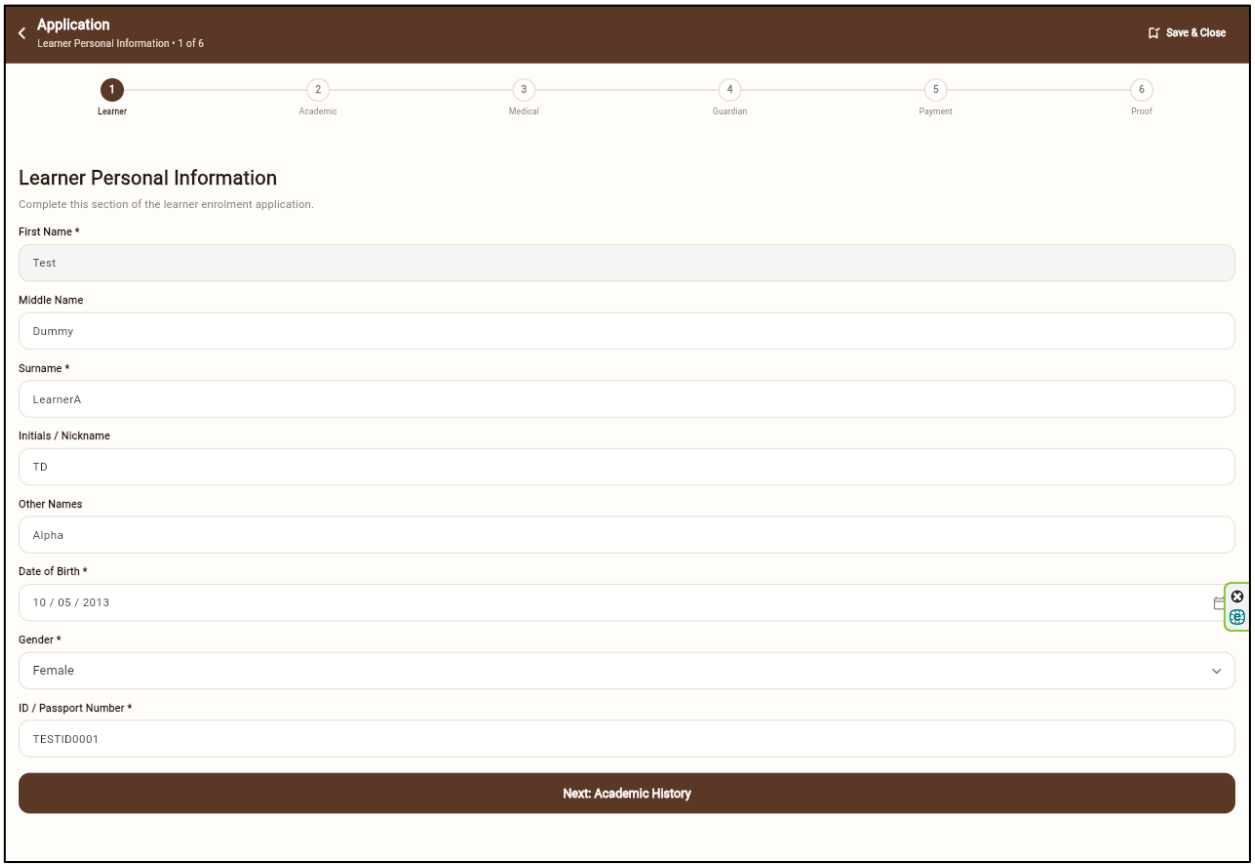


Figure 4.21: Application wizard step 1, Learner Personal Information. Step 1 of the application wizard, Learner Personal Information, where the parent enters the learner's names, initials, date of birth, gender and ID or passport number. A progress bar at the top shows the six steps (FR2).

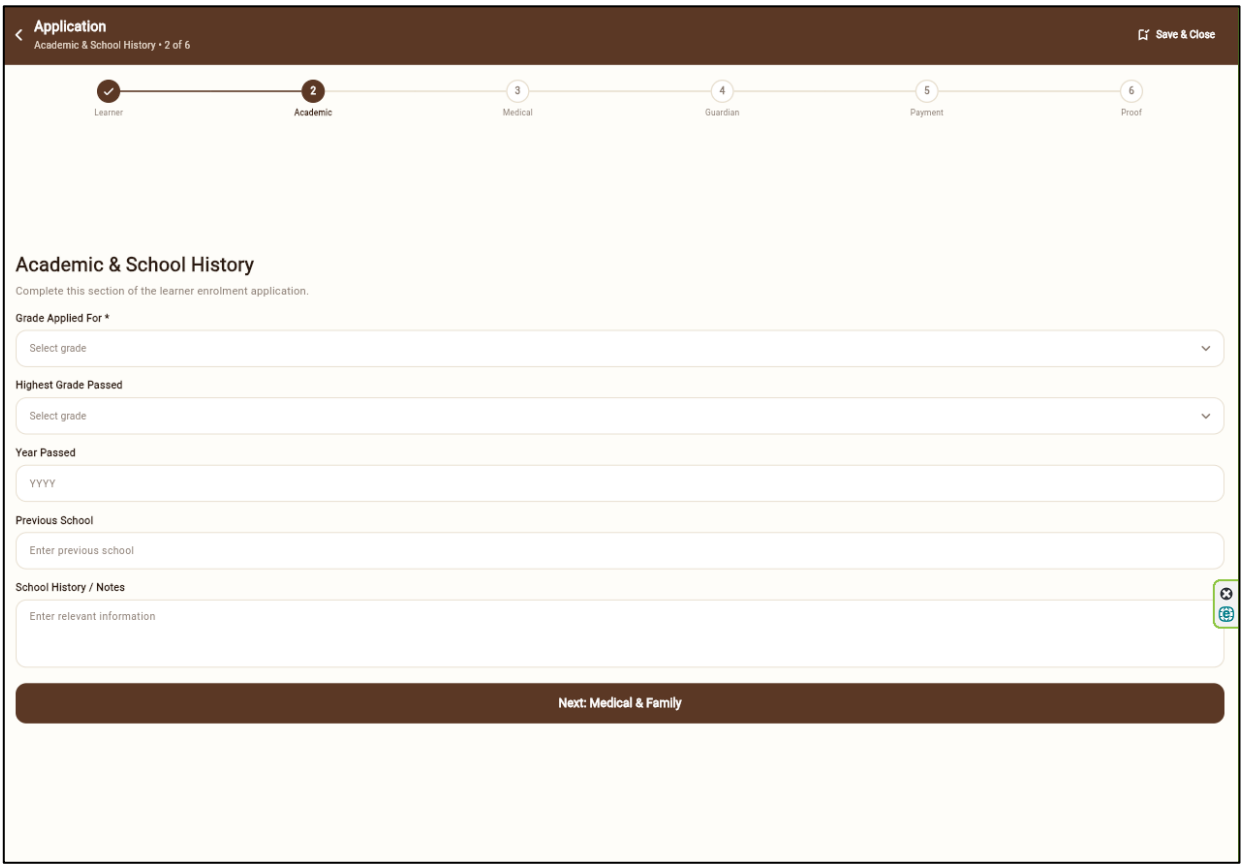


Figure 4.22: Application wizard step 2, Academic and School History. Step 2 of the wizard, Academic and School History, where the parent enters the grade applied for, the highest grade passed, the year it was passed, the previous school and any school-history notes.

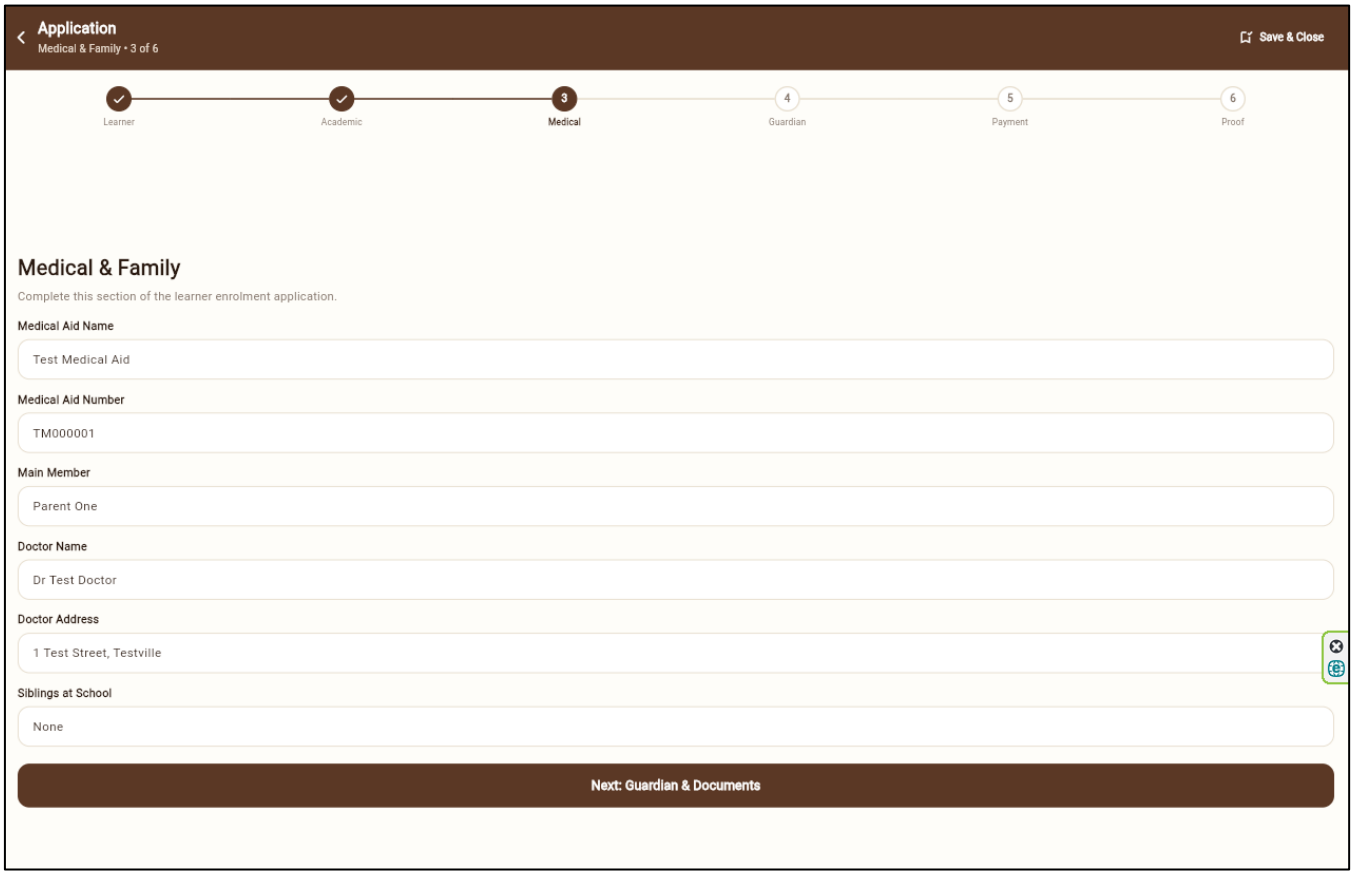


Figure 4.23: Application wizard step 3, Medical and Family. Step 3 of the wizard, Medical and Family, which captures the medical aid details, the doctor's details, any medical condition and siblings at the school, as required by the school's enrolment form.

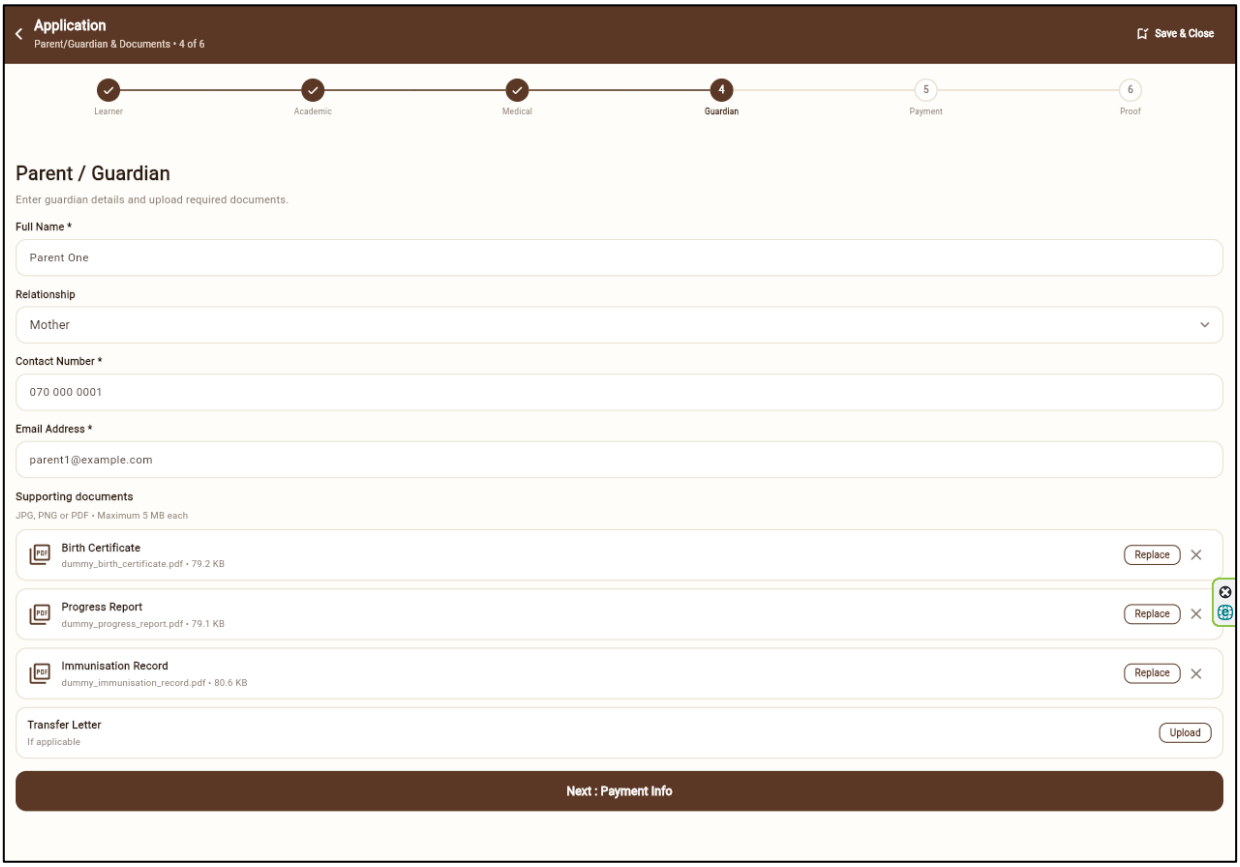


Figure 4.24: Application wizard step 4, Parent/Guardian and Documents. Step 4 of the wizard, Parent/Guardian and Documents, where the guardian's details are entered and the required supporting documents (birth certificate, progress report, immunisation record and transfer letter) are uploaded (FR3).

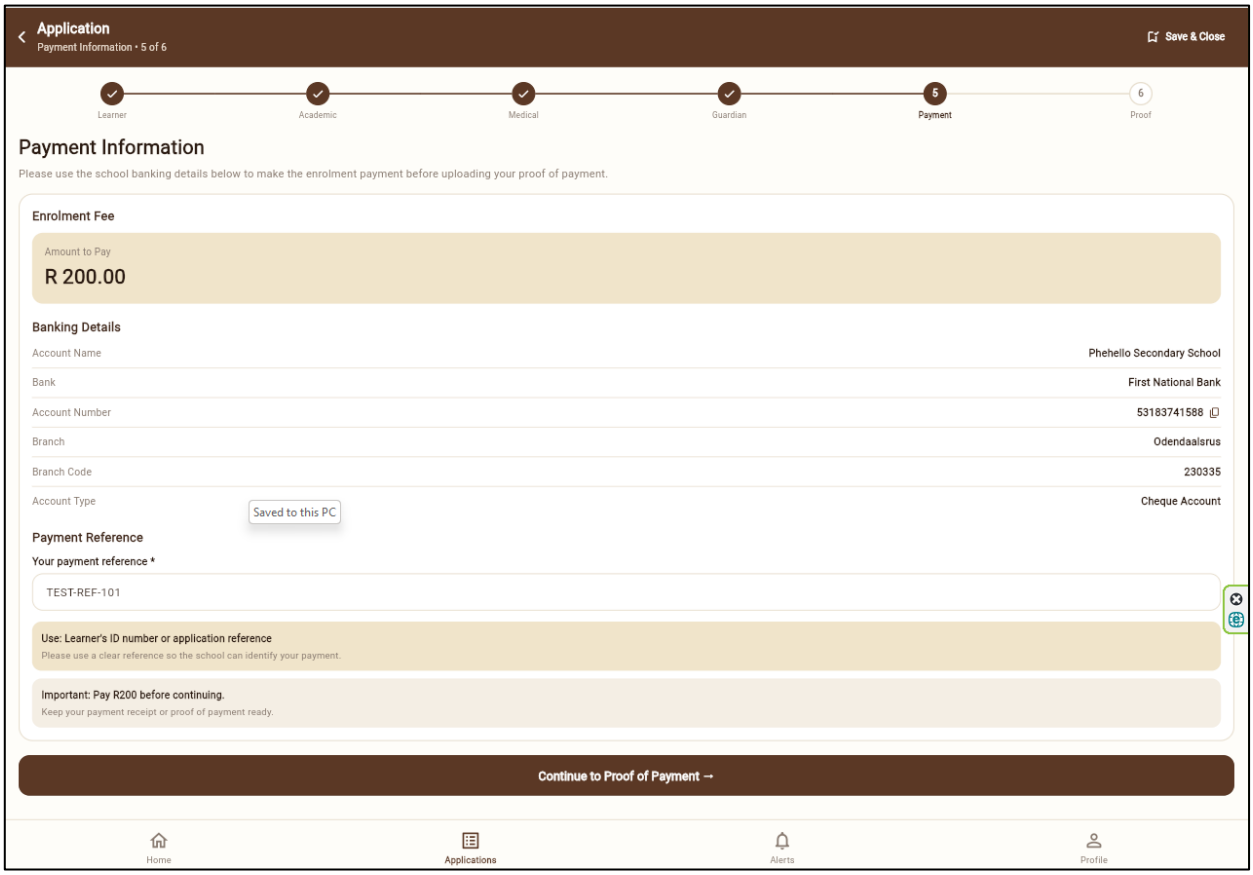


Figure 4.25: Application wizard step 5, Payment Information. Step 5 of the wizard, Payment Information, which shows the enrolment fee and the school's banking details and explains which payment reference to use before the parent continues to proof of payment (FR15).

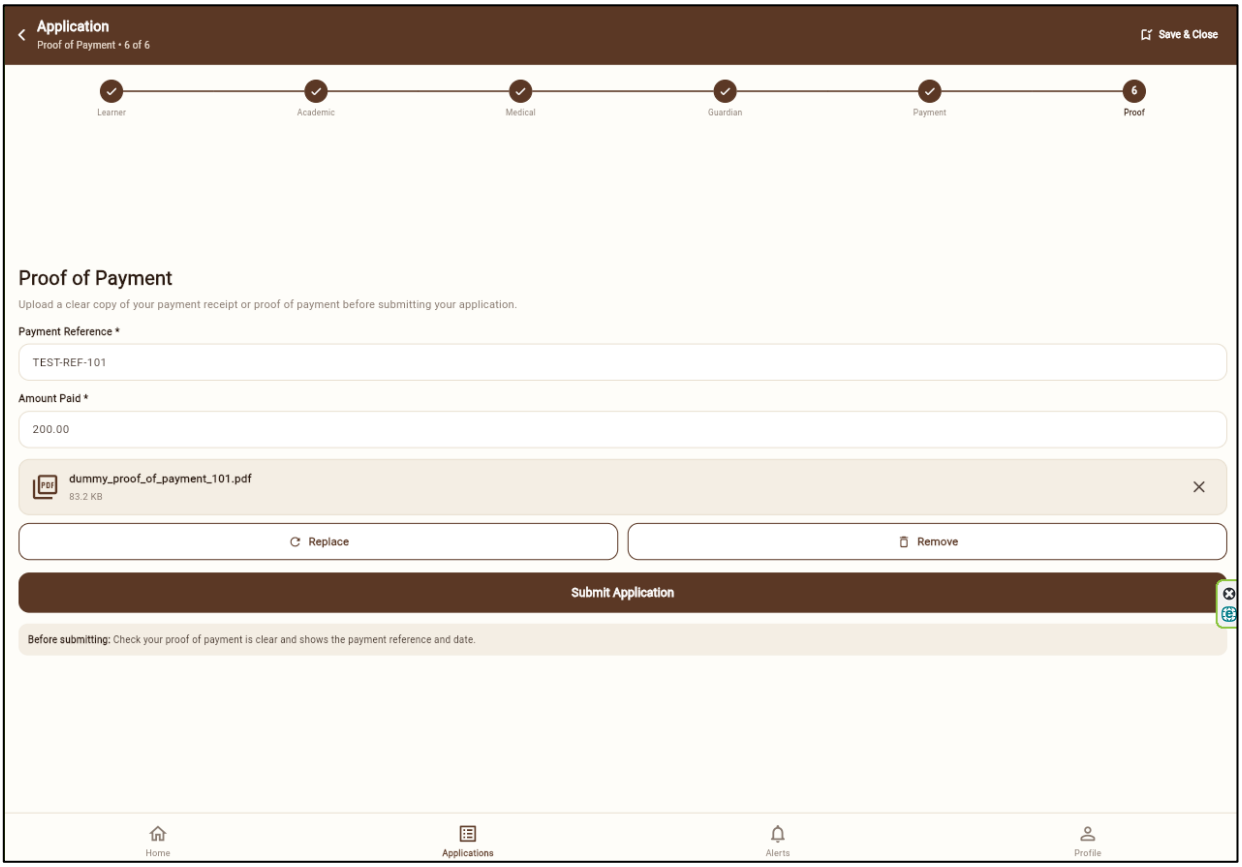


Figure 4.26: Application wizard step 6, Proof of Payment. Step 6 of the wizard, Proof of Payment, where the parent enters the payment reference and the amount paid, uploads the proof-of-payment file and submits the application (FR4).

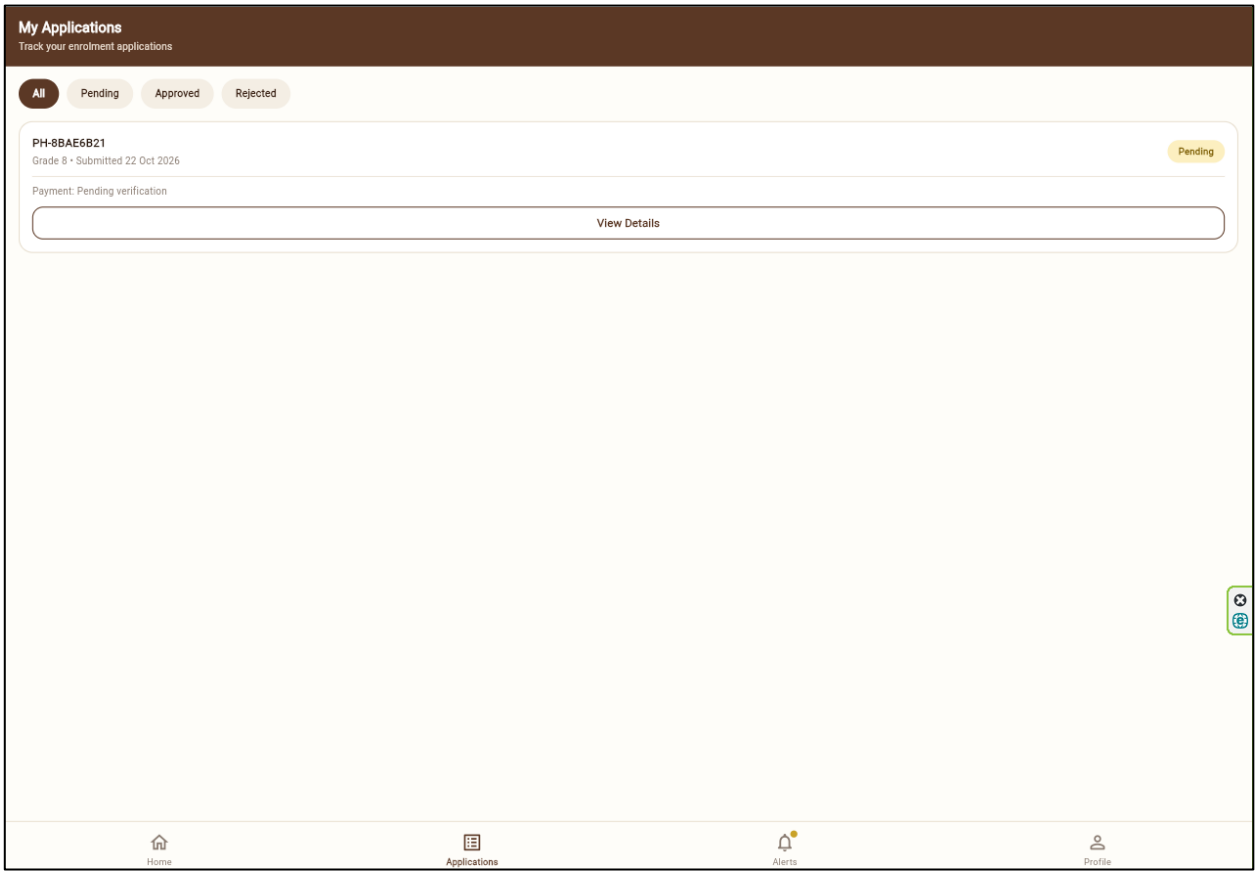


Figure 4.27: My Applications screen. The My Applications screen, which lists the parent's applications with status filters (All, Pending, Approved, Rejected) and a View Details button for each.

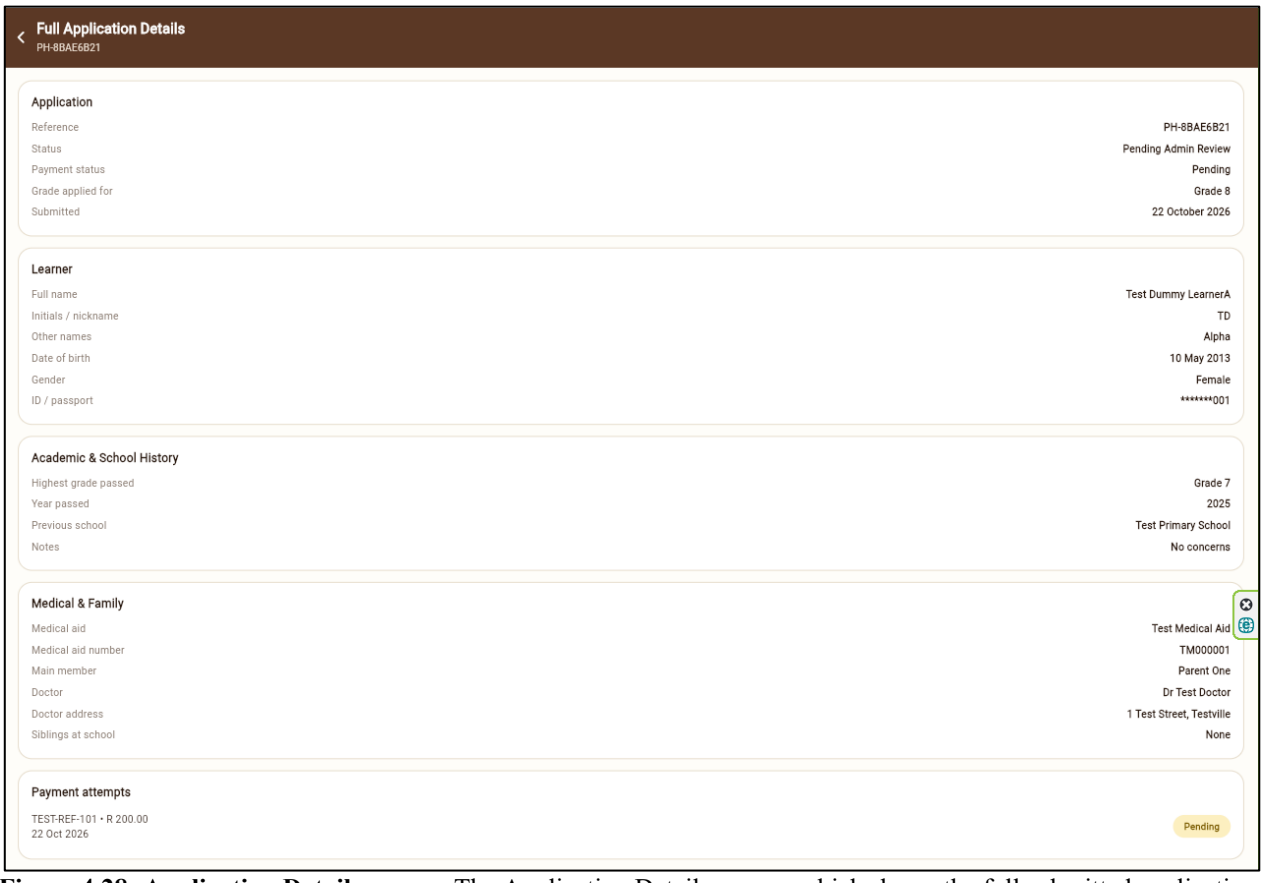


Figure 4.28: Application Details screen. The Application Details screen, which shows the full submitted application: its status, the learner details, the academic and medical information and the payment status (FR5).

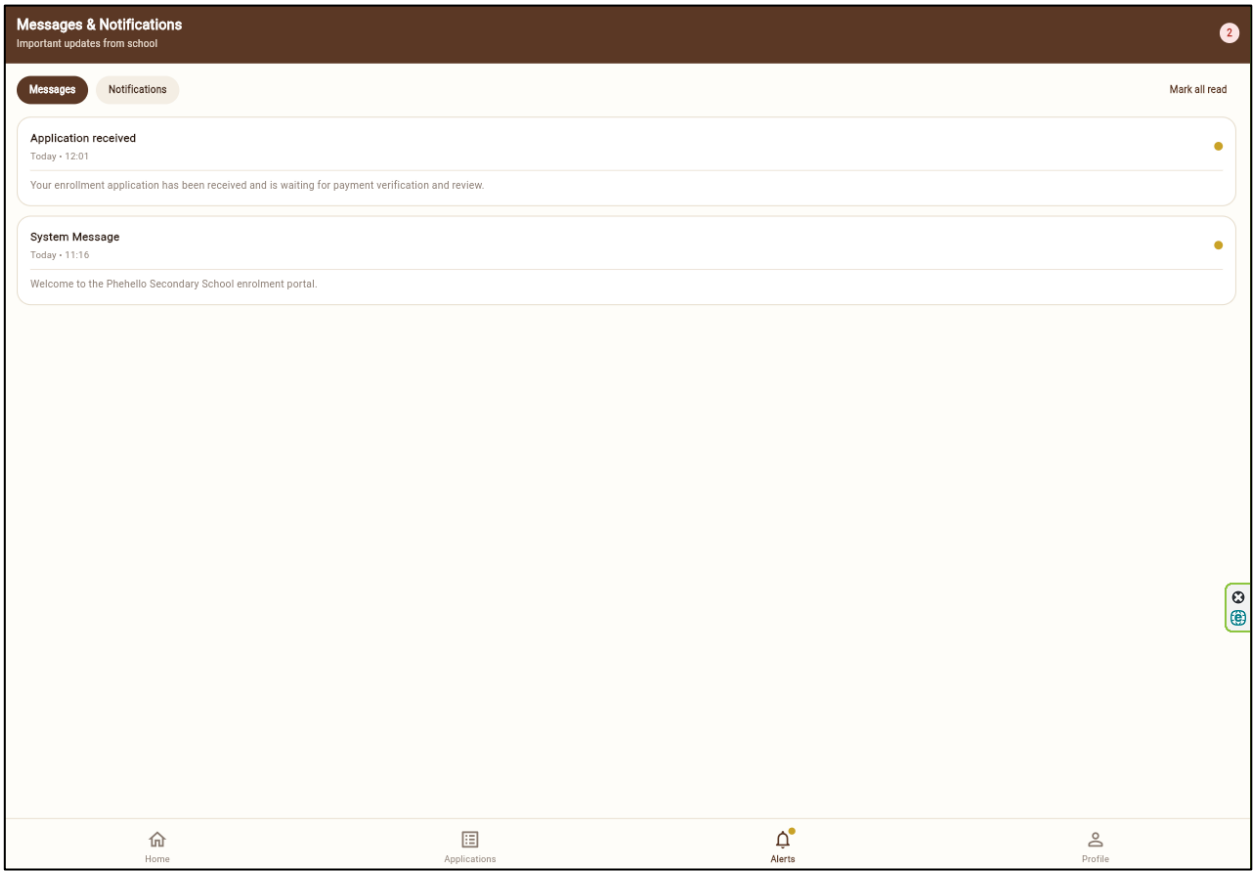


Figure 4.29: Messages and Notifications screen. The Messages and Notifications screen, where the parent reads messages and reminders sent by school staff, such as application updates and document reminders (FR10, FR11).

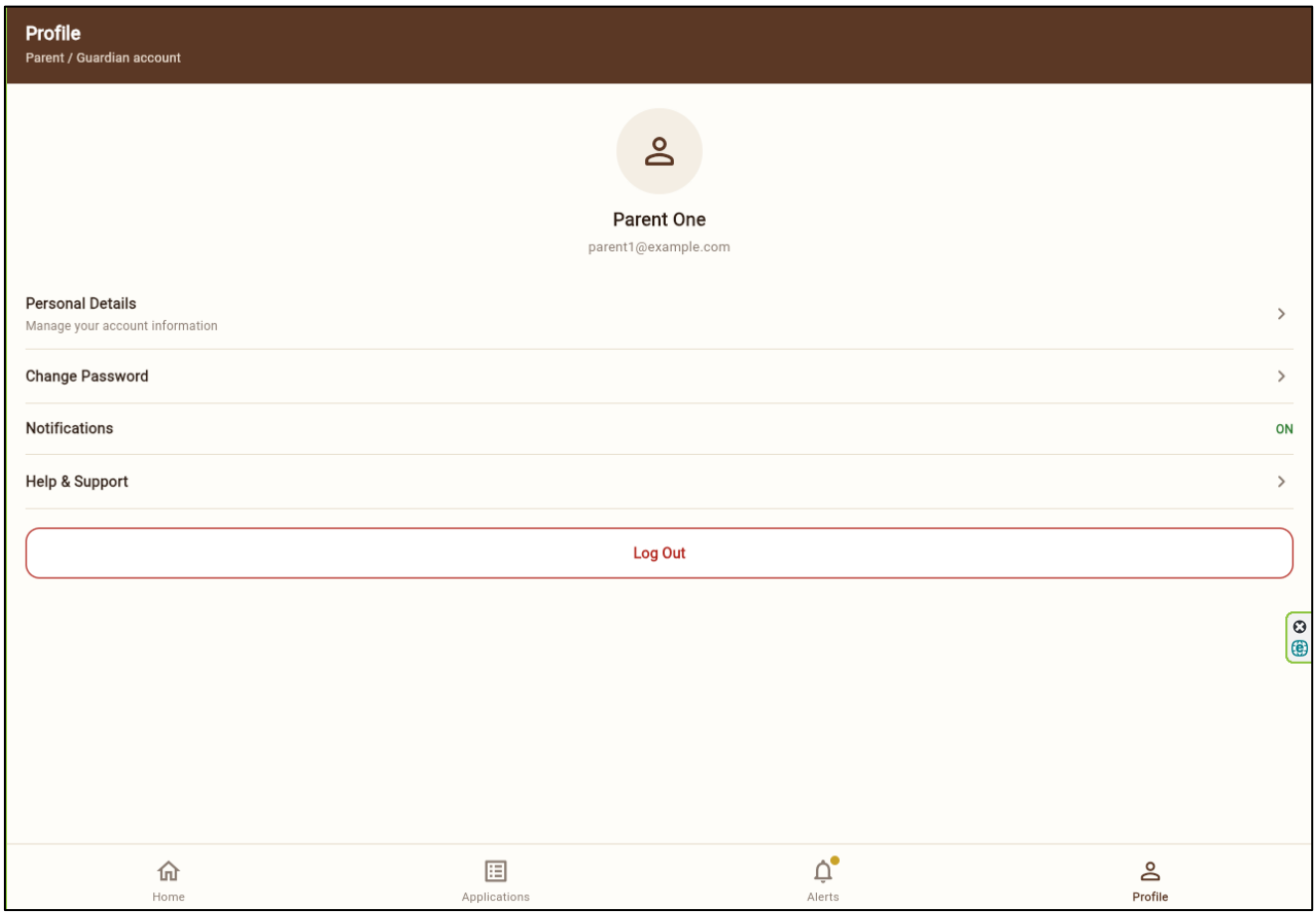


Figure 4.30: Profile screen. The Profile screen, which shows the parent's name and email with options for personal details, changing the password, notifications and Help & Support, and a Log Out button.

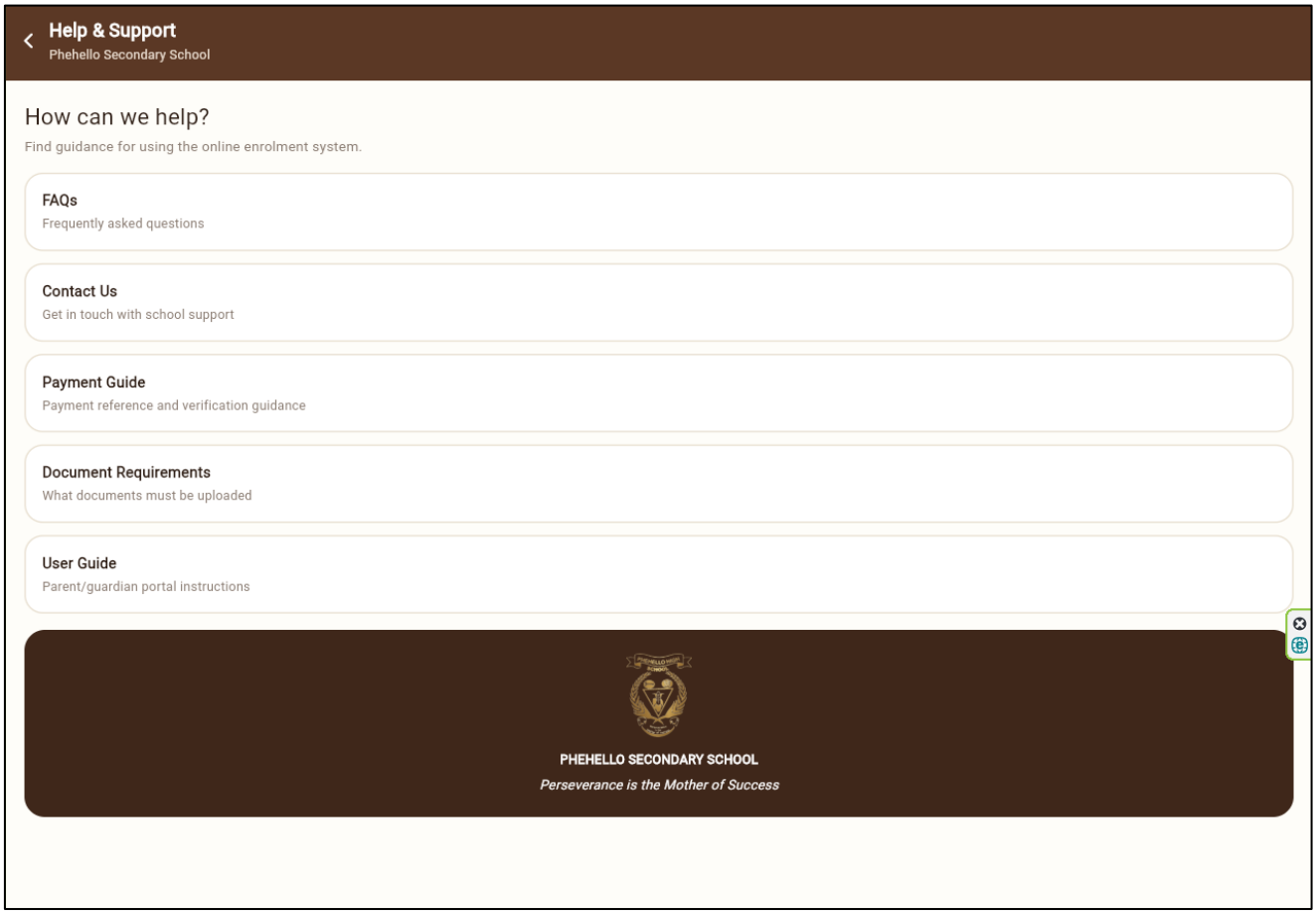


Figure 4.31: Help and Support screen. The Help & Support screen, which gives guidance on applying: frequently asked questions, how to contact the school, a payment guide, the required documents and a user guide.

Recording file name or link: Phehello Systems (Flutter recording)

Figure 4.32: Navigation through the application. A screen recording of navigation through the Flutter app, moving between the main screens such as sign-in, the application wizard, application status and notifications.

Function screenshots or short recordings

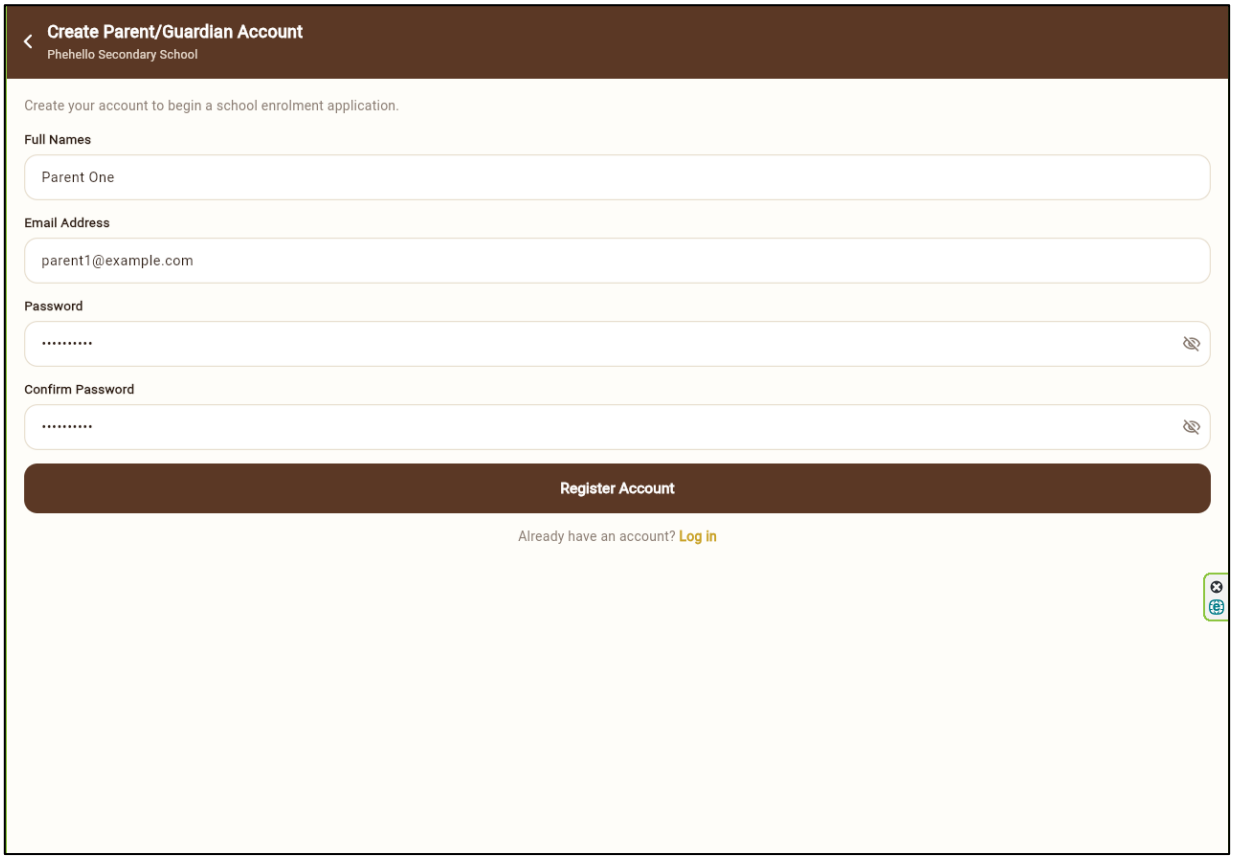


Figure 4.33: Registration. Registering a new parent account in the Flutter app: the Create Parent/Guardian Account form completed with the test parent's details (FR1).

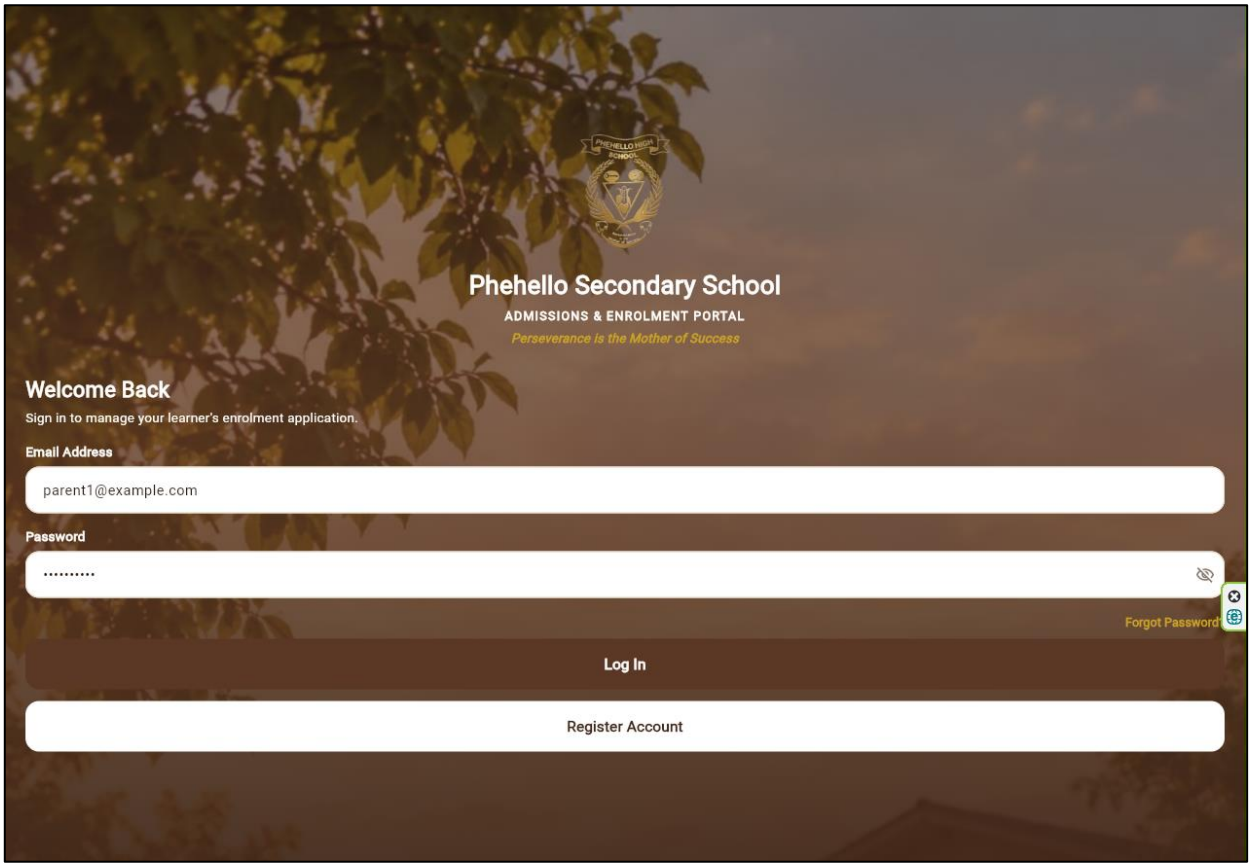


Figure 4.34: Sign in, (a) a successful sign in. Signing in to the Flutter app. Part (a) shows a parent signing in with their registered test account.

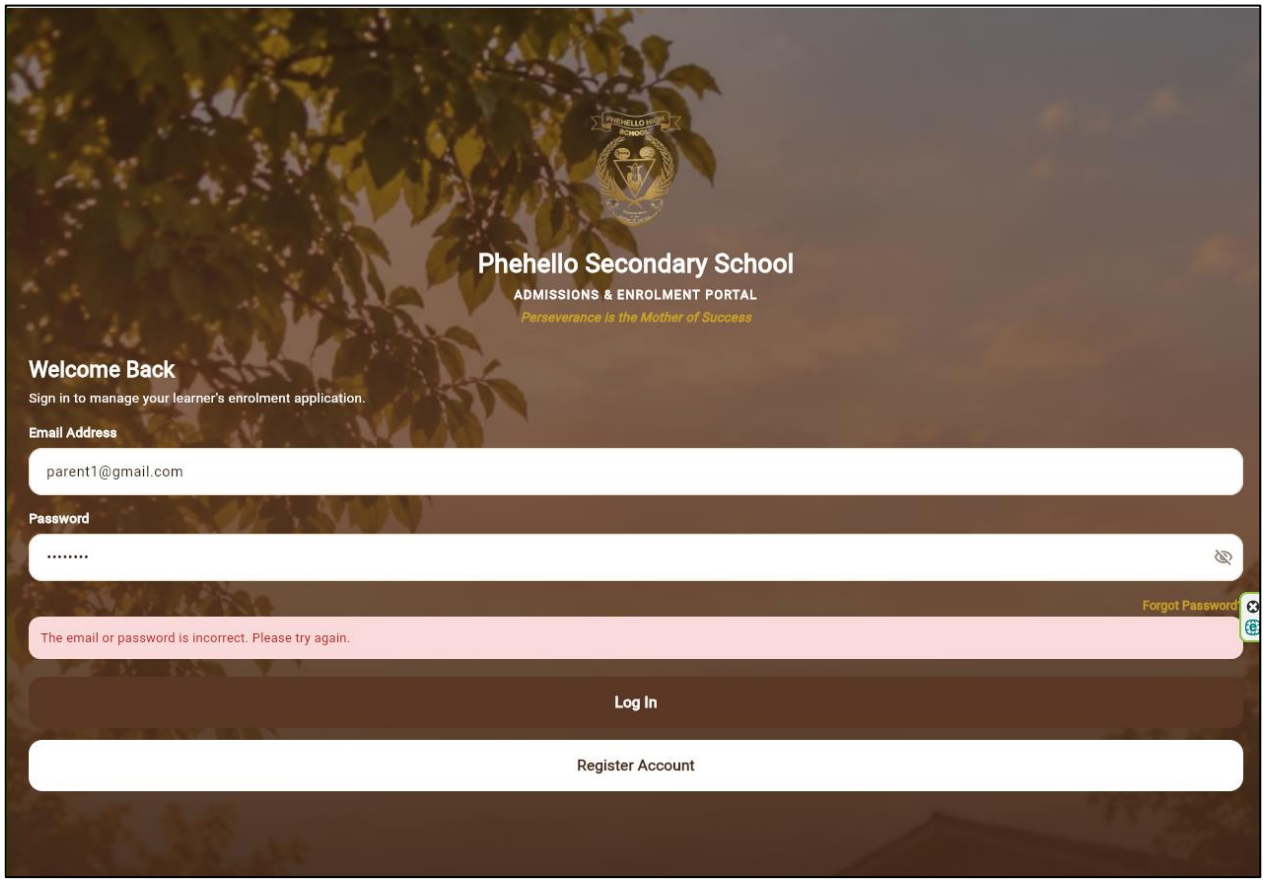


Figure 4.34: Sign in, (b) the error message for a wrong password. Signing in to the Flutter app. part (b) shows the error message displayed when the wrong password is entered.

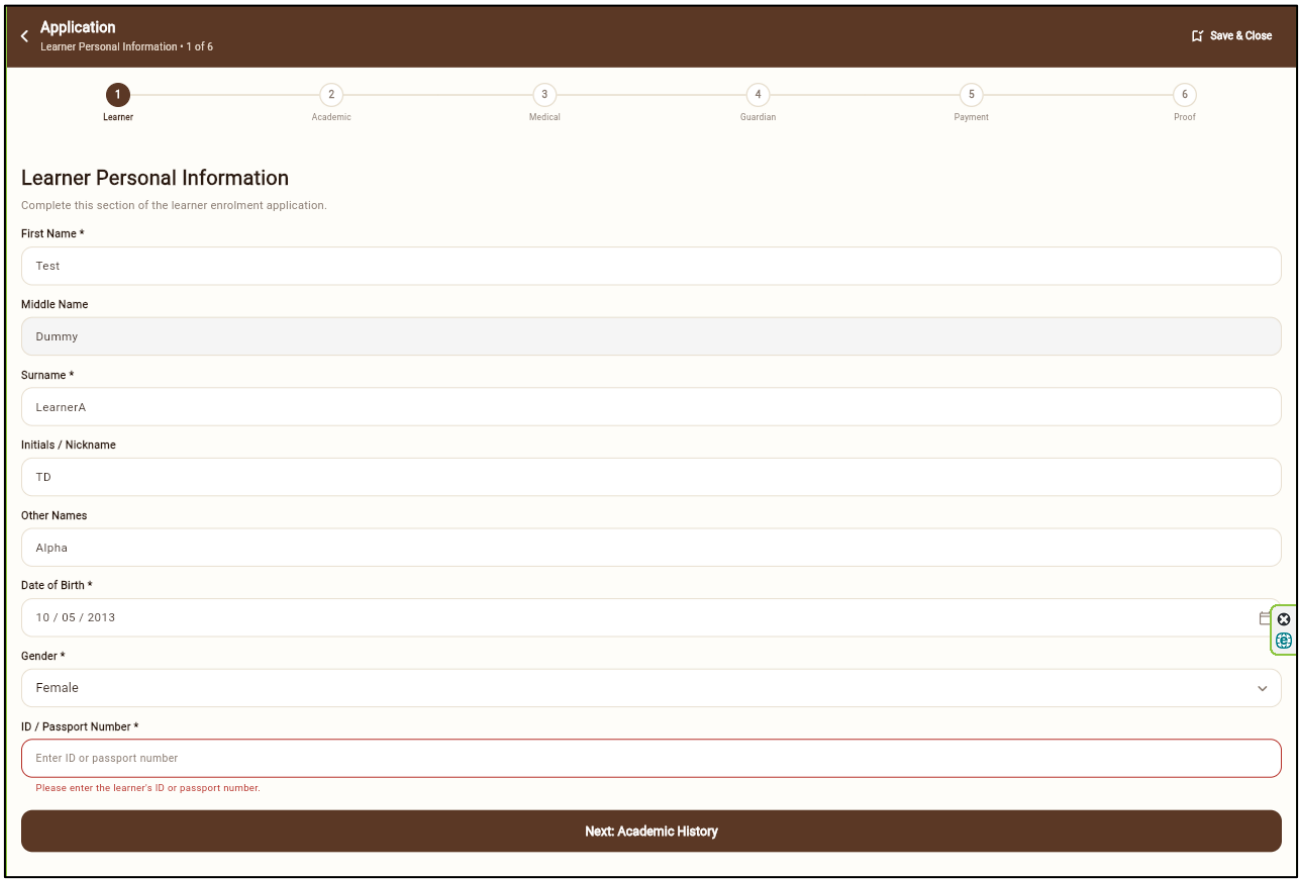


Figure 4.35: Form validation, (a) an empty required field with its message and an invalid email message. Form validation in the Flutter app. Part (a) shows an empty or invalid field with its error message.

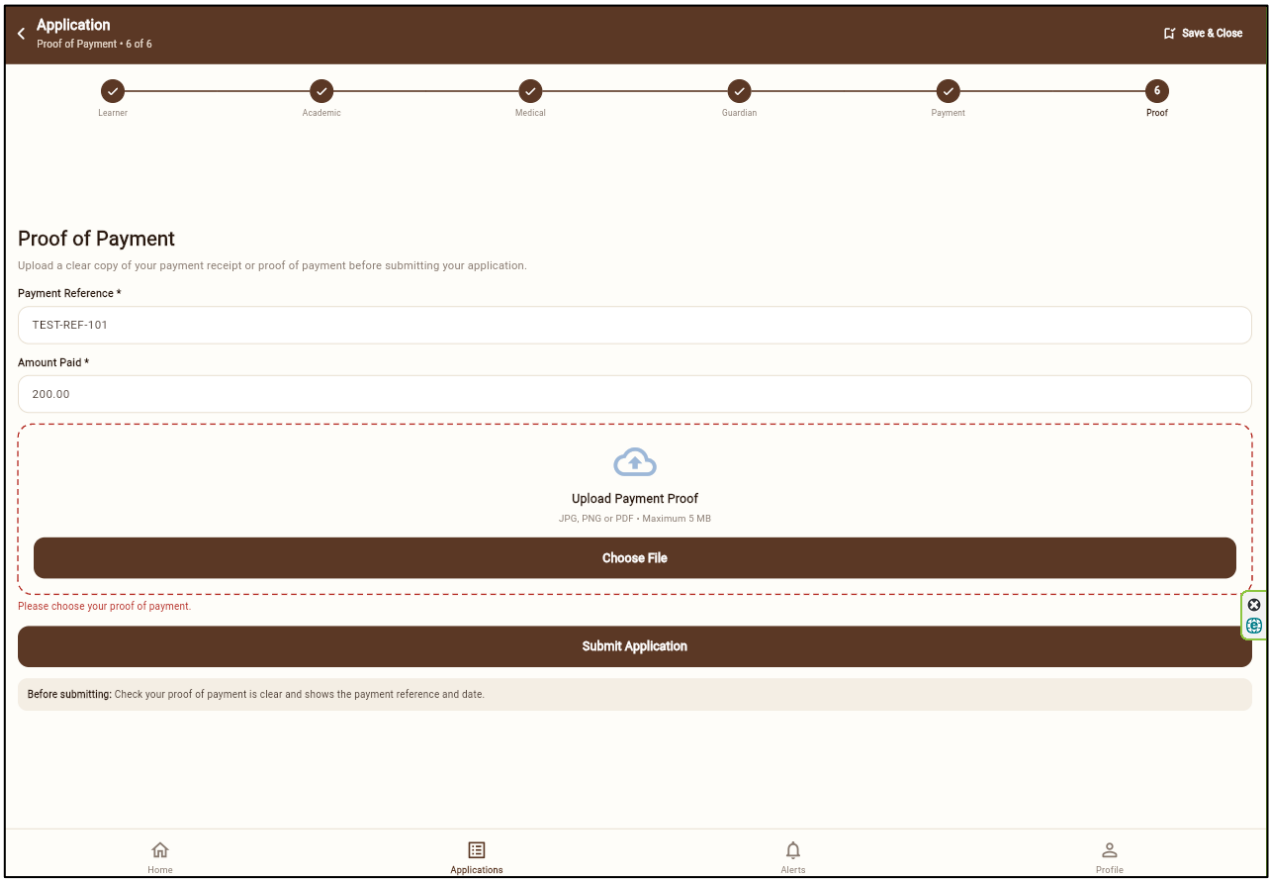


Figure 4.35: Form validation, (b) a missing document or proof of payment message. Form validation in the Flutter app. Part (b) shows the message displayed when a required document or proof of payment is missing.

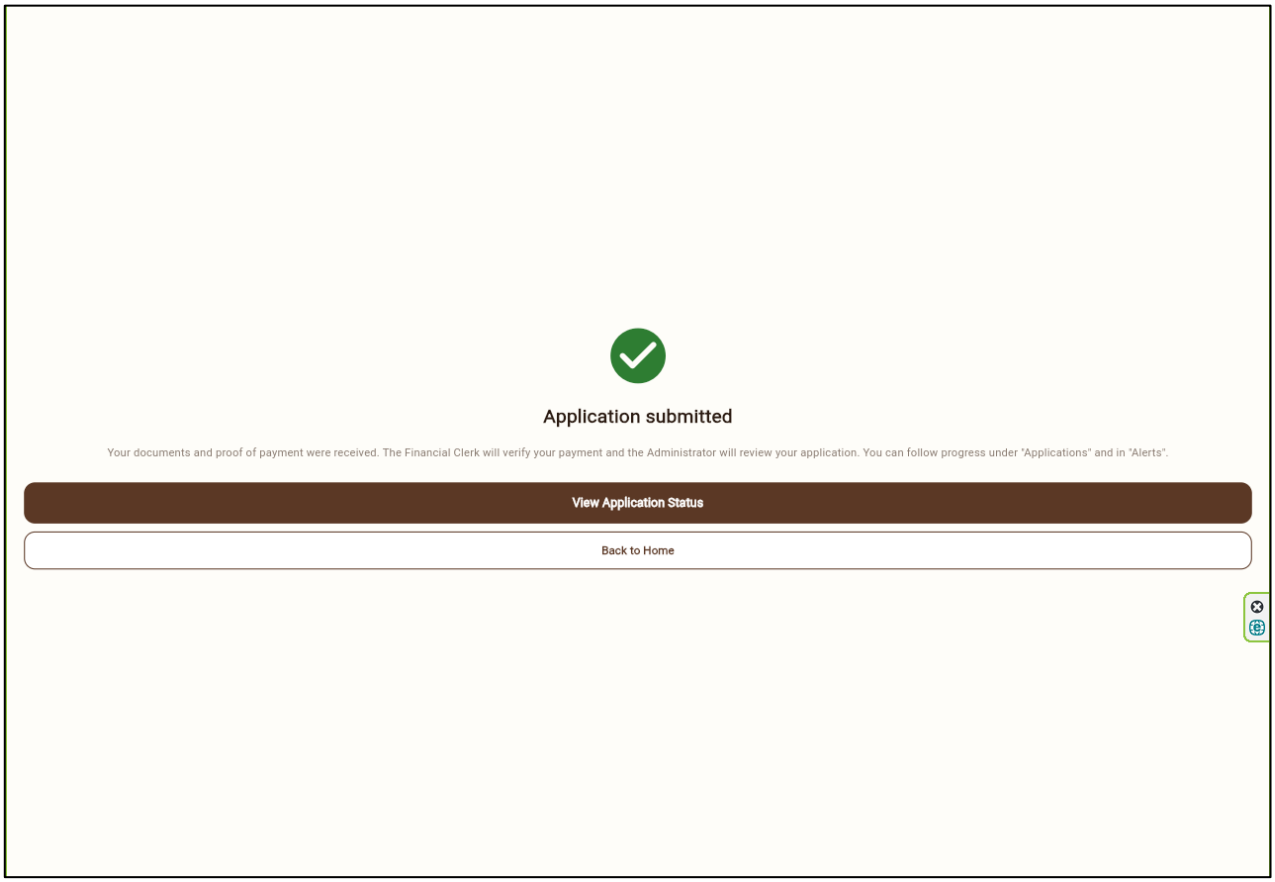


Figure 4.36: Application submission. The confirmation screen displayed after the parent submits an application. It confirms that the documents and proof of payment were received and offers View Application Status and Back to Home.

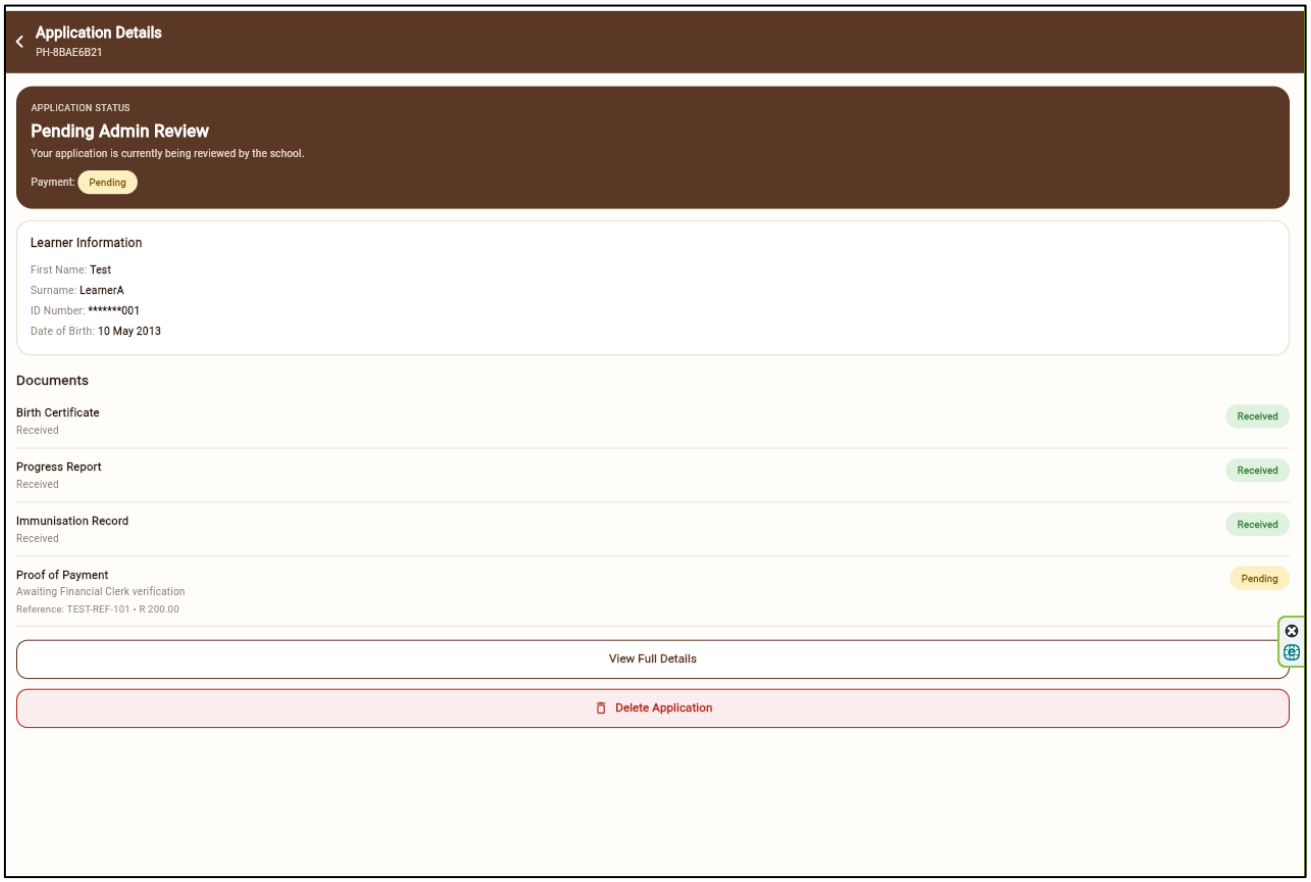
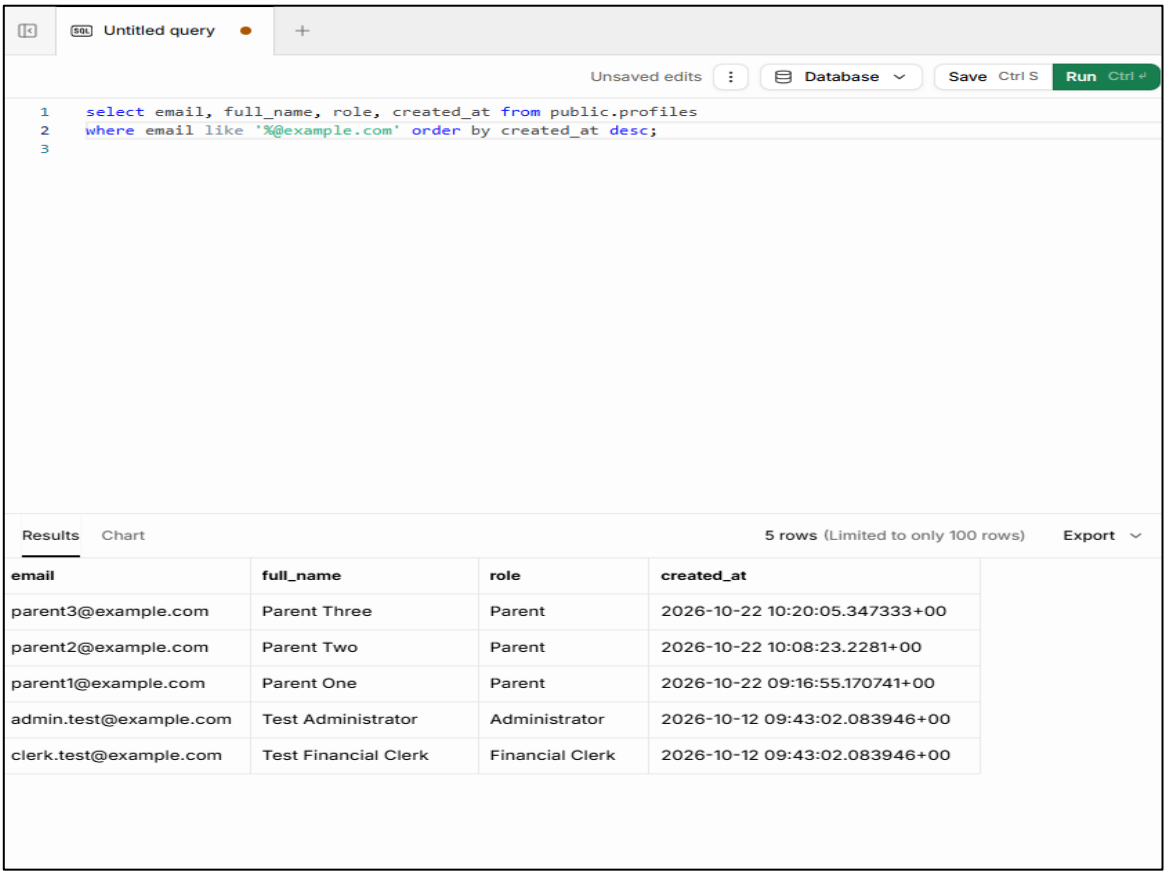


Figure 4.37: Status display. The status display: the parent's Application Details screen showing the application as Pending Admin Review, together with the payment status and the status of each document.

Supabase evidence



email	full_name	role	created_at
parent3@example.com	Parent Three	Parent	2026-10-22 10:20:05.347333+00
parent2@example.com	Parent Two	Parent	2026-10-22 10:08:23.2281+00
parent1@example.com	Parent One	Parent	2026-10-22 09:16:55.170741+00
admin.test@example.com	Test Administrator	Administrator	2026-10-12 09:43:02.083946+00
clerk.test@example.com	Test Financial Clerk	Financial Clerk	2026-10-12 09:43:02.083946+00

Figure 4.39: Profile created by registration. A query on the profiles table showing that registering in the app created a profile with the role Parent for each parent, alongside the test staff accounts.

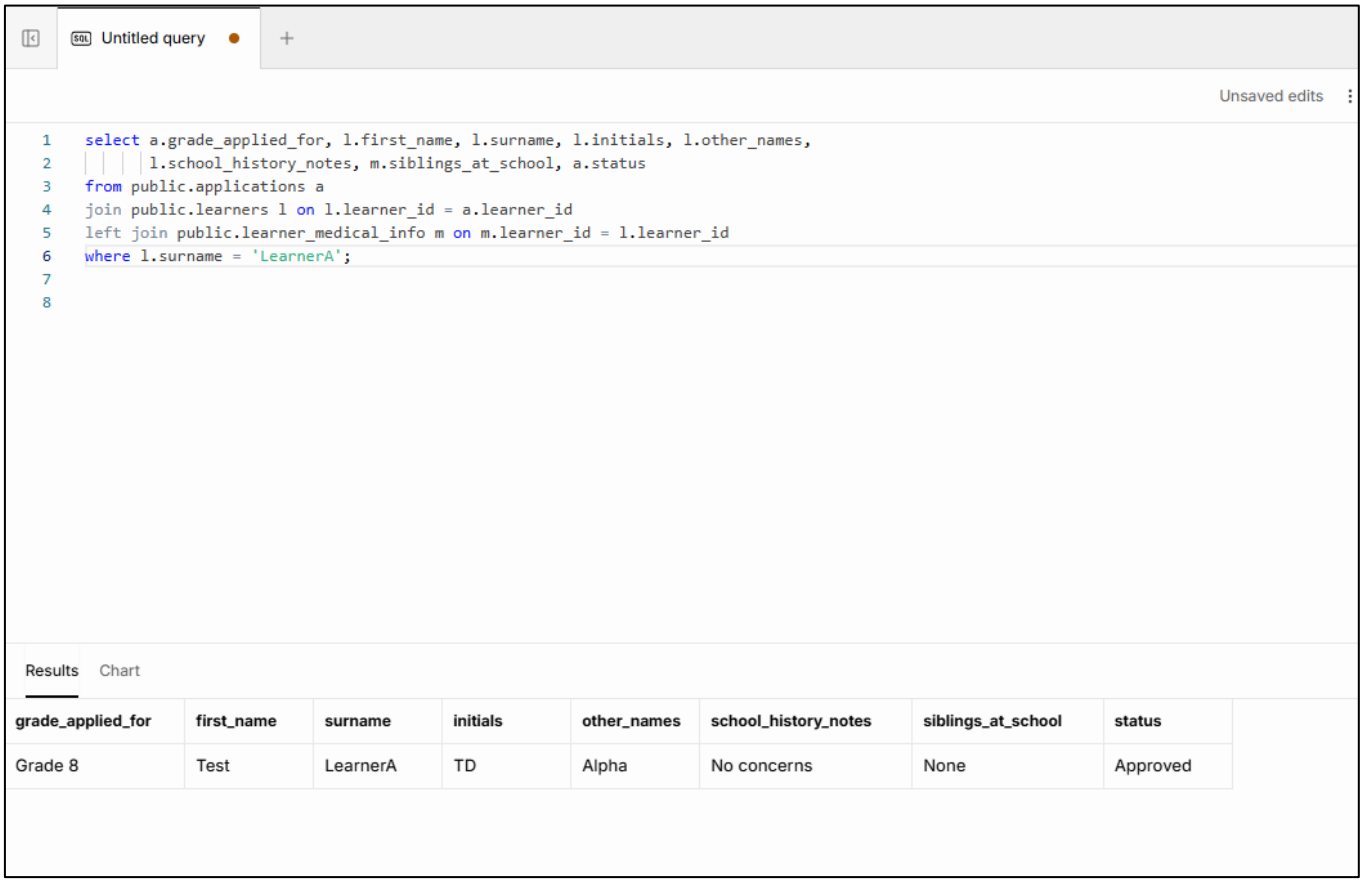


Figure 4.40: Application data saved by the app. A query joining applications, learners and learner_medical_info that shows the data the app saved: grade applied for, the learner's names and initials, school-history notes, siblings at school and the application status.

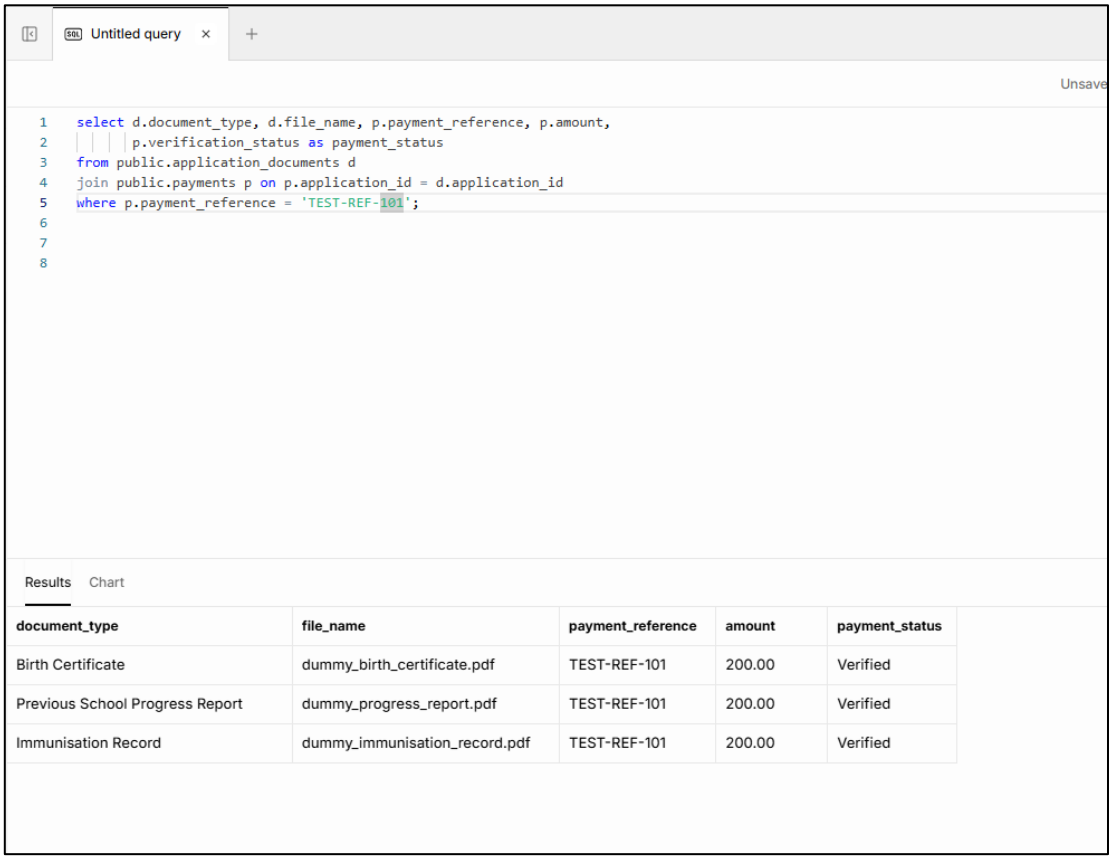


Figure 4.41: Documents and payment saved. A query joining application_documents and payments showing that the uploaded supporting documents (birth certificate, progress report and immunisation record) and the payment reference, amount and payment status were saved.

Untitled query

Unsaved edits

```
select case when name like '%/payment-proofs/%' then 'Proof of payment'
            else 'Supporting document' end as file_kind,
       split_part(name, '/', 1) as parent_id,
       regexp_replace(name, '^[^/]+/', '') as path_inside_parent_folder,
       created_at
from storage.objects
where bucket_id = 'enrolment-documents'
  and split_part(name, '/', 1) =
      (select id::text from auth.users where email = 'parent1@example.com')
order by created_at desc;
```

Results Chart

file_kind	parent_id	path_inside_parent_folder	created_at
Proof of payment	667b5aa6-5197-4b54-9c8c-4a2fa13745a0	payment-proofs/ead72eef-ba73-4e12-890f-12fda51b1ddd/proof_1792664332	2026-10-22 10:18:54.979398+00
Supporting document	667b5aa6-5197-4b54-9c8c-4a2fa13745a0	ead72eef-ba73-4e12-890f-12fda51b1ddd/documents/transferLetter_17926643	2026-10-22 10:18:54.359627+00
Supporting document	667b5aa6-5197-4b54-9c8c-4a2fa13745a0	ead72eef-ba73-4e12-890f-12fda51b1ddd/documents/immunisationRecord_179	2026-10-22 10:18:53.852127+00
Supporting document	667b5aa6-5197-4b54-9c8c-4a2fa13745a0	ead72eef-ba73-4e12-890f-12fda51b1ddd/documents/progressReport_179266	2026-10-22 10:18:53.309166+00
Supporting document	667b5aa6-5197-4b54-9c8c-4a2fa13745a0	ead72eef-ba73-4e12-890f-12fda51b1ddd/documents/birthCertificate_1792664	2026-10-22 10:18:52.794147+00
Proof of payment	667b5aa6-5197-4b54-9c8c-4a2fa13745a0	payment-proofs/8bae6b21-2e87-4c05-8968-53248e3ba03b/proof_17926633	2026-10-22 10:01:46.984141+00
Supporting document	667b5aa6-5197-4b54-9c8c-4a2fa13745a0	8bae6b21-2e87-4c05-8968-53248e3ba03b/documents/immunisationRecord_	2026-10-22 10:01:46.471673+00
Supporting document	667b5aa6-5197-4b54-9c8c-4a2fa13745a0	8bae6b21-2e87-4c05-8968-53248e3ba03b/documents/progressReport_1792	2026-10-22 10:01:45.953222+00
Supporting document	667b5aa6-5197-4b54-9c8c-4a2fa13745a0	8bae6b21-2e87-4c05-8968-53248e3ba03b/documents/birthCertificate_1792	2026-10-22 10:01:45.403571+00

Figure 4.42: Files uploaded to Storage. A query on the stored files showing what the app uploaded to the enrolment-documents bucket: supporting documents and proof-of-payment files, each kept in a folder belonging to its parent.

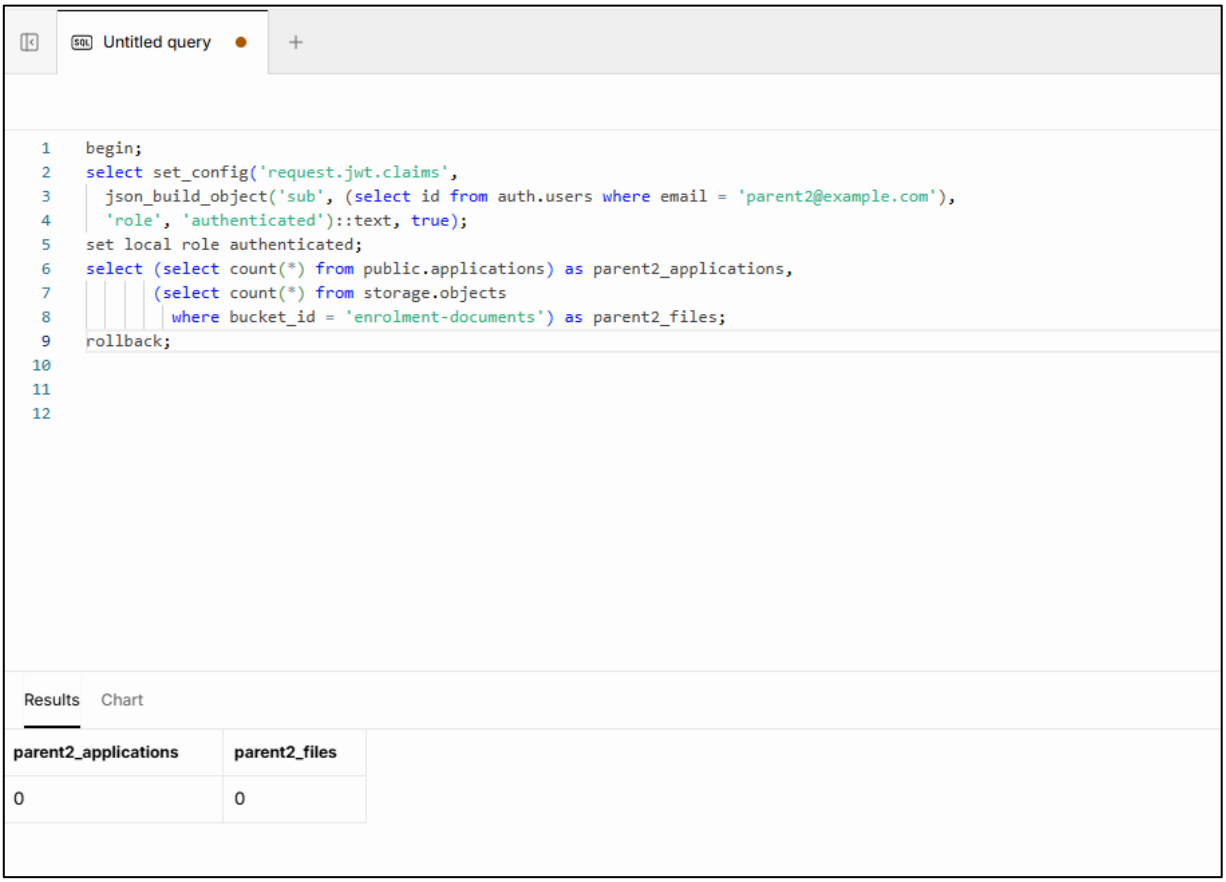


Figure 4.43: Access restriction, the SQL result showing 0 rows for that parent. An access-restriction test run as Parent Two. The query returns 0 applications and 0 stored files, showing that records and files belonging to other parents are not visible to this parent.

Develop the ASP.NET Web Application

Approved Web Application Requirements

Web function	Related requirement or use case	Required page
Authenticate staff and provide the appropriate role-based access path for the Administrator or Financial Clerk	Sign In (NFR1, NFR6)	Login
Provide access to payment-verification functions and a summary of payment-related activity	FR6 – Verify Proof of Payment	Financial Clerk Dashboard
Display the payment information and supporting details needed to verify the parent's proof of payment	FR6 – Verify proof of payment	Payment Details
View application summary counts, application trend and recent applications	FR13 – View Application Count	Administrator Dashboard
View all submitted applications with search and filters	FR7 – View Submitted Applications	Applications
View applications filtered by status	FR7	Approved / Rejected / Pending Applications
View learner and application details and documents; approve, reject or request corrections	FR7, FR8	Application Review
Add and view internal notes on an application	FR9 – Leave Notes on Application	Application Notes
Message a parent about missing or incorrect documents	FR10 – Message Parent About Documents	Message Parent
Send deadline reminders to parents	FR11 – Send Deadline Reminders	Send Reminder
Preview and generate the confirmation letter for an approved or rejected application	FR12 – Generate Confirmation Letter	Confirmation Letter
Review proof of payment next to the matching school payment record and verify it	FR6 – Verify Proof of Payment	Payment Verification
Show the uploaded proof of payment so that the clerk can approve or reject it (opened by clicking View from Payment Verification)	FR6 – Verify Proof of Payment	Proof of Payment
List payments the Financial Clerk has already verified, kept for their own record-keeping	FR6 – Verify Proof of Payment	Verified Payments

Initial Function Checks

Function or check	Result	Action required
Web application launches successfully	Pass	None
Main pages display correctly	Pass	None
Navigation works	Pass	None
Forms validate user input	Pass	None
Web application connects to Supabase	Pass	None
Required records can be retrieved	Pass	None
Records can be created or updated where required	Pass	None
Authentication works, where required	Pass	None
User access restrictions work as intended	Pass	None

Commit Your Work and Update Project Records

Document or tool	What to update	Status
GitHub	Flutter project committed to the mobile/ folder, with meaningful commit messages from each contributor	Done
Microsoft Project	Mobile development tasks and progress recorded in Project Schedule.mpp	Done
Project Progress Tracker	Completed tasks, responsible group members, progress and challenges	Done
SRS	Any approved requirement or design change recorded in the Requirements Change Log	Done

Changes made during implementation (for the SRS change log)

Requirement / Item	Original assumption	Change / Status	Reason
Dashboard bar charts (lecturer feedback)	The dashboard chart did not clearly show what it measured	Charts now show the days of the week on the x-axis and the number of applications or payments submitted on the y-axis, and the bars add up to the dashboard totals	Advised by the lecturer so the graphs reflect the applications made

Evidence

Web application screenshots

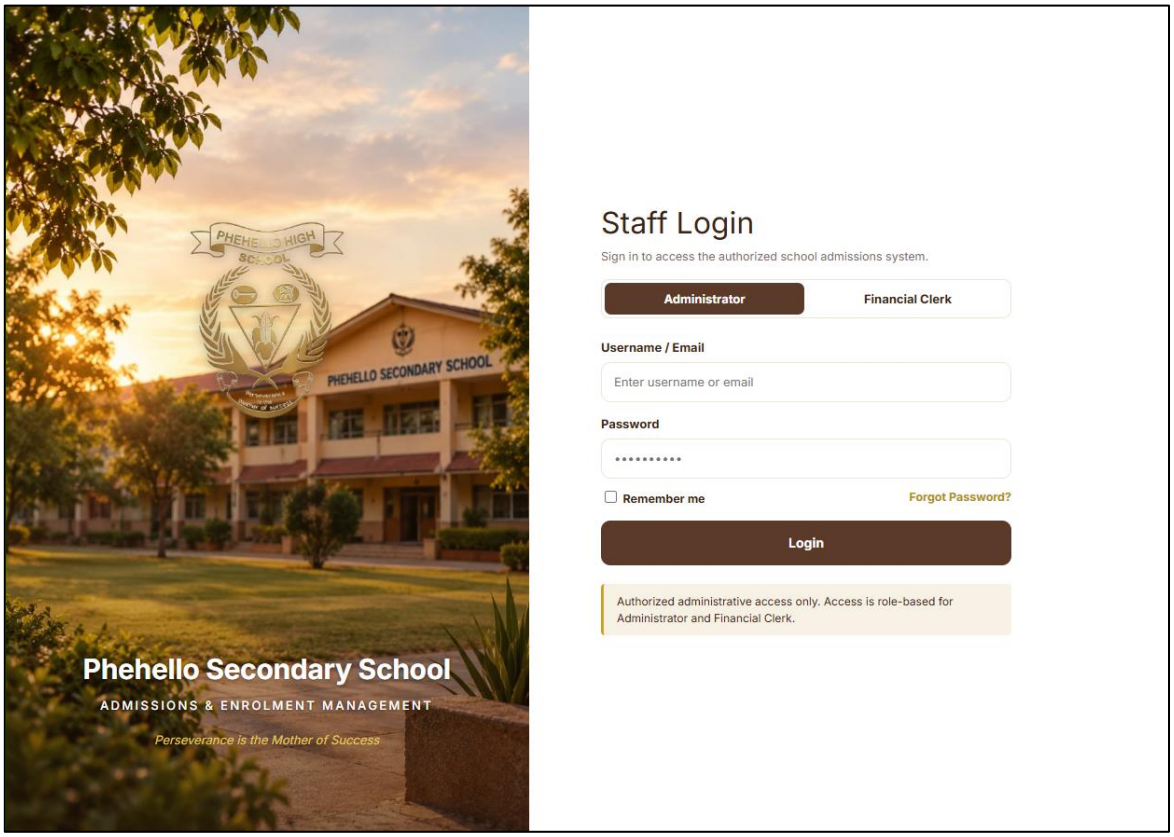


Figure 4.44: Login page. The staff Login page of the ASP.NET web app. Staff choose the Administrator or Financial Clerk tab and sign in with their email and password; access is limited to authorised staff (NFR1, NFR6).

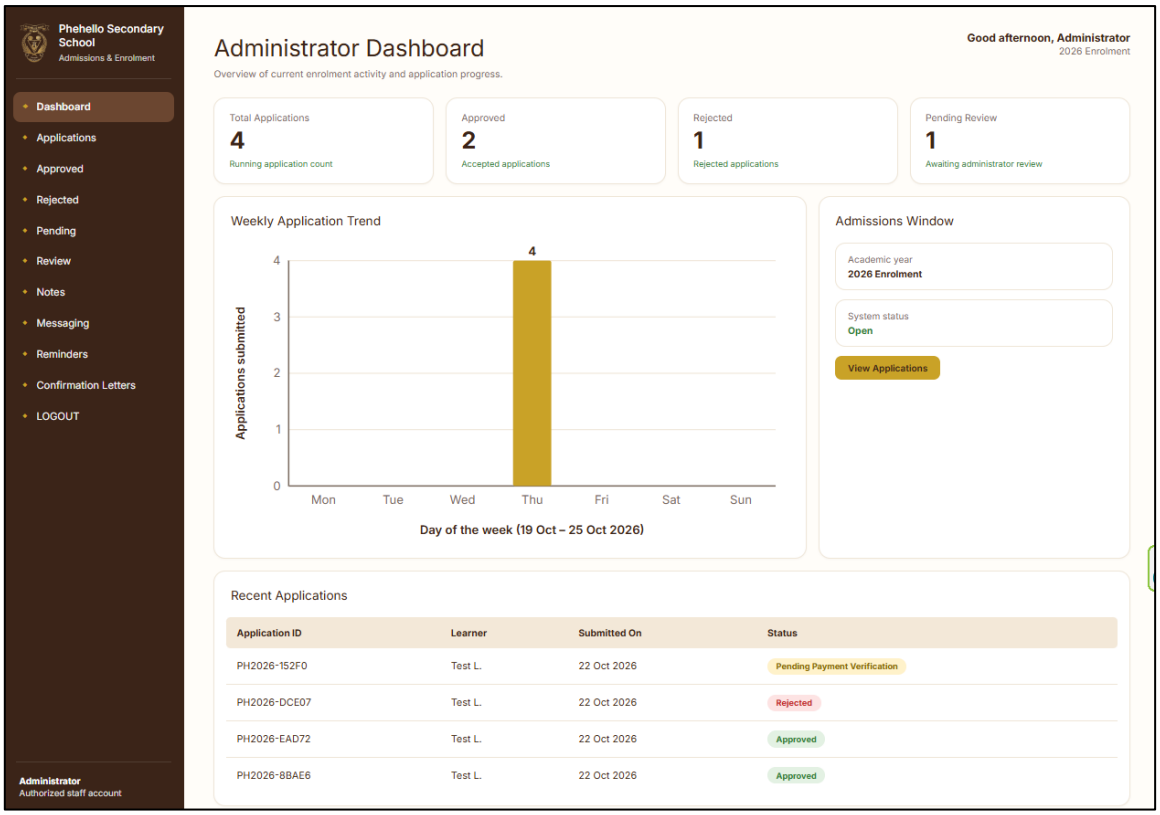


Figure 4.45: Administrator Dashboard. The Administrator Dashboard, showing the counts of total, approved, rejected and pending-review applications, a weekly application trend chart and a table of recent applications (FR13).

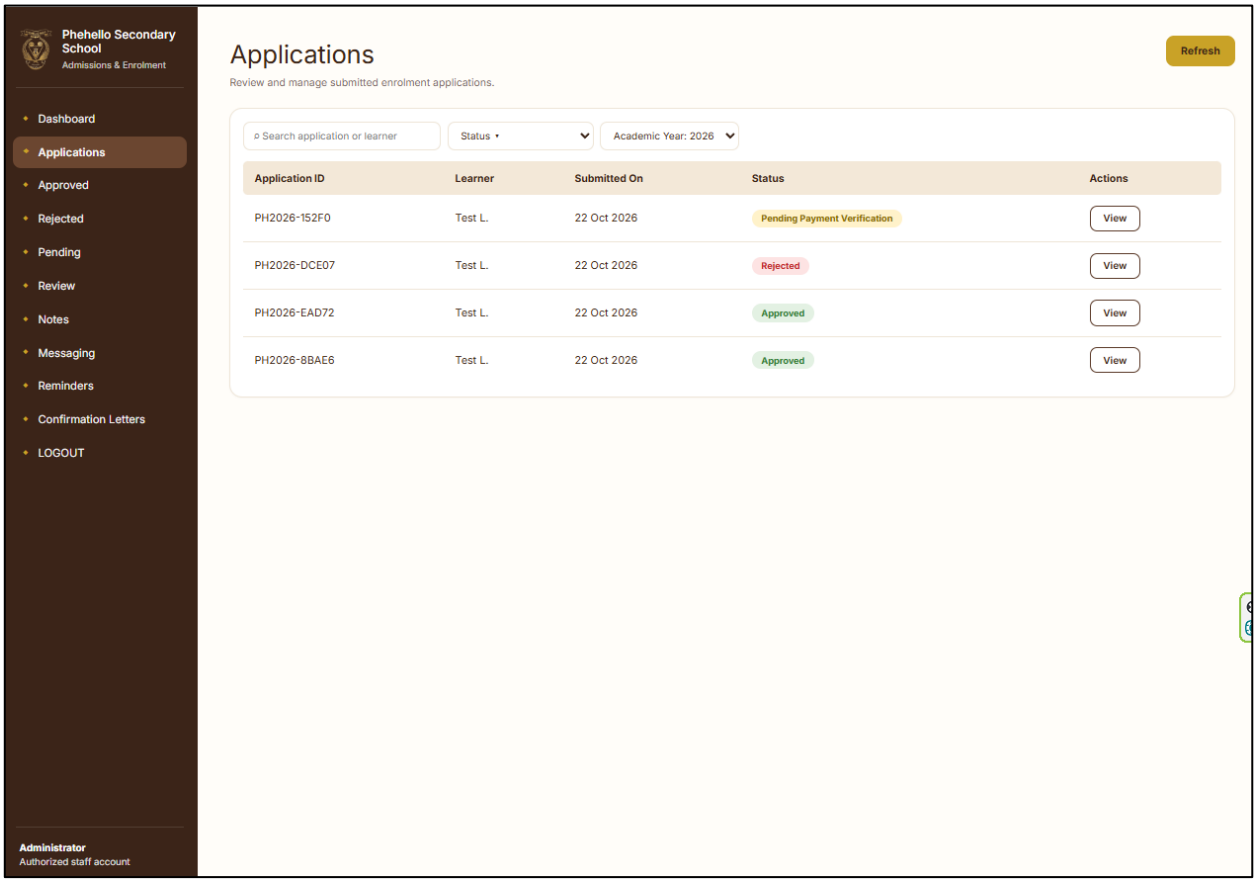


Figure 4.46: Applications page. The Applications page, which lists every submitted application with its learner, submission date and status, together with search and filter options and a View button (FR7).

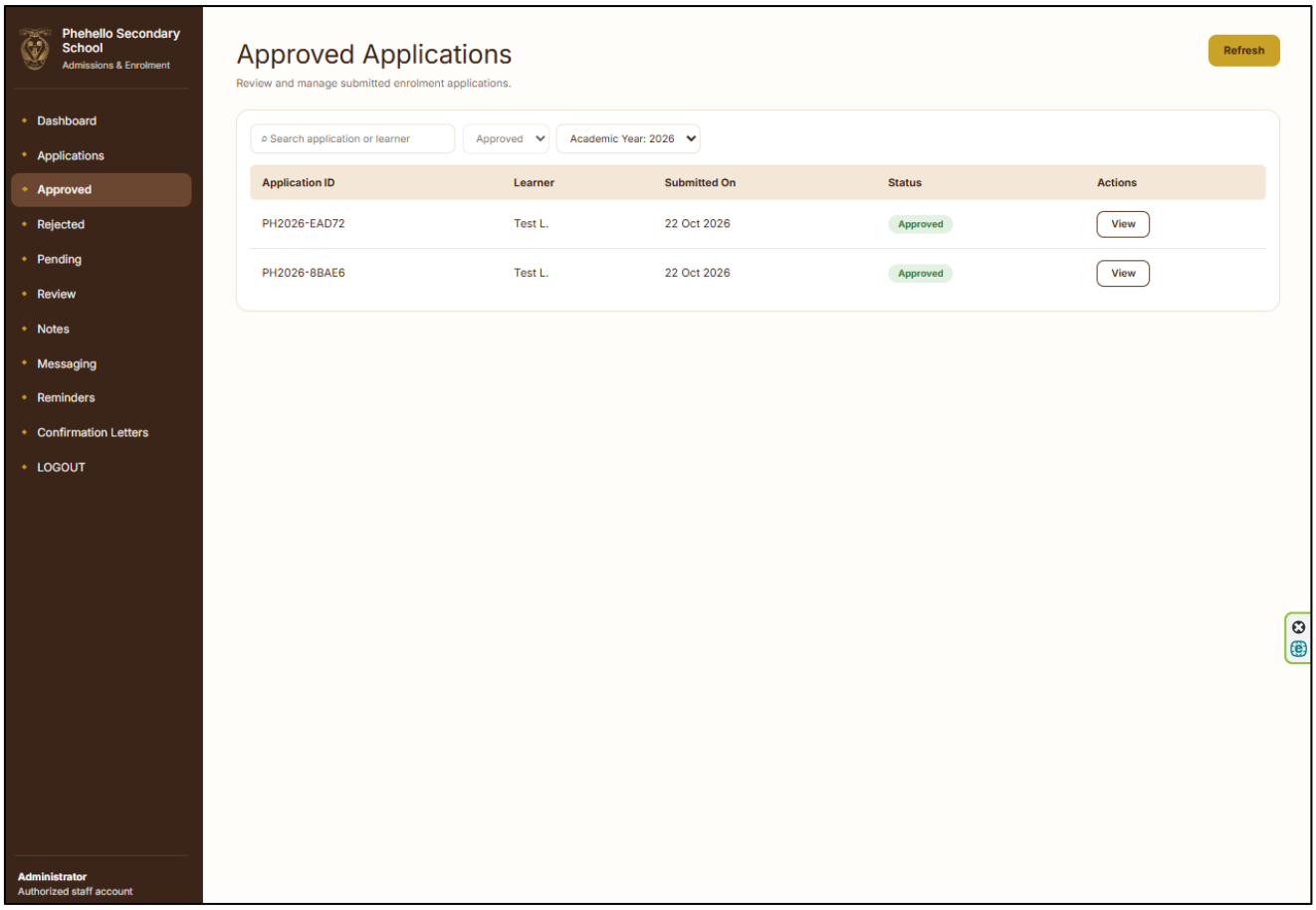


Figure 4.47: Approved Applications. The Approved Applications page, which lists only the applications the Administrator has approved.

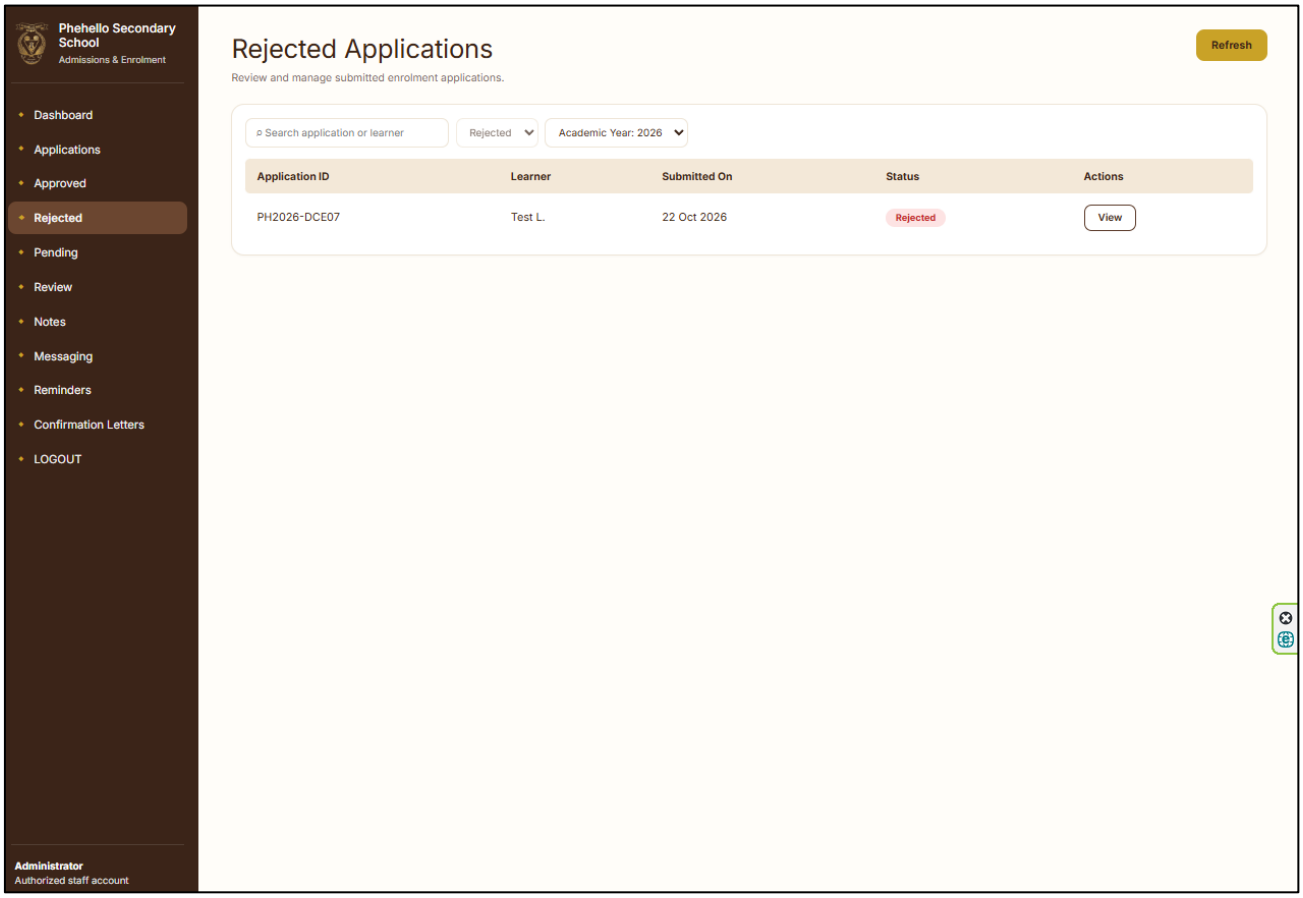


Figure 4.48: Rejected Applications. The Rejected Applications page, which lists only the applications the Administrator has rejected.

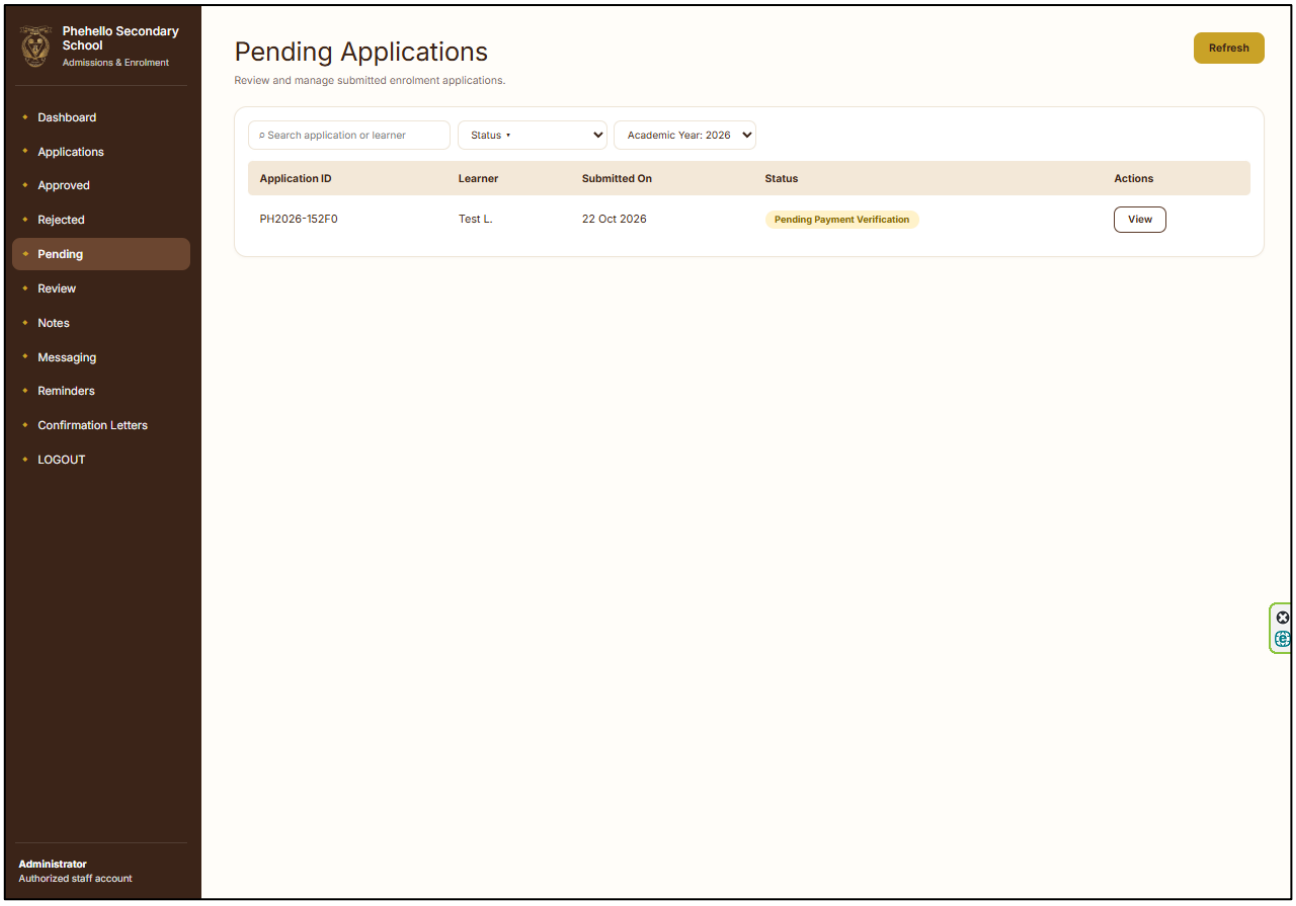


Figure 4.49: Pending Applications. The Pending Applications page, which lists the applications that are still waiting for a decision.

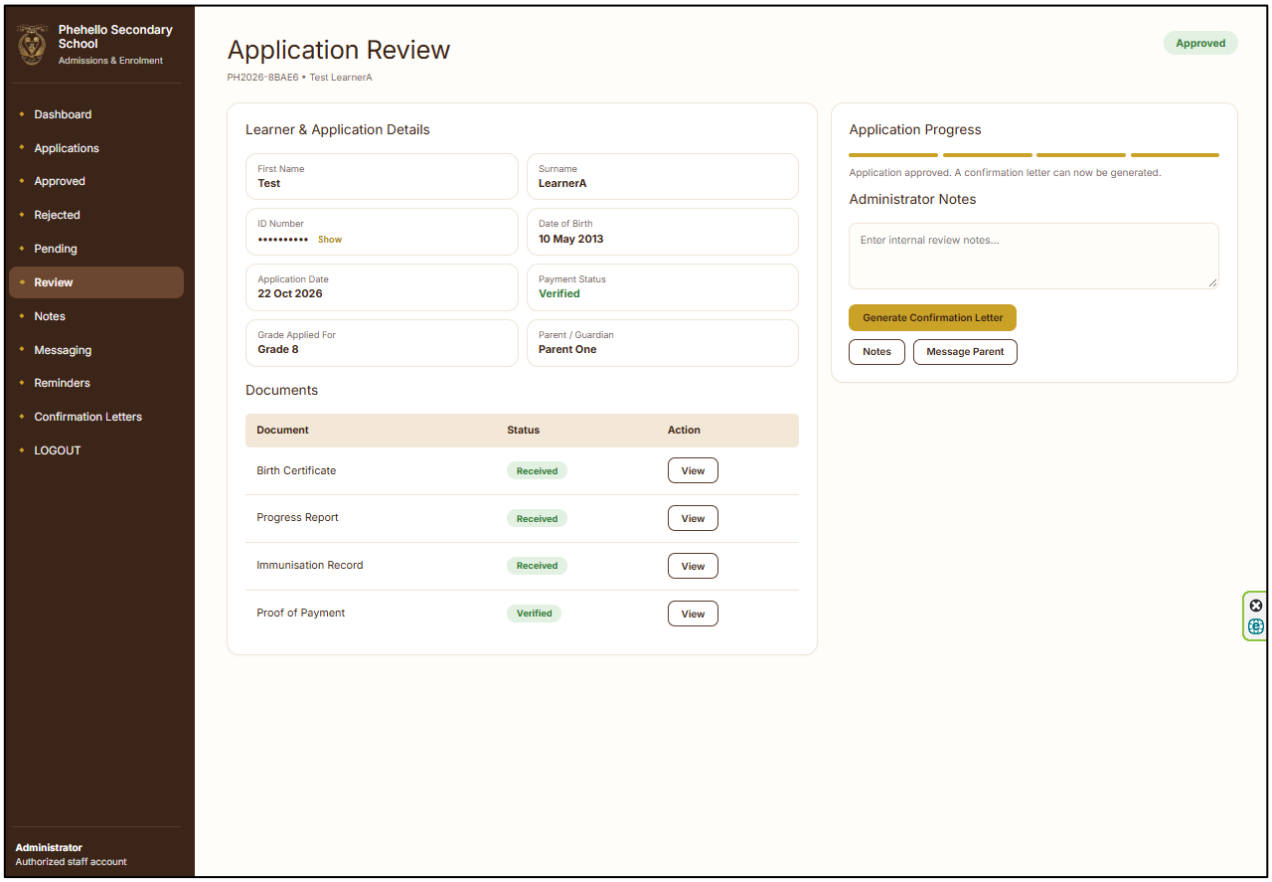


Figure 4.50: Application Review. The Application Review page, where the Administrator sees the learner and application details and the uploaded documents, and can approve, request corrections on or reject the application (FR7, FR8).

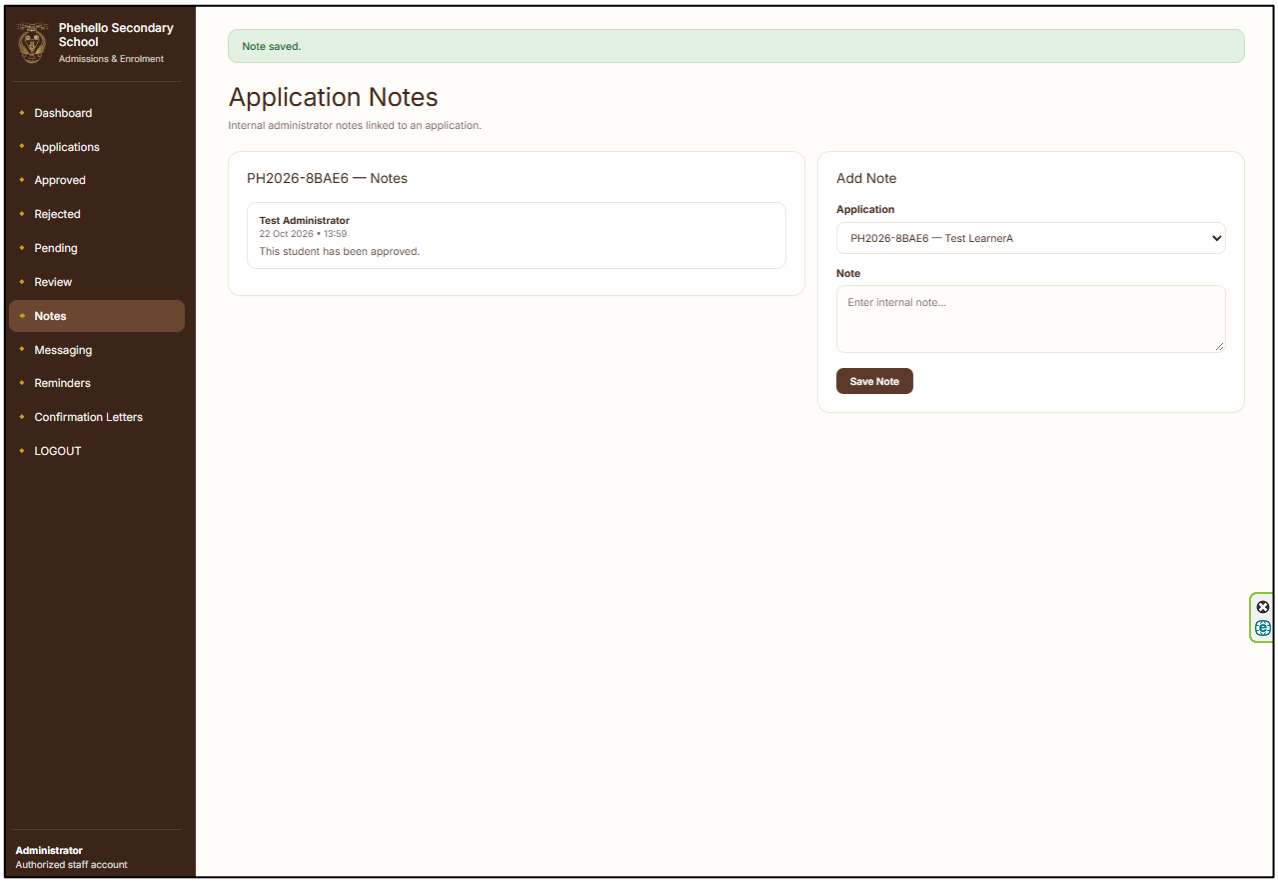


Figure 4.51: Application Notes. The Application Notes page, where the Administrator adds internal notes to an application and reads the notes already saved (FR9).

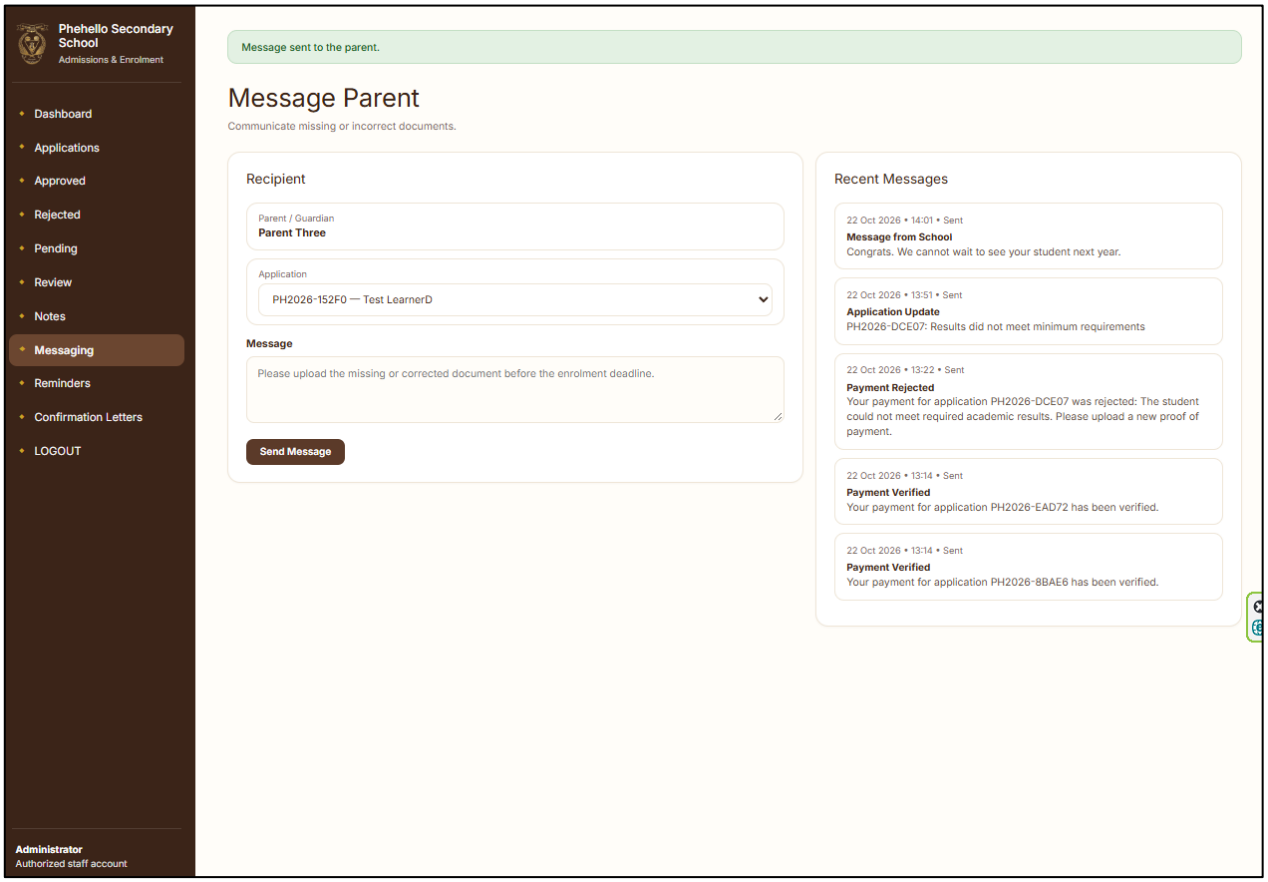


Figure 4.52: Message Parent. The Message Parent page, where the Administrator writes a message about missing or incorrect documents to the parent of the selected application. Sent messages are listed under Recent Messages (FR10).

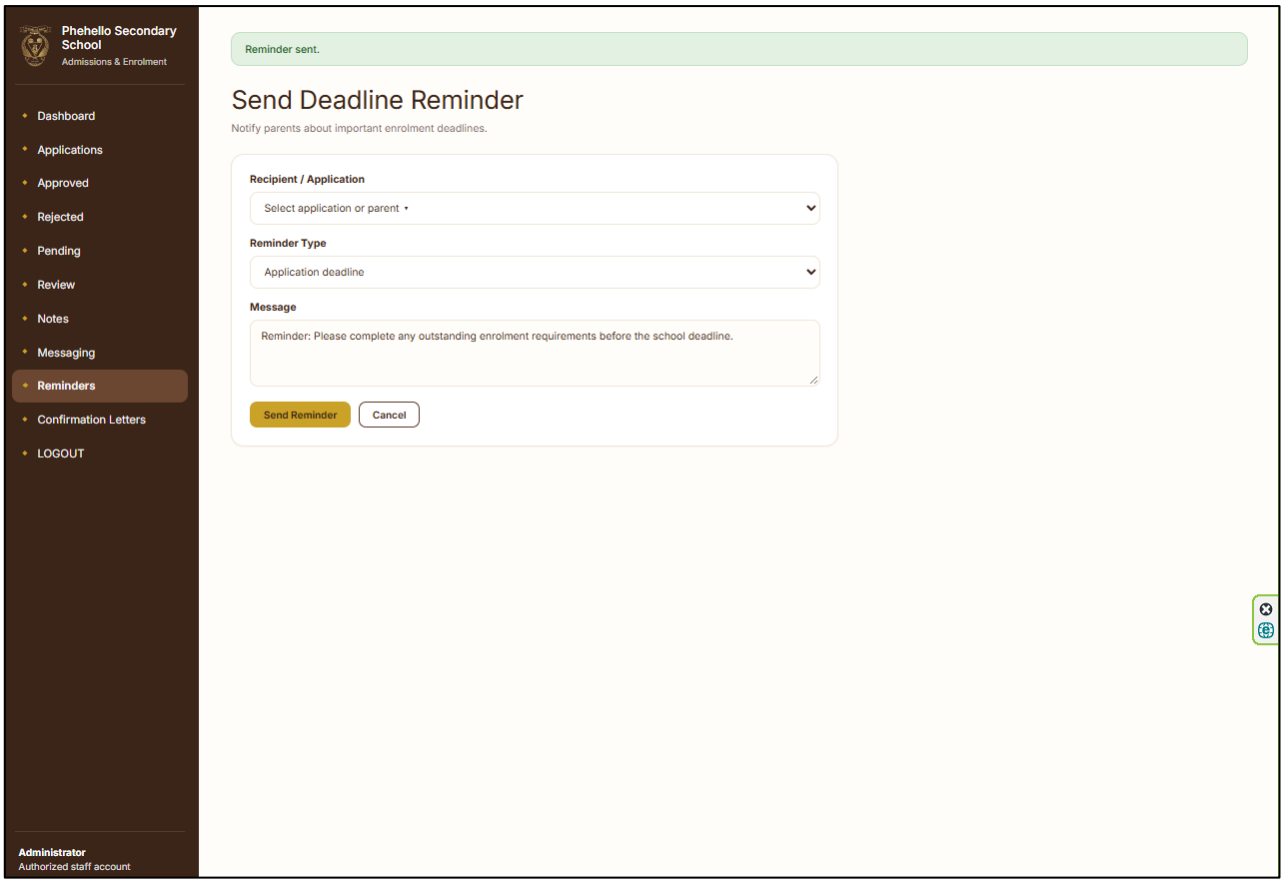


Figure 4.53: Send Reminder. The Send Deadline Reminder page, where the Administrator chooses an application or parent, a reminder type and a message, and sends a reminder about enrolment deadlines (FR11).

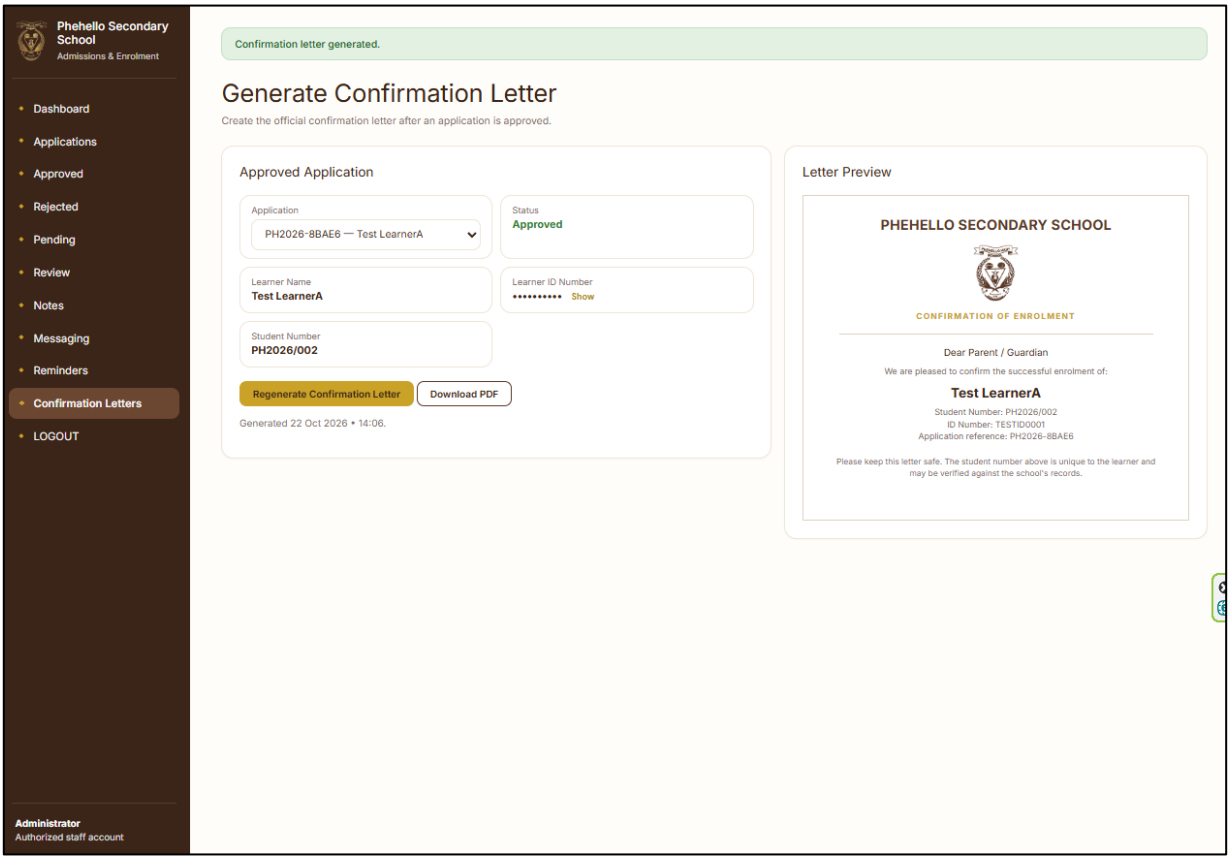


Figure 4.54: Confirmation Letter. The Confirmation Letter page, which shows the learner's name, ID number and student number for an approved or rejected application and previews the letter with the school logo before it is generated (FR12).

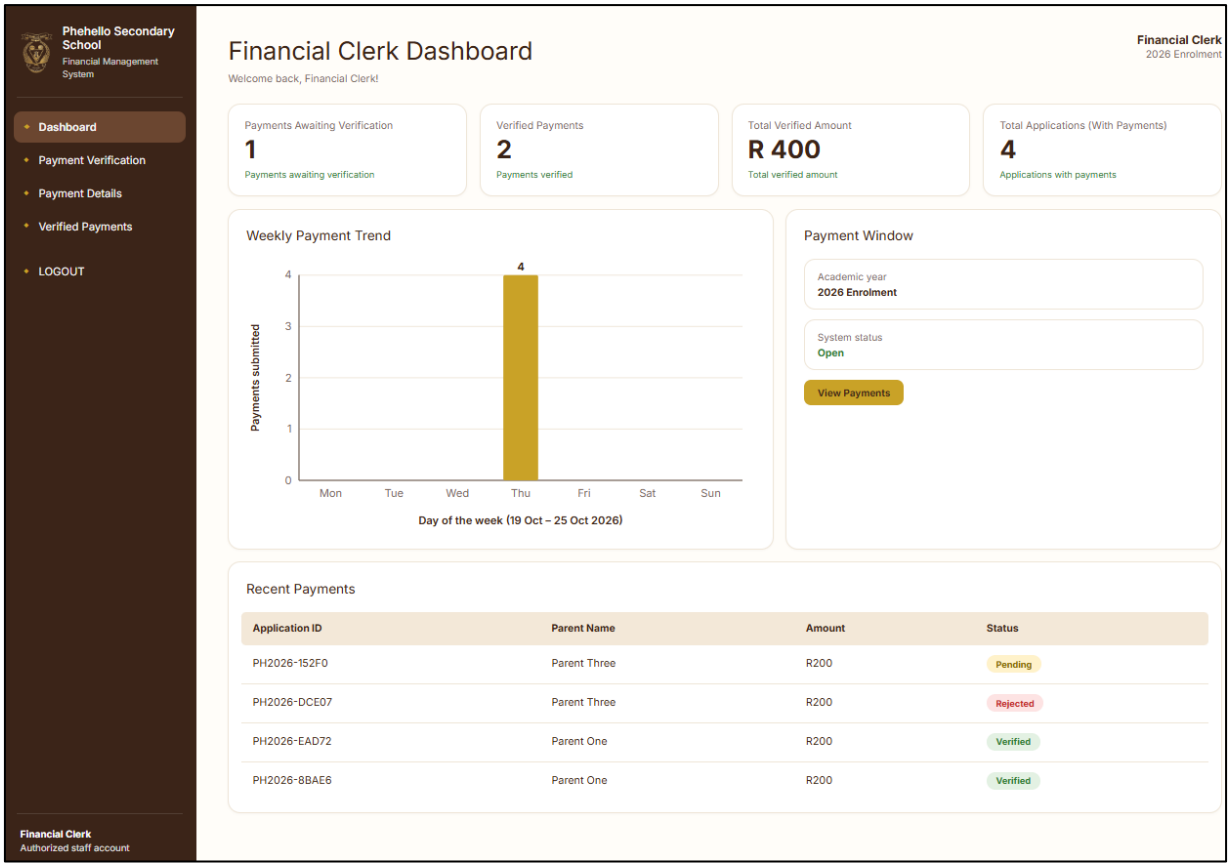


Figure 4.55: Financial Clerk Dashboard. The Financial Clerk Dashboard, showing payments awaiting verification, verified payments, the total verified amount, a weekly payment trend chart and recent payments (FR6).

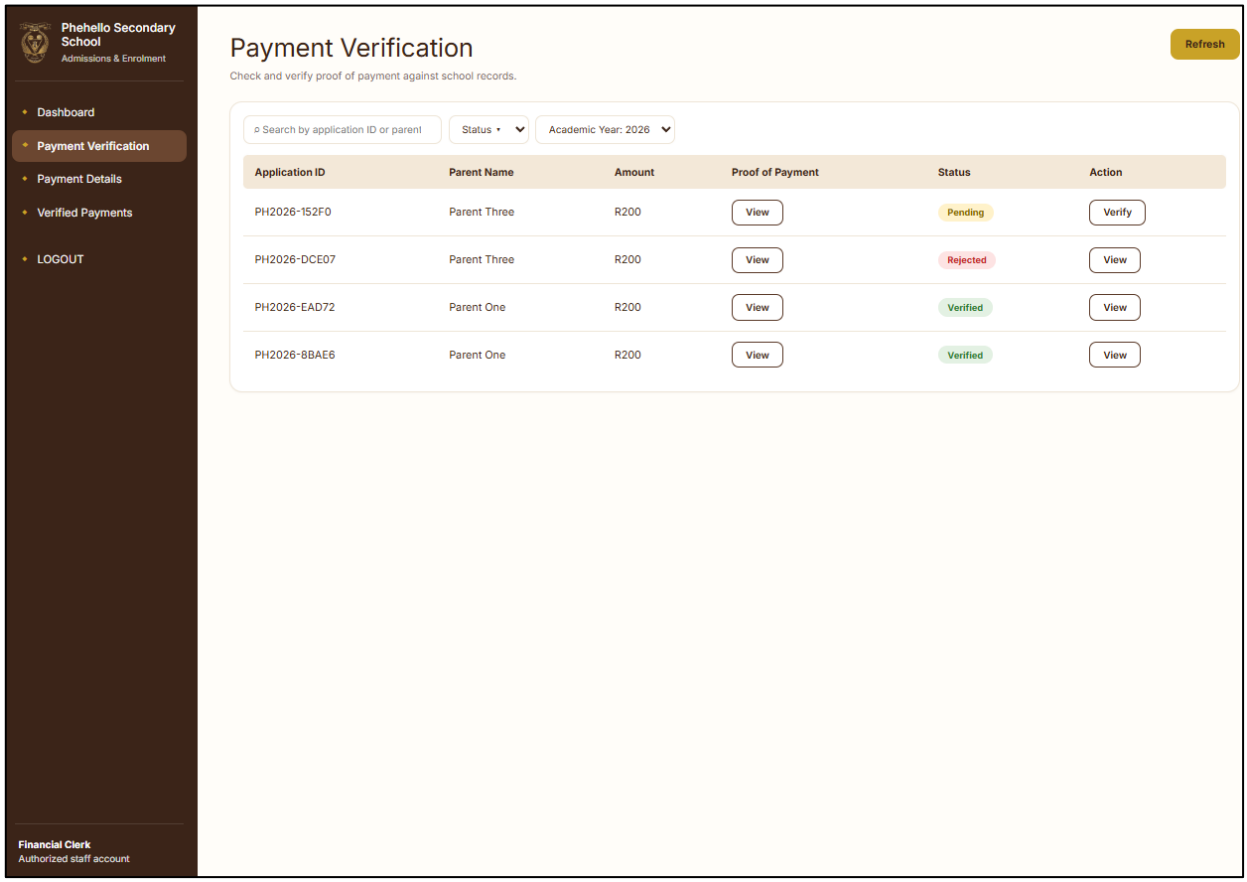


Figure 4.56: Payment Verification. The Payment Verification page, which lists submitted payments with the parent's name, the amount, a View button for the proof of payment, the status and the Verify and Reject actions (FR6).

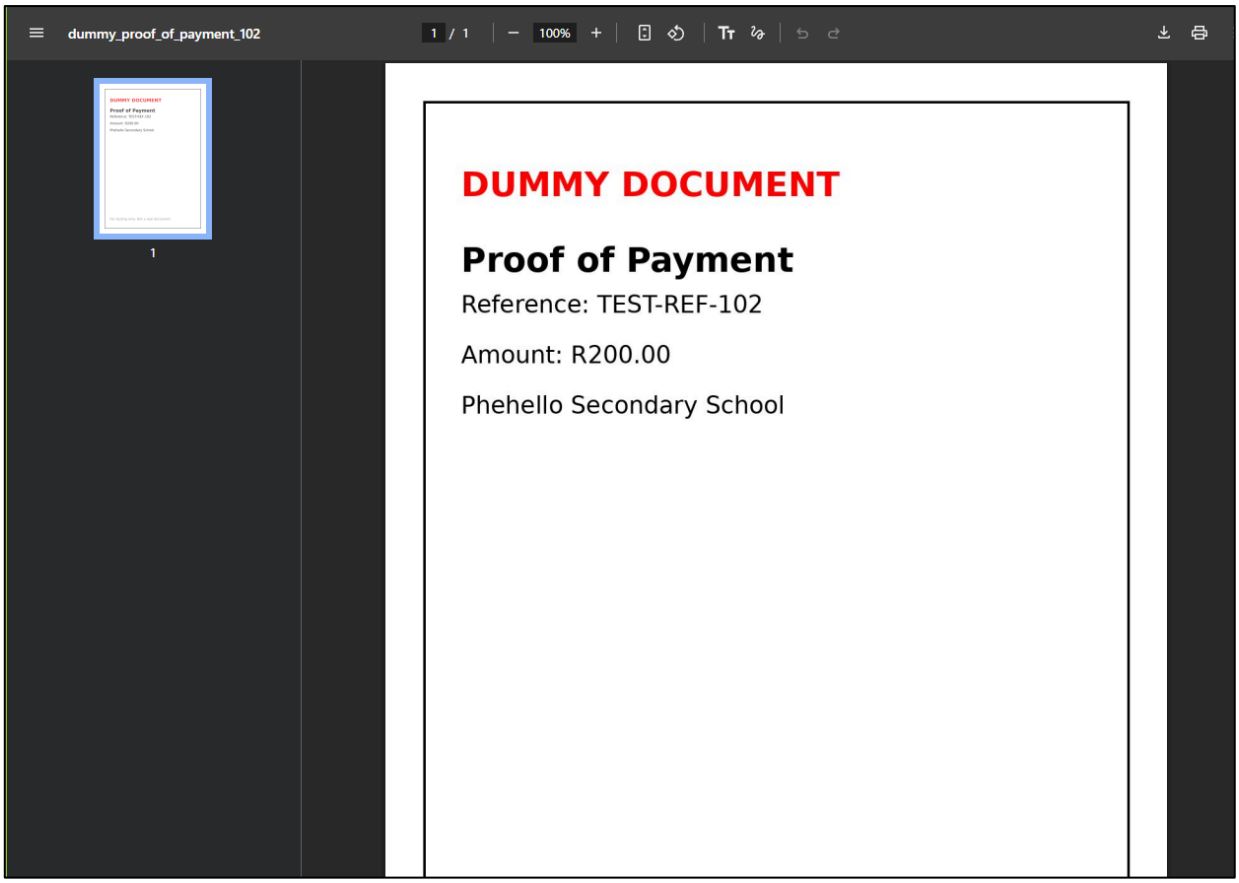


Figure 4.57: Proof of Payment. The Proof of Payment page, opened from Payment Verification. It displays the uploaded proof-of-payment file so that the Financial Clerk can check it against the school's records (FR6).

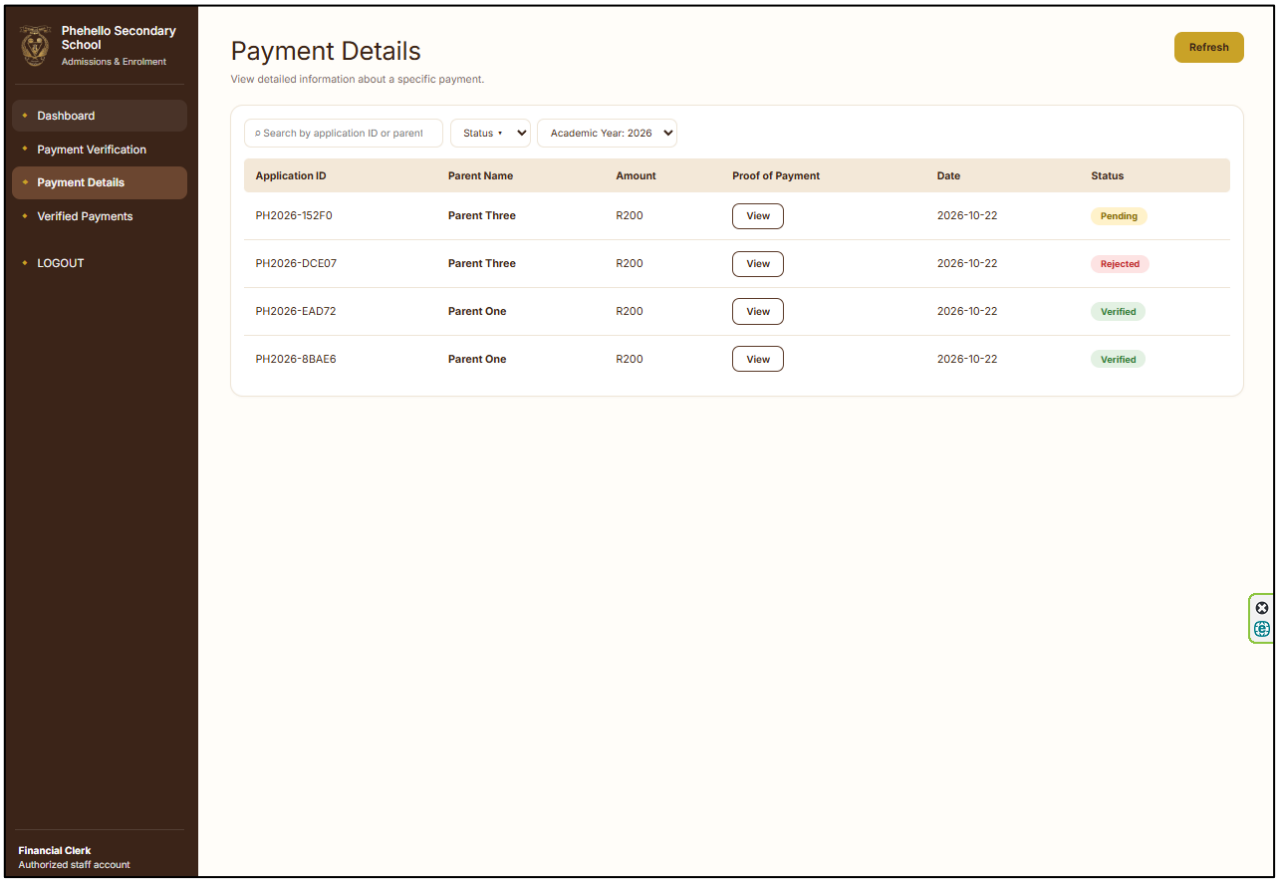


Figure 4.58: Payment Details. The Payment Details page, a searchable list of payments with the parent's name, amount, date and status, which the Financial Clerk uses to look up a specific payment.

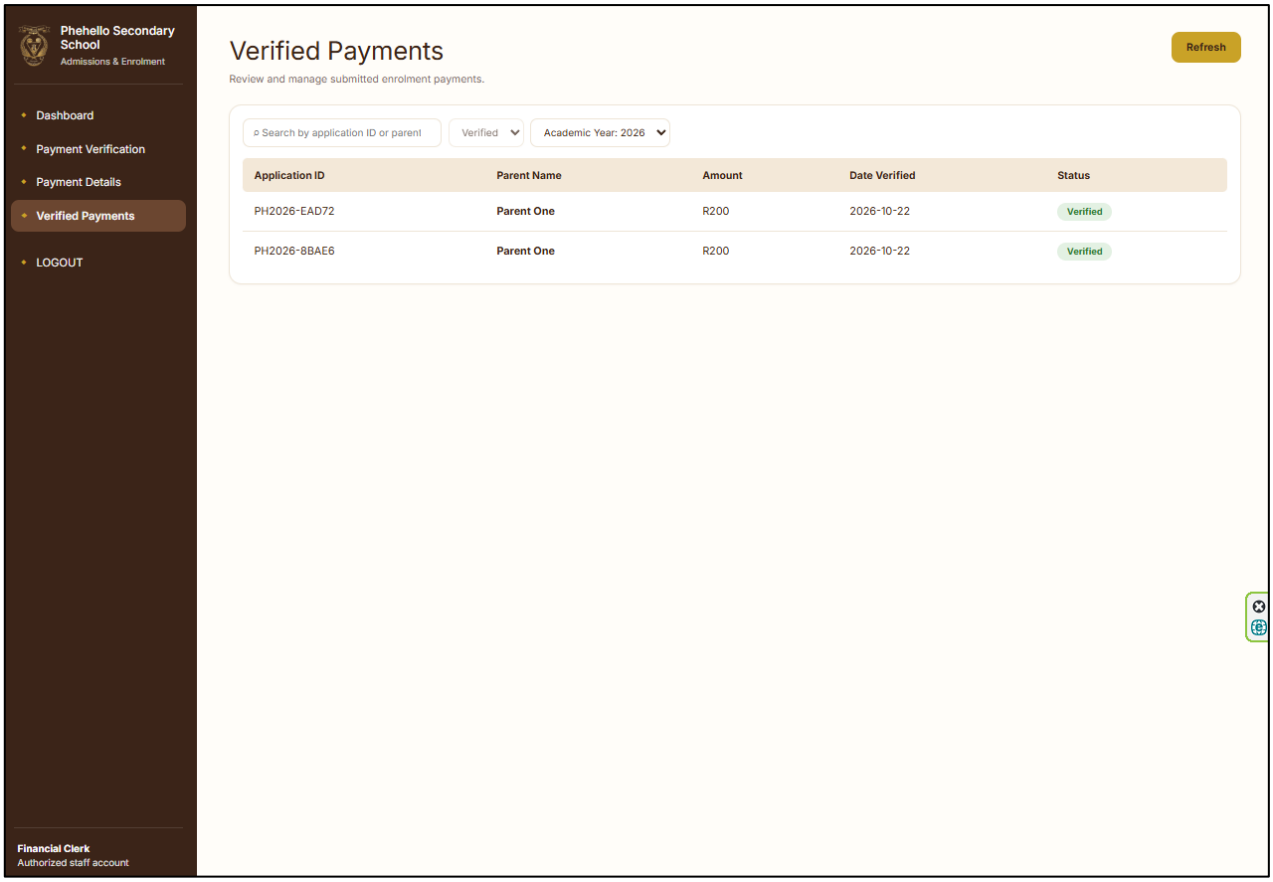


Figure 4.59: Verified Payments. The Verified Payments page, which lists the payments the Financial Clerk has already verified, kept for the Clerk's own record-keeping.

Recording file name or link: Phehello Systems (ASP.NET recording)

Figure 4.60: Navigation through the application. A screen recording of navigation through the ASP.NET web app, moving from the login page to the dashboards and the main staff pages.

Function screenshots or short recordings

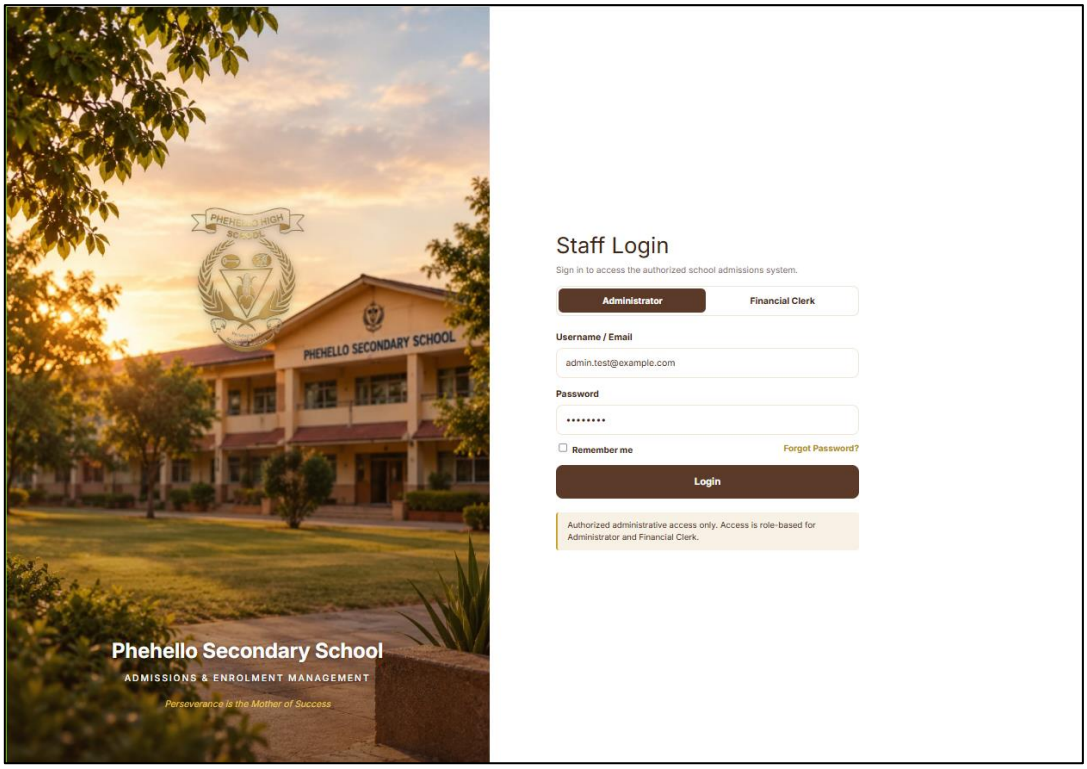


Figure 4.61: Staff sign in, (a) the Administrator signed in. Staff sign-in on the web app. Part (a) shows the Administrator signing in.

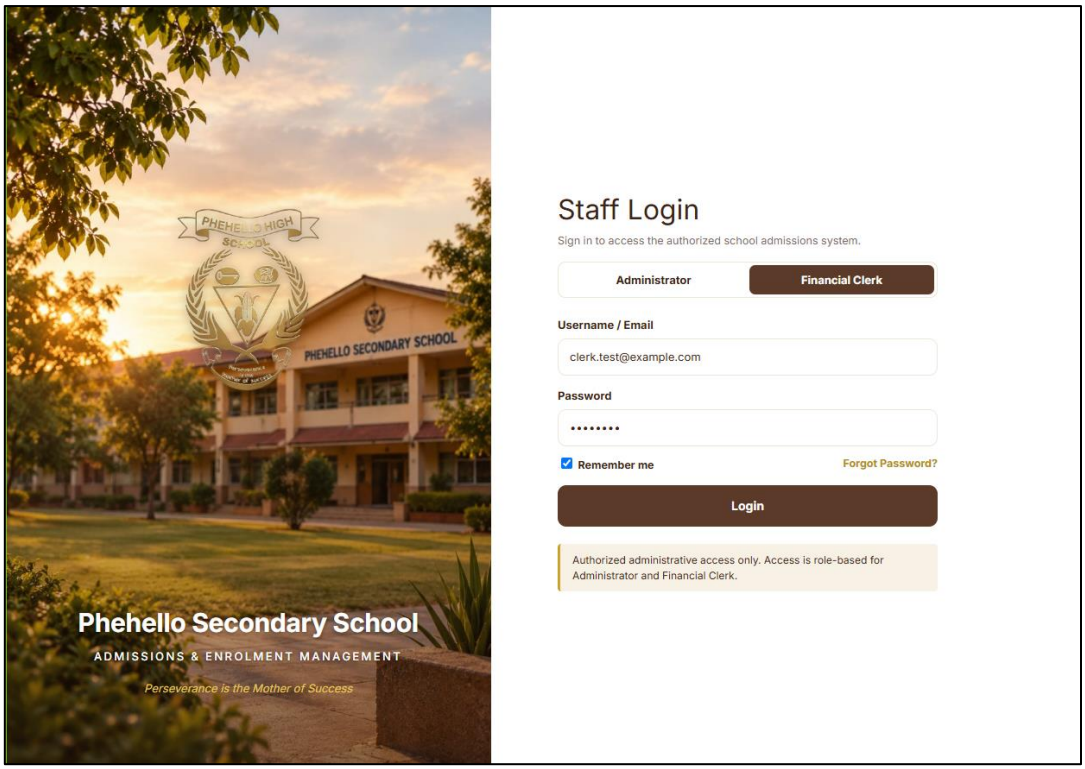


Figure 4.61: Staff sign in, (b) the Financial Clerk signed in. Staff sign-in on the web app. Part (b) shows the Financial Clerk signing in

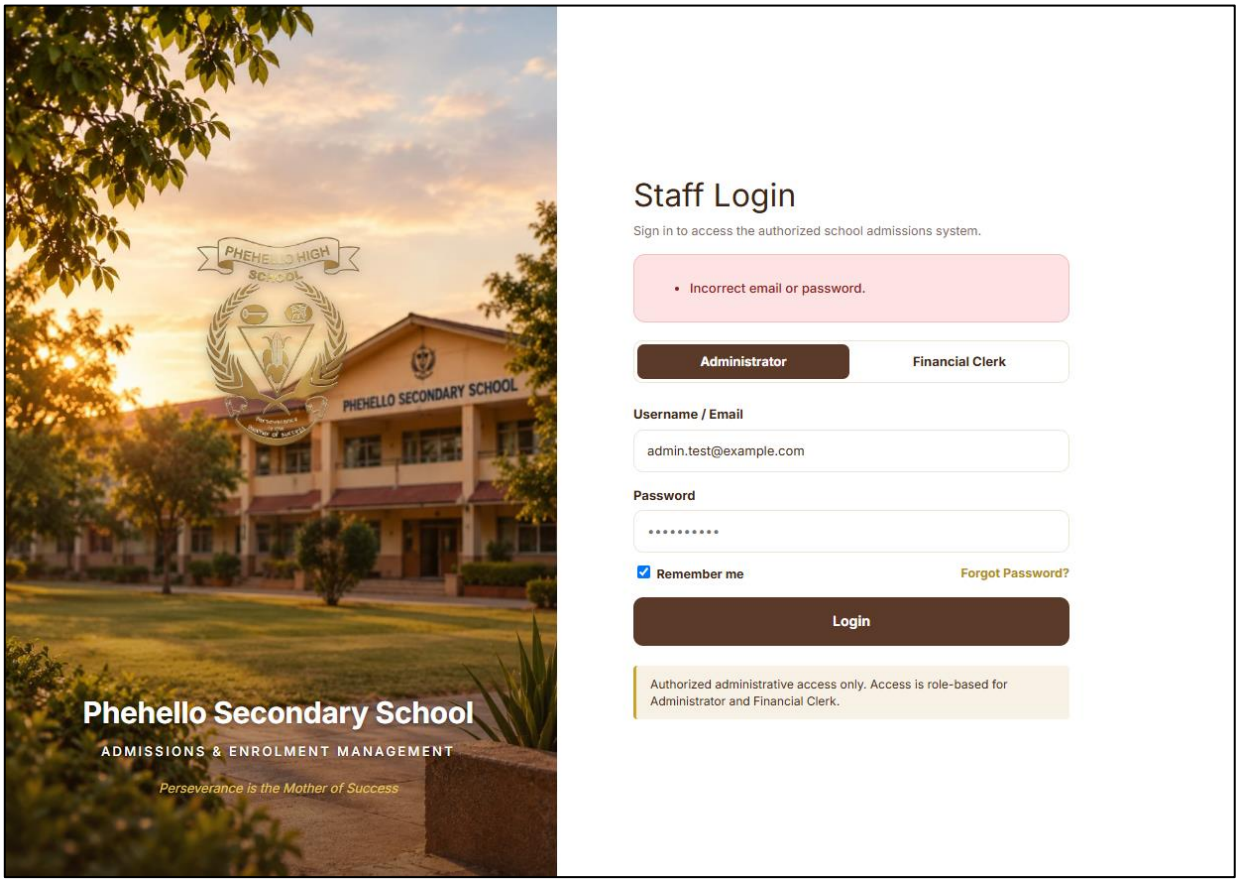


Figure 4.61: Staff sign in, (c) the error message for a wrong password. Staff sign-in on the web app. part (c) shows the error message displayed when a wrong password is entered.

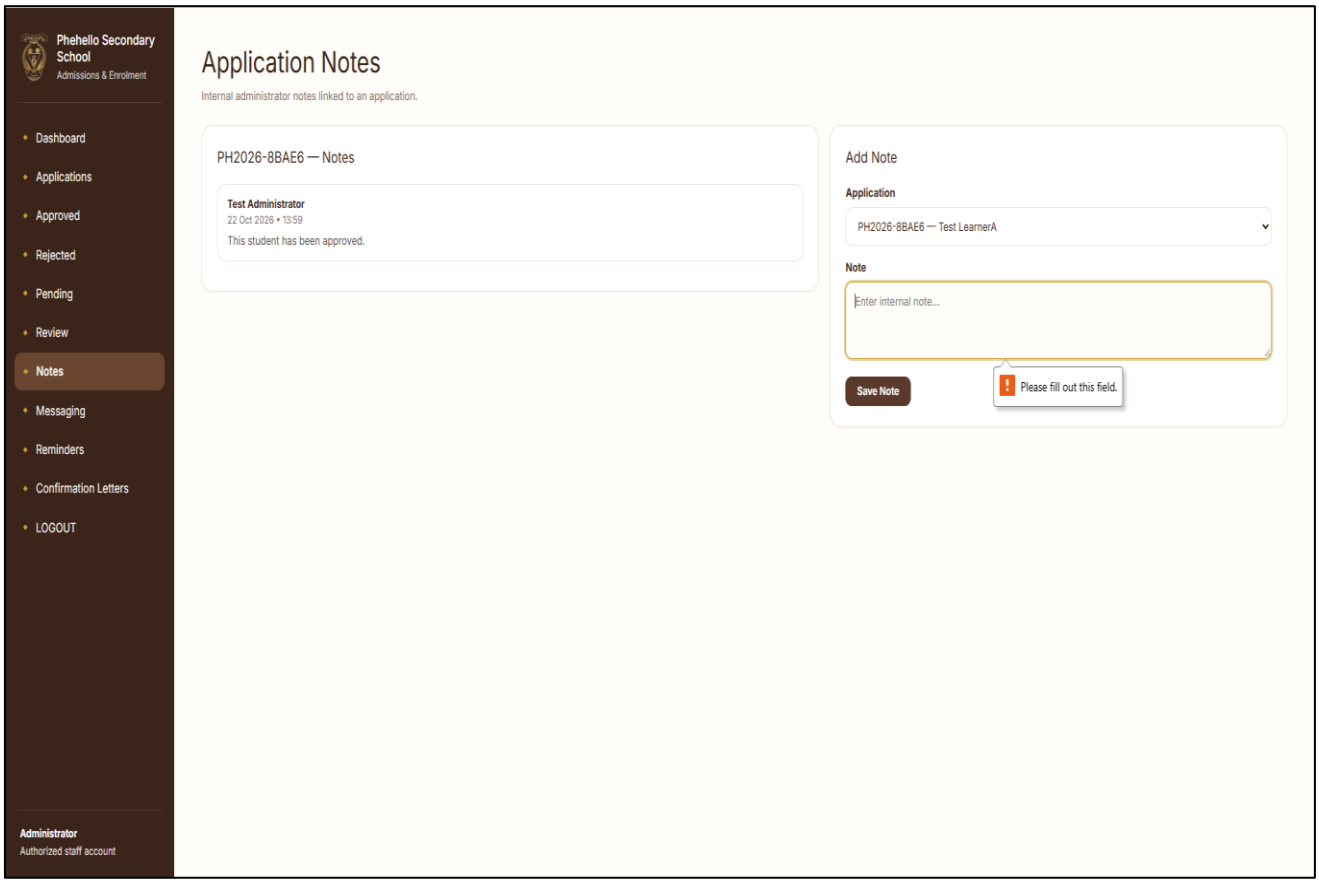


Figure 4.62: Form validation, (a) an empty note. Form validation on the staff pages. Each part shows a submission being blocked with its message: an empty note

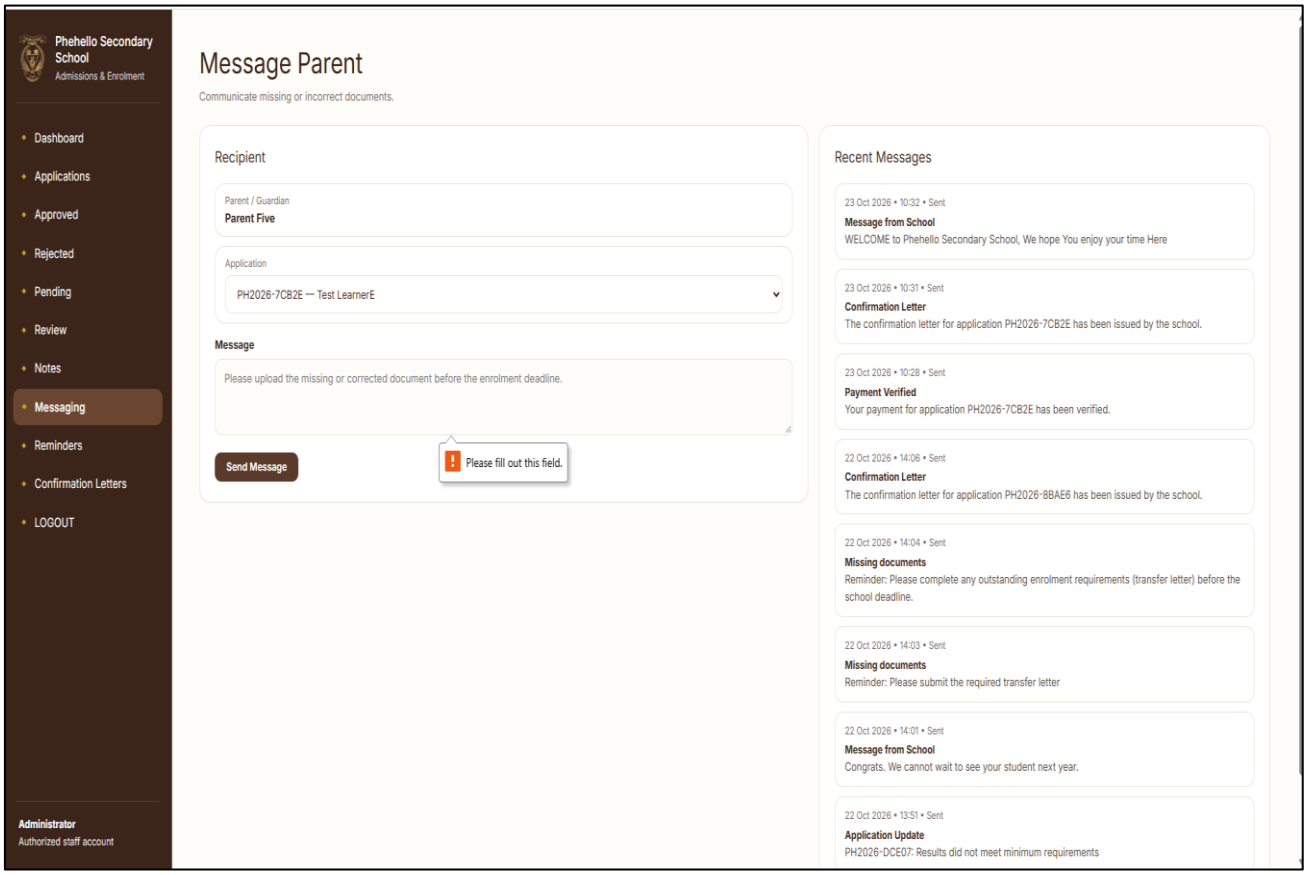


Figure 4.62: Form validation, (b) an empty message. Form validation on the staff pages. Each part shows a submission being blocked with its message: an empty message

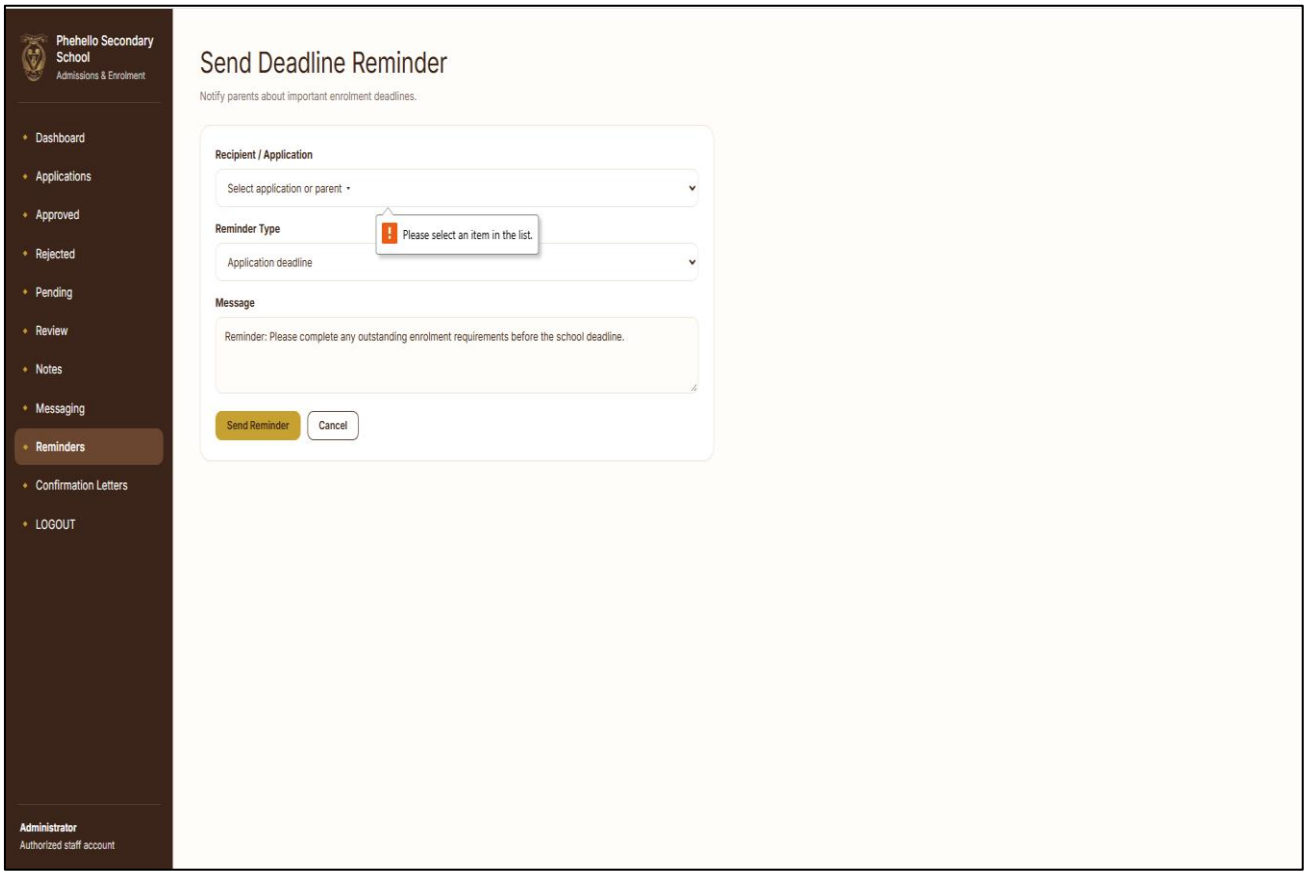


Figure 4.62: Form validation, *(c) an invalid reminder, each with its message.* Form validation on the staff pages. Each part shows a submission being blocked with its message: an invalid reminder.

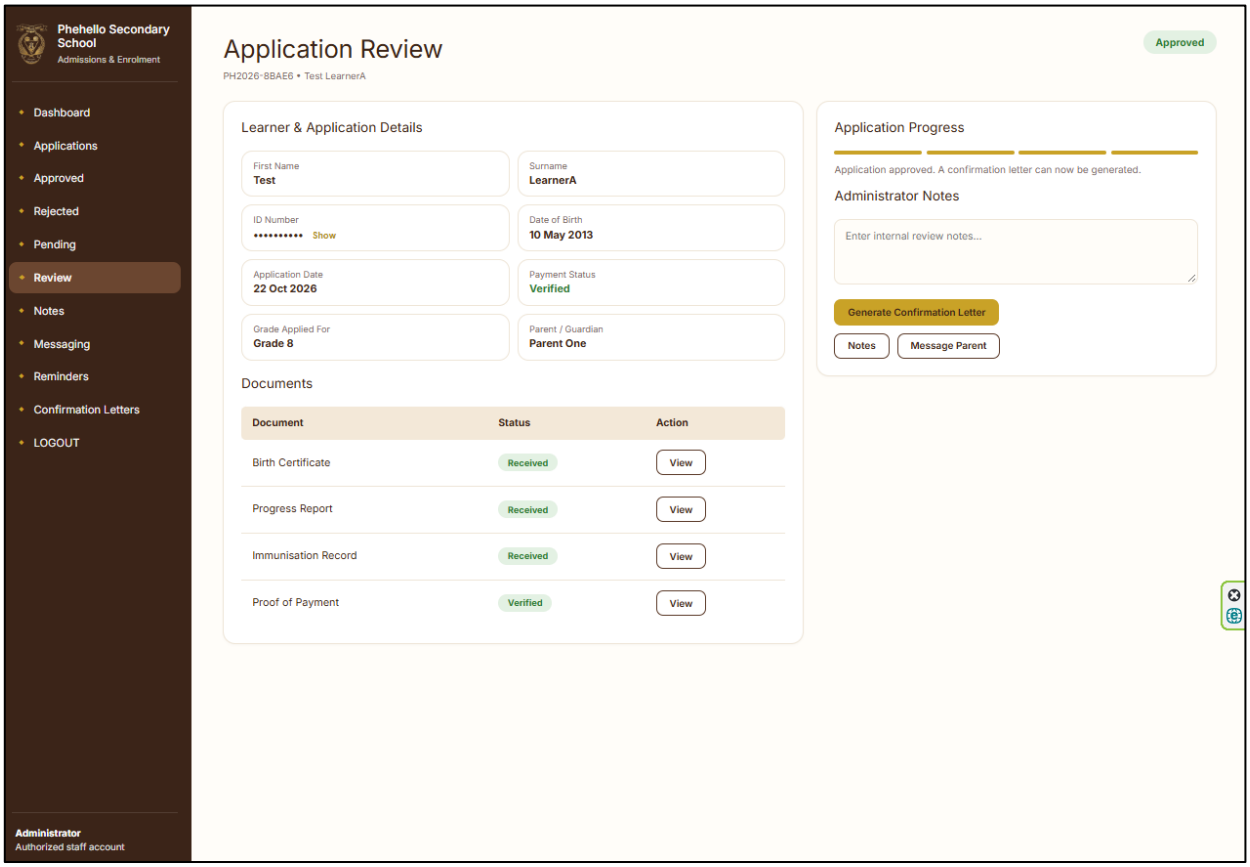


Figure 4.63: Approve, reject or request corrections. The Application Review page after the Administrator has made a decision. The status badge at the top right shows the new status, and the same page is used to reject an application or request corrections (FR8).

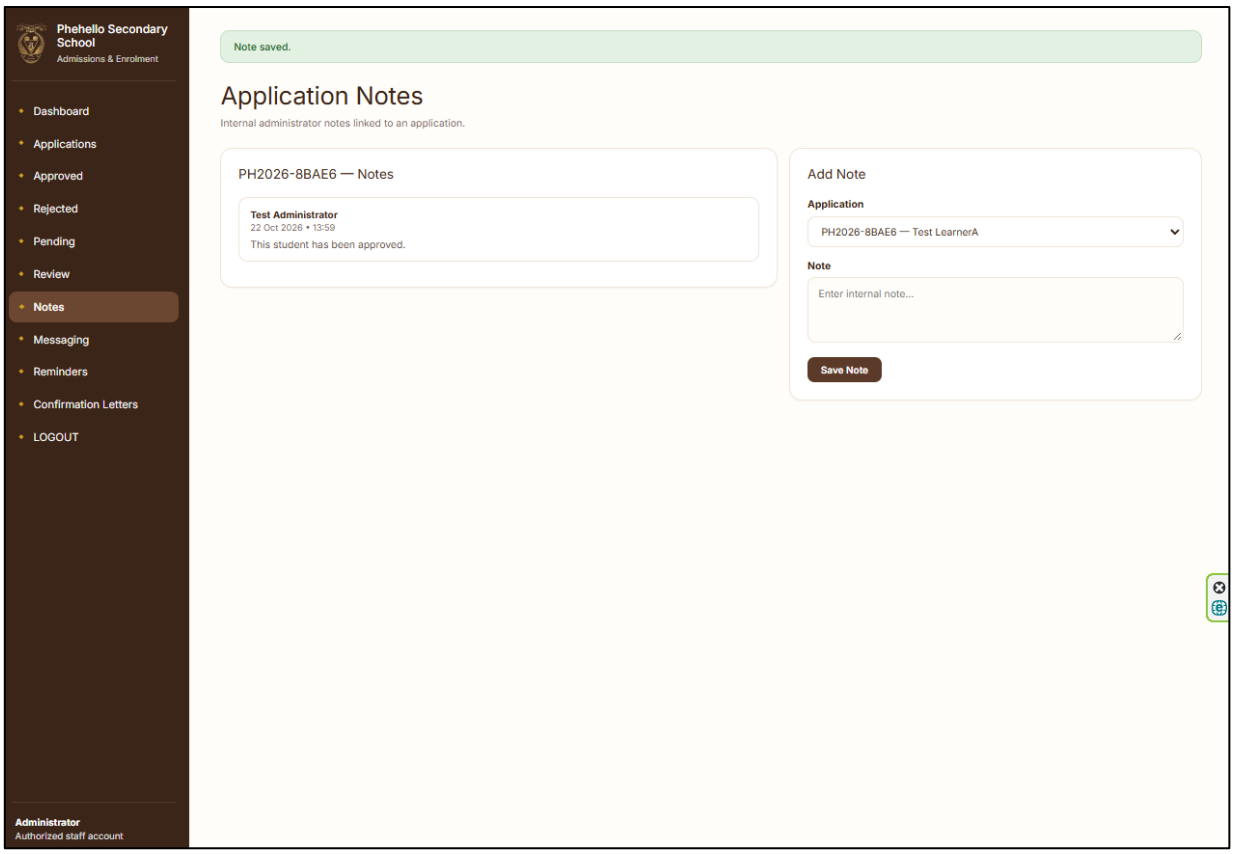


Figure 4.64: Add a note. Adding a note: the Application Notes page after the Administrator saves a note, showing the saved confirmation and the new note listed with its author and time (FR9).

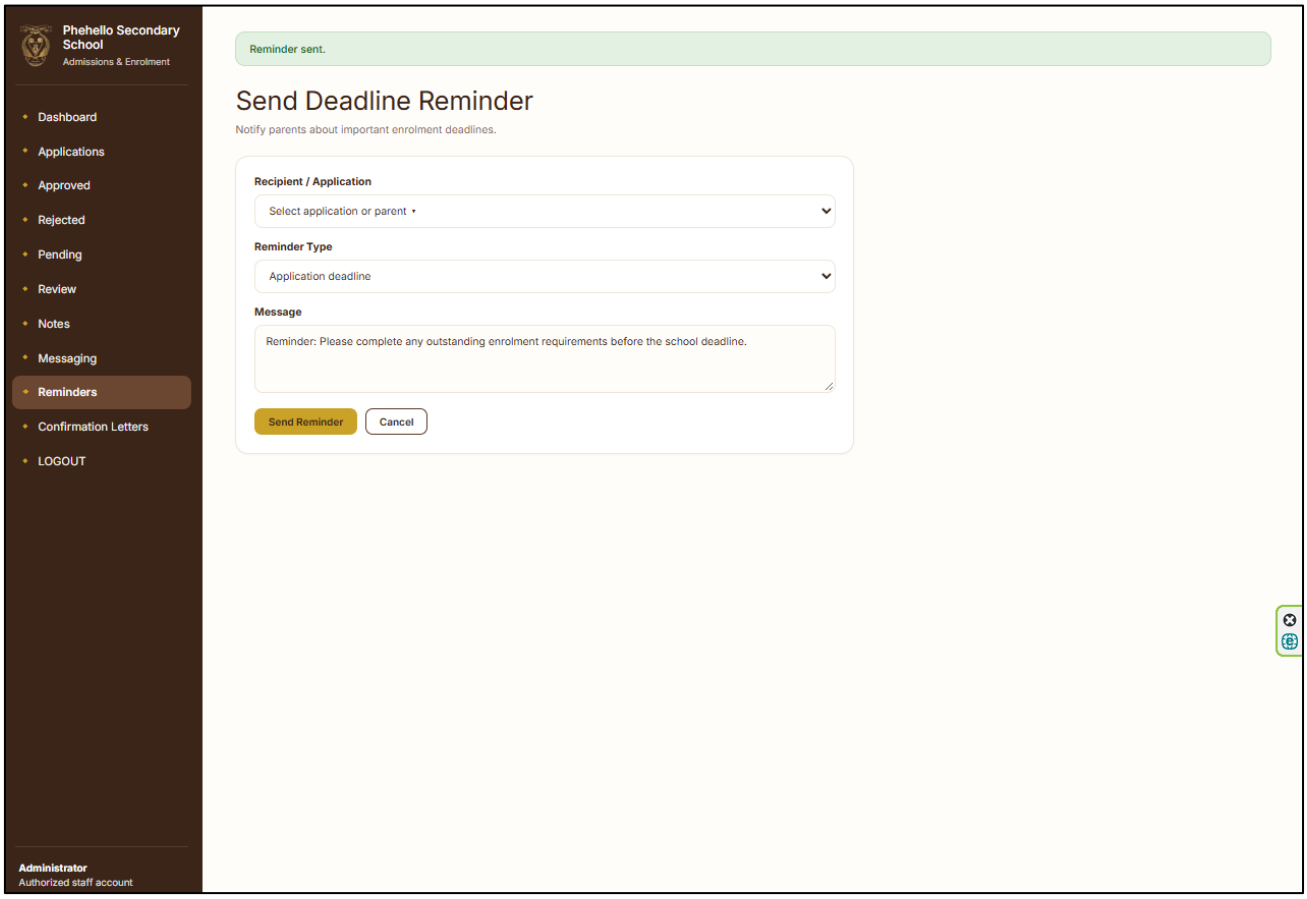


Figure 4.65: Message a parent and send a reminder, *(a) the reminder sent confirmation.* Communicating with parents. Part (a) shows the confirmation displayed after a deadline reminder is sent.

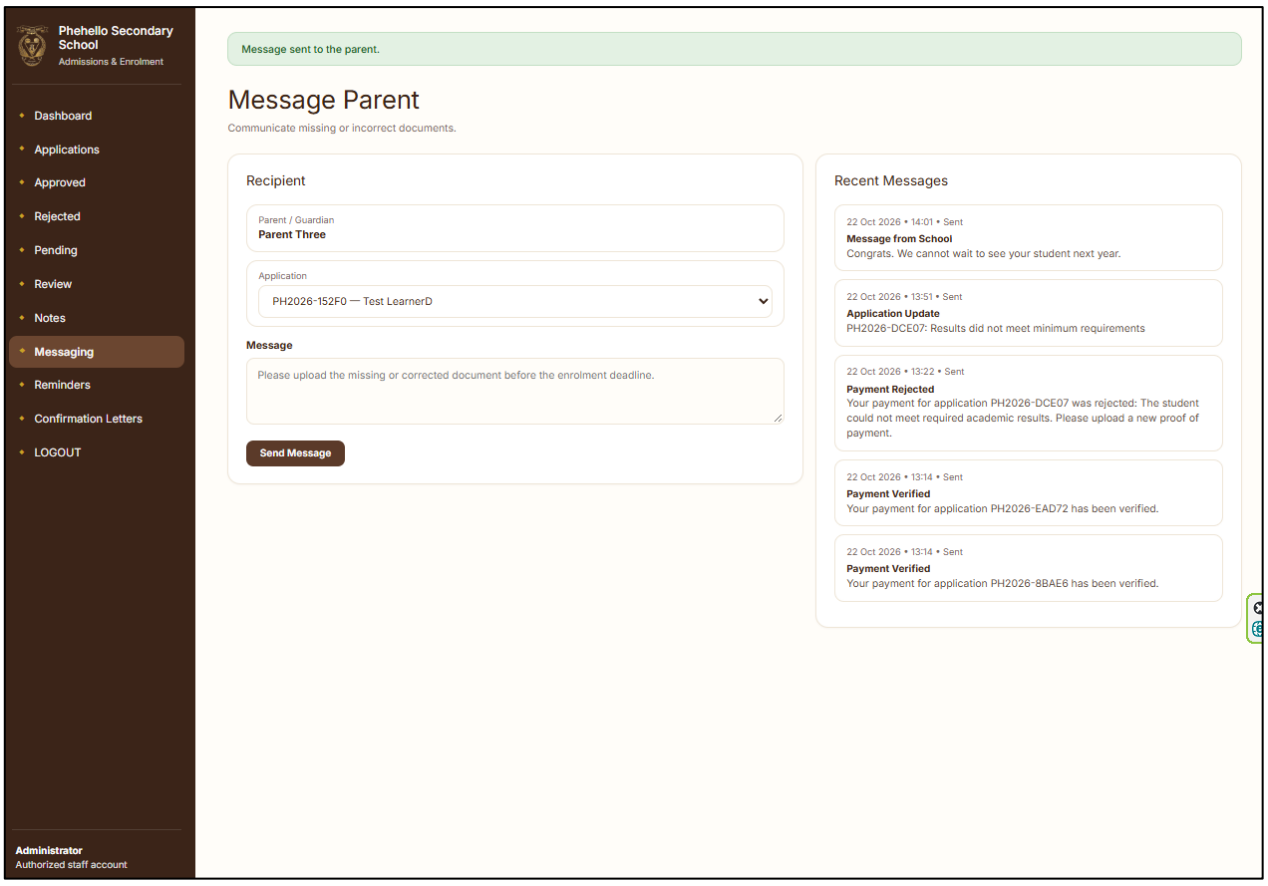


Figure 4.65, *(b) the message sent confirmation.* Communicating with parents. part (b) shows the confirmation after a message is sent to a parent, with the message listed under Recent Messages (FR10, FR11).

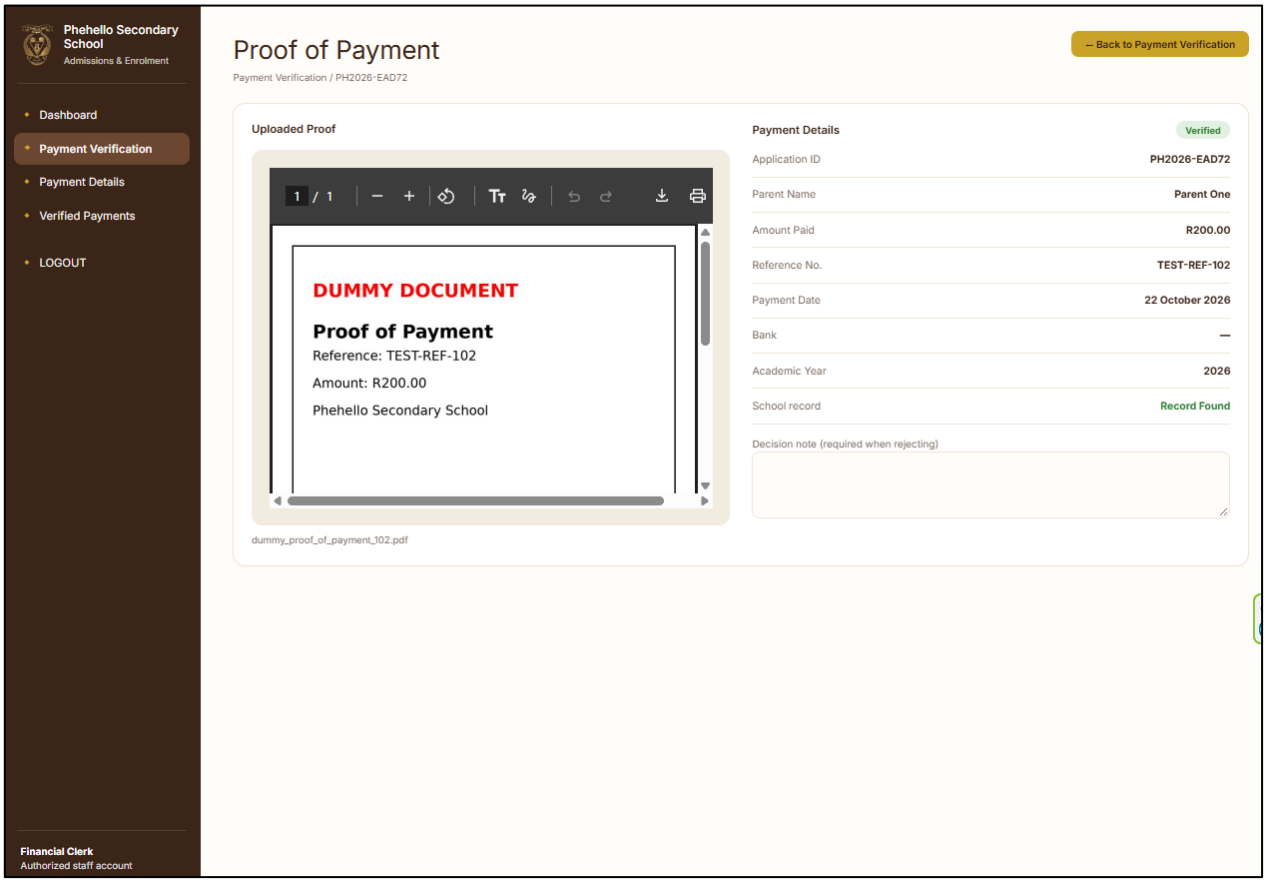


Figure 4.66: Verify a payment, bank not appearing due to proof of payment bot having it. Verifying a payment: the Proof of Payment page shows the uploaded proof next to the payment details. The Bank field is empty because the proof of payment that was uploaded does not contain a bank name.

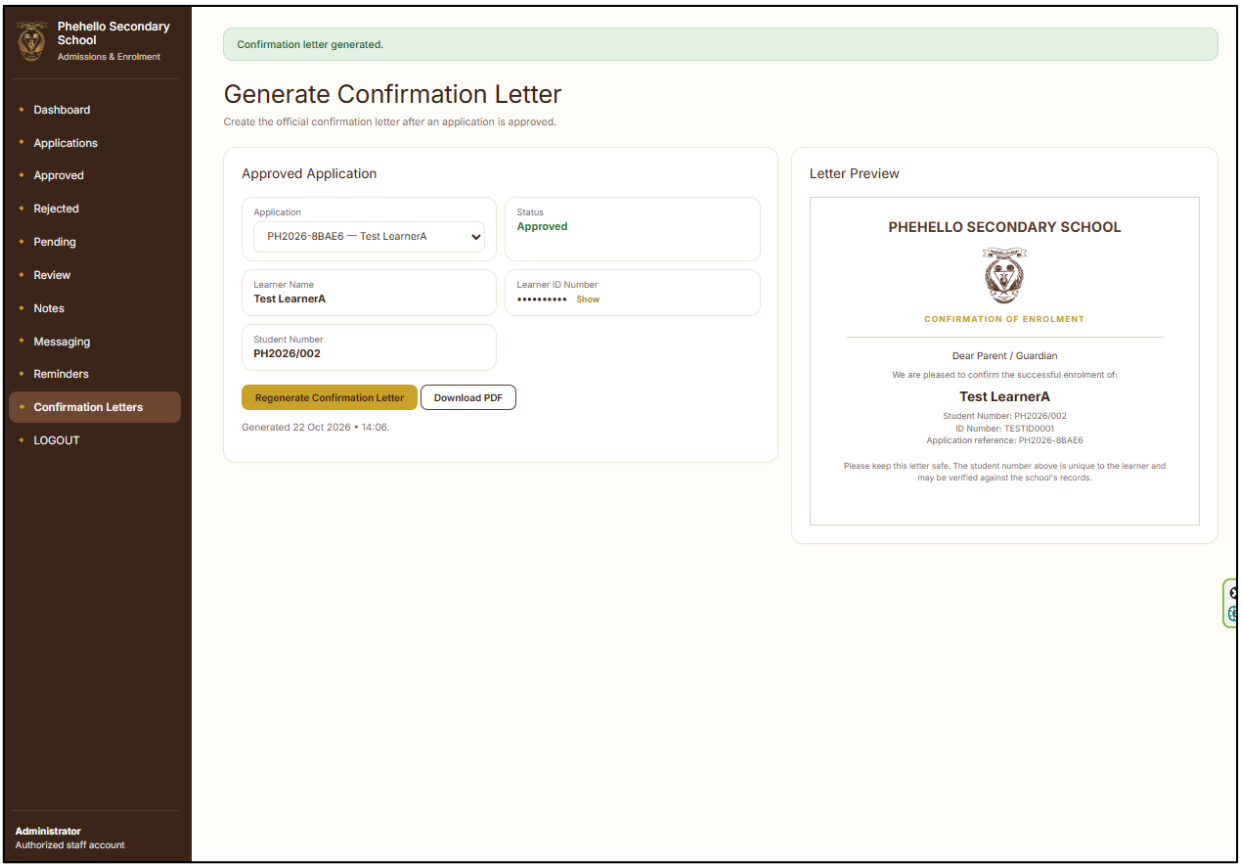


Figure 4.67: Generate a confirmation letter. Generating a confirmation letter: the Confirmation Letter page after the Administrator generates a letter, showing the confirmation message and the letter preview with the school logo, learner name and student number (FR12).

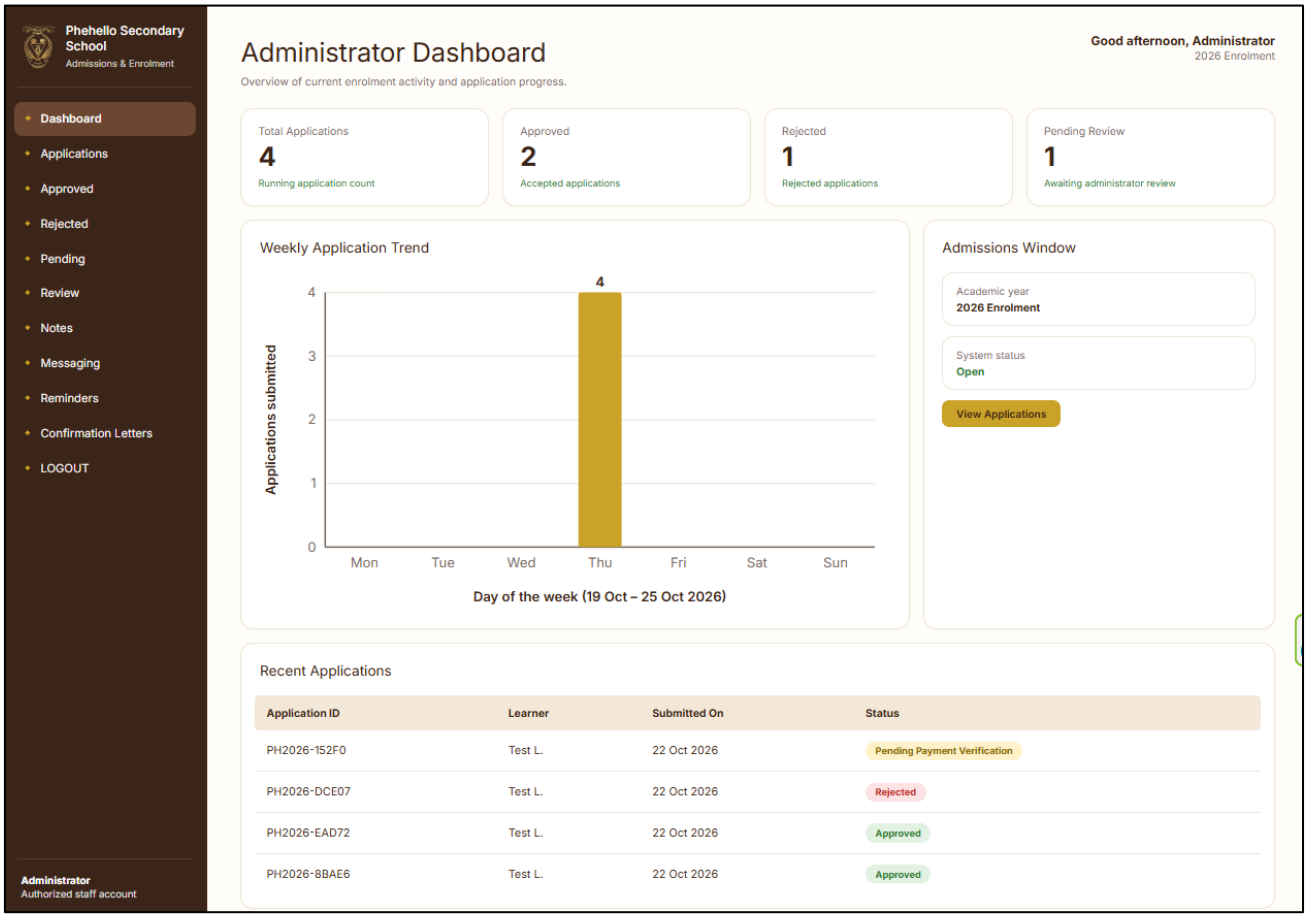


Figure 4.68: Chart accuracy, a dashboard chart whose bars add up to the total count shown on the same page. Chart accuracy: the Administrator Dashboard, where the weekly application chart shows the number of applications per day of the week and its bars add up to the total application count shown at the top of the page.

Code and project records

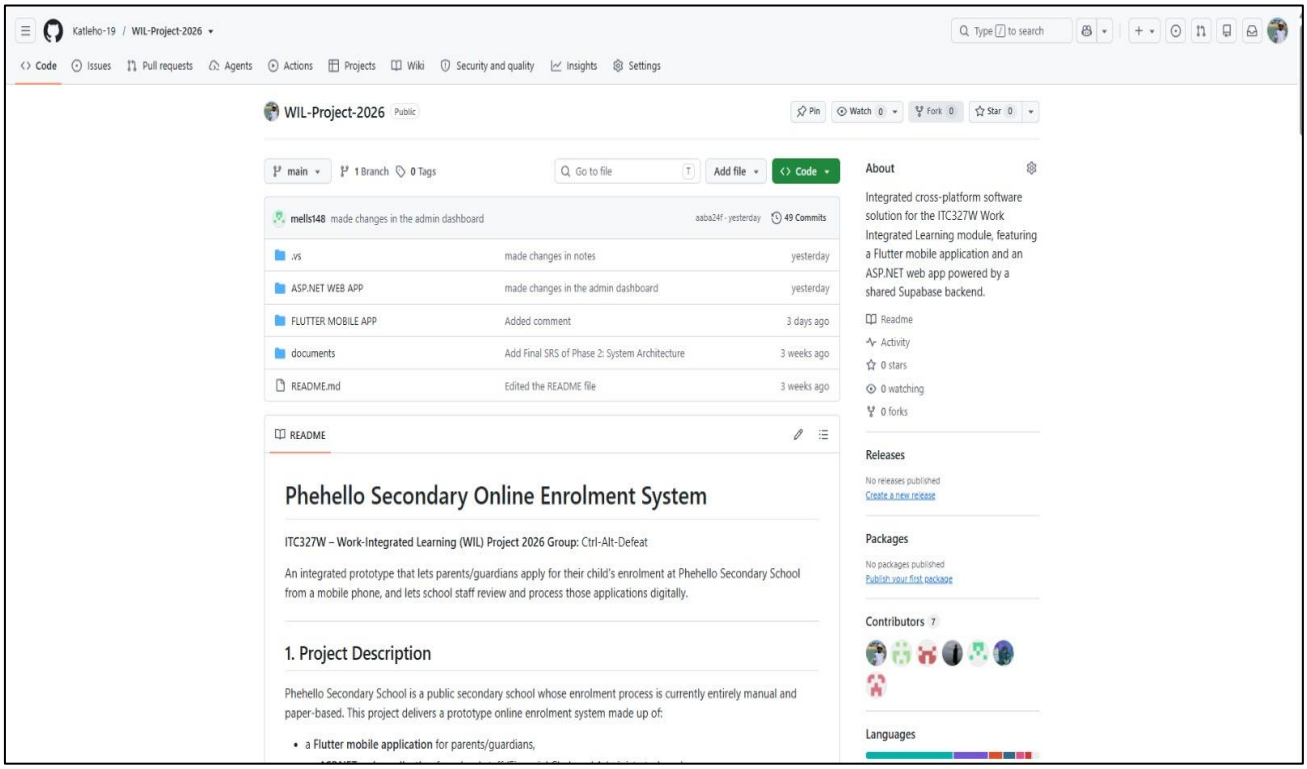


Figure 4.69: GitHub commits, *the mobile/ folder and the commit history with messages and contributors.* The GitHub repository showing the mobile/ folder and the commit history, with each contributor's commit messages.

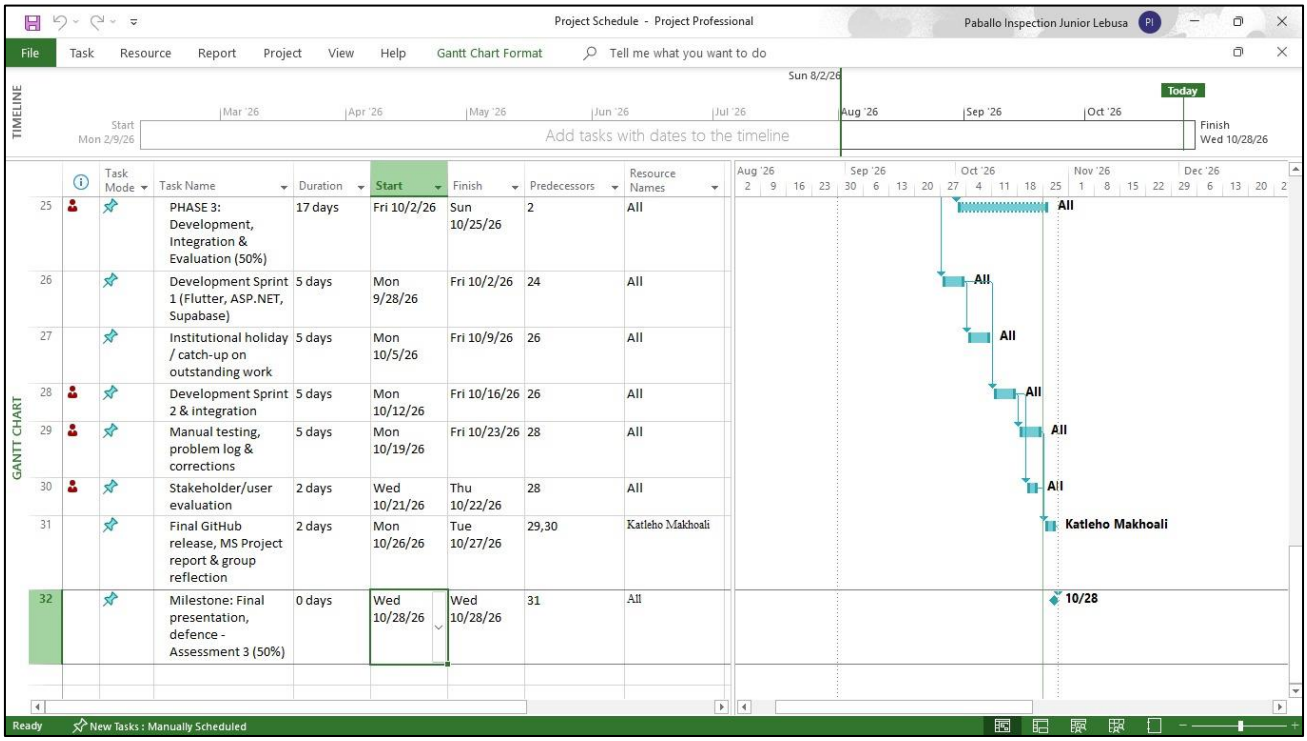
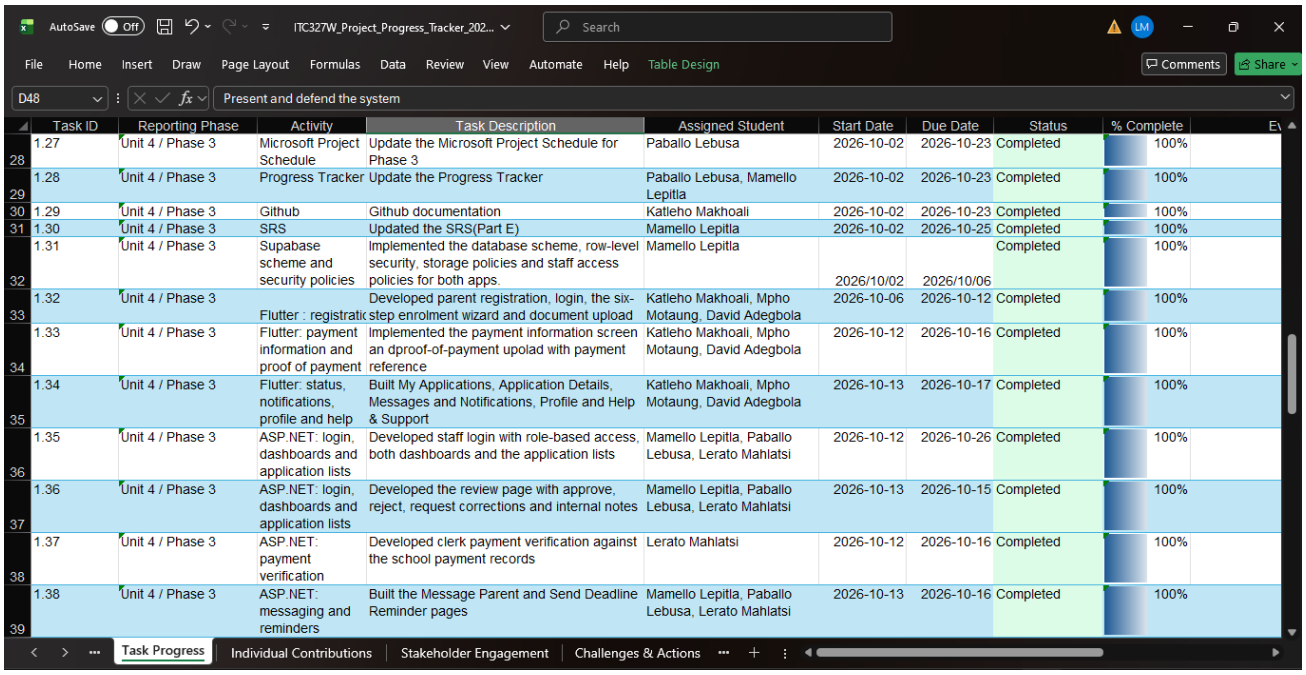


Figure 4.70: Updated Microsoft Project, ***the mobile development tasks with progress.*** The updated Microsoft Project schedule showing the mobile development tasks and their progress.



	Task ID	Reporting Phase	Activity	Task Description	Assigned Student	Start Date	Due Date	Status	% Complete
28	1.27	Unit 4 / Phase 3	Microsoft Project Schedule	Update the Microsoft Project Schedule for Phase 3	Paballo Lebusa	2026-10-02	2026-10-23	Completed	100%
29	1.28	Unit 4 / Phase 3	Progress Tracker	Update the Progress Tracker	Paballo Lebusa, Mamello Lepitla	2026-10-02	2026-10-23	Completed	100%
30	1.29	Unit 4 / Phase 3	Github	Github documentation	Katleho Makhoali	2026-10-02	2026-10-23	Completed	100%
31	1.30	Unit 4 / Phase 3	SRS	Updated the SRS(Part E)	Mamello Lepitla	2026-10-02	2026-10-25	Completed	100%
32	1.31	Unit 4 / Phase 3	Supabase scheme and security policies	Implemented the database scheme, row-level security, storage policies and staff access policies for both apps.	Mamello Lepitla	2026/10/02	2026/10/06	Completed	100%
33	1.32	Unit 4 / Phase 3	Flutter : registratic	Developed parent registration, login, the six-step enrolment wizard and document upload	Katleho Makhoali, Mpho Motaung, David Adegbola	2026-10-06	2026-10-12	Completed	100%
34	1.33	Unit 4 / Phase 3	Flutter: payment information and proof of payment	Implemented the payment information screen an dproof-of-payment upolad with payment reference	Katleho Makhoali, Mpho Motaung, David Adegbola	2026-10-12	2026-10-16	Completed	100%
35	1.34	Unit 4 / Phase 3	Flutter: status, notifications, profile and help	Built My Applications, Application Details, Messages and Notifications, Profile and Help & Support	Katleho Makhoali, Mpho Motaung, David Adegbola	2026-10-13	2026-10-17	Completed	100%
36	1.35	Unit 4 / Phase 3	ASP.NET: login, dashboards and application lists	Developed staff login with role-based access, both dashboards and the application lists	Mamello Lepitla, Paballo Lebusa, Lerato Mahlatsi	2026-10-12	2026-10-26	Completed	100%
37	1.36	Unit 4 / Phase 3	ASP.NET: login, dashboards and application lists	Developed the review page with approve, reject, request corrections and internal notes	Mamello Lepitla, Paballo Lebusa, Lerato Mahlatsi	2026-10-13	2026-10-15	Completed	100%
38	1.37	Unit 4 / Phase 3	ASP.NET: payment verification	Developed clerk payment verification against the school payment records	Lerato Mahlatsi	2026-10-12	2026-10-16	Completed	100%
39	1.38	Unit 4 / Phase 3	ASP.NET: messaging and reminders	Built the Message Parent and Send Deadline Reminder pages	Mamello Lepitla, Paballo Lebusa, Lerato Mahlatsi	2026-10-13	2026-10-16	Completed	100%

Figure 4.71: Updated Project Progress Tracker, ***completed tasks, who did them, progress and challenges.*** The updated Project Progress Tracker showing the completed tasks, who completed them, the progress and the challenges.

Stakeholder Confirmation of the Developed System

At the end of Unit 4, a confirmation form covering the system developed in Part E was sent to the stakeholder for signing. The stakeholder's signed form is included below as evidence.

Date the form was sent	23 October 2026
Method of communication	WhatsApp
Date signed and returned	25 October 2026
Stakeholder name and role	Maleshoane Gweneth Adegbola)– Principal, Phehello Secondary School (official stakeholder)
What was confirmed	That the developed system – the Flutter mobile application for parents/guardians, the ASP.NET web application for the Financial Clerk and Administrator, and the shared Supabase backend – reflects the enrolment process and requirements confirmed by the school.
Corrections or comments from the stakeholder	She liked that the app could easily approve and reject learners .
Confirmation accepted?	Yes
Evidence available	Figure 4.72 (returned form), Figure 4.73 (WhatsApp conversation), video file under Phehello systems(Stakeholder Confirmation Video) included in ZIP

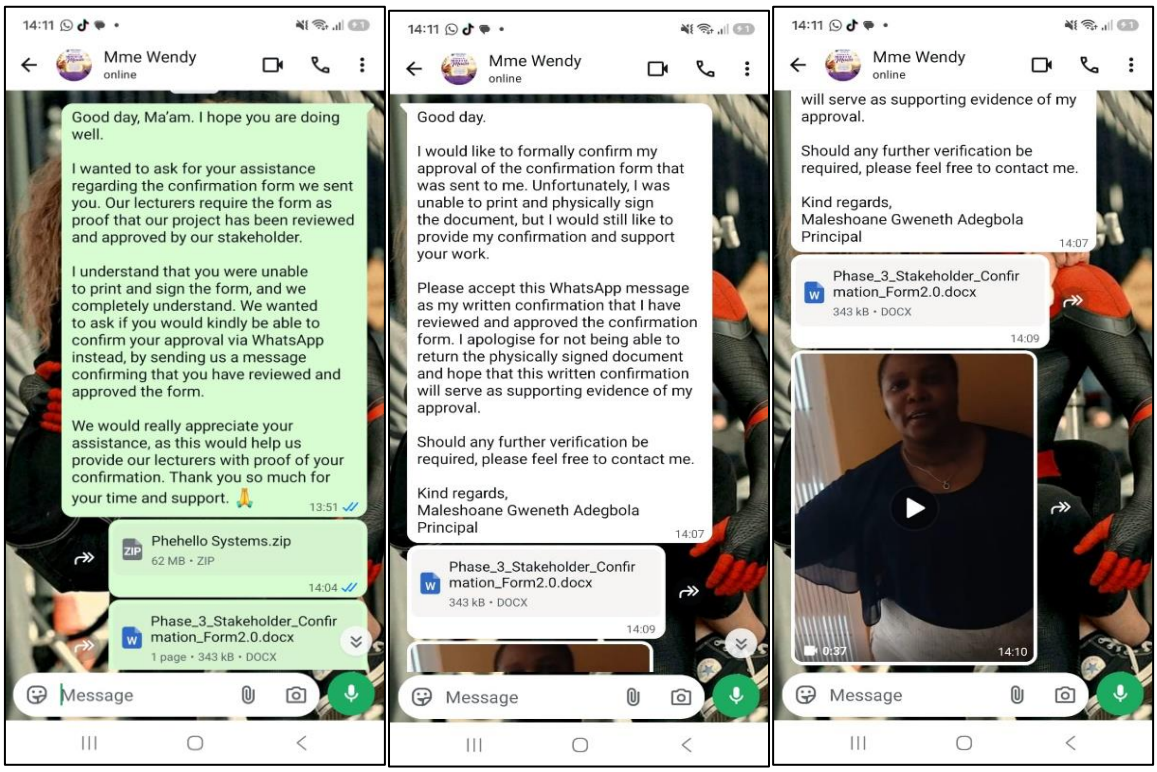
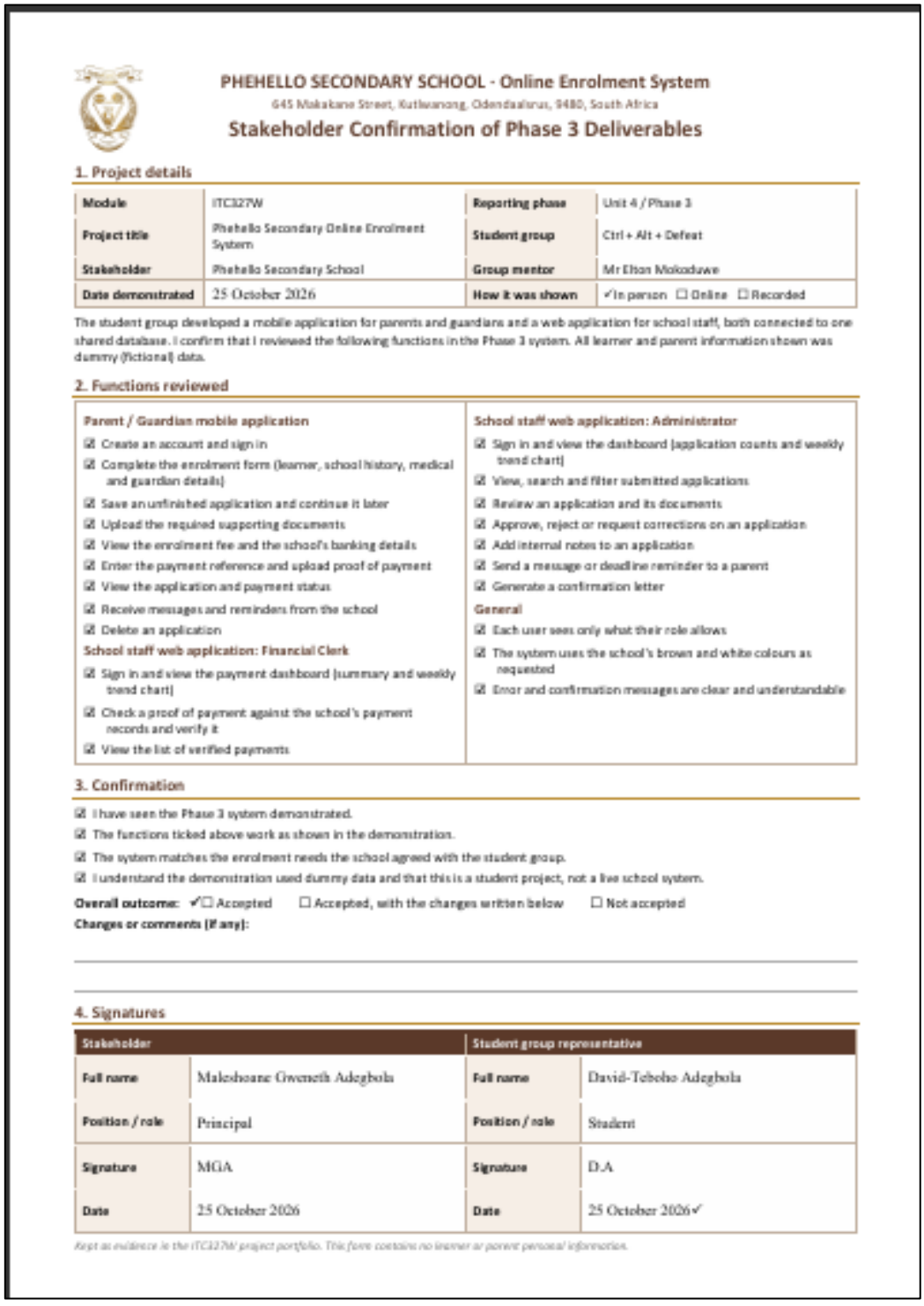


Figure 4.73– WhatsApp conversation with the Principal.The WhatsApp messages showing the form being sent to the Principal and returned to the group, with her video confirmation.



PHEHELLO SECONDARY SCHOOL - Online Enrolment System

645 Makakane Street, Kutlwanong, Odendaalsrus, 9480, South Africa

Stakeholder Confirmation of Phase 3 Deliverables

1. Project details

Module	ITC327W	**Reporting phase**	Unit 4 / Phase 3
Project title	Phehello Secondary Online Enrolment System	**Student group**	Ctrl + Alt + Defeat
Stakeholder	Phehello Secondary School	**Group mentor**	Mr Elton Mokoduwe
Date demonstrated	25 October 2026	**How it was shown**	✓In person ☐ Online ☐ Recorded

The student group developed a mobile application for parents and guardians and a web application for school staff, both connected to one shared database. I confirm that I reviewed the following functions in the Phase 3 system. All learner and parent information shown was dummy (fictional) data.

2. Functions reviewed

Parent / Guardian mobile application

- ☑ Create an account and sign in
- ☑ Complete the enrolment form (learner, school history, medical and guardian details)
- ☑ Save an unfinished application and continue it later
- ☑ Upload the required supporting documents
- ☑ View the enrolment fee and the school's banking details
- ☑ Enter the payment reference and upload proof of payment
- ☑ View the application and payment status
- ☑ Receive messages and reminders from the school
- ☑ Delete an application

School staff web application: Financial Clerk

- ☑ Sign in and view the payment dashboard (summary and weekly trend chart)
- ☑ Check a proof of payment against the school's payment records and verify it
- ☑ View the list of verified payments

School staff web application: Administrator

- ☑ Sign in and view the dashboard (application counts and weekly trend chart)
- ☑ View, search and filter submitted applications
- ☑ Review an application and its documents
- ☑ Approve, reject or request corrections on an application
- ☑ Add internal notes to an application
- ☑ Send a message or deadline reminder to a parent
- ☑ Generate a confirmation letter

General

- ☑ Each user sees only what their role allows
- ☑ The system uses the school's brown and white colours as requested
- ☑ Error and confirmation messages are clear and understandable

3. Confirmation

- ☑ I have seen the Phase 3 system demonstrated.
- ☑ The functions ticked above work as shown in the demonstration.
- ☑ The system matches the enrolment needs the school agreed with the student group.
- ☑ I understand the demonstration used dummy data and that this is a student project, not a live school system.

Overall outcome: ✓☐ Accepted ☐ Accepted, with the changes written below ☐ Not accepted

Changes or comments (if any):

4. Signatures

Stakeholder		Student group representative	
Full name	Makeshoane Gwyneth Adegbola	**Full name**	David-Teboho Adegbola
Position / role	Principal	**Position / role**	Student
Signature	MGA	**Signature**	D.A
Date	25 October 2026	**Date**	25 October 2026✓

Kept as evidence in the ITC327W project portfolio. This form contains no learner or parent personal information.

Figure 4.72 – Returned stakeholder confirmation form. Description: The stakeholder confirmation form as returned by the Principal over WhatsApp, initialled by her.